The Geometry of Motion

Volume 0: The Mathematical Primer

A Foundational Guide to Kinematics, Configuration, and State Space

Preface

Before we can begin to analyze the geometry of moving physical bodies or command them to navigate the world via control theory, we must establish a common language. The physics of movement happens in the real world, but our understanding of it happens entirely within mathematical spaces.

This primer, **Volume 0** of *The Geometry of Motion*, is designed to bridge the gap between basic calculus/algebra and the rigorous differential geometry required for modern nonlinear control theory. By intuitively introducing State Space, Configuration Manifolds, SO(3) Rotations, and Screw Theory, this text ensures that the "lay user" possesses the foundational armor needed to attack the subsequent volumes on Tangent Spaces (Volume I) and Trajectory Control (Volume II).

The philosophy of this primer maintains our overarching theme: mathematics should be driven by the physical reality of motion, not abstract algebra for its own sake.

Contents

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{nx}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ **or** $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

$\mathbf{R}$ Control penalty matrix in LQR/DDP.

$\mathbf{K}$ Feedback gain matrix ($\delta\mathbf{u} = -\mathbf{K}\delta\mathbf{x}$).

$\boldsymbol{\gamma}$ A trajectory or flow, mapping time and initial conditions to states.

Affine Control and Counterfactuals

$f(\mathbf{x})$ Drift vector field: complete autonomous evolution of the declared effective plant when $\mathbf{u} = \mathbf{0}$.

$G(\mathbf{x})$ Input (control) matrix field mapping control inputs to state derivatives.

$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$ Control-affine system equations of motion.

$\mathbf{x}^{\mathrm{ZTCF}}(t)$ Forward Zero-Torque Counterfactual trajectory under $\mathbf{u}(t) \equiv \mathbf{0}$.

$\mathbf{a}_{\mathrm{ZVCF}}(\mathbf{q}, \mathbf{z})$ Instantaneous Zero-Velocity Counterfactual acceleration with velocity and declared control set to zero.

$\mathrm{DCR}_{W,\mathcal{U}}(\mathbf{x})$ Drift-Control Ratio in a declared acceleration or task-projected space: $\frac{\|W\mathbf{a}_{\mathrm{drift}}\|_2}{\sup_{\mathbf{u}\in\mathcal{U}} \|WB_a\mathbf{u}\|_2 + \varepsilon}$.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\mathbf{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

A Primer on Linear Algebra

A useful model connects a mathematical description to measurements, forces and motion.

In Plain Language 1.1. Context and Use Case: Linear Algebra for Motion

The Math You Will See: We will cover Vectors ($\mathbf{v}$), Matrices ($\mathbf{M}$), Vector Spaces and their axioms, Linear Independence and Basis, Eigenvalues/Eigenvectors ($\lambda, \mathbf{v}$), the Singular Value Decomposition (SVD), Quadratic Forms ($\mathbf{x}^T\mathbf{M}\mathbf{x}$), Positive Definiteness, and Dyads/Dyadics (the tensor products that underpin inertia, stress, and stiffness).
Why We Need It: Joint velocities, task velocities, forces and measured signals have multiple interacting components. Linear maps describe how those components relate at a specified state; matrix decompositions reveal directions, sensitivity and numerical structure. Predicting a collision or a golf shot additionally requires the dynamics, initial condition, inputs, constraints and contact event. Eigenvalues of one matrix do not answer all of those questions.
All Variables Defined: Check the **Nomenclature** at the start of the book for standard vector and matrix conventions.

1.1 Historical Context

Linear algebra connects the problem of solving simultaneous equations with the geometric study of linear transformations. Its importance in mechanics comes from reusing the same structure for different tasks: resolving velocities, assembling inertias, fitting measurements and propagating local perturbations.

Kalman's 1960 paper *A New Approach to Linear Filtering and Prediction Problems* used state-transition methods to derive a recursive linear estimation scheme and its error-covariance equations [20]. It was a major contribution to estimation, not the first invention of state representation or a claim that every physical system is linear. Controllability and observability make the role of matrix rank precise for declared dynamical models.

Modern simulation and optimization combine kinematic maps, linear solves and nonlinear updates. Solving a linear system within a nonlinear simulation does not make the entire simulated motion linear. The algebra keeps those uses distinct.

1.2 Vectors, Matrices, and Vector Spaces

Before mapping physics, we must understand the fundamental data structures of control theory: vectors and matrices. More importantly, we must understand the abstract *space* in which these objects live, because the space and its metric determine which operations and comparisons are meaningful. Readers seeking a comprehensive undergraduate treatment of these ideas will find Strang's textbook [45] an excellent companion to this chapter.

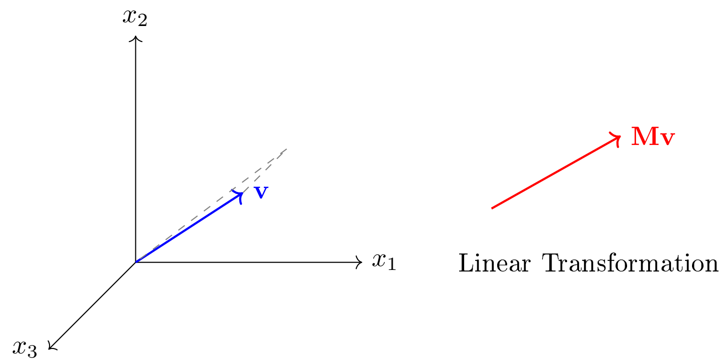


Figure 1.1: Vector in $\mathbb{R}^3$ and its transformation under a linear map $T(\mathbf{v}) = \mathbf{Mv}$.

1.2.1 Vectors

An element of a vector space is a **vector**; after choosing a finite basis, its coordinates form an ordered array of numbers. It can represent a position in space, a velocity profile, a list of sensor readings, or the activation levels of every muscle in a golfer's forearm. In physics, vectors implicitly assume a coordinate frame (e.g., X, Y, Z). A generic column vector in n-dimensional space is denoted:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n \tag{1.1}$$

It is crucial to distinguish two perspectives on what a vector "is." The *algebraic* perspective treats a vector as a column of numbers. The *geometric* perspective treats a vector as a directed arrow in space, possessing both magnitude and direction. Both perspectives are useful when the basis and metric are stated. A position is a point in an affine space; displacement and velocity are vectors. Muscle activations can be stored in an array, but their physically admissible bounded, nonnegative set is not itself a real vector space. When we write $\mathbf{v} \in \mathbb{R}^3$, we mean both the triple of numbers (v_1, v_2, v_3) and the arrow pointing from the origin to the point (v_1, v_2, v_3) in three-dimensional Euclidean space.

1.2.2 Matrices

A **matrix M** is a rectangular grid of numbers arranged in rows and columns. If a vector represents a point or a direction, a matrix acts as a set of instructions on how to *transform* that point—stretching it, rotating it, projecting it, or mapping it into an entirely different dimensional space. An $n \times m$ matrix maps vectors from $\mathbb{R}^m$ to $\mathbb{R}^n$:

$$\mathbf{M} = \begin{bmatrix} m_{11} & m_{12} & \cdots & m_{1m} \\ m_{21} & m_{22} & \cdots & m_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{nm} \end{bmatrix} \in \mathbb{R}^{n \times m} \tag{1.2}$$

When we multiply a matrix by a vector ($\mathbf{Mv}$), the result is a *linear combination of the columns of* $\mathbf{M}$, using the elements of $\mathbf{v}$ as weights. Specifically, if $\mathbf{m}_1, \mathbf{m}_2, \ldots, \mathbf{m}_m$ denote the columns of $\mathbf{M}$, then:

$$\mathbf{Mv} = v_1\mathbf{m}_1 + v_2\mathbf{m}_2 + \cdots + v_m\mathbf{m}_m \tag{1.3}$$

This "column view" of matrix-vector multiplication is one of the most important conceptual tools in linear algebra. It means that the *range* (or image) of a matrix—the set of all possible outputs $\mathbf{Mv}$ as $\mathbf{v}$ varies over all of $\mathbb{R}^m$—is precisely the *span* of the columns of $\mathbf{M}$. Throughout this book, when we rotate a robot arm, we are physically applying a matrix to the vectors extending along its links.

1.2.3 The Axioms of a Vector Space

We have described vectors informally as "arrays of numbers." But the true power of linear algebra lies in the abstract concept of a **vector space**, which captures the essential algebraic properties that make vectors useful—regardless of whether the "vectors" are columns of numbers, functions, or even infinite-dimensional signals.

Definition 1.1. Vector Space

A *vector space* V over the real numbers $\mathbb{R}$ is a set of objects (called vectors) together with two operations—**addition** ($\mathbf{u} + \mathbf{v}$) and **scalar multiplication** ($\alpha\mathbf{v}$, where $\alpha \in \mathbb{R}$)—satisfying the following eight axioms for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all $\alpha, \beta \in \mathbb{R}$:

(i) **Closure under addition:** $\mathbf{u} + \mathbf{v} \in V$.

(ii) **Commutativity:** $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$.

(iii) **Associativity of addition:** $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$.

(iv) **Existence of a zero vector:** There exists $\mathbf{0} \in V$ such that $\mathbf{v} + \mathbf{0} = \mathbf{v}$.

(v) **Existence of additive inverses:** For each $\mathbf{v} \in V$, there exists $-\mathbf{v} \in V$ such that $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$.

(vi) **Closure under scalar multiplication:** $\alpha\mathbf{v} \in V$.

(vii) **Distributivity:** $\alpha(\mathbf{u} + \mathbf{v}) = \alpha\mathbf{u} + \alpha\mathbf{v}$ and $(\alpha + \beta)\mathbf{v} = \alpha\mathbf{v} + \beta\mathbf{v}$.

(viii) **Compatibility:** $\alpha(\beta\mathbf{v}) = (\alpha\beta)\mathbf{v}$, and $1 \cdot \mathbf{v} = \mathbf{v}$.

In Plain Language 1.2. Why Do We Care About Abstract Axioms?

You might wonder: why bother with these formal rules? Because the same mathematical structures appear over and over in radically different contexts. The set $\mathbb{R}^n$ of column vectors satisfies these axioms—but so does the set of all continuous functions $f : [0, T] \to \mathbb{R}$ (where "addition" means adding functions pointwise). In optimal control, the "vectors" we optimize over are often entire *trajectories*, which are functions of time. By proving a theorem about abstract vector spaces, we prove it simultaneously for column vectors, for functions, and for trajectories. The axioms are the price of generality. The ambient function space may be a vector space, but trajectories satisfying nonlinear dynamics, unilateral contact or bounded actuation generally form a constrained subset that is not closed under addition and scaling.

Example 1.1. Verifying $\mathbb{R}^n$ is a Vector Space

Let us verify that $\mathbb{R}^n$ with standard component-wise addition and scalar multiplication satisfies the axioms.
Given $\mathbf{u} = (u_1, \dots, u_n)$ and $\mathbf{v} = (v_1, \dots, v_n)$ in $\mathbb{R}^n$, and $\alpha \in \mathbb{R}$:

- *Closure under addition:* $\mathbf{u}+\mathbf{v} = (u_1 + v_1, \dots, u_n + v_n)$. Since each $u_i + v_i \in \mathbb{R}$, the sum is in $\mathbb{R}^n$. ✓
- *Zero vector:* $\mathbf{0} = (0, 0, \dots, 0)$. Clearly $\mathbf{v} + \mathbf{0} = \mathbf{v}$. ✓
- *Additive inverse:* $-\mathbf{v} = (-v_1, \dots, -v_n)$. Then $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$. ✓
- *Closure under scalar multiplication:* $\alpha\mathbf{v} = (\alpha v_1, \dots, \alpha v_n) \in \mathbb{R}^n$. ✓

The remaining axioms (commutativity, associativity, distributivity, compatibility) follow immediately from the corresponding properties of real number arithmetic. Therefore $\mathbb{R}^n$ is a vector space. □

1.2.4 Linear Independence and Span

Definition 1.2. Linear Independence

A set of vectors $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k\}$ in a vector space V is called *linearly independent* if the only solution to the equation

$$\alpha_1\mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \cdots + \alpha_k\mathbf{v}_k = \mathbf{0} \tag{1.4}$$

is the trivial solution $\alpha_1 = \alpha_2 = \cdots = \alpha_k = 0$. If a nontrivial solution exists (i.e., at least one $\alpha_i \neq 0$), the vectors are *linearly dependent*.

Intuitively, a set of vectors is linearly independent if none of them can be "built" from the others. In $\mathbb{R}^2$, two nonzero vectors are independent exactly when neither is a scalar multiple of the other. In $\mathbb{R}^3$, three vectors are independent exactly when they do not lie in one plane through the origin.

Example 1.2. Linear Independence in $\mathbb{R}^3$

Consider the vectors:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \tag{1.5}$$

Are these linearly independent? We solve $\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 = \mathbf{0}$:

$$\alpha_1 + \alpha_3 = 0 \tag{1.6}$$
$$\alpha_2 + \alpha_3 = 0 \tag{1.7}$$
$$0 = 0 \tag{1.8}$$

The third equation provides no constraint. Setting $\alpha_3 = 1$ yields $\alpha_1 = -1$, $\alpha_2 = -1$. A nontrivial solution exists: $-\mathbf{v}_1 - \mathbf{v}_2 + \mathbf{v}_3 = \mathbf{0}$, confirming that $\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$. The vectors are linearly *dependent*. Geometrically, all three vectors lie in the xy-plane, and the third is the diagonal of the rectangle formed by the first two.

1.2.5 Basis and Dimension

Definition 1.3. Basis

A finite *basis* for a finite-dimensional vector space V is a set of vectors $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ that is both linearly independent and spans V (i.e., every vector in V can be written as a linear combination of the basis vectors). The number n of vectors in any basis is called the *dimension* of V, written $\dim(V) = n$.

A fundamental theorem of linear algebra states that *every basis for a given vector space has exactly the same number of elements.* This is why "dimension" is well-defined—it is an intrinsic property of the space, not of any particular choice of basis.

The standard basis for $\mathbb{R}^n$ consists of the unit coordinate vectors $\mathbf{e}_1 = (1, 0, \ldots, 0)^T$, $\mathbf{e}_2 = (0, 1, 0, \ldots, 0)^T$, through $\mathbf{e}_n = (0, \ldots, 0, 1)^T$. Any vector $\mathbf{v} = (v_1, \ldots, v_n)^T$ can be written uniquely as $\mathbf{v} = v_1\mathbf{e}_1 + \cdots + v_n\mathbf{e}_n$. The numbers $v_1, \ldots, v_n$ are the *coordinates* of $\mathbf{v}$ with respect to this basis.

In Plain Language 1.3. Why Basis Matters for Robotics

Joint angles are coordinates on a configuration space, not basis vectors. A task map $\mathbf{y} = h(\mathbf{q})$ is generally nonlinear and can change dimension. Its Jacobian $J(\mathbf{q}) = Dh(\mathbf{q})$ maps an instantaneous joint velocity to a task velocity, $\dot{\mathbf{y}} = J\dot{\mathbf{q}}$. This is a linear map between tangent spaces at a specified configuration, and may have a nontrivial kernel. A change of basis within one finite-dimensional vector space is instead square and invertible. A seven-joint-to-three-position Jacobian cannot be a change of basis.

1.2.6 Linear Transformations

Definition 1.4. Linear Transformation

A function $T : V \to W$ between two vector spaces is called a *linear transformation* (or *linear map*) if it preserves vector addition and scalar multiplication:

(i) $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ for all $\mathbf{u}, \mathbf{v} \in V$.

(ii) $T(\alpha \mathbf{v}) = \alpha T(\mathbf{v})$ for all $\alpha \in \mathbb{R}$ and $\mathbf{v} \in V$.

The fundamental connection between linear transformations and matrices is this: *every linear transformation between finite-dimensional vector spaces can be represented by a matrix, and every matrix represents a linear transformation.* Specifically, if $T : \mathbb{R}^m \to \mathbb{R}^n$ is linear, then there exists a unique $n \times m$ matrix $\mathbf{M}$ such that $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$ for all $\mathbf{v} \in \mathbb{R}^m$. The columns of $\mathbf{M}$ are simply $T(\mathbf{e}_1), T(\mathbf{e}_2), \ldots, T(\mathbf{e}_m)$—the images of the standard basis vectors.

Theorem 1.1. Matrix Multiplication is Linear

Let $\mathbf{M} \in \mathbb{R}^{n\times m}$. Then the function $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$ is a linear transformation from $\mathbb{R}^m$ to $\mathbb{R}^n$.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ and $\alpha \in \mathbb{R}$. By the distributive property of matrix multiplication:

$$T(\mathbf{u} + \mathbf{v}) = \mathbf{M}(\mathbf{u} + \mathbf{v}) = \mathbf{M}\mathbf{u} + \mathbf{M}\mathbf{v} = T(\mathbf{u}) + T(\mathbf{v}) \tag{1.9}$$

and by the compatibility of matrix multiplication with scalar multiplication:

$$T(\alpha\mathbf{v}) = \mathbf{M}(\alpha\mathbf{v}) = \alpha(\mathbf{M}\mathbf{v}) = \alpha T(\mathbf{v}) \tag{1.10}$$

Both conditions of linearity are satisfied. □

1.2.7 Rank, Null Space, and the Rank-Nullity Theorem

Not all linear transformations preserve full information about the input. Some transformations "collapse" certain directions to zero.

Definition 1.5. Rank

The *rank* of a matrix $\mathbf{M}$, denoted $\operatorname{rank}(\mathbf{M})$, is the number of linearly independent columns (equivalently, the number of linearly independent rows) of $\mathbf{M}$. It equals the dimension of the column space (the range of the corresponding linear transformation).

If a 3×3 matrix has rank 2, it collapses full 3D space into a 2D plane. In robotics, the rank of the Jacobian matrix $\mathbf{J}(\mathbf{q})$ tells you how many independent directions the end-effector can instantaneously move in. If the rank drops below its maximum value, the robot is at a *singularity*—some previously available instantaneous task velocities are no longer in its image. This statement concerns the chosen task and admissible joint velocities; it does not by itself prove finite-time unreachability or loss of dynamical controllability.

Definition 1.6. Null Space (Kernel)

The *null space* (or *kernel*) of a matrix $\mathbf{M}$, denoted $\mathcal{N}(\mathbf{M})$, is the set of all vectors that the matrix maps to zero:

$$\mathcal{N}(\mathbf{M}) = \{\mathbf{x} \in \mathbb{R}^m \mid \mathbf{M}\mathbf{x} = \mathbf{0}\} \tag{1.11}$$

For a redundant arm, $\ker J(\mathbf{q})$ contains instantaneous joint velocities with zero velocity in the chosen hand task. A finite self-motion must keep satisfying the evolving constraint $J(\mathbf{q}(t))\dot{\mathbf{q}}(t) = 0$; a straight step along the initial kernel need not do so. At regular configurations the constant-rank theorem describes the local task-level set. At a singular configuration, even an instantaneous kernel direction need not integrate into a finite constant-task motion.

Definition 1.7. The Determinant (Volumetric Interpretation)

For a square matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$, the *determinant* $\det(\mathbf{M})$ is a scalar whose absolute value measures the factor by which $\mathbf{M}$ scales n-dimensional volume. Imagine a unit hypercube $[0,1]^n$. Applying $\mathbf{M}$ to every point in that cube warps it into a parallelepiped whose signed volume equals $\det(\mathbf{M})$.

- If $\det(\mathbf{M}) = 1$, volume is perfectly preserved (like a pure rotation in space).
- If $\det(\mathbf{M}) = 0$, volume is crushed completely flat—the matrix is *singular* and loses rank. The transformation is not invertible.
- If $\det(\mathbf{M}) < 0$, orientation is reversed and volume is multiplied by $|\det(\mathbf{M})|$. For example, $\mathrm{diag}(-2, 3)$ reverses orientation and multiplies area by six; only absolute determinant one preserves volume.

Now we arrive at one of the most important structural results in linear algebra:

Theorem 1.2. Rank-Nullity Theorem

For any $n \times m$ matrix $\mathbf{M}$:

$$\underbrace{\mathrm{rank}(\mathbf{M})}_{\text{dimension of range}} + \underbrace{\dim\big(\mathcal{N}(\mathbf{M})\big)}_{\text{dimension of null space}} = m \quad \text{(number of columns)} \tag{1.12}$$

Proof. Let $r = \mathrm{rank}(\mathbf{M})$ and let $\{\mathbf{n}_1, \ldots, \mathbf{n}_k\}$ be a basis for $\mathcal{N}(\mathbf{M})$, where $k = \dim(\mathcal{N}(\mathbf{M}))$. Extend this to a basis $\{\mathbf{n}_1, \ldots, \mathbf{n}_k, \mathbf{w}_1, \ldots, \mathbf{w}_{m-k}\}$ for all of $\mathbb{R}^m$ (this extension is always possible by a standard result in linear algebra).

We claim that $\{\mathbf{M}\mathbf{w}_1, \ldots, \mathbf{M}\mathbf{w}_{m-k}\}$ is a basis for the column space of $\mathbf{M}$, which would establish $r = m - k$, i.e., $r + k = m$.

Spanning: Any $\mathbf{y}$ in the range of $\mathbf{M}$ can be written as $\mathbf{y} = \mathbf{M}\mathbf{v}$ for some $\mathbf{v} \in \mathbb{R}^m$. Expanding $\mathbf{v}$ in our basis: $\mathbf{v} = \sum_{i=1}^{k} \alpha_i \mathbf{n}_i + \sum_{j=1}^{m-k} \beta_j \mathbf{w}_j$. Since $\mathbf{M}\mathbf{n}_i = \mathbf{0}$, we get $\mathbf{y} = \sum_{j=1}^{m-k} \beta_j \mathbf{M}\mathbf{w}_j$. So the $\mathbf{M}\mathbf{w}_j$ span the range.

Independence: Suppose

$$\sum_{j=1}^{m-k} \beta_j \mathbf{M}\mathbf{w}_j = 0.$$

Then $M(\sum_j \beta_j \mathbf{w}_j) = 0$, so the vector $\sum_j \beta_j \mathbf{w}_j$ belongs to the kernel. It therefore equals $\sum_{i=1}^{k} \gamma_i \mathbf{n}_i$ for some scalars γ_i. The combined list of kernel vectors and extension vectors is a basis for $\mathbb{R}^m$, so it is independent. All β_j and γ_i must be zero.

Thus $r = m - k$, establishing $\text{rank}(\mathbf{M}) + \dim(\mathcal{N}(\mathbf{M})) = m$. □

In Plain Language 1.4. The Physical Meaning of Rank-Nullity

For a robot with m independent joint-velocity coordinates and an n-dimensional task:

- The **rank** of the Jacobian tells you the dimension of the instantaneous task-velocity image when all declared joint velocities are admissible.
- The **nullity** (dimension of the null space) tells you how many "extra" internal motions remain after the task is specified.

If a 7-DOF arm ($m = 7$) controls a hand in 3D position space ($n = 3$), and the Jacobian has rank 3, then the nullity is $7 - 3 = 4$. The robot has 4 degrees of internal freedom to reposition its elbow, shoulder, and wrist with zero instantaneous hand-position velocity. This does not fix hand orientation or guarantee a finite self-motion without updating the Jacobian.

1.3 The Dot Product, Cross Product, and Inner Product Spaces

Two fundamental operations exist for combining vectors. Understanding them geometrically is essential for everything from computing kinetic energy to defining stability metrics.

1.3.1 The Dot Product (Inner Product)

The dot product (or inner product) takes two vectors and returns a scalar. It measures how much two vectors "align" with each other.

Definition 1.8. Dot Product

In an orthonormal Euclidean basis, the *dot product* of vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ is:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^{n} a_i b_i \tag{1.13}$$

In $\mathbb{R}^3$, this expands to $a_x b_x + a_y b_y + a_z b_z$.

The geometric interpretation connects the algebraic formula to angles:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos(\theta) \tag{1.14}$$

where θ is the angle between $\mathbf{a}$ and $\mathbf{b}$, and $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$ is the Euclidean norm (length) of $\mathbf{a}$.

Derivation of Equation (1.14). Consider two vectors $\mathbf{a}$ and $\mathbf{b}$ with an angle θ between them. The vector $\mathbf{a} - \mathbf{b}$ forms the third side of a triangle. By the law of cosines:

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2 \, \|\mathbf{a}\| \, \|\mathbf{b}\| \cos \theta \tag{1.15}$$

Expanding the left side using the definition of the norm:

$$\|\mathbf{a} - \mathbf{b}\|^2 = (\mathbf{a} - \mathbf{b})^T (\mathbf{a} - \mathbf{b}) \tag{1.16}$$

$$= \mathbf{a}^T \mathbf{a} - 2\mathbf{a}^T \mathbf{b} + \mathbf{b}^T \mathbf{b} \tag{1.17}$$

$$= \|\mathbf{a}\|^2 - 2 \, \mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2 \tag{1.18}$$

Equating the two expressions:

$$\|\mathbf{a}\|^2 - 2 \, \mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2 \, \|\mathbf{a}\| \, \|\mathbf{b}\| \cos \theta \tag{1.19}$$

Canceling $\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2$ from both sides and dividing by -2:

$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos \theta \tag{1.20}$$

□

Key consequences of the dot product formula:

- If $\theta = 0$ (parallel vectors): $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\|$ (maximum positive value).
- If $\theta = 90^\circ$ (perpendicular vectors): $\mathbf{a} \cdot \mathbf{b} = 0$. This is the fundamental test for **orthogonality**.
- If $\theta = 180^\circ$ (antiparallel vectors): $\mathbf{a} \cdot \mathbf{b} = - \|\mathbf{a}\| \, \|\mathbf{b}\|$ (maximum negative value).

In mechanics, $T = \frac{1}{2} \dot{\mathbf{q}}^T M(\mathbf{q}) \dot{\mathbf{q}}$ is a kinetic-energy quadratic form. An optimal-control running cost may be $\ell = \mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}$; a trajectory cost integrates it and may add a terminal cost. Generalized-force projection $\tau_i = \mathcal{F}^T \mathcal{S}_i$ is a dual pairing of a wrench and a joint-motion direction, with consistent frame, reference point and component order. These expressions have different units and meanings; none is automatically a Euclidean angle between like physical quantities.

Orthogonal Projection

A common operation is projecting one vector onto another. For $\mathbf{a} \neq 0$, the **scalar projection** is $\mathbf{a}^T \mathbf{b} / \|\mathbf{a}\|$. The **vector projection** is:

$$\text{proj}_{\mathbf{a}} \mathbf{b} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}} \, \mathbf{a} = \frac{\mathbf{a}^T \mathbf{b}}{\|\mathbf{a}\|^2} \, \mathbf{a} \tag{1.21}$$

The residual $\mathbf{b} - \text{proj}_{\mathbf{a}} \mathbf{b}$ is orthogonal to $\mathbf{a}$. This decomposition—splitting a vector into a component along a direction and a component perpendicular to it—is the geometric core of the Gram-Schmidt process and underlies the QR decomposition used in numerical linear algebra.

1.3.2 The Cross Product and the Skew-Symmetric Matrix

The standard cross product used here is defined in oriented three-dimensional Euclidean space. Its result is orthogonal to both inputs. Parallel inputs, or a zero input, give the zero vector, which has no direction.

Definition 1.9. Cross Product

For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$, the *cross product* is:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix} \tag{1.22}$$

The direction of $\mathbf{a} \times \mathbf{b}$ is determined by the *right-hand rule*: curl the fingers of your right hand from $\mathbf{a}$ toward $\mathbf{b}$; your thumb points in the direction of the cross product. The magnitude satisfies:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \, \|\mathbf{b}\| \sin(\theta) \tag{1.23}$$

which equals the area of the parallelogram spanned by $\mathbf{a}$ and $\mathbf{b}$.

The cross product is *anti-commutative*: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. This has direct physical consequences. If you apply a force $\mathbf{F}$ at a displacement $\mathbf{r}$ from a pivot, the resulting torque is $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$. Reversing the order reverses the torque direction.

Crucially, the cross product can be rewritten as a matrix multiplication using the *skew-symmetric matrix*:

$$[\mathbf{a}_\times] = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \quad \Longrightarrow \quad \mathbf{a} \times \mathbf{b} = [\mathbf{a}_\times]\mathbf{b} \tag{1.24}$$

Proposition 1. *The skew-symmetric matrix $[\mathbf{a}_\times]$ satisfies:*

(i) $[\mathbf{a}_\times]^T = -[\mathbf{a}_\times]$ *(skew-symmetry; equivalently, all diagonal entries are zero and the off-diagonal entries are negated across the diagonal).*

(ii) $\mathbf{a}^T[\mathbf{a}_\times]\mathbf{b} = 0$ *for all* $\mathbf{b}$ *(the cross product is always perpendicular to both input vectors).*

(iii) The eigenvalues of $[\mathbf{a}_\times]$ *are* $\{0, +i\,\|\mathbf{a}\|, -i\,\|\mathbf{a}\|\}$ *(purely imaginary, reflecting the rotational nature of the cross product).*

Proof. (i) is immediate from inspection of Equation (1.24): transposing negates every off-diagonal element while the diagonal remains zero.

(ii) We compute $\mathbf{a}^T([\mathbf{a}_\times]\mathbf{b}) = \mathbf{a}^T(\mathbf{a} \times \mathbf{b})$. Since $\mathbf{a} \times \mathbf{b}$ is perpendicular to $\mathbf{a}$ (by the geometric definition of the cross product), the dot product is zero.

(iii) The characteristic polynomial $\det([\mathbf{a}_\times] - \lambda\mathbf{I}) = -\lambda^3 - \|\mathbf{a}\|^2 \lambda = -\lambda(\lambda^2 + \|\mathbf{a}\|^2)$. The roots are $\lambda = 0$ and $\lambda = \pm i\,\|\mathbf{a}\|$. □

This skew-symmetric matrix representation is absolutely pivotal for Chapters 4 and 6, where we show that the Lie algebra $\mathfrak{so}(3)$—the tangent space to the rotation group at the identity—consists entirely of 3×3 skew-symmetric matrices. An angular-velocity vector corresponds to such a matrix after choosing a body or spatial trivialization; a tangent matrix at a general rotation is not itself necessarily skew-symmetric.

The Scalar Triple Product

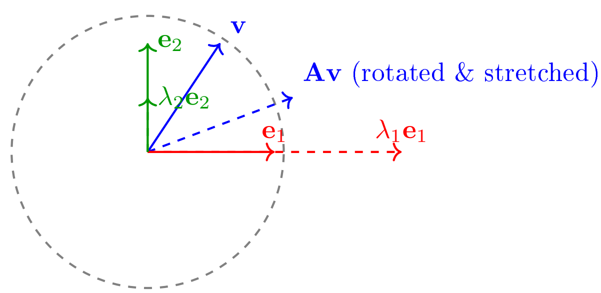
The scalar triple product $\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c})$ computes the signed volume of the parallelepiped formed by three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$. It can be computed as a determinant:

$$\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c}) = \det\begin{bmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \\ a_z & b_z & c_z \end{bmatrix} \tag{1.25}$$

If this determinant is zero, the three vectors are coplanar and linearly dependent—a direct connection between the cross product and the rank of the matrix formed by stacking the vectors as columns.

1.4 Eigenvalues, Eigenvectors, and Quadratic Forms

When you multiply a general vector by a matrix, the resulting vector typically changes both its magnitude and its direction. Over the complex numbers, every square matrix of positive size has an eigenvector. Real invariant lines need not exist: a planar quarter-turn has eigenvalues $\pm i$ and no real eigenvector. A real eigenvector spans a line mapped into itself; a positive eigenvalue preserves its ray, a negative one reverses it, and zero annihilates it. These invariant directions reveal the deepest structural properties of the linear transformation.



Example: positive eigenvalues preserve eigenvector rays

Figure 1.2: Eigenvectors and a Generic Vector Under $\mathbf{A} = \operatorname{diag}(2, 1/2)$

1.4.1 The Eigenvalue Equation

Definition 1.10. Eigenvalues and Eigenvectors

Given a square matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$, a nonzero vector $\mathbf{v} \in \mathbb{C}^n$ is called an *eigenvector* of $\mathbf{M}$ if there exists a scalar $\lambda \in \mathbb{C}$ such that:

$$\mathbf{M}\mathbf{v} = \lambda\mathbf{v} \tag{1.26}$$

The scalar λ is the corresponding *eigenvalue*. The requirement that $\mathbf{v} \neq \mathbf{0}$ is essential; otherwise the equation would be trivially satisfied for any λ.

Deriving the Characteristic Polynomial

How do we actually find the eigenvalues? We rearrange Equation (1.26):

$$\mathbf{M}\mathbf{v} - \lambda\mathbf{v} = \mathbf{0} \quad \Longrightarrow \quad (\mathbf{M} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0} \tag{1.27}$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. For a nonzero solution $\mathbf{v}$ to exist, the matrix $(\mathbf{M} - \lambda\mathbf{I})$ must be singular. By our earlier discussion, this means its determinant must vanish:

$$\det(\mathbf{M} - \lambda\mathbf{I}) = 0 \tag{1.28}$$

This is the **characteristic equation**. Expanding the determinant produces a polynomial of degree n in λ, called the **characteristic polynomial**. By the Fundamental Theorem of Algebra, this polynomial has exactly n roots (counted with multiplicity, possibly complex), giving us n eigenvalues.

Example 1.3. Computing Eigenvalues of a 2×2 Matrix

Consider the dynamics matrix of a simple spring system:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -4 & -5 \end{bmatrix} \tag{1.29}$$

Step 1: Form $\mathbf{A} - \lambda\mathbf{I}$:

$$\mathbf{A} - \lambda\mathbf{I} = \begin{bmatrix} -\lambda & 1 \\ -4 & -5 - \lambda \end{bmatrix} \tag{1.30}$$

Step 2: Compute the determinant and set it to zero:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = (-\lambda)(-5 - \lambda) - (1)(-4) \tag{1.31}$$

$$= \lambda^2 + 5\lambda + 4 \tag{1.32}$$

$$= (\lambda + 1)(\lambda + 4) = 0 \tag{1.33}$$

Step 3: The eigenvalues are $\lambda_1 = -1$ and $\lambda_2 = -4$. Both are negative, meaning this system is **asymptotically stable**—any perturbation decays exponentially back to equilibrium. The mode associated with $\lambda_1 = -1$ decays slowly (time constant $\tau_1 = 1$ second), while the mode associated with $\lambda_2 = -4$ decays four times faster ($\tau_2 = 0.25$ seconds).
Step 4: Find the eigenvectors. For $\lambda_1 = -1$, solve $(\mathbf{A} + \mathbf{I})\mathbf{v}_1 = \mathbf{0}$:

$$\begin{bmatrix} 1 & 1 \\ -4 & -4 \end{bmatrix} \mathbf{v}_1 = \mathbf{0} \quad \Longrightarrow \quad \mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \tag{1.34}$$

For $\lambda_2 = -4$, solve $(\mathbf{A} + 4\mathbf{I})\mathbf{v}_2 = \mathbf{0}$:

$$\begin{bmatrix} 4 & 1 \\ -4 & -1 \end{bmatrix} \mathbf{v}_2 = \mathbf{0} \quad \Longrightarrow \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -4 \end{bmatrix} \tag{1.35}$$

1.4.2 Physical Interpretation of Eigenvalues

1. **Inertia tensors and principal axes.** The symmetric rotational inertia tensor about the centre of mass has orthogonal principal axes and corresponding principal moments.

For an isolated rigid body with three distinct principal moments, steady torque-free rotation about the intermediate axis is unstable, whereas the minimum and maximum axes are stable to small rotational perturbations. An eigenvector of inertia alone does not establish stability under applied torques, damping or feedback.

2. **Constant linear dynamics.** For $\dot{\mathbf{x}} = A\mathbf{x}$, all eigenvalues strictly in the open left half-plane imply exponential asymptotic stability. Any eigenvalue with positive real part implies instability. With all real parts nonpositive, stability additionally requires each eigenvalue on the imaginary axis to be semisimple: its Jordan blocks must have size one. Such modes need not decay.

3. **Nonlinear and time-varying systems.** At an equilibrium of a continuously differentiable autonomous system, a Hurwitz linearization establishes local exponential stability, and an eigenvalue with positive real part establishes instability. A zero real part can be inconclusive: $\dot{x} = -x^3$ and $\dot{x} = x^3$ have the same zero linearization and opposite stability. Instantaneous eigenvalues of a time-varying matrix are not a substitute for its state-transition operator.

For example,

$$A_0 = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad e^{tA_0}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} t \\ 1 \end{pmatrix}$$

is unbounded despite two zero eigenvalues. By contrast, $A = 0$ has bounded constant solutions.

Even negative eigenvalues do not guarantee monotone decrease in a chosen norm. For

$$A = \begin{pmatrix} -1 & 10 \\ 0 & -1 \end{pmatrix}, \qquad e^{tA} = e^{-t}\begin{pmatrix} 1 & 10t \\ 0 & 1 \end{pmatrix},$$

the initial vector $(0,1)^T$ has norm $\sqrt{101}/e > 3.6$ at $t = 1$ before eventually decaying. The transient comes from the nonorthogonal modal structure. Mixed physical units require a declared metric before interpreting its magnitude. For a fixed positive-definite P, the quadratic rate is

$$\frac{d}{dt}(\mathbf{x}^T P\mathbf{x}) = \mathbf{x}^T(A^T P + PA)\mathbf{x}.$$

Asymptotic stability and short-horizon amplification answer different questions in a finite swing.

Example 1.4. Visualizing Eigenvectors: The Stretching Rubber Sheet

A general invertible planar map sends a unit circle to an ellipse. Its output axes are *left singular vectors*, while the corresponding input directions are right singular vectors. They need not be the same directions.

The shear

$$S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

has eigenvalues $1, 1$ but singular values $(1+\sqrt{5})/2$ and $(\sqrt{5}-1)/2$. It stretches and contracts different directions even though both eigenvalues are one.

For a symmetric matrix one can choose an orthonormal eigenbasis. The image of the unit ball has semiaxis lengths $|\lambda_i|$; a negative eigenvalue also reverses its eigenvector, and zero gives a collapsed direction. For a positive-definite symmetric matrix the semiaxis lengths equal the positive eigenvalues. This is the special case in which the rubber-sheet picture directly matches the eigenvalue magnitudes.

1.4.3 The Spectral Theorem for Symmetric Matrices

Among all matrices, *symmetric* matrices ($\mathbf{M} = \mathbf{M}^T$) occupy a privileged position. They arise naturally throughout mechanics (mass matrices, stiffness matrices, inertia tensors) and control theory (cost matrices, Lyapunov functions).

Theorem 1.3. The Spectral Theorem

If $\mathbf{M} \in \mathbb{R}^{n\times n}$ is symmetric ($\mathbf{M} = \mathbf{M}^T$), then:

(i) All eigenvalues of $\mathbf{M}$ are real.

(ii) Eigenvectors corresponding to distinct eigenvalues are orthogonal.

(iii) $\mathbf{M}$ can be factored as $\mathbf{M} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^T$, where $\mathbf{Q}$ is an orthogonal matrix ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$) whose columns are the eigenvectors, and $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is a diagonal matrix of eigenvalues.

Proof that eigenvalues of a real symmetric matrix are real. Let $\mathbf{M}$ be real symmetric. Suppose $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq 0$, allowing complex entries. We will show that λ must be real.

Take the complex conjugate transpose (denoted by *) of both sides of $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$:

$$\mathbf{v}^*\mathbf{M}^T = \bar{\lambda}\,\mathbf{v}^* \tag{1.36}$$

Since $\mathbf{M} = \mathbf{M}^T$ (and $\mathbf{M}$ is real, so $\mathbf{M}^* = \mathbf{M}^T = \mathbf{M}$):

$$\mathbf{v}^*\mathbf{M} = \bar{\lambda}\,\mathbf{v}^* \tag{1.37}$$

Now multiply the original equation on the left by $\mathbf{v}^*$:

$$\mathbf{v}^*\mathbf{M}\mathbf{v} = \lambda\,\mathbf{v}^*\mathbf{v} \tag{1.38}$$

And multiply the conjugated equation on the right by $\mathbf{v}$:

$$\mathbf{v}^*\mathbf{M}\mathbf{v} = \bar{\lambda}\,\mathbf{v}^*\mathbf{v} \tag{1.39}$$

Equating the right-hand sides:

$$\lambda\,\mathbf{v}^*\mathbf{v} = \bar{\lambda}\,\mathbf{v}^*\mathbf{v} \tag{1.40}$$

Since $\mathbf{v} \neq \mathbf{0}$, we have $\mathbf{v}^*\mathbf{v} = \sum_i |v_i|^2 > 0$. Dividing both sides by $\mathbf{v}^*\mathbf{v}$ yields $\lambda = \bar{\lambda}$, which is true if and only if λ is real. □

Proof of Orthogonality for Distinct Eigenvalues. For real symmetric $\mathbf{M}$, choose real eigenvectors with distinct eigenvalues:

$$\mathbf{M}\mathbf{v}_1 = \lambda_1\mathbf{v}_1, \qquad \mathbf{M}\mathbf{v}_2 = \lambda_2\mathbf{v}_2, \qquad \lambda_1 \neq \lambda_2.$$

Compute:

$$\lambda_1(\mathbf{v}_1^T\mathbf{v}_2) = (\mathbf{M}\mathbf{v}_1)^T\mathbf{v}_2 = \mathbf{v}_1^T\mathbf{M}^T\mathbf{v}_2 = \mathbf{v}_1^T\mathbf{M}\mathbf{v}_2 = \mathbf{v}_1^T(\lambda_2\mathbf{v}_2) = \lambda_2(\mathbf{v}_1^T\mathbf{v}_2) \tag{1.41}$$

Therefore $(\lambda_1 - \lambda_2)(\mathbf{v}_1^T\mathbf{v}_2) = 0$. Since $\lambda_1 \neq \lambda_2$, we must have $\mathbf{v}_1^T\mathbf{v}_2 = 0$. The eigenvectors are orthogonal. □

The spectral decomposition $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ applies an orthogonal coordinate change to eigenvector axes, scales each coordinate by its eigenvalue, and applies the inverse orthogonal change. Negative eigenvalues reverse signs. Unlike the SVD's nonnegative singular scaling, this diagonal scaling retains the eigenvalue signs.

1.4.4 Quadratic Forms and Positive Definiteness

In optimization and energy mechanics, we frequently encounter *quadratic forms*—scalar-valued functions that are quadratic in a vector argument:

Definition 1.11. Quadratic Form

Given a symmetric matrix $\mathbf{M} \in \mathbb{R}^{n\times n}$, the associated *quadratic form* is the scalar function $q : \mathbb{R}^n \to \mathbb{R}$ defined by:

$$q(\mathbf{x}) = \mathbf{x}^T\mathbf{M}\mathbf{x} = \sum_{i=1}^{n}\sum_{j=1}^{n} M_{ij}\, x_i\, x_j \tag{1.42}$$

Quadratic forms appear throughout this text series:

- **Kinetic energy:** $T = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$, where $\mathbf{M}(\mathbf{q})$ is the mass matrix (Chapter 11).
- **Optimal control running cost:** $\ell = \mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{u}^T\mathbf{R}\mathbf{u}$, where $\mathbf{Q}$ penalizes state deviations and $\mathbf{R}$ penalizes control effort.
- **Lyapunov functions:** $V(\mathbf{x}) = \mathbf{x}^T\mathbf{P}\mathbf{x}$, used to prove stability (Volume I).
- **Contraction metrics:** $\delta\mathbf{x}^T\mathbf{M}(x)\,\delta\mathbf{x}$, measuring squared infinitesimal length on manifolds (Volume I).

Definition 1.12. Positive Definiteness

A symmetric matrix $\mathbf{M}$ is called:

- **Positive definite** ($\mathbf{M} \succ 0$) if $\mathbf{x}^T\mathbf{M}\mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$.
- **Positive semi-definite** ($\mathbf{M} \succeq 0$) if $\mathbf{x}^T\mathbf{M}\mathbf{x} \geq 0$ for all $\mathbf{x}$.
- **Negative definite** ($\mathbf{M} \prec 0$) if $\mathbf{x}^T\mathbf{M}\mathbf{x} < 0$ for all $\mathbf{x} \neq \mathbf{0}$.
- **Indefinite** if $\mathbf{x}^T\mathbf{M}\mathbf{x}$ takes both positive and negative values.

Proposition 2. *A symmetric matrix* $\mathbf{M}$ *is positive definite if and only if all of its eigenvalues are strictly positive.*

Proof. By the Spectral Theorem (Theorem 1.3), $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_n)$ and $\mathbf{Q}$ is orthogonal. For any nonzero $\mathbf{x}$, define $\mathbf{y} = \mathbf{Q}^T\mathbf{x}$. Since $\mathbf{Q}$ is invertible and $\mathbf{x} \neq \mathbf{0}$, we have $\mathbf{y} \neq \mathbf{0}$. Then:

$$\mathbf{x}^T\mathbf{M}\mathbf{x} = \mathbf{x}^T\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T\mathbf{x} = \mathbf{y}^T\mathbf{\Lambda}\mathbf{y} = \sum_{i=1}^{n} \lambda_i y_i^2 \tag{1.43}$$

($\Rightarrow$) If $\mathbf{M} \succ 0$, then for each eigenvector $\mathbf{v}_i$ (which is nonzero): $\mathbf{v}_i^T\mathbf{M}\mathbf{v}_i = \lambda_i \|\mathbf{v}_i\|^2 > 0$, so $\lambda_i > 0$.

($\Leftarrow$) If all $\lambda_i > 0$ and $\mathbf{y} \neq \mathbf{0}$, then at least one $y_i \neq 0$, so $\sum_i \lambda_i y_i^2 > 0$. □

In Plain Language 1.5. Why Positive Definiteness Matters for Control

Let the origin be an equilibrium and $V(\mathbf{x}) = \mathbf{x}^T P \mathbf{x}$ with $P \succ 0$. Nonincrease of V along solutions gives a stability bound on a suitable invariant neighborhood; it does not imply convergence. An undamped oscillator has constant positive energy and never comes to rest. A negative-definite derivative, with the usual autonomous-system regularity and domain hypotheses, gives asymptotic convergence; a merely semidefinite derivative needs an additional invariant-set argument. V is a mathematical measure and need not be physical energy. Time-varying or global conclusions require their corresponding uniformity, domain and growth assumptions.

The Cholesky Decomposition

A practical consequence of positive definiteness: any positive definite matrix $\mathbf{M}$ can be uniquely factored as:

$$\mathbf{M} = \mathbf{L}\mathbf{L}^T \tag{1.44}$$

where $\mathbf{L}$ is a lower triangular matrix with strictly positive diagonal entries. This is the **Cholesky decomposition**. It is numerically efficient (roughly half the cost of a general LU factorization) and is the standard method for solving linear systems involving positive definite matrices in physics engines. One usually solves the system rather than explicitly forming its inverse. MuJoCo's documented implementation uses a sparse L^TDL factorization of joint-space inertia and back-substitution; this is related to, but not literally the displayed LL^T algorithm.[1]

For a real symmetric input in exact arithmetic, Cholesky succeeds with strictly positive pivots precisely when the input is positive definite. In floating-point arithmetic, failure can also reflect rounding near the definiteness boundary, incorrect symmetry/storage assumptions or numerical scaling; inspect the data and residual before interpreting it physically. A routine that reads only one triangle does not by itself verify symmetry of the supplied array.

1.5 The Singular Value Decomposition (SVD)

What if a matrix is not square? What if it does not have a full set of eigenvectors? The **Singular Value Decomposition** (SVD) is the ultimate, universal factorization of *any* matrix $\mathbf{M} \in \mathbb{R}^{n \times m}$, regardless of its shape or rank.

[1] https://mujoco.readthedocs.io/en/stable/computation/index.html

Theorem 1.4. The Singular Value Decomposition

Every matrix $\mathbf{M} \in \mathbb{R}^{n\times m}$ can be factored as:

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{1.45}$$

where:

- $\mathbf{U} \in \mathbb{R}^{n\times n}$ is an orthogonal matrix ($\mathbf{U}^T\mathbf{U} = \mathbf{I}$). Its columns are called the *left singular vectors*.
- $\mathbf{\Sigma} \in \mathbb{R}^{n\times m}$ is a diagonal matrix with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(n,m)} \geq 0$ on the diagonal. These are the *singular values*.
- $\mathbf{V} \in \mathbb{R}^{m\times m}$ is an orthogonal matrix. Its columns are the *right singular vectors*.

1.5.1 Derivation of the SVD From the Eigendecomposition

The SVD can be derived from the spectral theorem applied to related symmetric matrices. Here is the construction:

Step 1. Form the symmetric matrix $\mathbf{M}^T\mathbf{M} \in \mathbb{R}^{m\times m}$. This matrix is always positive semi-definite because for any $\mathbf{x}$:

$$\mathbf{x}^T(\mathbf{M}^T\mathbf{M})\mathbf{x} = (\mathbf{M}\mathbf{x})^T(\mathbf{M}\mathbf{x}) = \|\mathbf{M}\mathbf{x}\|^2 \geq 0 \tag{1.46}$$

Step 2. By the Spectral Theorem, $\mathbf{M}^T\mathbf{M}$ has an eigendecomposition:

$$\mathbf{M}^T\mathbf{M} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \tag{1.47}$$

where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_m)$ with $\lambda_i \geq 0$ (since $\mathbf{M}^T\mathbf{M}$ is positive semi-definite). Order these eigenvalues decreasingly and take their square roots. Only $\min(n, m)$ singular values are listed in the rectangular diagonal Σ; when $m > n$, the remaining eigenvalues of M^TM are necessarily zero.

Step 3. For each nonzero singular value σ_i, define the corresponding left singular vector:

$$\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{M}\mathbf{v}_i \tag{1.48}$$

These vectors are orthonormal (a straightforward verification using $\mathbf{M}^T\mathbf{M}\mathbf{v}_i = \sigma_i^2\mathbf{v}_i$).

Step 4. Extend $\{\mathbf{u}_1, \ldots, \mathbf{u}_r\}$ to a complete orthonormal basis for $\mathbb{R}^n$ (where $r = \text{rank}(\mathbf{M})$). Assemble these into the columns of $\mathbf{U}$. Then $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ follows by construction. This is an existence proof, not a recommendation to form M^TM numerically: doing so squares the condition number in the full-column-rank case and can obscure small singular values.

1.5.2 Geometric Interpretation of the SVD

The SVD reveals that *any* linear transformation, no matter how complex, can be decomposed into exactly three simple geometric operations:

1. **Reorient the input space by a rotation or reflection** (apply $\mathbf{V}^T$): align the input with the principal directions.

2. **Scale each axis independently** (apply $\mathbf{\Sigma}$): stretch or shrink along each principal direction.

3. **Reorient the output space by a rotation or reflection** (apply $\mathbf{U}$): reorient the result in the output space.

The image of the unit ball is a solid, possibly degenerate ellipsoid in the output space. Rank loss collapses some dimensions; for a rectangular map, the image of a sphere need not be only an ellipsoid's boundary. The singular values are the semi-axis lengths of this ellipsoid, and the left singular vectors point along the ellipsoid's principal axes.

1.5.3 The Condition Number

For a nonzero, full-rank rectangular matrix, its 2-norm condition number is

$$\kappa_2(M) = \frac{\sigma_{\max}}{\sigma_{\min}},$$

where the denominator is the smallest of the $\min(n, m)$ listed singular values. Rank deficiency is conventionally assigned infinite condition number here. A wide, full-row-rank matrix may have finite condition number and a nontrivial input kernel simultaneously.

For a nonsingular square system $M\mathbf{x} = \mathbf{b}$ with fixed M and nonzero $\mathbf{b}$, the relative perturbation bound is

$$\frac{\|\delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq \kappa_2(M)\frac{\|\delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2}.$$

Absolute inverse gain is instead $\|M^{-1}\|_2 = 1/\sigma_{\min}$. For $M = 10^{-3}I$, the condition number is one but the absolute inverse gain is 1000. Forward gain is $\sigma_{\max}$, not the condition number. Matrix perturbations and general least-squares problems need additional bounds.

Singular values depend on coordinate units and chosen norms. A task Jacobian mixing translation and rotation, or joint coordinates measured in degrees rather than radians, needs explicit scaling or a metric before its condition number is a physical comparison.

1.5.4 Truncated SVD and Low-Rank Approximation

One of the most powerful applications of the SVD is data compression. Given a matrix $\mathbf{M}$ with singular values $\sigma_1 \geq \sigma_2 \geq \cdots$, the best rank-$k$ approximation (in the Frobenius norm) is:

$$\mathbf{M}_k = \sum_{i=1}^{k} \sigma_i\, \mathbf{u}_i \mathbf{v}_i^T \tag{1.49}$$

This result (the Eckart-Young-Mirsky theorem) underpins dimensionality reduction techniques like Principal Component Analysis (PCA), which identifies the dominant modes of variation in high-dimensional data such as motion capture recordings of golf swings.

Example 1.5. The SVD of the Jacobian (Manipulability Ellipsoid)

For seven independent joint velocities and a three-dimensional hand-position task, $J \in \mathbb{R}^{3\times 7}$ maps $\dot{\mathbf{q}}$ to $\mathbf{v}_H$. With the declared budget $\|\dot{\mathbf{q}}\|_2 \leq 1$:

- The columns of U are task-space singular directions. A direction with zero singular value is absent from the instantaneous velocity image.

- The three singular values give the semiaxes of the image of that joint-speed ball. They are not actual maximum speeds without this budget, its units and actuator feasibility.
- The columns of V, not of V^T, give the corresponding joint-velocity directions. Its remaining columns span the input kernel.

For a positive-definite joint-speed metric H and task metric W, normalize the map as

$$\widetilde{J} = W^{1/2} J H^{-1/2}, \qquad \dot{\mathbf{q}}^T H \dot{\mathbf{q}} \le 1.$$

Singular values of this normalized map compare declared physical budgets; unscaled Euclidean values need not survive a change of units.
Virtual work gives $\boldsymbol{\tau} = J^T \mathbf{F}$ and power equality $\boldsymbol{\tau}^T \dot{\mathbf{q}} = \mathbf{F}^T \mathbf{v}_H$. Under the dual torque budget $\boldsymbol{\tau}^T H^{-1} \boldsymbol{\tau} \le 1$, feasible quasistatic task forces satisfy

$$\mathbf{F}^T J H^{-1} J^T \mathbf{F} \le 1.$$

At full row rank this gives reciprocal force/velocity semiaxes in consistent normalized coordinates. At rank loss it may be unbounded in a task-force direction supported by ideal constraints. It does not predict unlimited real force: structural strength, joint/contact limits, compliance, actuation, balance and dynamics still matter.

1.5.5 Instantaneous Rank and Finite-Time Control

A full-row-rank 3×7 task Jacobian has a four-dimensional kernel without being singular. More generally, a kinematic singularity is a drop below the maximum rank attainable by the specified mechanism and task; an inherently lower-dimensional image is not itself evidence of a singular configuration. This kinematic redundancy differs from underactuation, which concerns the rank and admissible set of the actuator map in the dynamics.

Initial task-null velocity also need not preserve finite position. For two unit planar links,

$$\mathbf{p}(\mathbf{q}) = \begin{pmatrix} \cos q_1 + \cos(q_1 + q_2) \\ \sin q_1 + \sin(q_1 + q_2) \end{pmatrix},$$

the straight configuration has $J(0) = \begin{pmatrix} 0 & 0 \\ 2 & 1 \end{pmatrix}$. Its kernel contains $(1, -2)^T$, but the straight coordinate step $\mathbf{q}(t) = t(1, -2)^T$ gives $\mathbf{p}(t) = (2\cos t, 0)^T$, not a stationary hand.

Conversely, a one-dimensional instantaneous input can control more than one state over time. For the double integrator

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

B has rank one while $[B, AB]$ has rank two. Its controllability Gramian is

$$W_c(T) = \begin{pmatrix} T^3/3 & T^2/2 \\ T^2/2 & T \end{pmatrix}, \qquad \det W_c(T) = T^4/12 > 0 \quad (T > 0).$$

This unconstrained linear result does not remove finite torque bounds or contact requirements. It shows why instantaneous rank, finite-time reachability, observation and a successful golf delivery must be evaluated using their respective maps and assumptions.

Observation has the same temporal distinction. If position alone is measured, $C = [1, 0]$ has an instantaneous kernel containing velocity. Yet the observability matrix $\mathcal{O} = [C^T, (CA)^T]^T = I_2$ has full rank: position measurements over time reveal velocity in this ideal model. A kernel of C is not necessarily an unobservable dynamical subspace.

1.6 Dyads, Dyadics, and Tensor Products

The matrix decompositions we have studied so far—eigendecomposition, SVD—break a matrix into pieces that reveal its geometric action. In this section we meet a complementary idea: building matrices up from the simplest possible pieces, called *dyads*. This viewpoint is not merely algebraic; it is the language that mechanics has used for over a century to describe inertia, stress, and stiffness without ever choosing a coordinate frame [35].

1.6.1 What Is a Dyad?

Definition 1.13. Dyad (Outer Product)

Given two vectors $\mathbf{a} \in \mathbb{R}^m$ and $\mathbf{b} \in \mathbb{R}^n$, the **dyad** (or **outer product**) is the $m \times n$ matrix

$$\mathbf{D} \;=\; \mathbf{a}\,\mathbf{b}^T.$$

This matrix has rank at most one: every column is a scalar multiple of $\mathbf{a}$, and every row is a scalar multiple of $\mathbf{b}^T$.

A dyad encodes *two* directions simultaneously. When it acts on a vector $\mathbf{v}$,

$$\mathbf{D}\,\mathbf{v} \;=\; \big(\mathbf{a}\,\mathbf{b}^T\big)\mathbf{v} \;=\; \mathbf{a}\,\langle \mathbf{b},\, \mathbf{v}\rangle\,,$$

it first *senses* how much $\mathbf{v}$ aligns with $\mathbf{b}$ (via the inner product $\langle \mathbf{b},\, \mathbf{v}\rangle$) and then *responds* along the direction $\mathbf{a}$, scaled by that alignment.

In Plain Language 1.6. Dyads: Direction of Action and Direction of Sensitivity

The dyad $\mathbf{a}\mathbf{b}^T$ measures the component of an input along $\mathbf{b}$ and produces a response along $\mathbf{a}$. It has rank one if both vectors are nonzero, and rank zero otherwise. This is an algebraic response channel, not automatically a passive spring.

For an unprestressed conservative spring along a unit direction $\mathbf{n}$, stored energy $\frac{1}{2}k(\mathbf{n}^T\mathbf{u})^2$ gives restoring force $-k\mathbf{n}\mathbf{n}^T\mathbf{u}$. Its stiffness is symmetric. A generic cross-direction dyad is nonsymmetric and cannot by itself be the Hessian of a smooth elastic potential. For example, the field $(0, x)$ does one unit of work around the positively oriented unit square; a conservative spring would do zero net closed-path work.

Example 1.6. A Rank-1 Linear Response

Let $\mathbf{a} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ (vertical) and $\mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ (horizontal). Then

$$\mathbf{D} = \mathbf{a}\,\mathbf{b}^T = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Applied to a displacement $\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 2 \end{bmatrix}$:

$$\mathbf{D}\,\mathbf{v} = \begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix}.$$

The dyad extracts the horizontal component (3) and produces a purely vertical response—exactly the one-channel behaviour described above.

1.6.2 From Dyads to Dyadics

A nonzero dyad has rank one. Physical stiffness, inertia and stress maps may have full rank or may have null directions. We get there by *summing* dyads.

Definition 1.14. Dyadic (Tensor Product Expansion)

A **dyadic** is a finite sum of dyads:

$$\mathbf{T} \;=\; \sum_{k=1}^{r} \mathbf{a}_k\,\mathbf{b}_k^T.$$

Every $m \times n$ matrix can be written in this form with $r \le \min(m, n)$ terms.

This is not a new fact—it is the column–row factorisation you already know from the SVD. If $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$, then

$$\mathbf{A} \;=\; \sum_{k=1}^{r} \sigma_k\,\mathbf{u}_k\,\mathbf{v}_k^T,$$

which is a dyadic with r terms (the rank of $\mathbf{A}$). The SVD merely chooses the *orthogonal* dyad decomposition; infinitely many other decompositions exist.

Abstractly, a linear map $V \to W$ belongs to $W \otimes V^*$: its input slot is a covector. In standard Euclidean coordinates the inner product identifies $\mathbb{R}^n$ with its dual, so $\mathbf{a}\mathbf{b}^T$ can be written using two vector coordinate arrays and identified with an elementary tensor in $\mathbb{R}^m \otimes \mathbb{R}^n$. Under general coordinate changes one must retain which slots are vectors and which are covectors.

1.6.3 Physical Meaning in Mechanics

The power of the dyadic viewpoint is that it names the two "slots" of a matrix: input direction and output direction. Three central objects in mechanics are dyadics.

The Inertia Tensor. About the centre of mass, the rotational inertia of a rigid body maps angular velocity to angular momentum:

$$\mathbf{L} = \mathbf{I}\,\boldsymbol{\omega}. \tag{1.50}$$

Here $\boldsymbol{\omega}$ is the "input" (the axis and rate of spin) and $\mathbf{L}$ is the "output" (the direction and magnitude of angular momentum). For a generic direction in a body with unequal principal moments, $\mathbf{L}$ and $\boldsymbol{\omega}$ need not be parallel; they are parallel along every principal axis even without an isotropic inertia; the inertia tensor is the dyadic that encodes this directional coupling.

The Stress Tensor. The Cauchy stress tensor $\boldsymbol{\sigma}$ maps a surface-normal direction $\hat{\mathbf{n}}$ to the traction (force per unit area) $\mathbf{t}$ acting on that surface:

$$\mathbf{t} = \boldsymbol{\sigma}\,\hat{\mathbf{n}}.$$

Input: the orientation of a small area element. Output: the force that the material on one side exerts on the other.

The Stiffness Matrix. For a small-displacement, unprestressed truss, let $\mathbf{u}$ collect all nodal displacements and $B_e\mathbf{u}$ be member e's axial extension. The external balancing force is $\mathbf{f} = K\mathbf{u}$, with

$$K = \sum_e k_e B_e^T B_e, \qquad k_e = \frac{E_e A_e}{L_e}.$$

For a member between two three-dimensional nodes, its nonzero incidence blocks are $B_e = [-\mathbf{n}_e^T, \mathbf{n}_e^T]$. Thus one member contributes a rank-one *nodal* matrix, not merely a 3×3 direction matrix. A single anchored-node star is a special case in which the free-node stiffness reduces to $\sum_e k_e \mathbf{n}_e \mathbf{n}_e^T$.

Unrestrained rigid translations give null modes of the assembled matrix. Prestress, geometric stiffness, constraints and nonlinear deformation require additional terms or a tangent linearization. Summing member directions alone does not encode the connectivity of a general truss.

In Plain Language 1.7. Why Engineers Say "Tensor"

When a mechanical engineer refers to the "inertia tensor" or "stress tensor," the usual rigid-body inertia and classical Cauchy stress have symmetric 3×3 representations. Symmetry of Cauchy stress here assumes the classical angular-momentum balance without independent couple stresses; not every tensor or linear response is symmetric. The word *tensor* signals that the object transforms in a specific, predictable way under changes of coordinate frame. It is not just a bag of nine numbers; it is a physical machine that takes one vector in and produces another vector out, regardless of how you choose your axes.

1.6.4 The Coordinate-Free Perspective

A dyadic is a geometric object that lives in the space of linear maps $\mathbb{R}^n \to \mathbb{R}^m$. Choosing a basis gives it a matrix representation, but the underlying map does not depend on that choice.

Example 1.7. Inertia Tensor of a Point Mass

A point mass m at displacement $\mathbf{r}$ from a chosen reference origin contributes inertia tensor

$$\mathbf{I} \;=\; m\big(\|\mathbf{r}\|^2\,\mathbf{1} - \mathbf{r}\,\mathbf{r}^T\big), \tag{1.51}$$

where $\mathbf{1}$ is the 3×3 identity. Notice that no basis vectors appear—$\mathbf{r}\,\mathbf{r}^T$ is a dyad formed from the position vector with itself, and $\mathbf{1}$ is the identity map. This expression is *coordinate-free*: it is valid in every frame simultaneously.

When we change from a body-fixed frame $\{B\}$ to a world frame $\{W\}$ via a rotation $\mathbf{R} \in \mathrm{SO}(3)$, the matrix representation of the inertia tensor transforms by the **congruence transformation**:

$$\mathbf{I}^W \;=\; \mathbf{R}\,\mathbf{I}^B\,\mathbf{R}^T. \tag{1.52}$$

The congruence form $\mathbf{R}\,(\cdot)\,\mathbf{R}^T$ rotates both the input slot and the output slot of the dyadic. It preserves symmetry, positive-definiteness, and—for rotations—eigenvalues (the principal moments of inertia). This formula changes orientation about the same reference point; changing the origin also requires the parallel-axis contribution.

1.6.5 Basis, Metric and Mechanical Duality

If old velocity coordinates and new coordinates satisfy $\mathbf{v} = S\widetilde{\mathbf{v}}$ for an invertible S, preservation of kinetic energy and power requires

$$\widetilde{M} = S^T M S, \qquad \widetilde{\boldsymbol{\tau}} = S^T \boldsymbol{\tau},$$

$$\widetilde{\mathbf{v}}^T \widetilde{M} \widetilde{\mathbf{v}} = \mathbf{v}^T M \mathbf{v}, \qquad \widetilde{\boldsymbol{\tau}}^T \widetilde{\mathbf{v}} = \boldsymbol{\tau}^T \mathbf{v}.$$

This congruence law describes the kinetic bilinear form and its vector-to-covector map. A linear endomorphism instead has the similarity law $\widetilde{A} = S^{-1} A S$. For an orthogonal S, the two transformation patterns can look alike, which can hide the distinction.

Congruence preserves definiteness but generally changes ordinary matrix eigenvalues. Physical comparisons in nonorthogonal or differently scaled coordinates require the corresponding metric, not raw eigenvalues or a dot product chosen only because the arrays have equal length. This distinction connects velocity transmission through J to force transmission through J^T, and prevents coordinate-dependent numerical norms from being labeled energy or effort without a model.

1.6.6 Connection to the Rest of This Book

The dyadic point of view recurs throughout our study of rigid-body mechanics:

- **Screw axes and wrenches (Chapter 5).** A wrench combines a force and a moment into a single six-dimensional object. The spatial inertia is a 6×6 map from twist to spatial momentum, not directly to wrench. A wrench is related to momentum rate, including the transport terms required by a moving coordinate frame.

- **Spatial algebra (Chapter 8).** With twist ordered as $(\boldsymbol{\omega}, \mathbf{v}_O)$ and centre-of-mass displacement $\mathbf{c}$ from the reference origin O, the spatial inertia is

$$\mathcal{I}_O = \begin{bmatrix} I_O & m[\mathbf{c}]_\times \\ -m[\mathbf{c}]_\times & mI_3 \end{bmatrix}.$$

  Here I_O is rotational inertia about O, not about the centre of mass unless the origins coincide. The matrix maps twist to angular momentum about O and linear momentum, and is SPD when every nonzero rigid-body twist has positive kinetic energy. In a body-fixed frame attached to the rigid body, $\mathbf{w} = \mathcal{I}_O\dot{\mathbf{v}} + \mathbf{v} \times^* (\mathcal{I}_O\mathbf{v})$, where the force cross-product operator includes the required frame transport. The momentum map itself contains no acceleration.

- **The mass matrix (Chapter 11).** The joint-space mass matrix $\mathbf{M}(\mathbf{q})$ that appears in the manipulator equation $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{c} = \boldsymbol{\tau}$ is a configuration-dependent dyadic on joint-velocity space. Each link contributes a term built from Jacobians and spatial inertias through $M(\mathbf{q}) = \sum_i J_i^T \mathcal{I}_i J_i$, with each twist Jacobian and inertia expressed about matching origins in matching frames. This is the kinetic-energy counterpart of assembling nodal stiffness through a compatibility map.

- **The articulated-body algorithm (Chapter 10).** Featherstone's algorithm propagates *articulated-body inertias* through a kinematic tree. These are 6×6 dyadics that account for the coupling between a link's motion and the reactions of all its descendants.

Understanding that all these objects are dyadics—linear maps built from sums of outer products—provides a unifying thread: the same algebraic structure appears whether we are analysing a truss, spinning a gyroscope, or computing the forward dynamics of a humanoid robot.

1.7 Matrix Norms and Conditioning

Having decomposed matrices via the eigendecomposition and SVD, we now turn to a fundamental practical question: *how large is a matrix, and how sensitive is a linear system to perturbations?* Matrix norms quantify the first question; the *condition number* answers the second.

Definition 1.15. Frobenius Norm $\|\mathbf{A}\|_F$

The **Frobenius norm** of a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2} = \sqrt{\operatorname{tr}\big(\mathbf{A}^T\mathbf{A}\big)} = \sqrt{\sigma_1^2 + \sigma_2^2 + \cdots + \sigma_{\min(m,n)}^2}. \tag{1.53}$$

The last equality follows directly from the SVD. The Frobenius norm treats the matrix as a long vector and computes its Euclidean length.

Definition 1.16. Spectral Norm $\|\mathbf{A}\|_2$

The **spectral norm** (also called the *operator 2-norm* or *induced 2-norm*) is

$$\|\mathbf{A}\|_2 \;=\; \max_{\|\mathbf{x}\|_2=1} \|\mathbf{A}\mathbf{x}\|_2 \;=\; \sigma_{\max}(\mathbf{A}). \tag{1.54}$$

This is the maximum factor by which $\mathbf{A}$ can stretch a unit vector—exactly the length of the longest semi-axis of the image ellipsoid from the SVD's geometric interpretation (Section 1.5).

Definition 1.17. Condition Number $\kappa(\mathbf{A})$

For a matrix $\mathbf{A}$ with $\sigma_{\min} > 0$, the **condition number** with respect to the 2-norm is

$$\kappa(\mathbf{A}) \;=\; \frac{\sigma_{\max}}{\sigma_{\min}}. \tag{1.55}$$

When $\sigma_{\min} = 0$ (i.e., $\mathbf{A}$ is singular), we define $\kappa(\mathbf{A}) = \infty$. A well-conditioned matrix has $\kappa \approx 1$; an ill-conditioned matrix has $\kappa \gg 1$.

1.7.1 What the Pseudoinverse Does Not Guarantee

For $J = U\Sigma V^T$, the Moore–Penrose pseudoinverse is $J^+ = V\Sigma^+U^T$, with each nonzero singular value inverted. The command $J^+\mathbf{v}$ is the minimum-Euclidean-norm least-squares solution. It exactly realizes $\mathbf{v}$ only when $\mathbf{v} \in \operatorname{im} J$; its notion of minimum norm depends on the declared units and metric.

The pseudoinverse does not remove near-singular amplification. With $J = \operatorname{diag}(1, 10^{-3})$ and desired velocity $(1,1)^T$, it requests joint velocity $(1,1000)^T$. Error in the weak task direction is amplified; error in every direction need not be. A condition-number threshold alone also misses an overall loss of scale such as $J = \epsilon I$, whose condition number stays one as $\epsilon \to 0$.

A damped least-squares alternative solves

$$\min_{\dot{\mathbf{q}}} \;\|J\dot{\mathbf{q}} - \mathbf{v}\|_2^2 + \lambda^2\|\dot{\mathbf{q}}\|_2^2, \qquad \lambda > 0,$$

and applies singular-direction gains

$$\frac{\sigma_i}{\sigma_i^2 + \lambda^2} \le \frac{1}{2\lambda}.$$

This bounds inverse gain by accepting a task residual. It does not enforce joint-speed limits, contact constraints or closed-loop stability by itself. Truncated SVD likewise requires an explicit tolerance for treating small measured singular values as zero; numerical rank is not automatically exact physical rank.

For a golf model, a small singular value can explain why a chosen hand-velocity correction demands a large joint-velocity change at one configuration. Predicting actual motion additionally requires the inertial equations, actuator limits and controller. Neither shaking nor a beneficial release follows from the condition number alone.

1.8 Matrix Decompositions for Computation

The eigendecomposition and SVD reveal geometric structure. This section surveys three additional decompositions whose primary purpose is *computational efficiency* when solving the linear systems that arise throughout dynamics and control.

1.8.1 LU Decomposition

Any invertible matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ can (with row permutations) be factored as

$$\mathbf{PA} = \mathbf{LU}, \tag{1.56}$$

where $\mathbf{P}$ is a permutation matrix, $\mathbf{L}$ is unit lower triangular, and $\mathbf{U}$ is upper triangular. Solving $\mathbf{Ax} = \mathbf{b}$ then reduces to two triangular solves (forward and back substitution), each costing $O(n^2)$ after the $O(n^3)$ factorization. When multiple right-hand sides share the same $\mathbf{A}$, LU is the workhorse: factor once, solve many times.

1.8.2 QR Decomposition

For any $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$,

$$\mathbf{A} = \mathbf{QR}, \tag{1.57}$$

where $\mathbf{Q} \in \mathbb{R}^{m\times m}$ is orthogonal and $\mathbf{R} \in \mathbb{R}^{m\times n}$ is upper trapezoidal: an upper-triangular $n \times n$ block above zero rows. QR is the preferred method for solving least-squares problems $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$ because, unlike the normal equations $\mathbf{A}^T\mathbf{Ax} = \mathbf{A}^T\mathbf{b}$, it avoids squaring the condition number ($\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$ for full column rank). A backward-stable QR solve avoids that additional conditioning penalty. Rank-deficient problems need an appropriate rank-revealing or SVD treatment, particularly when a minimum-norm solution is required.

1.8.3 Cholesky Decomposition

If $\mathbf{M} \in \mathbb{R}^{n\times n}$ is symmetric positive definite (SPD), then there exists a unique lower-triangular matrix $\mathbf{L}$ with positive diagonal entries such that

$$\mathbf{M} = \mathbf{LL}^T. \tag{1.58}$$

Dense Cholesky costs roughly half the leading-order flops of dense LU and is backward stable under the usual floating-point assumptions for a sufficiently well-resolved SPD input. No pivoting is needed in exact arithmetic. Near-singularity, underflow, overflow and loss of definiteness through rounding remain numerical concerns.

Mathematical Principle 1.1. Cholesky and the Mass Matrix

In independent generalized-velocity coordinates, a model with strictly positive kinetic energy for every nonzero admissible velocity has an SPD mass matrix. Redundant embedding coordinates, massless idealizations or singular coordinate charts can violate that premise. With ideal constraints one may instead solve a reduced system or an indefinite saddle-point system; Cholesky does not directly apply to every augmented matrix.

For the unconstrained SPD solve $M\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{h} - \mathbf{g}$, Cholesky or a structure-exploiting factorization is appropriate. An articulated-body algorithm can solve tree dynamics recursively without explicitly forming the full dense matrix. Choose the algorithm for the model and sparsity rather than treating one factorization as universal [36].

1.9 Worked Example: Eigenvalue and SVD Analysis in Python

Let us bring together the theory developed in this chapter with a concrete computational example. The following Python code constructs a 3×3 symmetric matrix (representing a simplified 3D inertia tensor), computes its eigendecomposition and SVD, and verifies the key properties we have proven.

```python
import numpy as np

# ---- Define a symmetric positive definite matrix ----
# This could represent a simplified 3D inertia tensor
M = np.array([
    [5.0, 1.0, 0.5],
    [1.0, 4.0, 0.3],
    [0.5, 0.3, 3.0]
])

# Verify symmetry
assert np.allclose(M, M.T), "Matrix is not symmetric!"

# ---- Eigendecomposition ----
eigenvalues, eigenvectors = np.linalg.eigh(M)  # eigh for symmetric matrices
print("Eigenvalues:", np.round(eigenvalues, 4))
print("Eigenvectors (columns):\n", np.round(eigenvectors, 4))

# Positive principal moments plus their triangle inequalities
# are required for a physical three-dimensional inertia tensor.
assert eigenvalues[-1] <= eigenvalues[0] + eigenvalues[1]

# Verify: all eigenvalues positive => positive definite
assert all(eigenvalues > 0), "Matrix is not positive definite!"
print("Matrix is POSITIVE DEFINITE (all eigenvalues > 0)")

# Verify: eigenvectors are orthogonal
QTQ = eigenvectors.T @ eigenvectors
print("Q^T Q (should be identity):\n", np.round(QTQ, 10))

# Verify spectral decomposition: M = Q * Lambda * Q^T
M_reconstructed = eigenvectors @ np.diag(eigenvalues) @ eigenvectors.T
print("Reconstruction error:", np.linalg.norm(M - M_reconstructed))

# ---- Singular Value Decomposition ----
U, sigma, VT = np.linalg.svd(M)
print("\nSingular values:", np.round(sigma, 4))
print("Eigenvalues (sorted descending):", np.round(sorted(eigenvalues,
    reverse=True), 4))
print("For symmetric PD matrices, singular values = eigenvalues (sorted)")

# ---- Condition Number ----
```

```python
kappa = sigma[0] / sigma[-1]
print(f"\nCondition number: {kappa:.4f}")
print("(Close to 1 means well-conditioned; >>1 means ill-conditioned)")

# ---- Cholesky Decomposition ----
L = np.linalg.cholesky(M)
print("\nCholesky factor L:\n", np.round(L, 4))
print("Verify L @ L^T = M, error:", np.linalg.norm(M - L @ L.T))
```

Listing 1.1: Eigendecomposition and SVD of a Symmetric Matrix

1.10 Chapter Summary

Mathematical Principle 1.2. Key Takeaways from Chapter 1

1. A **vector space** is defined by eight axioms guaranteeing closure, commutativity, and compatibility of addition and scalar multiplication. $\mathbb{R}^n$ is the prototypical example.
2. The **rank** of a matrix counts independent columns; the **null space** captures the "lost" dimensions. The **Rank-Nullity Theorem** connects them: rank + nullity = number of columns.
3. The **dot product** measures alignment between vectors and defines orthogonality. The **cross product** in $\mathbb{R}^3$ produces a perpendicular vector and can be written as a *skew-symmetric matrix* multiplication—a representation central to Lie Algebra ($\mathfrak{so}(3)$).
4. **Eigenvalues** characterize asymptotic stability of a constant continuous-time linear system, with Jordan structure needed on the imaginary-axis boundary. Eigenvectors of a symmetric inertia tensor give principal axes; its eigenvalues give principal moments. The **Spectral Theorem** guarantees that symmetric matrices have real eigenvalues and orthogonal eigenvectors.
5. **Positive definiteness** ($\mathbf{M} \succ 0$) guarantees a quadratic form has a strict minimum—the cornerstone of Lyapunov stability proofs and energy-based control design.
6. The **SVD** decomposes a matrix into orthogonal transformations and nonnegative scaling. Applied to a Jacobian with declared input and output metrics, it describes instantaneous velocity transmission. Relative conditioning, absolute inverse gain and finite-time controllability answer different questions.

1.11 Further Reading

The material in this chapter draws on a long tradition of textbooks at the intersection of linear algebra and applied mathematics. For a comprehensive undergraduate treatment that emphasizes geometric intuition alongside computational technique, Strang [45] remains the standard reference; his "four fundamental subspaces" perspective directly informs the rank-nullity discussion presented here.

In the robotics context, Chapter 2 of Murray, Li, and Sastry [35] develops the specific linear-algebraic tools—rotation matrices, skew-symmetric operators, and the Lie bracket—that recur throughout the rest of this textbook. Readers who wish to pursue the numerical aspects of linear algebra, particularly iterative solvers and the role of condition numbers in optimization, will find Nocedal and Wright [36] an invaluable companion.

The dyadic and tensor-product viewpoint introduced in Section 1.6 connects directly to continuum mechanics, where stress and strain are second-order tensors built from exactly the same outer-product construction. Students interested in these connections should consult any graduate text on continuum mechanics or tensor analysis.

Finally, the eigenvalue structures and matrix exponentials introduced in this chapter are the first hints of *Lie theory*. Hall [19] provides a mathematically rigorous bridge from linear algebra to Lie groups and Lie algebras, foreshadowing the rotation groups (SO(3)(3)) and rigid-body transformation groups (SE(3)(3)) that appear in Chapters 4 and 5.

1.12 Exercises

Computational Exercises

1. **(Rank and Null Space)** Given the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

compute the rank, determinant, and a basis for the null space. Verify the Rank-Nullity Theorem. *Hint: row reduce to echelon form.*

2. **(Eigenvalue Computation)** For the matrix $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$, compute the eigenvalues and eigenvectors by hand. Verify that $\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i$ for each pair.

3. **(SVD by Hand)** Compute the SVD of $\mathbf{M} = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$. What are the singular values, and how do they relate to the eigenvalues?

4. **(Cholesky)** Verify that $\mathbf{M} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ is positive definite by: (a) checking all eigenvalues are positive, (b) computing the Cholesky factorization $\mathbf{L}\mathbf{L}^T$.

5. **(Python)** Write a Python script that generates a random 5×5 symmetric matrix $\mathbf{M} = \mathbf{A}^T\mathbf{A} + \epsilon\mathbf{I}$ with $\epsilon > 0$ (guaranteeing positive definiteness in exact arithmetic), computes its eigendecomposition, SVD, and Cholesky factorization, and verifies all three decompositions reproduce the original matrix within numerical tolerance.

Conceptual Exercises

6. **(Vector Spaces)** Is the set of all 2×2 symmetric matrices a vector space under standard matrix addition and scalar multiplication? Prove or disprove by checking the axioms. What is its dimension?

7. **(Physical Interpretation)** A robot Jacobian $\mathbf{J} \in \mathbb{R}^{3\times 5}$ has singular values $\sigma_1 = 2.0$, $\sigma_2 = 0.8$, $\sigma_3 = 0.01$. Describe the shape of the velocity manipulability ellipsoid. Is the robot near a singularity? What does the condition number tell you about the reliability of inverse kinematics computations?

8. **(Positive Definiteness in Control)** Explain why strictly positive kinetic energy for every nonzero independent generalized velocity implies an SPD mass matrix. How can redundant coordinates, a massless idealization or a singular coordinate chart instead produce a zero eigenvalue? Distinguish this from an ordinary massive rigid body having a physically accessible motion with zero kinetic energy.

9. **(Skew-Symmetric Matrices)** Prove that for any skew-symmetric matrix $\mathbf{S}$ ($\mathbf{S}^T = -\mathbf{S}$), the matrix $e^{\mathbf{S}} = \mathbf{I} + \mathbf{S} + \frac{1}{2!}\mathbf{S}^2 + \cdots$ is an orthogonal matrix (i.e., $(e^{\mathbf{S}})^T e^{\mathbf{S}} = \mathbf{I}$). *Hint: use the fact that* $(e^{\mathbf{S}})^T = e^{\mathbf{S}^T} = e^{-\mathbf{S}}$. This result is the mathematical foundation of Chapter 6.

Proof-Based Exercises

10. **(Rank-Nullity Application)** Prove that if $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m < n$, then $\mathbf{A}$ must have a nontrivial null space (i.e., $\mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$). Interpret this result for a robot with more independent joint coordinates than task-space dimensions. Explain why this kinematic redundancy does not establish underactuation, and why a full-row-rank task Jacobian can have a nontrivial kernel.

11. **(Trace and Eigenvalues)** Prove that the trace of a matrix (sum of diagonal elements) equals the sum of its eigenvalues: $\mathrm{tr}(\mathbf{M}) = \sum_{i=1}^{n} \lambda_i$. Count eigenvalues with algebraic multiplicity, over $\mathbb{C}$ if necessary. *Hint: use trace invariance under similarity and a complex Schur triangularization; do not assume that every matrix is diagonalizable.*

12. **(Determinant and Eigenvalues)** Prove that the determinant of a matrix equals the product of its eigenvalues: $\det(\mathbf{M}) = \prod_{i=1}^{n} \lambda_i$. *Hint: evaluate the characteristic polynomial at* $\lambda = 0$.

13. **(Uniqueness of SVD)** The sorted singular values are unique. Even for a distinct positive singular value, a corresponding left/right pair can have both signs reversed. For a repeated positive singular value, both bases can undergo the same orthogonal change within their paired subspaces. Demonstrate this with I_2; also explain why bases for the left and right zero-singular-value subspaces can be chosen independently.

14. **(Schur Complement and Positive Definiteness)** Let $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^T & \mathbf{C} \end{bmatrix}$ be a symmetric block matrix with $\mathbf{A} \succ 0$. Prove that $\mathbf{M} \succ 0$ if and only if $\mathbf{C} - \mathbf{B}^T\mathbf{A}^{-1}\mathbf{B} \succ 0$ (the *Schur complement* condition). This result is used extensively in the Articulated Body Algorithm (Chapter 10).

Chapter 2

The Transition to State Space

A photograph of a golfer tells us where the visible segments are. It does not tell us their velocities, the shaft's deformation rate, or the muscle activation that will persist into the next instant. Two swings can pass through nearly the same pose and then produce different clubhead motion. State-space modeling asks what information must be carried forward to predict that continuation under specified future inputs.

This is a way to organize physics, not to leave physical space behind. Joint coordinates describe configuration; forward kinematics gives the clubhead pose; equations of motion describe how forces change the state; measurement equations describe what cameras or sensors reveal. These maps must agree. A useful swing model connects them before making claims about passive motion, control effort or impact accuracy.

2.1 State, Input, and Output

2.1.1 A State Is Sufficient Information for a Declared Model

For a deterministic model, a state at time t_0, together with the future input and the model, determines the subsequent solution wherever that solution exists uniquely. State variables need not be a minimal list: redundant coordinates can be useful if their constraints are enforced. Minimal realization is a separate input–output concept introduced below. A state can be sufficient for one prediction task and insufficient for another.

For a point mass with known force law, position q and velocity v suffice:

$$x = \begin{bmatrix} q \\ v \end{bmatrix}, \qquad \dot{x} = \begin{bmatrix} v \\ F(q, v, u, t)/m \end{bmatrix}, \qquad m > 0. \tag{2.1}$$

Acceleration is computed from this model. It need not be an independent state here. But a model with commanded jerk, $\dot{a} = u$, has state (q, v, a); an actuator with current, pressure, activation or hysteresis needs its own memory variables unless a justified reduction removes them. An accelerometer measures an output related to acceleration and gravity; using that measurement does not by itself make acceleration an independent state.

An unconstrained natural mechanical system with an n-dimensional configuration manifold Q has state $(q, v) \in TQ$, of intrinsic dimension $2n$. Constraints, additional electrical or elastic states, delay, distributed deformation and hybrid modes change this description. Quaternion coordinates can contain more scalar entries than the intrinsic dimension. Finite-dimensional ordinary differential equations are a valuable model class, not a universal representation of every physical system.

Prediction also requires the future force or command history. A known instantaneous torque determines an instantaneous acceleration, not an entire future swing. Local Lipschitz continuity of the vector field in state is a standard sufficient condition for local uniqueness;

continuity alone is insufficient. Even a smooth field can escape in finite time. Stochastic models specify conditional distributions rather than exact future realizations.

2.1.2 What Can Be Observed Is a Different Question

Write a smooth model as

$$\dot{x} = F(x, u, t), \qquad y = h(x, u, t). \tag{2.2}$$

The output may be a joint encoder, a camera projection, a clubhead position, or a force measurement. A measurement can be many-to-one: one camera image does not generally determine a three-dimensional configuration, much less all its velocities. A known input is information supplied to an observer, not automatically an additional state measurement. Parameters such as segment inertia also need specification or estimation.

For golf, distinguish the modeled state, the available measurements, the uncertain estimate and the quantities of interest at impact. The same visible clubhead path can be compatible with different internal loads or activation histories. State estimation and model identification are related but different problems: reconstructing $x(t)$ for a known model does not establish that the model's parameters or zero-input counterfactual are correct.

2.2 Geometry of Position, Velocity, and Acceleration

2.2.1 Configuration Points and Tangent Vectors

A configuration is a point of Q. A velocity is a vector in T_qQ, attached to that configuration. Two velocities at the same point can be added; vectors at different points require a specified identification or transport. General manifolds have no canonical addition of points. In a chosen Euclidean vector-space model, however, addition of position coordinates is legitimate, and solutions of a linear position equation can superpose. The validity of dynamical superposition depends on the equations, not simply on naming a variable position or velocity.

For a pendulum, $Q = S^1$ and TQ is a cylinder. A real-valued angle is a local coordinate or an unwrapped lift; θ and $\theta + 2\pi$ denote the same configuration. The state curve $t \mapsto x(t)$ is parameterized by one time variable, but its image may be a single equilibrium point, a periodic curve, or a nonperiodic path. Under a prescribed time-dependent input, the state alone need not define an autonomous phase portrait.

2.2.2 Coordinate Acceleration Is Not a Vector Under General Changes

For new coordinates $z = \phi(q)$,

$$\dot{z} = D\phi(q)v, \qquad \ddot{z} = D\phi(q)\ddot{q} + D^2\phi(q)[v, v]. \tag{2.3}$$

The Hessian term prevents bare coordinate acceleration from transforming like a tangent vector. For example, $q(t) = 1 + 2t$ has $\ddot{q} = 0$, while $z = q^2$ has $\ddot{z} = 8$ near $q = 1$. With a chosen connection, covariant acceleration $\nabla_v v$ is a tangent vector; the lifted state derivative $(v, \ddot{q})$ is a vector in $T_{(q,v)}TQ$. These are related objects, not interchangeable names for the same coordinate array.

For natural mechanics the kinetic metric supplies the Levi-Civita connection, so the coordinate equation contains its Christoffel velocity products. Generalized forces are covectors paired with virtual displacement or velocity; the inverse mass metric converts them into acceleration contributions. At a fixed (q, v), differences between accelerations produced by different inputs transform linearly because the common Hessian term cancels. Adding two entire nonlinear trajectories has no such guarantee.

Clubhead position $r = h(q)$ makes the connection concrete:

$$\dot{r} = J(q)v, \qquad \ddot{r} = J(q)\dot{v} + \dot{J}(q, v)v. \tag{2.4}$$

A large Cartesian radial acceleration can coexist with modest angular acceleration. Forces, coordinate acceleration, task acceleration and control authority must be compared in declared coordinates and units. The neighboring configuration and Lagrangian chapters develop this geometry further.

2.3 Building Models From Physical Balances

2.3.1 A Point-Mass Pendulum

Take a point mass m on a massless rod of length ℓ, with a fixed pivot and angle measured from downward vertical. With applied pivot torque τ and viscous coefficient $b \geq 0$,

$$m\ell^2\ddot{\theta} + b\dot{\theta} + mg\ell\sin\theta = \tau, \qquad \dot{x} = \begin{bmatrix} v \\ -(g/\ell)\sin\theta - bv/(m\ell^2) + \tau/(m\ell^2) \end{bmatrix}. \tag{2.5}$$

Here $x = (\theta, v)$ and the input is torque, in N m. Writing the right side as τ after division by $m\ell^2$ would instead require redefining that symbol as an angular-acceleration input. A uniform rod has different inertia and center-of-mass distance. A moving pivot requires its prescribed motion and boundary force; it is not this fixed-pivot model.

2.3.2 A DC Motor: An Electrical State Matters

For an ideal voltage-driven motor without load torque,

$$L\dot{i} = V_{\text{in}} - Ri - K_e\omega, \qquad J\dot{\omega} = K_t i - b\omega, \qquad \dot{\theta} = \omega. \tag{2.6}$$

Inductance L and inertia J are positive; R and b are nonnegative losses. The state order (i, θ, ω) gives

$$A = \begin{bmatrix} -R/L & 0 & -K_e/L \\ 0 & 0 & 1 \\ K_t/J & 0 & -b/J \end{bmatrix}, \qquad B = \begin{bmatrix} 1/L \\ 0 \\ 0 \end{bmatrix}. \tag{2.7}$$

For speed and current predictions alone, θ can be omitted because it does not feed back into those equations. To predict absolute angle, it must be retained. A state variable does not need a derivative directly containing the input: $\dot{\theta} = \omega$ is enough to make angle part of that prediction.

With $L = .01$ H, $R = 1\,\Omega$, $K_e = K_t = .02$ in coherent SI units, $J = .001$ kg m^2 and $b = .001$ N m s/rad,

$$A = \begin{bmatrix} -100 & 0 & -2 \\ 0 & 0 & 1 \\ 20 & 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 100 \\ 0 \\ 0 \end{bmatrix}, \quad \lambda(A) \approx \{0, -99.594297, -1.405703\}\,\mathrm{s}^{-1}. \tag{2.8}$$

The zero mode is the retained angle offset. The zero-voltage full state is stable but not asymptotically stable to one selected angle; current and rate decay, and angle tends to an initial-condition-dependent constant. Voltage still controls angle through current and rate. An ideal motor with numerically equal K_t, K_e satisfies

$$\frac{d}{dt}\left(\frac{1}{2}Li^2 + \frac{1}{2}J\omega^2\right) = V_{\text{in}}i - Ri^2 - b\omega^2. \tag{2.9}$$

This power identity checks the coupling signs. Neither the matrix B alone nor a state derivative is a measure of energy injection.

2.3.3 Two Tanks: State Domains and Flow Direction

Let two open tanks have equal area $A_t > 0$, nonnegative heights, and connections at the same zero-height datum. With nonnegative inflow Q_{in} and positive coefficients c_t, c_o,

$$\begin{aligned} Q_{12} &= c_t \operatorname{sgn}(h_1 - h_2)\sqrt{|h_1 - h_2|}, & Q_o &= c_o\sqrt{h_2}, \\ A_t\dot{h}_1 &= Q_{\text{in}} - Q_{12}, & A_t\dot{h}_2 &= Q_{12} - Q_o. \end{aligned} \tag{2.10}$$

Both flow coefficients have units $\mathrm{m}^{5/2}/\mathrm{s}$. A check valve would require a different law. The signed square root reverses transfer when tank 2 is higher. Adding the balances gives $A_t(\dot{h}_1+\dot{h}_2) = Q_{\text{in}} - Q_o$, so internal transfer cancels. At a dry tank the vector field points into the nonnegative domain. Overflow, finite connecting-pipe inertia and empty-pipe behavior are excluded.

The square-root law is continuous but not locally Lipschitz at equal heads or zero outlet height. That fact alone does not establish nonuniqueness. For two solutions with the same inflow, monotonicity of each flow law gives

$$A_t\frac{d}{dt}\frac{1}{2}\|h - \tilde{h}\|^2 = -\Delta(h_1 - h_2)\Delta Q_{12} - \Delta h_2 \Delta Q_o \le 0. \tag{2.11}$$

Here each Δ is the difference between the two solutions. Equal initial states therefore remain equal. Local existence, the inward boundary field and finite volume growth under bounded inflow complete the physical continuation argument. Numerical methods must respect the domain and resolve reversals; silently replacing a negative numerical height with zero can hide mass-balance error.

For $A_t = .1\ \mathrm{m}^2$, $c_t = .05$ and $c_o = .03\ \mathrm{m}^{5/2}/\mathrm{s}$, $Q_{\text{in}} = .01\ \mathrm{m}^3/\mathrm{s}$ and $h(0) = (.5, .2)$ m, integration gives approximately $h(10) = (.194816, .147879)$ m and $h(25) = (.153289, .112933)$ m. The equilibrium is

$$h_2^* = (Q_{\text{in}}/c_o)^2 = .111111\,\mathrm{m}, \qquad h_1^* = h_2^* + (Q_{\text{in}}/c_t)^2 = .151111\,\mathrm{m}. \tag{2.12}$$

These are illustrative hydraulic calculations, included to show why a state-space model must retain physical balance and admissibility.

2.4 From Differential Equations to Linear State Models

For an explicit scalar equation $y^{(n)} = g(y, \dot{y}, \ldots, y^{(n-1)}, u, t)$, set $x_j = y^{(j-1)}$. Then $\dot{x}_j = x_{j+1}$ for $j < n$ and $\dot{x}_n = g$. The construction requires an equation solvable for its highest derivative; an implicit singular relation may instead be a differential-algebraic system.

For constant coefficients,

$$y^{(n)} + a_{n-1}y^{(n-1)} + \cdots + a_0 y = u, \quad A = \begin{bmatrix} 0 & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{n-1} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}. \tag{2.13}$$

The shift equations prove equivalence, with initial derivatives included. This companion construction is not a uniqueness theorem for every realization of an input–output map. A nonsingular constant change $z = Tx$ gives $A_z = TAT^{-1}$, $B_z = TB$ and $C_z = CT^{-1}$; nonminimal realizations can even have different dimensions.

2.4.1 Matrices, Units, and Ports

An LTI model is

$$\dot{x} = Ax + Bu, \qquad y = Cx + Du, \quad A \in \mathbb{R}^{n\times n},\ B \in \mathbb{R}^{n\times m},\ C \in \mathbb{R}^{p\times n},\ D \in \mathbb{R}^{p\times m}. \tag{2.14}$$

Time-varying linear models use $A(t), B(t), C(t), D(t)$. Additive offsets produce an affine model; deviations about a feasible reference can remove its nominal offset. The output equation includes direct feedthrough where the physical measurement requires it. A block diagram must feed the state back into Ax and the input into both Bu and Du; neither path is optional merely because the system is open loop.

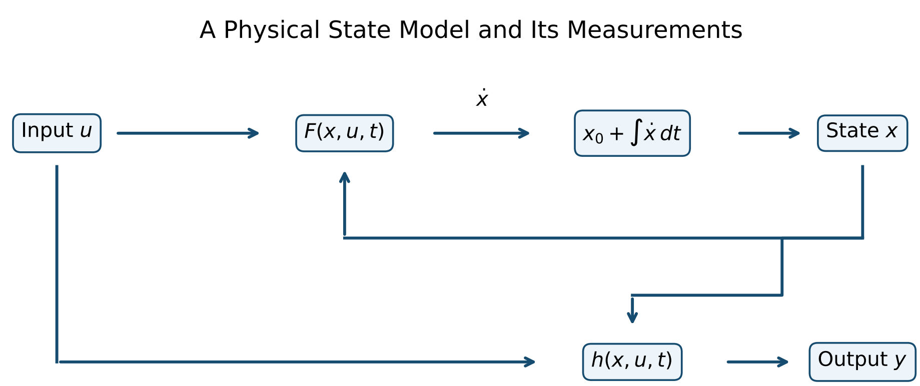


Figure 2.1: State, Input, and Measurement Paths. State and Input Enter Both the Dynamics and the Output Map; Integration Also Requires an Initial State.

Different state components can have different units. For $m\ddot{q} + c\dot{q} + kq = u$,

$$x = (q, v), \qquad A = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}. \tag{2.15}$$

The input force changes the velocity derivative; it does not instantaneously jump position under a bounded ordinary input. An impulse needs an impulse law. Actual input power is uv, and mechanical energy satisfies $\dot{E} = uv - cv^2$. Thus signed work may be negative even for a nonzero positive input. Scaled coordinates are needed before interpreting a Euclidean norm of position and velocity as a physical error or energy.

2.5 Solutions, Equilibria, and Local Models

2.5.1 The LTI Solution and Its Limits

The matrix exponential is defined for every finite square matrix, including defective and singular ones:

$$e^{At} = \sum_{j=0}^{\infty} \frac{A^j t^j}{j!}, \qquad x(t) = e^{At}x_0 + \int_0^t e^{A(t-s)}Bu(s)\,ds. \tag{2.16}$$

Uniform convergence of the series and its derivative on bounded time intervals permits termwise differentiation; it gives $\frac{d}{dt}e^{At} = Ae^{At}$ and $e^{A0} = I$. Differentiating $e^{-At}x(t)$ and integrating proves the forced formula. This establishes superposition for this linear model, including its position components, with consistent initial data and inputs.

If $A = P\Lambda P^{-1}$ is diagonalizable, exponentiating its diagonal entries is an identity. It is not a universal numerical algorithm: defective matrices lack that decomposition and ill-conditioned eigenvectors amplify error. Established matrix-exponential routines use more robust constructions. For LTV systems replace $e^{A(t-s)}$ by the state-transition matrix $\Phi(t,s)$; in general Φ is not the exponential of the time integral of $A(t)$.

2.5.2 Linearization About a State or a Swing

An equilibrium under constant u^* satisfies $F(x^*, u^*) = 0$. For an explicitly time-dependent model, a constant state requires that equality for all relevant times. A steady rotation may be stationary in selected reduced variables while angle keeps changing; it is not an equilibrium of the full angle state.

For a twice continuously differentiable field near an equilibrium,

$$\delta\dot{x} = A\delta x + B\delta u + r, \qquad A = F_x(x^*, u^*), \quad B = F_u(x^*, u^*), \quad \|r\| \le c(\|\delta x\| + \|\delta u\|)^2 \tag{2.17}$$

on a sufficiently small neighborhood with a bounded Hessian. Continuous differentiability gives a little-o remainder instead. A swing is usually a moving nominal solution, so its perturbations obey an LTV equation with derivatives evaluated along $(\bar{x}(t), \bar{u}(t))$. Feedback $u = \pi(x,t)$ contributes $F_u\pi_x$ to the closed-loop Jacobian. A reference that does not satisfy the dynamics leaves a residual forcing term.

For the unforced undamped pendulum, the downward and upward local matrices are

$$A_{\text{down}} = \begin{bmatrix} 0 & 1 \\ -g/\ell & 0 \end{bmatrix}, \qquad A_{\text{up}} = \begin{bmatrix} 0 & 1 \\ g/\ell & 0 \end{bmatrix}. \tag{2.18}$$

At the top, the displacement is $\eta = \theta - \pi$, so $\ddot{\eta} = (g/\ell)\eta$ to first order. It is incorrect to interpret that equation as an equation for the original absolute angle with equilibrium zero.

2.6 Stability Requires More Than an Eigenvalue List

2.6.1 Linear Classification and Jordan Blocks

For an autonomous finite-dimensional LTI system, the origin is exponentially asymptotically stable exactly when A is Hurwitz: all eigenvalues have negative real part. It is Lyapunov stable exactly when every eigenvalue has nonpositive real part and every eigenvalue on the imaginary axis is semisimple (all its Jordan blocks have size one). A positive real part or a nontrivial imaginary-axis Jordan block gives instability.

In two real dimensions, real negative eigenvalues produce a stable node; real positive ones produce an unstable node. They do not produce spirals. Opposite signs give a saddle. Nonreal conjugates with negative or positive real part give inward or outward spirals. A nonzero purely imaginary pair gives a linear center with periodic elliptical orbits. Repeated real eigenvalues require the eigenvector/Jordan structure to distinguish node types. Zero eigenvalues require their own examination.

For example,

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad e^{At} = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}. \tag{2.19}$$

Both eigenvalues are zero, but a nonzero initial velocity makes position grow without bound. Even when A is Hurwitz, nonnormal matrices can amplify the Euclidean norm transiently. Eigenvalues describe asymptotic rates and modal structure; they do not determine every finite-time error or monotonicity property.

2.6.2 Nonlinear Borderline Cases

For a continuously differentiable autonomous nonlinear field, a Hurwitz equilibrium Jacobian implies local exponential stability; an eigenvalue with positive real part implies instability. If there are no positive real parts but some zero real parts, linearization can be inconclusive. The scalar systems $\dot{x} = -x^3$, $\dot{x} = 0$ and $\dot{x} = x^3$ all have zero derivative at the origin, yet respectively have asymptotic stability, stability without attraction, and instability.

The undamped pendulum's bottom is stable but not attractive because its locally positive energy is conserved. The linear center alone is not a proof that every nonlinear system with the same imaginary eigenvalues has a center. A phase portrait can suggest a global picture, but simulations cover selected initial states and times, not every possible trajectory.

2.7 Reachability, Observability, and Hidden Modes

2.7.1 Reachability and the Gramian

For a continuous-time LTI system with unrestricted inputs, controllability means that any initial state can be driven to any terminal state over any prescribed positive horizon. Its equivalent rank condition is

$$\mathcal{C} = [B \ AB \ \cdots \ A^{n-1}B], \qquad \operatorname{rank} \mathcal{C} = n. \tag{2.20}$$

To see why, define

$$W(T) = \int_0^T e^{As} B B^T e^{A^T s}\, ds, \qquad z^T W(T) z = \int_0^T \|B^T e^{A^T s} z\|^2\, ds. \tag{2.21}$$

If the right side vanishes, continuity makes the integrand zero throughout the interval. Differentiating its analytic factor at zero gives $B^T(A^T)^j z = 0$ for every j. Conversely, vanishing through $j = n-1$ implies vanishing for every power by Cayley–Hamilton, and hence for the exponential. Thus W is positive definite exactly when $\mathcal{C}$ has full row rank. For $d = x_f - e^{AT}x_0$, the input

$$u(t) = B^T e^{A^T(T-t)} W(T)^{-1} d \tag{2.22}$$

reaches x_f. Substitution into the solution formula verifies it. It minimizes the unweighted integral of $u^T u$ among such inputs, by orthogonality to the nullspace of the endpoint operator. This is a squared-input cost, not generally mechanical energy or metabolic effort.

Rank alone says nothing about feasibility under torque, speed, contact or state constraints. Gramian conditioning and the remaining time matter, and require declared state/input scales. For a free unit mass, bounded force $|u| \le U$ from rest gives $|v(T)| \le UT$ and $|q(T) - q(0)| \le UT^2/2$, despite full rank. The two bounds are necessary, not a complete independent description of the joint reachable set.

2.7.2 PBH Tests Use the Correct Side

The equivalent Popov–Belevitch–Hautus test is

$$\operatorname{rank}[\lambda I - A \quad B] = n \quad \text{for every } \lambda \in \mathbb{C}. \tag{2.23}$$

A failure means there is a nonzero complex row w^* with $w^*A = \lambda w^*$ and $w^*B = 0$: a left eigenvector annihilates the input. Right eigenvectors do not give this test. The double integrator above with $B = (0,1)^T$ is controllable, even though its right zero-eigenvector $(1,0)^T$ is orthogonal to B. The rank and Gramian formulations, with unrestricted-input assumptions, are also developed in Kia's lecture notes [23].

2.7.3 Observability Needs a Known Input and an Output History

Subtract the known forced response and direct feedthrough from the measured output. The remaining signal is $Ce^{At}x_0$. The initial state is uniquely recoverable from its noiseless history over a positive interval exactly when

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}, \qquad \operatorname{rank}\mathcal{O} = n. \tag{2.24}$$

The observability Gramian $\int_0^T e^{A^T t} C^T C e^{At} dt$ proves this by the same squared-norm argument. Formally differentiating the residual output generates the rows of $\mathcal{O}$; directly differentiating noisy measurements is usually a poor estimator. Independent sample noise of variance σ^2 gives variance $2\sigma^2/\Delta t^2$ in a forward difference. An observer needs a noise model, conditioning and bandwidth decisions in addition to structural observability.

The duality is between reachability of (A, B) and observability of (A^T, B^T); similarly, observability of (A, C) is reachability of (A^T, C^T). In the motor example the controllability rank is three, but current-only measurement $C = (1, 0, 0)$ has observability rank two. An unknown constant angle offset leaves all future current measurements unchanged. Adding an angle measurement changes observability, not the open-loop pair (A, B).

Stabilizability allows uncontrollable modes if they are already strictly stable; detectability allows unobservable modes if they are strictly stable. Under those conditions a continuous LTI observer-based stabilizing controller exists without constraints. An unobservable angle integrator is not strictly stable, so current-only sensing does not provide asymptotic regulation of an unknown absolute angle. This distinction matters when a swing outcome depends on a face-angle component that the measurement setup cannot resolve.

2.8 Transfer Functions and Realization

For zero initial conditions, Laplace transformation gives

$$G(s) = C(sI - A)^{-1}B + D, \qquad Y(s) = G(s)U(s). \tag{2.25}$$

With a nonzero initial state, add $C(sI - A)^{-1}x_0$ to the output. For multiple inputs and outputs, G is a transfer matrix; frequency-domain methods do not fail merely because a system is MIMO. State-space and transfer descriptions answer complementary questions. A realization may use physical coordinates or abstract internal coordinates; exposing state variables does not automatically expose an energy balance.

A finite-dimensional proper rational transfer matrix has state-space realizations. A minimal one is reachable and observable, and minimal realizations of the same map are related by similarity. Larger realizations can contain hidden modes. For $A = \mathrm{diag}(-1, 1)$, $B = (1, 0)^T$ and $C = (1, 0)$, the transfer function is $1/(s + 1)$, although a nonzero second state grows as e^t. Input–output stability does not establish internal stability of a nonminimal realization. The realization discussion in *Feedback Systems* explains this distinction [2].

Kalman's 1960 paper cited here is specifically *A New Approach to Linear Filtering and Prediction Problems* [20]. It develops estimation from a state viewpoint and connects filtering with regulation. It should not be described as a paper that invented every state-space concept and only later led to a filter. This chapter uses the established framework without attributing an entire history to that one publication. Nor does it claim that transfer functions were replaced or that all optimal control problems have analytic solutions.

2.9 Sampling a Plant and Implementing Feedback

For a fixed sample period $T_s > 0$ and an input held constant between samples, exact LTI sampling gives

$$x_{k+1} = A_d x_k + B_d u_k, \qquad A_d = e^{AT_s}, \qquad B_d = \int_0^{T_s} e^{A\tau} B \, d\tau. \tag{2.26}$$

If A is invertible, $B_d = A^{-1}(A_d - I)B$; a block exponential works without that restriction:

$$\exp\left(T_s \begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix}\right) = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix}. \tag{2.27}$$

The zero-order-hold assumption concerns the input waveform, not every possible digital controller [33]. Real controllers can have different holds, multiple rates, delays, jitter and internal states. The discrete plant model predicts sampled states; it need not be an equation literally executed inside the controller.

Discrete autonomous stability requires eigenvalues strictly inside the unit circle for asymptotic stability. Boundary eigenvalues additionally need semisimple Jordan structure for Lyapunov stability. Since $\lambda(A_d) = e^{\lambda(A)T_s}$, exact sampling of a Hurwitz autonomous plant remains stable for every positive sample period. A slow sample rate does not move those exact open-loop poles outside the unit circle. An unstable Euler approximation is a numerical issue, not a new physical instability.

Held feedback is different. With $u_k = -Kx_k$, its exact sampled matrix is $A_d - B_dK$, generally different from $e^{(A-BK)T_s}$. For $\dot{x} = -x + u$ and $K = 2$,

$$x_{k+1} = (3e^{-T_s} - 2)x_k. \tag{2.28}$$

Continuous feedback is stable with pole -3, but held feedback is asymptotically stable only for $0 < T_s < \log 3$ in these time units. This is an explicit implementation effect.

Sampling can also lose information or actuation directions. For $A = \left[\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right]$, $B = (0,1)^T$, $C = (1,0)$ and $T_s = \pi$, $A_d = -I$ and $B_d = (2,0)^T$. The continuous system is reachable and observable, while both sampled ranks are one. Samples taken half a period apart miss the velocity component. There is no universal adequate sample rate: the relevant dynamics, hold, sensors, latency, noise and required accuracy must be stated.

2.10 Lyapunov Functions, Domains, and Convergence

Let F be locally Lipschitz on an open neighborhood of an equilibrium at zero, and let V be continuously differentiable, zero at the equilibrium and positive elsewhere in that neighborhood. Then

$$\dot{V} = \nabla V^T F. \tag{2.29}$$

Nonpositive $\dot{V}$ establishes local Lyapunov stability; negative definite $\dot{V}$ establishes local asymptotic stability. Global conclusions require global conditions and boundedness/completeness, for example a proper positive-definite V with negative-definite derivative on all of $\mathbb{R}^n$. A local candidate is not automatically a global certificate. For an LTI Hurwitz matrix, solving $A^TP + PA = -Q$ with $Q > 0$ gives $P > 0$ and $V = x^TPx$, a metric that need not equal the physical inertia.

With a nonstrict derivative, LaSalle's principle uses a compact forward-invariant set Ω: solutions approach the largest invariant subset of $\{x \in \Omega : \dot{V} = 0\}$. They need not approach a single point. Khalil's invariance lecture gives the hypotheses and the pendulum application [21].

For the fixed-pivot damped pendulum,

$$V = \frac{1}{2}m\ell^2v^2 + mg\ell(1 - \cos\theta), \qquad \dot{V} = -bv^2 \leq 0. \tag{2.30}$$

Choose the connected sublevel around the bottom with $V \leq c$ and $0 < c < 2mg\ell$. It stays strictly inside $|\theta| < \pi$, bounds velocity, and is compact and forward invariant. For $b > 0$, remaining in $\dot{V} = 0$ requires $v = 0$ and $\sin\theta = 0$; only the bottom equilibrium lies in this component. LaSalle therefore proves attraction there. The upright equilibrium prevents a global claim to the bottom over the entire cylinder. With $b = 0$, energy is conserved and nearby nonstationary trajectories do not converge.

For a golf swing, attraction to a rest equilibrium is usually not the objective. One may study a moving trajectory, a periodic orbit or a finite-time impact event. Those require tracking, orbital or event-conditioned analysis with specified inputs and uncertainty. A decreasing energy-like function is not a universal explanation of a successful swing.

2.11 A Reproducible Oscillator and Its Interpretation

For $m, k > 0$ and $c \geq 0$, define $\omega_0 = \sqrt{k/m}$ and $\zeta = c/(2\sqrt{km})$. The eigenvalues satisfy $\lambda^2 + (c/m)\lambda + k/m = 0$. In the unforced underdamped case $0 < \zeta < 1$,

$$q(t) = e^{-\zeta\omega_0 t}\left[q_0 \cos(\omega_d t) + \frac{v_0 + \zeta\omega_0 q_0}{\omega_d}\sin(\omega_d t)\right], \qquad \omega_d = \omega_0\sqrt{1-\zeta^2}. \tag{2.31}$$

At critical damping, $q(t) = [q_0 + (v_0 + \omega_0 q_0)t]e^{-\omega_0 t}$. For overdamping the solution is a combination of two decaying real exponentials. Nonoscillatory modal structure does not mean position always approaches zero monotonically: at critical damping with $\omega_0 = 1$, $q_0 = 1$ and $v_0 = -3$, $q = (1-2t)e^{-t}$ crosses zero and later turns back. The usual fastest-nonoscillatory statement concerns a specified response comparison, not every initial condition, norm or settling threshold.

The illustrative model $m = 1$ kg, $c = .5$ N s/m and $k = 2$ N/m has $\zeta \approx .176777$ and poles $-.25 \pm 1.391941i$ s^{-1}. Compare numerical integration against the closed form, check $E = q^2 + v^2/2$ decreases for zero input, and label position and velocity with their different units.

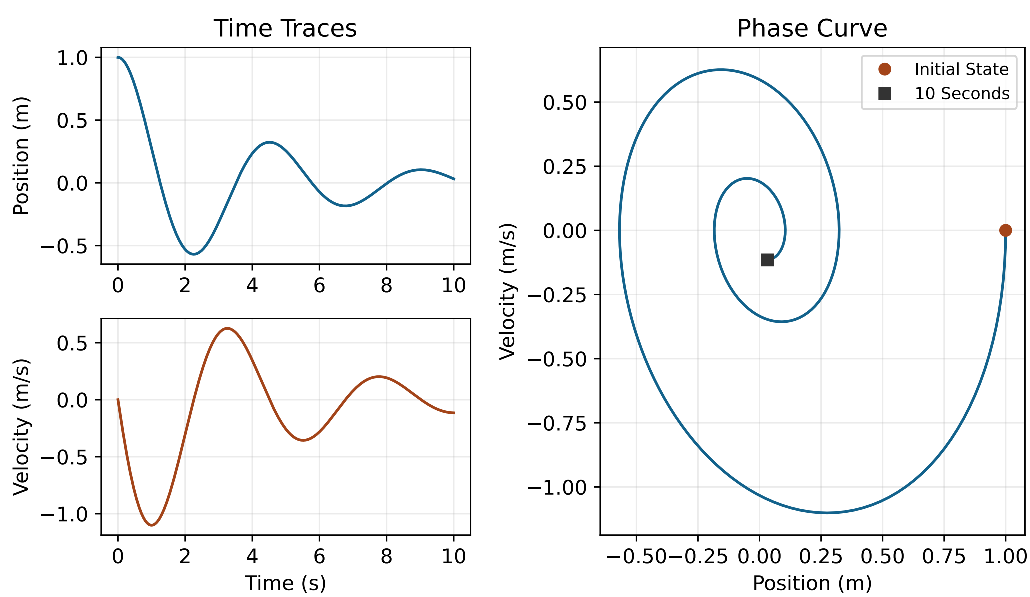


Figure 2.2: An Unforced Damped Oscillator From the Analytic Solution. Time Traces Use Separate Units; the Phase Curve Connects Position and Velocity Without Treating Their Norm as Energy.

2.11.1 Running the Checks

Run this example from the repository root with Python 3.12 and the project dependencies. The small numerical module implements the declared motor, signed tank balance and singular-safe hold construction; it is not a swing simulator. The independent regression tests compare these results with balance identities, analytic solutions and rank counterexamples.

```python
import numpy as np
from scipy.integrate import solve_ivp
from scipy.linalg import expm
from src.tools.state_space_examples import (
    TankParameters, motor_matrices, zoh_matrices,
)

motor_a, motor_b = motor_matrices()
print("Motor poles:", np.linalg.eigvals(motor_a))
tank = TankParameters()
times = np.array([0., 10., 25., 50., 100.])
for tolerance in (1e-8, 1e-10):
    solution = solve_ivp(
        lambda t, h: tank.rate(h, .01), (0, 100), [.5, .2],
        method="DOP853", t_eval=times,
        rtol=tolerance, atol=tolerance / 100,
    )
    if not solution.success:
        raise RuntimeError(solution.message)
    print(tolerance, solution.y.T)

oscillator = np.array([[0., 1.], [-2., -.5]])
frequency = np.sqrt(2 - .25**2)
time = 10.
closed_q = np.exp(-.25*time) * (
    np.cos(frequency*time) + .25/frequency*np.sin(frequency*time)
)
print("Position residual:", (expm(time*oscillator) @ [1, 0])[0] -
    closed_q)
ad, bd = zoh_matrices(np.array([[-1.]]), np.array([[1.]]), 2.)
print("Held feedback pole:", (ad - 2*bd).item())
```

Convergence with tighter numerical tolerances verifies the computation of this model. It does not validate its parameters against measurements. A simulator and an estimator can agree because they share the same wrong assumptions; independent physical checks remain necessary.

2.12 Connecting the State to a Golf Outcome

Start with the prediction: clubhead pose and velocity at ball contact, a shaft strain history, or a joint load. Choose a state that carries the information relevant to that quantity, and specify supports and contact modes. Recover the observable parts from measurements, attach uncertainty to the estimate, and identify the input channels and their limits. The history before a torque-removal branch is retained in its initial state; removing a command does not erase stored motion or activation.

Then propagate through the remaining dynamics. With a smooth nominal motion, a perturbation obeys $\delta\dot{x} = A(t)\delta x + B(t)\delta u$. Its terminal effect depends on the propagated input directions, not just the size of B or the instantaneous passive acceleration. For an impact guard $\psi(x,t) = 0$, changes in state also change event time. Comparing two states at one clock time is different from comparing their respective ball encounters. A contact-mode change can invalidate a smooth local approximation.

Preparation, sensing and late correction are therefore intertwined. Preparation changes the state from which the rest of the swing unfolds; sensing constrains which errors can be recognized; actuator dynamics and the remaining horizon constrain which corrections are feasible. A state-space model helps make these statements testable. It does not establish that a particular muscle should relax, that active torque is negligible, or that an observed proximal-to-distal sequence measures energy transfer.

2.13 Chapter Synthesis and Further Reading

A state is sufficient information for a declared prediction model under specified future inputs. A coordinate chart, a measurement and a minimal realization are different constructions. Mechanical balances determine the state equations; geometry determines how their components transform; reachability and observability determine structural possibilities; constraints, sampling, conditioning and uncertainty determine practical limits. Stability results must retain their domains and hypotheses. The Lagrangian chapter explains where the mechanical equations come from, and the counterfactual chapter explains how to compare precisely defined interventions along a swing.

2.14 Exercises

1. **Motor State and Sensing.** Derive the three-state numerical motor matrices and their eigenvalues. Verify its controllability rank and its current-only observability rank. Explain why an angle offset is both a state and a hidden measurement mode. Add an angle sensor and recompute observability. Check the electrical–mechanical power identity.

2. **Two-Tank Balance.** Integrate the stated tank example for 100 s. Repeat from $(.2, .5)$ m to exercise reversed flow. Verify total volume balance and nonnegative heights, derive the equilibrium, and compare two solver tolerances. Explain the square-root regularity issue and the monotonicity uniqueness argument.

3. **Damped Motion.** For $m = 2$ kg, $c = 4$ N s/m and $k = 3$ N/m, compute ζ and the poles, and compare phase curves from $(1, 0)$ and $(.5, 1)$. State axis units. Contrast modal oscillation, position monotonicity and energy decay; test the chapter's critical-damping counterexample.

4. **Pendulum Equilibria.** With $\ell = 1$ m and $g = 9.81$ m/s^2, linearize the zero-torque undamped model at the downward angle zero and upward angle π. Use a local displacement at each. Distinguish what the eigenvalues prove from what the energy argument proves.

5. **State Completeness.** Give two motions with the same position and different futures under the same force law. Then construct an actuator example with the same (q, v) but different internal state and different acceleration. Contrast it with a jerk-controlled model and with an accelerometer output.

6. **Representation and History.** Explain the complementary roles of state-space and MIMO transfer descriptions. For the hidden unstable-mode example, derive $G(s)$ and the nonzero-initial-state response. State the specific subject of the cited Kalman 1960 paper.

7. **A Saddle With Rotation.** For a real three-state LTI matrix with eigenvalues $-2, 1+2i, 1-2i$, describe the stable subspace and unstable rotating plane. Distinguish initial conditions on the stable subspace from generic nearby states. Does the eigenvalue list specify Euclidean transient magnitudes?

8. **Feedback With Incomplete Observation.** Explain controllability without observability. Give one unobservable stable mode that is compatible with detectability and one nondecaying hidden mode that prevents asymptotic estimation. State the information and assumptions required for a stabilizing observer-based LTI controller.

9. **The Rank Test and Its Limits.** Use Cayley–Hamilton and the Gramian identity to prove the controllability rank condition. Derive the double-integrator bounded-input inequalities. Explain why full rank does not guarantee a chosen impact target with limited torque and time.

10. **Exponential Propagation.** Prove the diagonalization identity where it applies, then calculate the exponential of the defective zero-eigenvalue matrix without diagonalization. Explain why both are covered by the defining series and why numerical eigendecomposition can be ill-conditioned.

11. **Linearization and Geometry.** Derive the forced pendulum Jacobians with physical torque input. Derive the Hessian term under $z = \phi(q)$ and verify $q = 1 + 2t$, $z = q^2$. Explain when a same-state difference in acceleration transforms as a vector.

12. **Two-Dimensional Stability.** For $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$, derive $\lambda = ((a+d) \pm \sqrt{(a-d)^2 + 4bc})/2$. Include repeated and zero roots in the classification. Construct nonlinear systems with the same zero Jacobian but different stability.

13. **The PBH Criterion.** Prove the left-eigenvector condition from the rank of $[\lambda I - A \quad B]$, and connect it to the invariant orthogonal complement of the reachable subspace. Use the double integrator to disprove the right-eigenvector version.

14. **Variation of Constants and Sampling.** Derive the forced solution and the block-exponential hold formula. Verify the scalar held-feedback threshold $T_s < \log 3$ and the oscillator's sampled rank loss at $T_s = \pi$. Distinguish numerical discretization error from actual sample-and-hold dynamics.

15. **Observability and Invariance.** Prove the observability rank test with its Gramian, including subtraction of known input terms. For the damped pendulum, construct a compact invariant sublevel below the upright energy and identify the largest invariant subset of $\dot{V} = 0$. Explain why nonincreasing energy alone does not prove convergence or global attraction.

Chapter 3

Configuration Space and Degrees of Freedom

A golfer can place the clubhead at the same point using different elbow, shoulder and torso configurations. Those postures need not permit the same next motion, require the same joint torques, or produce the same impact. Configuration space provides the language for distinguishing these questions. It describes the modeled arrangement of the bodies; kinematics maps that arrangement to a task; dynamics determines how forces change the motion.

This chapter develops local coordinates, constraint counts, tangent vectors, forward and inverse kinematics, workspace and non-holonomic motion. The examples are specified mechanisms, not measurements of an anatomical golfer. Keeping that boundary explicit lets us transfer the mathematics without transferring unjustified performance claims.

3.1 Historical Context: From Lagrange to Riemann

Generalized coordinates let mechanics describe a constrained arrangement without carrying every particle coordinate independently. Manifold theory explains why a useful local coordinate description need not extend over the entire configuration space. Modern robotics combines these ideas with maps between joint arrangements and end-effector tasks [35, 30]. A coordinate system is a representation; it does not determine the underlying physical freedom.

Nor does curvature define a manifold: Euclidean space is a manifold, and a torus can carry an intrinsically flat metric despite its familiar curved embedding in three dimensions. Topology, smooth structure and metric answer different questions. A model's mass matrix later supplies one physically meaningful metric; the configuration manifold itself does not select it.

3.2 Degrees of Freedom (DOF) and Grübler's Formula

3.2.1 Local Freedom and Independent Constraints

At a regular configuration, the number of degrees of freedom is the local dimension of the configuration space. A free rigid body has three translational and three rotational freedoms. Six local coordinates suffice near a regular chart point, but there is no globally nonsingular, unique three-angle description of all orientations. A rotation matrix or unit quaternion stores more scalar entries with constraints; that storage does not add physical freedoms.

For an ambient configuration manifold of dimension D, suppose smooth equations $\phi(q) = 0$ have locally constant derivative rank r and define a regular level set. Then

$$\dim \mathcal{Q} = D - r, \qquad T_q\mathcal{Q} = \ker D\phi(q). \tag{3.1}$$

This follows from the constant-rank theorem: locally, r independent coordinates are fixed by the equations and $D-r$ remain free. Counting written equations instead of their independent rank can overcount constraints.

At a singular point the linearized kernel can be misleading. The equation $x^2 + y^2 = 0$ has the single real solution $(0, 0)$, while its derivative there is zero and its kernel is all of $\mathbb{R}^2$. A two-dimensional linearized kernel does not make the solution set locally two-dimensional. Closed linkages may similarly have singular branches or instantaneous motions that do not extend to finite motion.

Two welded floating bodies form one rigid assembly with six freedoms. A hinge between two otherwise free spatial bodies leaves six assembly freedoms plus one relative rotation. Fixing one body to ground removes the six assembly freedoms. These counts concern the chosen mechanical model, not the number of motors or available control inputs.

3.2.2 Grübler's Formula

Let n count moving rigid links, excluding a fixed ground link. Let joint i permit d_i relative freedoms within a planar or spatial model. Under independent regular joint constraints, a planar count is

$$\mathrm{DOF} = 3n - \sum_i c_i, \qquad c_i = 3 - d_i, \tag{3.2}$$

or, for j joints,

$$\mathrm{DOF} = 3n - 3j + \sum_{i=1}^{j} d_i. \tag{3.3}$$

Each moving planar body begins with two translations and one rotation. A revolute or prismatic joint removes two independent relative freedoms. Subtracting their independent constraints gives the formula. For spatial bodies, replace each 3 by 6:

$$\mathrm{DOF} = 6n - \sum_{i=1}^{j} (6 - d_i). \tag{3.4}$$

With $N = n + 1$ total links including ground, both forms are $\lambda(N - 1 - j) + \sum_i d_i$, where $\lambda = 3$ or 6. This is a useful count, not a proof that a particular linkage has independent constraints. Redundant constraints, special geometry and singular configurations require a rank or geometric analysis [30].

3.2.3 Worked Example 1: Planar Four-Bar Linkage

Three moving links and four revolute joints give $3(3) - 4(2) = 1$ regular freedom. Write a closure equation using absolute link angles and a fixed ground displacement de_x:

$$ae(\theta_1) + be(\theta_2) - ce(\theta_3) - de_x = 0, \qquad e(\theta) = (\cos\theta, \sin\theta)^T. \tag{3.5}$$

These are two scalar equations in three angles. Their derivative normally has rank two. The input θ_1 determines the other angles locally only when the two-column derivative with respect to (θ_2, θ_3) is invertible. Its determinant is $-bc\sin(\theta_3 - \theta_2)$, so that choice of input coordinate fails when the coupler and output link are parallel.

Different assembly branches can share an input angle. The branch must be specified and followed continuously; one motor does not make the global closure solution unique. At some singular postures the full closure derivative also loses rank, and finite mobility needs more than a first-order count.

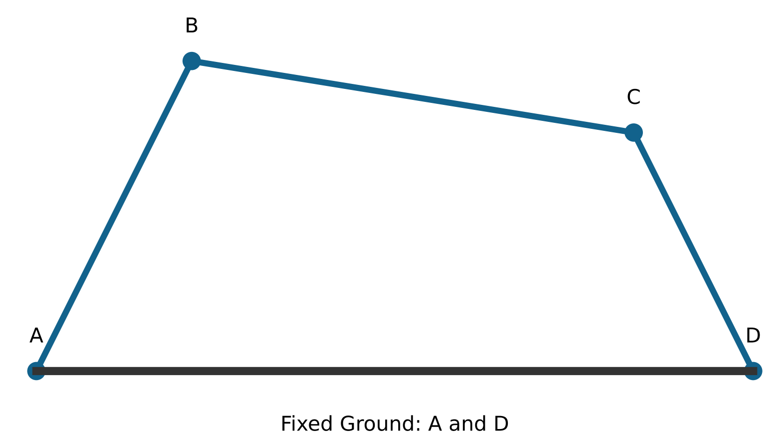


Figure 3.1: Four-Bar Closure on One Regular Assembly Branch. Ground Contains Two Fixed Pivots; the Input Angle Is Only a Local Coordinate Where the Selected Closure Derivative Is Nonsingular.

3.2.4 Worked Example 2: Spatial Stewart Platform

For an ideal six-leg UPS platform, each leg has a universal joint, a prismatic joint and a spherical joint, with $2+1+3$ freedoms. There are $N = 14$ bodies including ground, $n = 13$ moving bodies and $j = 18$ joints. Thus the regular independent-constraint count is

$$6(14 - 1 - 18) + 6(2 + 1 + 3) = 6. \tag{3.6}$$

This is a mobility count for the mechanism with variable leg lengths. Local control of all platform velocities also requires the actuator-length derivative to have rank six.

If base anchors are a_i, platform-frame anchors are b_i, and the platform pose is (p, R), the required lengths are directly

$$\ell_i = \|p + Rb_i - a_i\|. \tag{3.7}$$

No coupled root-finding is needed to obtain lengths from a specified pose. The reverse problem, pose from lengths, may have multiple solutions. Joint-angle limits, leg interference and stroke limits must still be checked. A singular length-to-pose relation is not diagnosed by merely recounting bodies.

3.2.5 Worked Example 3: Human Arm Kinematics

A common reduced arm model assigns three shoulder rotations, one elbow flexion angle, one forearm pronation/supination angle and two wrist angles. That gives seven coordinates relative to a fixed shoulder base. Forearm rotation must not be relabeled as a third rotation of a ball-and-socket wrist. Scapular motion, shoulder translation, compliant tissues and hand joints change a more detailed model's configuration.

For a six-dimensional hand-pose task, a seven-coordinate model has a one-dimensional local inverse-kinematics family near a reachable regular point where the task derivative has rank six. This conclusion follows locally from the regular-level-set theorem. It does not say that every target is reachable or that every posture admits the same redundancy. Joint limits, singularities and grasp constraints can change the admissible set.

With both hands holding a rigid club, the arms and torso form a closed kinematic loop. Counting two unconstrained seven-coordinate arms and adding a club would ignore the grip constraints. Compliant grips require a different model. In either case, configuration redundancy is distinct from muscle-force redundancy and from the ability to alter a selected impact outcome.

3.3 The Configuration Space and Topological Manifolds

The configuration space $\mathcal{Q}$ is the set of modeled arrangements satisfying the selected geometric conditions. Its regular unconstrained regions are often smooth manifolds. Joint limits can introduce boundaries and corners; collision exclusions remove configurations; singular loop closures can create sets that are not manifolds. A contact-mode change can alter which equations apply. Therefore "every physical configuration space is a smooth manifold" is too strong.

A topological n-manifold without boundary is a Hausdorff, second-countable space in which every point has a neighborhood homeomorphic to an open subset of $\mathbb{R}^n$. Hausdorff means distinct points have disjoint neighborhoods; second-countable means a countable basis suffices. A manifold with boundary uses a local half-space at its boundary instead.

The circle $S^1 = \{(x, y) : x^2 + y^2 = 1\}$ has dimension one. The sphere S^2 has dimension two, including at its poles; longitude fails at the poles, not the manifold. Two independent unrestricted revolute angles give $S^1 \times S^1$, a torus. With restricted joint ranges, the admissible set can instead be a rectangle or another subset of that torus. Tracking cable wind-up or accumulated revolutions introduces additional model information; an unwrapped angle is not automatically the physical orientation itself.

For a free spatial rigid body, position and orientation give $\mathbb{R}^3 \times SO(3)$ as a manifold. Its group multiplication is the semidirect product $SE(3)$, because rotating one frame also changes how translations compose. The orientation space $SO(3)$ is three-dimensional and compact. Unit quaternions give a two-to-one representation: opposite quaternion points represent the same orientation.

3.4 Charts, Atlases, and Coordinate Patches

A chart is a homeomorphism $\varphi : U \to \varphi(U) \subset \mathbb{R}^n$, where both U and its image are open in their respective spaces. An atlas is a collection of charts covering the manifold. A smooth atlas has smooth transition maps on every overlap; the associated maximal compatible atlas defines the smooth structure. Here "smooth" means C^∞, although other differentiability classes can be specified.

3.4.1 Worked Example: Two Valid Charts on the Circle

No single chart covers all of S^1. A hypothetical homeomorphism would map the compact circle onto a nonempty compact open subset of $\mathbb{R}$, which does not exist. The half-open interval representation $[0, 2\pi)$ is a bijective labeling of circle points, but its inverse-angle assignment jumps across the cut and its image is not open. Boundedness alone is not the problem: a bounded open interval can be a valid chart image.

Let $U_N = S^1 \setminus \{(0,1)\}$ and $U_S = S^1 \setminus \{(0,-1)\}$. Stereographic coordinates are

$$u = \varphi_N(x, y) = \frac{x}{1-y}, \qquad v = \varphi_S(x, y) = \frac{x}{1+y}. \tag{3.8}$$

Both images are all of $\mathbb{R}$. Their explicit inverses are

$$\varphi_N^{-1}(u) = \frac{(2u, u^2-1)}{1+u^2}, \qquad \varphi_S^{-1}(v) = \frac{(2v, 1-v^2)}{1+v^2}. \tag{3.9}$$

Substitution verifies unit norm and recovers the original coordinate. Each point other than its chart's omitted pole has one coordinate. The two chart domains cover the circle and overlap away from both poles.

In contrast, projecting to x on a region $y > -\epsilon$ is generally not injective: for a small positive $\eta < \epsilon$, both $(\sqrt{1-\eta^2}, \eta)$ and $(\sqrt{1-\eta^2}, -\eta)$ belong to the region and share x. A smooth formula is not a valid chart unless it is also a homeomorphism onto an open image.

3.5 Transition Maps and Smooth Compatibility

For overlapping charts, the transition map is $\varphi_j \circ \varphi_i^{-1}$ on $\varphi_i(U_i \cap U_j)$. It changes coordinates of the same point. It need not preserve their numerical values, Euclidean lengths or component velocities.

3.5.1 Worked Example: Circle Transition and Velocity

The circle inverses give, on $u \neq 0$,

$$v = \frac{1}{u}, \qquad u = \frac{1}{v}, \qquad \dot{v} = -\frac{\dot{u}}{u^2}. \tag{3.10}$$

These transition maps are smooth on their open, disconnected overlap images $\mathbb{R} \setminus \{0\}$. Their singularity at zero lies outside the overlap: zero is the opposite chart's omitted pole.

For example, $u = 1$ corresponds to $(1, 0)$ and $v = 1$. If $\dot{u} = 2$, then $\dot{v} = -2$. These represent the same physical tangent vector. Differentiating the inverse map also gives the circle's induced line element

$$ds^2 = \frac{4}{(1+u^2)^2}\, du^2. \tag{3.11}$$

The same speed is obtained in the v chart after transforming both the metric and the velocity. Comparing raw coordinate rates without this transformation can create an apparent difference that is purely representational.

3.6 Tangent Vectors, Curves, and Derivations

The tangent space $T_q\mathcal{Q}$ is an n-dimensional vector space attached to a particular configuration. It describes unconstrained instantaneous directions on the smooth configuration manifold. Additional non-holonomic restrictions may allow only a subspace of these directions; an active unilateral boundary may instead admit a one-sided cone.

Two smooth curves through q represent the same tangent vector when their coordinate derivatives agree at zero in one chart. The chain rule for a transition map then makes them agree in every chart. The tangent vector is the equivalence class, while $\dot{q}$ is its coordinate representation.

Alternatively, a tangent vector is an $\mathbb{R}$-linear derivation v on smooth real functions satisfying

$$v(fg) = v(f)g(q) + f(q)v(g). \tag{3.12}$$

A curve γ supplies $v(f) = \frac{d}{dt}|_0 f(\gamma(t))$. Conversely, in coordinates z centered at q, a smooth function locally has the form

$$f(z) = f(0) + \sum_i z_i h_i(z), \qquad h_i(0) = \partial_i f(0). \tag{3.13}$$

The product rule gives $v(f) = \sum_i v(z_i)\partial_i f(0)$. Constants have zero derivative, and a derivation depends only on the function near q; smooth cutoff functions justify using local coordinate functions. The coordinate curve $z(t) = t(v(z_1), \dots, v(z_n))$ realizes this derivation for sufficiently small t. Thus curve classes and derivations are isomorphic, with n independent components.

The tangent bundle is the disjoint union

$$T\mathcal{Q} = \bigsqcup_{q\in\mathcal{Q}} T_q\mathcal{Q}. \tag{3.14}$$

Its elements are pairs (q, v_q) and it has dimension $2n$. Locally its coordinates transform as $(z, \dot{z}) \mapsto (\tau(z), D\tau(z)\dot{z})$. In general it is not a global product $\mathcal{Q} \times \mathbb{R}^n$.

For an unconstrained second-order mechanical model, configuration and velocity can form the state in $T\mathcal{Q}$. With a rank-k homogeneous allowed-velocity distribution, the constrained mechanical state lies in that distribution and has dimension $n + k$, under the regularity assumptions. Muscle activation, tendon deformation, flexible-shaft modes, controller memory or contact-mode variables add states. Consequently "state always has twice the configuration dimension" is not a general modeling rule.

3.7 Generalized Coordinates and Forward Kinematics

Generalized coordinates parameterize configurations locally; they need not all be motor angles or all be independently actuated. Prismatic displacement, a passive hinge angle and a floating-base coordinate are examples. A redundant constrained representation can also be computationally useful, even though it is not a minimal chart.

For a fixed-length arm model, joint coordinates and a base pose determine Cartesian segment positions. A shoulder angle alone is not enough for a general spatial shoulder: its orientation requires the chosen multi-coordinate representation. The anatomical elbow is approximated as a hinge only within a declared model.

Forward kinematics is a map $f : \mathcal{Q} \to \mathcal{Y}$ to a selected task space. Position may lie in $\mathbb{R}^3$, planar pose in $SE(2)$, and spatial pose in $SE(3)$. These targets are different maps, with potentially different derivative ranks.

For planar links of positive lengths L_1, L_2 and relative joint angles,

$$\begin{aligned} x &= L_1 \cos q_1 + L_2 \cos(q_1 + q_2), \\ y &= L_1 \sin q_1 + L_2 \sin(q_1 + q_2). \end{aligned} \tag{3.15}$$

The final-link orientation is $q_1 + q_2$ modulo 2π. The position equations by themselves do not include it as a controlled task coordinate.

3.8 The Jacobian Matrix and the Kinematic Derivative

The intrinsic derivative is $Df_q : T_q\mathcal{Q} \to T_{f(q)}\mathcal{Y}$. In local coordinates its matrix has entries

$$J_{ij}(q) = \frac{\partial f_i}{\partial q_j}, \qquad \dot{y} = J(q)\dot{q}. \tag{3.16}$$

In particular, the bottom-left entry is $\partial f_m/\partial q_1$, not $\partial f_m/\partial q_n$. Changing nonsingular coordinate charts changes the matrix but preserves its rank. Singular coordinate representations can introduce an apparent singularity that is not a loss of physical task freedom.

For a spatial pose, Euler-angle derivatives are not generally the angular-velocity vector. A geometric Jacobian maps the declared generalized velocities to a declared twist, with its frame, reference point and angular/linear ordering specified. A six-coordinate pose derivative and a six-component twist Jacobian are related through the relevant chart map, not interchangeable by notation alone.

3.8.1 Worked Example: Two-Link Position Jacobian

Differentiating the two-link equations gives

$$J_p = \begin{bmatrix} -L_1 \sin q_1 - L_2 \sin(q_1 + q_2) & -L_2 \sin(q_1 + q_2) \\ L_1 \cos q_1 + L_2 \cos(q_1 + q_2) & L_2 \cos(q_1 + q_2) \end{bmatrix}, \tag{3.17}$$

with $\det J_p = L_1 L_2 \sin q_2$. For $L_1 = 2, L_2 = 1, q = (0, 0)$,

$$J_p = \begin{bmatrix} 0 & 0 \\ 3 & 1 \end{bmatrix}. \tag{3.18}$$

Only vertical endpoint velocity is available to first order. The arm still has two joint configuration freedoms. Motion in its position-Jacobian null space can alter joint posture, and second-order endpoint changes can appear even when the first-order velocity is zero.

3.8.2 Worked Example: Three-Link Planar Pose

Let $\alpha_i = \sum_{j=1}^{i} q_j$. A three-link planar pose is

$$(x, y) = \sum_{i=1}^{3} L_i(\cos \alpha_i, \sin \alpha_i), \qquad \theta = \alpha_3 \pmod{2\pi}. \tag{3.19}$$

On a local angle chart, a compact expression for every Jacobian column is

$$J_{1j} = -\sum_{i=j}^{3} L_i \sin\alpha_i, \qquad J_{2j} = \sum_{i=j}^{3} L_i \cos\alpha_i, \qquad J_{3j} = 1. \tag{3.20}$$

Subtracting consecutive columns shows $\det J = L_1 L_2 \sin q_2$. Full rank gives a locally invertible planar pose map. It does not establish global uniqueness or a six-dimensional spatial pose capability.

3.9 Inverse Kinematics

Inverse kinematics asks for configurations satisfying $f(q) = y_d$ within the admissible set. Solutions may be absent, isolated, multiple or continuous. A mathematical solution may violate a joint limit or collision condition. Even an admissible configuration does not supply a dynamically feasible path to it.

3.9.1 Geometric Two-Link Inverse Kinematics

For a target (x_d, y_d) with $r^2 = x_d^2 + y_d^2$, the forward equations imply

$$r^2 = L_1^2 + L_2^2 + 2L_1L_2\cos q_2. \tag{3.21}$$

Therefore, defining $c_2 = (r^2 - L_1^2 - L_2^2)/(2L_1L_2)$, a reachable target requires $|c_2| \le 1$. The two generic branches are

$$\begin{aligned} s_2 &= \pm\sqrt{1-c_2^2}, \qquad q_2 = \operatorname{atan2}(s_2, c_2), \\ q_1 &= \operatorname{atan2}(y_d, x_d) - \operatorname{atan2}(L_2 s_2, L_1 + L_2 c_2). \end{aligned} \tag{3.22}$$

Angles are understood modulo 2π before applying physical joint ranges. The sign of the cosine numerator follows directly from the squared vector sum; using the opposite sign produces the wrong elbow angle.

For $L_1 = L_2 = 1$ and target $(1, 1)$, the branches are $(0, \pi/2)$ and $(\pi/2, -\pi/2)$. Substitution recovers the same position but different final-link orientations. At ordinary inner or outer boundaries, the two branches coalesce modulo angle periodicity. At the equal-link origin, $q_2 = \pi$ and any q_1 works, so the solution is a continuous family and $\operatorname{atan2}(0, 0)$ must not be treated as a unique shoulder solution.

3.9.2 Numerical Inverse Kinematics and Convergence

For local Euclidean task coordinates, set $e(q) = y_d - f(q)$. Linearizing gives $J(q)\delta q \approx e(q)$. A common iteration is

$$q_{k+1} = q_k + \alpha_k J(q_k)^{\dagger} e(q_k). \tag{3.23}$$

For an invertible square Jacobian, solve the linear system rather than explicitly forming an inverse. Classical Newton iteration has local quadratic convergence near a nonsingular root under suitable smoothness and sufficiently close initialization, with full steps eventually accepted. That statement is not a universal guarantee for singular, rectangular, damped or constrained iterations.

The pseudoinverse minimizes the linearized residual, and among tied solutions chooses the smallest Euclidean coordinate step. Both choices depend on scaling and metrics. A regularized alternative solves

$$\min_{\delta q} \|W^{1/2}(J\delta q - e)\|^2 + \lambda^2 \|R^{1/2}\delta q\|^2, \tag{3.24}$$

with declared positive-definite task and joint weights. It trades residual against step size; it does not make an unreachable target reachable. Use meaningful stopping tolerances, step limits and admissibility checks. On an orientation manifold, use a consistent local error or retraction rather than subtracting arbitrary wrapped angles across a cut.

3.10 Reachable and Dexterous Workspace

Let $\mathcal{Q}_{\text{adm}}$ be the admissible configuration set and p the position map. The reachable position workspace is $p(\mathcal{Q}_{\text{adm}})$. For a required orientation set $\mathcal{R}_{\text{req}}$, define the dexterous position workspace by

$$\begin{aligned} p_d \in \mathcal{W}_{\text{dex}} \quad \Longleftrightarrow \quad & \text{for every } R_d \in \mathcal{R}_{\text{req}}, \\ & \text{there exists } q \in \mathcal{Q}_{\text{adm}} \text{ such that} \\ & (p(q), R(q)) = (p_d, R_d). \end{aligned} \tag{3.25}$$

The configuration may differ for each orientation. Existence of one full-rank Jacobian at a position is a local differential property, not this global orientation quantifier. A determinant is also undefined for a rectangular Jacobian.

For two unrestricted positive-length planar links, vector geometry gives

$$|L_1 - L_2| \le \|p\| \le L_1 + L_2. \tag{3.26}$$

Every intermediate radius occurs as q_2 varies, and q_1 rotates the resultant through all directions. Unequal lengths give a closed annulus; equal lengths give a closed disk. With $L_1 = 2, L_2 = 1$, its area is $\pi(3^2 - 1^2) = 8\pi$ in squared length units. This analytic argument establishes coverage; a finite sample cloud only illustrates it.

For an equal-link planar 2R arm, the folded origin is reachable with every final-link orientation by varying q_1 while keeping $q_2 = \pi$. Yet its position Jacobian has rank one. At a generic nonzero point, the two inverse branches give only two final-link orientations, even when the position Jacobian is full rank. Thus, for the all-planar-orientations requirement and unrestricted equal links, the dexterous workspace is just the origin. The example directly separates dexterity in this defined sense from local position manipulability.

The magnitude $|\det J_p| = L_1 L_2 |\sin q_2|$ measures local area scaling for the selected coordinate metrics. It is not a geometric distance to singularity or a condition number. Singular values describe directional gains; their ratio describes conditioning after choosing units and metrics. Two matrices with equal determinant can have very different conditioning, such as I_2 and $\operatorname{diag}(100, 0.01)$. A colored determinant plot must say what it measures.

3.11 The Frobenius Theorem and Integrability

A smooth constant-rank distribution Δ assigns a smoothly varying k-dimensional subspace $\Delta(q) \subset T_q\mathcal{Q}$. It is integrable when it is tangent to a foliation by k-dimensional leaves. The regular Frobenius theorem states that this occurs exactly when the distribution is involutive:

$$X, Y \text{ smooth sections of } \Delta \quad \Longrightarrow \quad [X, Y] \text{ is a section of } \Delta. \tag{3.27}$$

Use the convention $[X, Y] = DY\,X - DX\,Y$, equivalent to the commutator of derivations. The brackets can leave the original distribution when it is nonintegrable. They need not span the entire ambient tangent space.

For example, on $\mathbb{R}^4$ with coordinates (x, y, z, w), take

$$X_1 = \partial_x, \qquad X_2 = \partial_y + x\partial_z, \qquad [X_1, X_2] = \partial_z. \tag{3.28}$$

The rank-two distribution is nonintegrable, but every generated motion keeps w fixed. Its Lie closure has rank three, not four. Nonintegrability alone cannot imply full configuration reachability.

3.11.1 A Specified Unicycle and Its Lie Bracket

For an ideal unicycle with independently commanded signed forward speed and heading rate, let $q = (x, y, \theta)$ and

$$X_1 = (\cos\theta, \sin\theta, 0)^T, \qquad X_2 = (0, 0, 1)^T. \tag{3.29}$$

The no-lateral-slip restriction is

$$-\sin\theta\,\dot{x} + \cos\theta\,\dot{y} = 0, \tag{3.30}$$

so the allowed-velocity distribution has rank two in a three-dimensional configuration space. Its bracket is

$$[X_1, X_2] = (\sin\theta, -\cos\theta, 0)^T. \tag{3.31}$$

These three vectors span the tangent space everywhere. For the smooth driftless system with reversible controls around zero, the bracket-generating condition yields local controllability; appropriate connected, obstacle-free models admit connections by concatenated admissible motions [35, 6]. This conclusion uses more than failure of Frobenius.

The four-step sequence forward, rotate, backward, reverse-rotate, each for duration ϵ with unit magnitudes, produces

$$\Delta q = \epsilon^2 [X_1, X_2] + O(\epsilon^3). \tag{3.32}$$

Every segment obeys the velocity constraint. The net sideways displacement is not instantaneous sideways slipping. Its order also matters: a small maneuver does not provide a first-order sideways velocity channel. Finite time, speed limits, collision clearance and lack of reverse motion can change the practical or local reachability question.

For two points connected by a positive-length rigid rod, $\|r_1 - r_2\|^2 = L^2$ is a regular position constraint in six-dimensional particle space. Its tangent constraint is integrable and the selected length level set has dimension five. The differentiated velocity equation alone preserves whichever separation was selected initially; an initial position constraint chooses the physical leaf.

3.12 Holonomic and Non-Holonomic Constraints

A holonomic equality constraint can be written $\phi(q,t)=0$. At regular points it defines the corresponding position set or time-dependent family. Differentiation gives $D_q\phi\,\dot q+\partial_t\phi=0$. Time-dependent constraints are generally affine in velocity. Joint-limit and separation inequalities need their own admissibility and active-set treatment; an inequality is not simply another equality that always subtracts a dimension.

A genuinely non-holonomic velocity restriction cannot locally be replaced by a family of position equalities of the same codimension. It restricts paths in the ambient configuration space. Which positions a controlled system can reach still depends on all its vector fields, their Lie closure, control bounds, drift and obstacles.

3.12.1 Upright Rolling Wheel

For an ideal upright wheel of radius $r>0$, use configuration (x,y,θ,φ), with heading θ and signed rolling angle φ. Choose the sign of φ so positive rolling advances along heading. No slip then requires

$$\dot x = r\dot\varphi\cos\theta, \qquad \dot y = r\dot\varphi\sin\theta. \tag{3.33}$$

These are two independent velocity constraints on a four-dimensional configuration. The factors have units of speed. Replacing the signed rolling relation by an unspecified length ratio times speed gives the wrong units and loses the direction of rotation. An actual coin that can lean needs additional configuration and contact dynamics; this is the upright-wheel idealization.

3.12.2 Unicycle Versus Bicycle Steering

The independently driven unicycle has two controls:

$$\dot x = v\cos\theta, \qquad \dot y = v\sin\theta, \qquad \dot\theta=\omega. \tag{3.34}$$

A kinematic bicycle with wheelbase $L>0$ and steering angle δ instead uses $\dot\theta = v\tan\delta/L$. Its admissible $(v,\dot\theta)$ pairs depend on steering bounds, and it cannot turn in place with finite steering angle. If steering rate is the input, δ becomes a state. Neither model is accurately described as a one-input unicycle while simultaneously treating speed and steering as independent inputs.

3.12.3 Ball and Moving-Surface Contact

Let C be the ball center and P its contact point, all expressed in the same inertial frame. No slip equates velocities of the contacting material points:

$$v_C + \omega\times r_{CP} = v_{\mathrm{surface}}(P,t). \tag{3.35}$$

Here r_{CP} is the vector from center to contact, not the contact's absolute position. For a sphere of radius r on a fixed plane with unit normal n toward the center, $r_{CP}=-rn$, giving $v_C = r\omega\times n$. Rotation about n is not excluded by no slip at an ideal point; a no-twist or finite-patch law would add a condition.

Maintained sphere-plane contact leaves two center-position coordinates plus three orientation freedoms, a five-dimensional configuration manifold. Tangential no slip removes two velocity directions, leaving rank three and an eight-dimensional constrained mechanical state under this idealization. A tilted fixed plane changes gravitational dynamics, not the algebra of the contact-velocity equality. A moving plate requires its actual surface velocity and maintained-contact geometry. Friction and normal-force feasibility must be checked separately.

3.12.4 Pfaffian Constraints

A homogeneous Pfaffian constraint is $A(q)\dot{q} = 0$. Its independent rows are covectors that annihilate allowed velocities. If the rank is r on an n-dimensional manifold, $\ker A$ has dimension $n - r$. Calling velocities "orthogonal to the rows" uses the Euclidean metric of a chosen coordinate representation; the intrinsic statement is covector-vector pairing equal to zero.

For the unicycle, $A = (-\sin\theta, \cos\theta, 0)$ has one independent row and a two-dimensional kernel. A holonomic constraint's differentiated equation can also be written in Pfaffian form. Thus the appearance of $A\dot{q} = 0$ alone does not prove non-holonomy; integrability is the additional question.

3.13 Python Example: Kinematics and Workspace

The repository module implements the stated planar chain, two-link inverse branches, circle charts and exact constant-input unicycle step. Its tests compare Jacobians with finite differences, close both inverse branches through the forward map, verify chart transitions and recover the finite-motion bracket sign. No numerical sample replaces the geometric proofs.

```python
import numpy as np
from src.tools.configuration_examples import (
    planar_pose, planar_jacobian, inverse_two_link,
    circle_point, circle_coordinate, unicycle_step,
)

lengths = np.ones(2)
angles = np.array([np.pi / 4, np.pi / 3])
pose = planar_pose(lengths, angles)
print(pose)
print(np.linalg.det(planar_jacobian(lengths, angles)[:2]))
for branch in inverse_two_link(lengths, pose[:2]):
    np.testing.assert_allclose(
        planar_pose(lengths, branch)[:2], pose[:2]
    )
    print(branch)
point = circle_point(0.7, 1)
print(point, circle_coordinate(point, -1))
state = np.zeros(3)
for speed, turn in [(1, 0), (0, 1), (-1, 0), (0, -1)]:
    state = unicycle_step(state, speed, turn, 0.01)
print(state / 0.01**2)
```

For the published unit-link example at $q = (\pi/4, \pi/3)$, the endpoint is approximately $(0.448287736, 1.673032607)$, with final-link orientation $7\pi/12$. The position determinant is

$\sqrt{3}/2$. The two-link inverse reproduces the position on both branches; comparing their orientations illustrates why a position task is not a pose task.

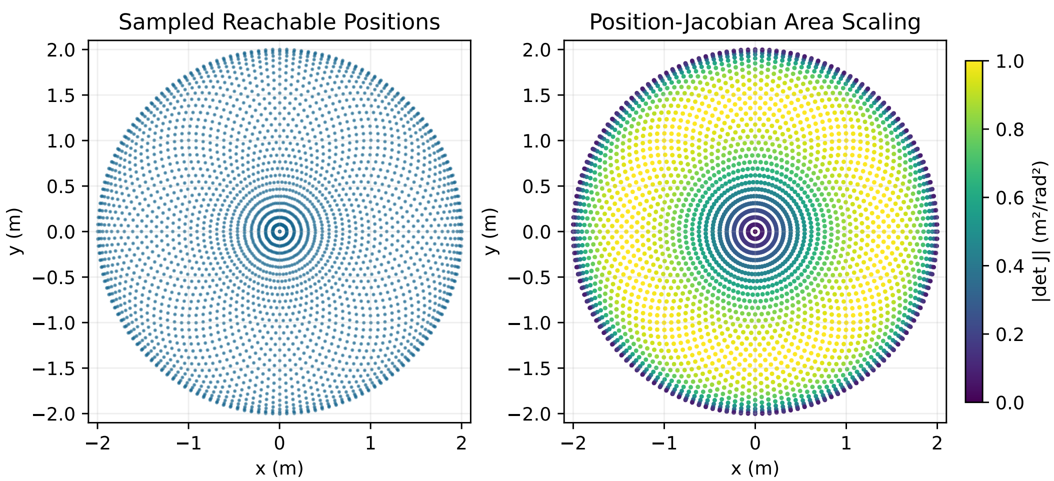


Figure 3.2: A Sampled Unit-Link Position Workspace and Its Local Area Scaling. The Exact Reachable Set Is a Disk; Determinant Magnitude Is Not Dexterous Workspace or Distance to Singularity.

The figure samples an unrestricted unit-link arm and colors configurations by $|\det J_p|$. The analytic reachable set is the disk of radius two. Joint-angle samples are not uniformly distributed in workspace, and overlapping points can hide branch information. This is an area-scaling illustration, not a dexterous-workspace plot, a singularity-distance map or evidence of a human swing strategy.

3.14 From Configuration Geometry to a Golf-Swing Argument

A defensible analysis first names the modeled bodies, joint coordinates, admissible contact and grip modes, and the task. Clubhead position, face orientation, path direction, impact speed and contact location are different outputs. Their Jacobians and tolerances differ. A change invisible to one task may matter to another.

At a fixed regular posture, $\delta y = J\delta q$ describes first-order geometric sensitivity. A null-space displacement can leave that task unchanged to first order while changing joint leverage, mass distribution, available muscle force or contact feasibility. It need not preserve the task after a finite displacement; following a level set requires integrating a compatible direction field or resolving the nonlinear constraint.

Kinematic freedom is also not torque authority. Ground reactions and two-hand grip forces couple the body and club. Joint torques act through the mass matrix and the constraint equations; muscle activation and tendon dynamics restrict how those torques can change. A desired endpoint velocity in the Jacobian image may demand unavailable joint rates or forces. Conversely, a singular instantaneous map need not eliminate all finite-motion options.

Finally, club delivery is evaluated along a trajectory, often at an impact event. The same posture reached with different velocities, activation states or shaft deformation can produce different outcomes. A claim that a geometric feature improves delivery must specify a feasible intervention, propagate the controlled dynamics and evaluate the selected outcome under measurement uncertainty. This connects configuration geometry to dynamics and control while keeping their distinct evidential roles clear.

3.15 Chapter Summary and Key Principles

Configuration dimension is local freedom under a stated model. Charts represent it locally; topology and constraints determine where those charts work. Tangent vectors transform with chart derivatives, and state requires all variables needed to predict motion. Kinematic maps and their ranks depend on the selected task. Inverse solutions and workspace require global admissibility checks beyond local rank.

Frobenius tests integrability of a regular distribution. Reachability additionally requires the controlled vector fields and their admissible inputs. Geometric freedom, instantaneous velocity authority and dynamically feasible golf delivery are related through explicit maps and equations, not interchangeable descriptions of the same quantity.

3.16 Further Reading

Lynch and Park [30] distinguish configuration topology, independent-constraint counts, task space and workspace. Their official task-space supplement [29] explains the orientation requirement in dexterous workspace. The numerical inverse-kinematics supplement [28] develops the pseudoinverse's local linear role; the mobile-robot controllability supplement [27] connects bracket rank to admissible control sets.

Murray, Li and Sastry [35], Siciliano and colleagues [42], and Bullo and Lewis [6] provide broader treatments of robot kinematics and geometric mechanics/control. The explicit chart inverses, triangle derivation and nonintegrability counterexample here can be checked directly. A source citation does not remove the need to state regularity, coordinate conventions or the physical model used in an application.

3.17 Exercises

1. Explain the six local freedoms of a free spatial rigid body. Why do a rotation matrix's nine entries and a unit quaternion's four entries not increase its three rotational freedoms?

2. Count the regular mobility of a planar four-bar with three moving links and four revolute joints. Differentiate its two closure equations. State when the input angle is a valid local coordinate and why assembly branches can share its value.

3. Prove that no single chart covers the circle using compactness and openness of a chart image. Explain why the angle labeling on $[0, 2\pi)$ is not such a chart. Give a bounded open interval that is a valid possible chart image.

4. Show that $T_{(x,y)}S^1 = \{(a,b) : xa + yb = 0\}$ has dimension one. Compare this tangent space with the derivative kernel of $x^2 + y^2 = 0$ at its isolated solution.

5. For two unit links at $q = (0,0)$, compute position and position Jacobian. Identify the available first-order endpoint velocity direction and a nonzero joint velocity giving zero endpoint velocity.

6. A standard planar slider-crank has three moving links, three revolute joints and one prismatic joint. Sketch its connectivity and obtain the regular count. Explain why the mobility count alone does not uniquely identify these joint types.

7. For two unit links at $q = (\pi/2, 0)$, compute the singular position Jacobian and its image. Describe the missing radial velocity direction. Does the mechanism lose a joint configuration coordinate?

8. Derive the inner radius, outer radius and area of the unrestricted planar workspace with $L_1 = 2, L_2 = 1$. State what changes when the lengths are equal or the joint angles are limited.

9. Verify both stereographic inverses, derive $v = 1/u$ on the overlap and transform the velocity. Recover the same circle speed using the induced metric in either chart.

10. Write the ideal unicycle's Pfaffian constraint and count its kernel dimension. Compute $[X_1, X_2]$ using the stated convention. Explain which additional control hypotheses permit a local controllability conclusion, and test the four-step finite maneuver numerically.

11. Prove the reachable-radius bounds for arbitrary positive L_1, L_2 and show every intermediate radius occurs. For equal links, determine the all-planar-orientations dexterous workspace and explain why it differs from the nonsingular position workspace.

12. For a planar arm with lengths $(1, 1, 0.5)$ and target $(x, y, \theta) = (2, 0.5, \pi/4)$, subtract the final-link vector to obtain the two-link wrist target. Solve both branches geometrically. Compare with a safeguarded Newton iteration starting at $(\pi/6, \pi/6, \pi/6)$, reporting residuals and any rejected steps rather than assuming convergence.

13. For a sphere of radius r on a fixed horizontal plane, write its five-dimensional configuration, no-slip velocity equation and rank-three allowed-velocity space. Explain the eight-dimensional constrained mechanical state and how no twist or a moving plate would change the model.

14. Complete the curve/derivation equivalence proof using the local expansion $f(z) = f(0) + \sum_i z_i h_i(z)$. Explain why the equivalence is independent of chart choice and why tangent vectors at different base points are not automatically interchangeable.

15. For specified Stewart platform anchors, derive the six lengths directly from a requested pose. Then differentiate them with respect to translational velocity and angular velocity, identifying each row's moment arm and direction. State the rank test for local platform control and distinguish direct length calculation from solving pose from lengths.

Chapter 4

Rotations, Quaternions, and SO(3)

The tragedy of Euclidean space is that it works perfectly until you spin.

In Plain Language 4.1. Context & Use Case: The Problem with Spinning

The Math You Will See: We introduce **Rotation Matrices** ($\mathbf{R}$), the **Special Orthogonal Group** SO(3)(3), and **Quaternions** ($\mathbf{q}$). **Why We Need It:** Moving a robot arm up, down, left, or right is easy; you just add numbers to X, Y, Z. But calculating what happens when a drone performs a backflip while spinning like a top is a mathematical nightmare. The most intuitive way to track rotation (Euler angles like Roll, Pitch, Yaw) literally breaks the laws of math in certain positions, causing flight controllers to crash. To solve this, we must map rotations using completely different objects: 3×3 matrices and 4-dimensional Quaternions. **All Variables Defined:** $\mathbf{R} \in \mathrm{SO}(3)(3)$ is a rotation matrix, and $\mathbf{q} \in \mathbb{R}^4$ is a quaternion.

4.1 Historical Context: From Euler to Hamilton

The mathematics of 3D rotation has been refined over centuries by some of history's greatest mathematicians. In **1775**, Leonhard Euler published his groundbreaking *Euler's Rotation Theorem*, establishing that any finite rotation in 3D space can be described as a single rotation about a fixed axis by some angle—a foundational geometric insight that remains central to modern rotation calculus.

However, Euler's representation (axis-angle form) requires solving nonlinear equations and computing trigonometric functions. In **1843**, the Irish mathematician **William Rowan Hamilton** invented **Quaternions** ($\mathcal{H}$), an extension of complex numbers to four dimensions. Quaternions elegantly encode rotations as multiplication operations on 4-tuples, bypassing the singularities that plague other representations. Almost simultaneously, **Olinde Rodrigues** (1795–1853) derived the famous **Rodrigues rotation formula**, expressing how a vector rotates about an axis—a result that connects Euler's axis-angle theorem to matrix algebra.

These three frameworks—rotation matrices, quaternions, and axis-angle representations—are *mathematically equivalent* but offer different computational and conceptual advantages. Modern robotics and computer graphics employ all three, often switching between them fluidly.

4.2 The Coordinate Frame

When we model movement or control robotic systems, we are not looking at the physical object itself; we are tracking a **Coordinate Frame** rigidly attached to the object, often denoted as the Body Frame ($\{b\}$).

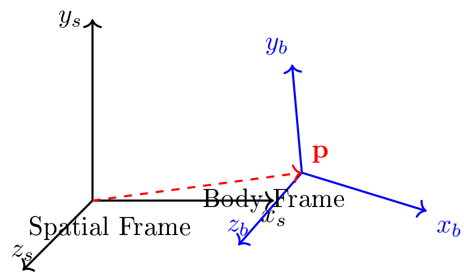


Figure 4.1: Spatial and body frames: The spatial frame is fixed in the world; the body frame is rigidly attached to a moving object. The relationship between them is encoded by a rotation matrix $\mathbf{R}$ and a position vector $\mathbf{p}$.

To know where the Body Frame $\{b\}$ is relative to the absolute stationary universe—the Spatial (or World) Frame ($\{s\}$)—we must define two mathematical properties:

1. The **translation** vector $\mathbf{p} \in \mathbb{R}^3$ pointing from the origin of $\{s\}$ to the origin of $\{b\}$.

2. The **rotation** of the axes of $\{b\}$ relative to the axes of $\{s\}$.

Unlike the straightforward (X, Y, Z) translation vector, rotation is mathematically subtle and surprisingly difficult to parameterize.

4.3 Rotation Matrices: From First Principles

4.3.1 What Is a Rotation Matrix?

To represent a rotation, we examine the three orthonormal basis vectors (axes) of the body frame $\{b\}$: the unit vectors $\hat{\mathbf{x}}_b, \hat{\mathbf{y}}_b$, and $\hat{\mathbf{z}}_b$, each expressed in the coordinates of the spatial frame $\{s\}$. Stack these three column vectors side-by-side:

$$\mathbf{R} = \begin{bmatrix} | & | & | \\ \hat{\mathbf{x}}_b & \hat{\mathbf{y}}_b & \hat{\mathbf{z}}_b \\ | & | & | \end{bmatrix} \in \mathbb{R}^{3\times 3} \tag{4.1}$$

This is a **Rotation Matrix**. Each column is a unit vector, and any two columns are orthogonal to each other. This orthonormality is the key property.

4.3.2 Orthonormality Constraints

Theorem 4.1. Orthonormality Conditions

A matrix $\mathbf{R} \in \mathbb{R}^{3\times 3}$ is a valid rotation matrix if and only if:

1. **Orthogonality:** $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, which means $\mathbf{R}^{-1} = \mathbf{R}^T$.
2. **Determinant Constraint:** $\det(\mathbf{R}) = +1$.

Proof. Let $\mathbf{R} = [\mathbf{c}_1 \mid \mathbf{c}_2 \mid \mathbf{c}_3]$, where $\mathbf{c}_i$ are the columns. The product $\mathbf{R}^T\mathbf{R}$ yields a 3×3 matrix whose (i, j) entry is $\mathbf{c}_i^T\mathbf{c}_j$. For this to equal $\mathbf{I}$:

- Diagonal: $\mathbf{c}_i^T\mathbf{c}_i = 1$ for each i (each column is unit length).
- Off-diagonal: $\mathbf{c}_i^T\mathbf{c}_j = 0$ for $i \neq j$ (columns are mutually orthogonal).

These are exactly 6 independent constraints on the 9 matrix entries, leaving $9 - 6 = 3$ degrees of freedom.

The determinant condition $\det(\mathbf{R}) = 1$ distinguishes *proper rotations* (right-handed orientation preserving) from reflections, which would have $\det(\mathbf{R}) = -1$. □

4.3.3 The Three Elementary Rotation Matrices

Any rotation in 3D can be decomposed into rotations about the principal axes. We derive the three elementary rotation matrices.

Theorem 4.2. Rotation About the x-Axis

A rotation by angle θ about the x-axis, with the rotation viewed as right-hand rule (counterclockwise when looking from positive x toward the origin), is given by:

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix} \tag{4.2}$$

Geometric Interpretation:

- **Column 1:** $[1, 0, 0]^T$ is the x-axis, unchanged by rotation about itself.
- **Column 2:** $[0, \cos\theta, \sin\theta]^T$ is where the original y-axis rotates to.
- **Column 3:** $[0, -\sin\theta, \cos\theta]^T$ is where the original z-axis rotates to.

Derivation. The x-axis unit vector $\hat{\mathbf{e}}_x = [1, 0, 0]^T$ is invariant. The y and z axes lie in the yz-plane, which undergoes a 2D rotation:

$$\begin{bmatrix} y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} y \\ z \end{bmatrix}.$$

Embedding this in 3D yields equation (4.2). □

Theorem 4.3. Rotation About the y-Axis

A rotation by angle θ about the y-axis is:

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \tag{4.3}$$

Theorem 4.4. Rotation About the z-Axis

A rotation by angle θ about the z-axis is:

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{4.4}$$

Each elementary matrix satisfies $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = 1$ by construction (they encode orthonormal bases).

4.4 The Special Orthogonal Group: SO(3)

4.4.1 SO(3) as a Group

Definition 4.1. The Group SO(3)(3)

The **Special Orthogonal Group** SO(3)(3) [35] is the set of all 3×3 real matrices satisfying:

$$\mathrm{SO(3)}(3) = \{\mathbf{R} \in \mathbb{R}^{3\times3} : \mathbf{R}^T\mathbf{R} = \mathbf{I},\ \det(\mathbf{R}) = 1\} \tag{4.5}$$

Theorem 4.5. SO(3)(3) is a Group

The set SO(3)(3) forms a mathematical group under matrix multiplication.

Proof. We verify the four group axioms:

Closure: If $\mathbf{R}_1, \mathbf{R}_2 \in \mathrm{SO(3)}(3)$, then:

$$(\mathbf{R}_1\mathbf{R}_2)^T(\mathbf{R}_1\mathbf{R}_2) = \mathbf{R}_2^T\mathbf{R}_1^T\mathbf{R}_1\mathbf{R}_2 = \mathbf{R}_2^T\mathbf{I}\mathbf{R}_2 = \mathbf{I}, \tag{4.6}$$

$$\det(\mathbf{R}_1\mathbf{R}_2) = \det(\mathbf{R}_1)\det(\mathbf{R}_2) = 1 \cdot 1 = 1. \tag{4.7}$$

Thus $\mathbf{R}_1\mathbf{R}_2 \in \mathrm{SO(3)}(3)$.

Associativity: Matrix multiplication is associative: $(\mathbf{R}_1\mathbf{R}_2)\mathbf{R}_3 = \mathbf{R}_1(\mathbf{R}_2\mathbf{R}_3)$.

Identity: The identity matrix $\mathbf{I}$ satisfies $\mathbf{I}^T\mathbf{I} = \mathbf{I}$ and $\det(\mathbf{I}) = 1$, so $\mathbf{I} \in \mathrm{SO(3)}(3)$.

Inverses: For $\mathbf{R} \in \mathrm{SO(3)}(3)$, we have $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, so $\mathbf{R}^{-1} = \mathbf{R}^T$. Moreover:

$$\det(\mathbf{R}^T) = \det(\mathbf{R}) = 1, \tag{4.8}$$

so $\mathbf{R}^{-1} \in \mathrm{SO(3)}(3)$. □

4.4.2 SO(3) as a Smooth Manifold

Theorem 4.6. SO(3)(3) is a 3-Dimensional Smooth Manifold

The set SO(3)(3) is a smooth, compact manifold of dimension 3.

Argument. The orthogonality constraint $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ imposes 6 independent scalar equations (the symmetric matrix $\mathbf{I}$ has 6 independent entries) on the 9 entries of $\mathbf{R}$. Additionally, $\det(\mathbf{R}) = 1$ is one equation, but it is redundant: for orthogonal matrices with real entries, $\det(\mathbf{R}) = \pm 1$ automatically, and we select the component with determinant $+1$ by continuity and connectedness.

Thus, the effective number of independent constraints is 6, leaving:

$$\dim(\mathrm{SO}(3)(3)) = 9 - 6 = 3. \tag{4.9}$$

Each point in SO(3)(3) admits a local parameterization by 3 parameters (e.g., Euler angles in a suitable domain), confirming that SO(3)(3) is a smooth 3-dimensional manifold. □

Mathematical Principle 4.1. The SO(3) Manifold

The continuous, infinite geometric space of all possible valid 3×3 rotation matrices satisfying $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = 1$ is a mathematical **manifold** called the **Special Orthogonal Group in 3 Dimensions**, denoted as SO(3)(3).
If a Configuration Space $\mathcal{Q}$ is simply a rigid body freely rotating around a fixed point, we say $\mathcal{Q} = \mathrm{SO}(3)(3)$.

4.5 Euler Angles: Gimbal Lock and Beyond

4.5.1 Definition of Euler Angles

The most intuitive way for a human to describe the orientation of an aircraft or robotic arm is to split rotation into three successive rotations about primary axes. This is known as an **Euler Angle** representation: $\boldsymbol{\theta} = [\phi, \theta, \psi]^T$.

While highly intuitive for human pilots, Euler Angles harbor a fatal mathematical flaw known as **Gimbal Lock**.

4.5.2 The ZYX Convention

One common convention is **ZYX Euler angles** (also called *yaw-pitch-roll*):

1. Rotate by ψ (yaw) about the original z-axis.
2. Rotate by θ (pitch) about the *current* y-axis (after yaw).
3. Rotate by ϕ (roll) about the *current* x-axis (after yaw and pitch).

The composite rotation matrix is:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_x(\phi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi). \tag{4.10}$$

Computing this product explicitly:

$$\mathbf{R}(\phi, \theta, \psi) = \begin{bmatrix} \cos\psi\cos\theta & -\sin\psi\cos\phi + \cos\psi\sin\theta\sin\phi & \sin\psi\sin\phi + \cos\psi\sin\theta\cos\phi \\ \sin\psi\cos\theta & \cos\psi\cos\phi + \sin\psi\sin\theta\sin\phi & -\cos\psi\sin\phi + \sin\psi\sin\theta\cos\phi \\ -\sin\theta & \cos\theta\sin\phi & \cos\theta\cos\phi \end{bmatrix} \tag{4.11}$$

4.5.3 The ZXZ Convention

Another standard convention is **ZXZ Euler angles** (symmetric, used in physics and astronomy):

1. Rotate by ϕ about the original z-axis.
2. Rotate by θ about the *current* x-axis.
3. Rotate by ψ about the *current* z-axis (after the first two rotations).

The composite rotation is:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi)\mathbf{R}_x(\theta)\mathbf{R}_z(\phi). \tag{4.12}$$

4.5.4 Gimbal Lock: Singularity and Proof

In Plain Language 4.2. Gimbal Lock: Physical Intuition

If an airplane pitches perfectly 90° straight up into the sky, its Roll axis (pointing out its nose) now perfectly aligns with its original Yaw axis (pointing perfectly down towards the Earth).
Mathematically, two independent degrees of freedom have become linearly dependent—they merged into the identical physical rotation axis. A degree of freedom is literally lost during the calculation. If you attempt to control the plane passing through this singularity, the Jacobian mapping Euler angle rates to angular velocity becomes singular, producing undefined or numerically unstable results (NaN values, wildly large rate commands).

Theorem 4.7. Gimbal Lock at $\theta = \pm 90°$

For ZYX Euler angles, the Jacobian matrix relating angular velocity components to Euler angle rate components becomes singular when $\theta = \pm 90°$.

Derivation. The relationship between angular velocity $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ in the spatial frame and Euler angle rates $\dot{\boldsymbol{\theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^T$ is given by:

$$\boldsymbol{\omega} = \mathbf{J}(\boldsymbol{\theta})\,\dot{\boldsymbol{\theta}}, \tag{4.13}$$

where $\mathbf{J}(\boldsymbol{\theta})$ is the Jacobian. For ZYX angles, it can be shown that:

$$\mathbf{J}(\phi, \theta, \psi) = \begin{bmatrix} 0 & -\sin\psi & \cos\psi\cos\theta \\ 0 & \cos\psi & \sin\psi\cos\theta \\ 1 & 0 & -\sin\theta \end{bmatrix} \tag{4.14}$$

When $\theta = 90^\circ$, we have $\cos\theta = 0$, so:

$$\mathbf{J}\left(\phi, 90^\circ, \psi\right) = \begin{bmatrix} 0 & -\sin\psi & 0 \\ 0 & \cos\psi & 0 \\ 1 & 0 & -1 \end{bmatrix}. \tag{4.15}$$

The third column is linearly dependent on the first two, so $\text{rank}(\mathbf{J}) < 3$ and $\det(\mathbf{J}) = 0$. The Jacobian is singular, making it impossible to invert: given $\boldsymbol{\omega}$, we cannot solve uniquely for $\dot{\boldsymbol{\theta}}$. Physically, the yaw and roll rates become indistinguishable. □

Since our control theory and geometric motion algorithms fundamentally rely on continuous trajectories twisting through all possible spatial angles (like chaotic golf club arcs or throwing motions), we must completely abandon Euler angles for internal calculations. We cannot afford mathematical singularities.

4.6 Differential Geometry Primer: Lie Groups and Lie Algebras

We have established that SO(3)(3) is a manifold. However, it is not just any manifold. Because you can combine two rotations to get a third rotation (via matrix multiplication), and every rotation can be smoothly inverted, SO(3)(3) possesses both algebraic structure (it is a mathematical *group*) and geometric structure (it is a *smooth manifold*).

Definition 4.2. Lie Group

A *Lie Group* is a smooth manifold that is also a mathematical group, where the group operations (multiplication and inversion) are smooth functions. As a result, we can perform calculus on them. SO(3)(3) is the preeminent example of a Lie Group in mechanics.

4.6.1 The Tangent Space and Lie Algebra

If SO(3)(3) is a curved manifold of positions (rotations), what is its velocity space? By our earlier definition in Chapter 3, the velocities must live in the Tangent Space.

If we examine the tangent space at the identity rotation (the "zero" rotation, matrix $\mathbf{I}$), we discover an incredibly special mathematical object called the **Lie Algebra**, denoted by lowercase $\mathfrak{so}(3)(3)$.

Definition 4.3. Lie Algebra $\mathfrak{so}(3)(3)$

The *Lie Algebra* $\mathfrak{so}(3)(3)$ is the tangent space to the Lie Group SO(3)(3) at the identity element. Physically, it represents the space of all possible angular velocities (spin rates) when the object is currently unrotated. Mathematically, it consists of all 3×3 *skew-symmetric* matrices: matrices $[\boldsymbol{\omega}]_\times$ satisfying $[\boldsymbol{\omega}]_\times^T = -[\boldsymbol{\omega}]_\times$.

A skew-symmetric matrix representing a vector $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ is:

$$[\boldsymbol{\omega}]_\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \tag{4.16}$$

In Plain Language 4.3. Mathematical Reminder: Group vs. Algebra

Whenever you hear **Lie Group** (like SO(3)(3) or SE(3)(3)), think of the curved, global manifold of *Positions/Configurations*. Whenever you hear **Lie Algebra** (like $\mathfrak{so}(3)(3)$ or $\mathfrak{se}(3)(3)$), think of the perfectly flat, linear tangent space of *Velocities*. We can do standard linear control math in the Lie Algebra, but we map the results back onto the curved Lie Group using the *Exponential Map* (covered next) to actually rotate the physical robot.

4.7 Rodrigues' Rotation Formula: Full Derivation

We now prove the celebrated **Rodrigues' rotation formula**, which gives a closed-form expression for the matrix exponential of a skew-symmetric matrix. This result, first obtained by Olinde Rodrigues in 1840 [39], is the computational backbone of the exponential map for SO(3)(3).

4.7.1 The Key Identity: $[\hat{\omega}]^3 = -[\hat{\omega}]$

Let $\hat{\boldsymbol{\omega}} \in \mathbb{R}^3$ be a unit vector ($\|\hat{\boldsymbol{\omega}}\| = 1$) and $[\hat{\omega}] \in \mathfrak{so}(3)(3)$ the corresponding skew-symmetric matrix. We first establish a crucial algebraic identity.

Theorem 4.8. Cyclic Property of Skew-Symmetric Matrices

For any unit vector $\hat{\boldsymbol{\omega}}$ with skew-symmetric matrix $[\hat{\omega}]$:

$$[\hat{\omega}]^3 = -[\hat{\omega}]. \tag{4.17}$$

Proof. Recall that for a skew-symmetric matrix, the action $[\hat{\omega}]\mathbf{v} = \hat{\boldsymbol{\omega}} \times \mathbf{v}$. Thus:

$$[\hat{\omega}]^2\mathbf{v} = \hat{\boldsymbol{\omega}} \times (\hat{\boldsymbol{\omega}} \times \mathbf{v}). \tag{4.18}$$

By the vector triple product identity $\mathbf{a} \times (\mathbf{a} \times \mathbf{v}) = \mathbf{a}(\mathbf{a} \cdot \mathbf{v}) - \mathbf{v}(\mathbf{a} \cdot \mathbf{a})$ with $\|\hat{\boldsymbol{\omega}}\| = 1$:

$$[\hat{\omega}]^2 = \hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}. \tag{4.19}$$

Therefore:

$$[\hat{\omega}]^3 = [\hat{\omega}](\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}) = [\hat{\omega}]\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - [\hat{\omega}]. \tag{4.20}$$

Since $[\hat{\omega}]\hat{\boldsymbol{\omega}} = \hat{\boldsymbol{\omega}} \times \hat{\boldsymbol{\omega}} = \mathbf{0}$, we obtain $[\hat{\omega}]^3 = -[\hat{\omega}]$. □

This identity implies that all higher powers of $[\hat{\omega}]$ reduce to either $[\hat{\omega}]$ or $[\hat{\omega}]^2$:

$$[\hat{\omega}]^4 = -[\hat{\omega}]^2, \quad [\hat{\omega}]^5 = [\hat{\omega}], \quad [\hat{\omega}]^6 = [\hat{\omega}]^2, \quad \dots \tag{4.21}$$

4.7.2 Taylor Expansion and Regrouping

The matrix exponential is defined by its Taylor series:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \theta[\hat{\omega}] + \frac{\theta^2}{2!}[\hat{\omega}]^2 + \frac{\theta^3}{3!}[\hat{\omega}]^3 + \frac{\theta^4}{4!}[\hat{\omega}]^4 + \cdots \tag{4.22}$$

Substituting $[\hat{\omega}]^3 = -[\hat{\omega}]$, $[\hat{\omega}]^4 = -[\hat{\omega}]^2$, $[\hat{\omega}]^5 = [\hat{\omega}]$, etc., and grouping by powers of $[\hat{\omega}]$:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \underbrace{\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \cdots\right)}_{\sin\theta}[\hat{\omega}] + \underbrace{\left(\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \frac{\theta^6}{6!} - \cdots\right)}_{1-\cos\theta}[\hat{\omega}]^2. \tag{4.23}$$

The odd-power series is exactly $\sin\theta$ and the even-power series is exactly $1 - \cos\theta$.

Theorem 4.9. Rodrigues' Rotation Formula

For a unit axis $\hat{\boldsymbol{\omega}} \in \mathbb{R}^3$ and angle $\theta \in \mathbb{R}$, the rotation matrix is:

$$\boxed{e^{[\hat{\omega}]\theta} = \mathbf{I} + \sin\theta\,[\hat{\omega}] + (1 - \cos\theta)\,[\hat{\omega}]^2.} \tag{4.24}$$

Equivalently, using (4.19), the action on a vector $\mathbf{v}$ is:

$$\mathbf{Rv} = \mathbf{v}\cos\theta + (\hat{\boldsymbol{\omega}} \times \mathbf{v})\sin\theta + \hat{\boldsymbol{\omega}}(\hat{\boldsymbol{\omega}} \cdot \mathbf{v})(1 - \cos\theta). \tag{4.25}$$

The vector form (4.25) has a geometric interpretation: decompose $\mathbf{v}$ into a component along $\hat{\boldsymbol{\omega}}$ (unchanged by rotation) and a component perpendicular to $\hat{\boldsymbol{\omega}}$ (rotated by θ in the plane normal to $\hat{\boldsymbol{\omega}}$). See [39] for the original derivation and [35] for the modern Lie-theoretic perspective.

4.8 Quaternion Algebra

4.8.1 Definition and Representation

A **quaternion** $\mathbf{q} \in \mathbb{H}$ is a 4-tuple written as:

$$\mathbf{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = q_w + q_x\mathbf{i} + q_y\mathbf{j} + q_z\mathbf{k} \tag{4.26}$$

where $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are the three *imaginary units* satisfying:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1, \quad \mathbf{ij} = \mathbf{k}, \quad \mathbf{jk} = \mathbf{i}, \quad \mathbf{ki} = \mathbf{j}. \tag{4.27}$$

Note that quaternion multiplication is *non-commutative*: $\mathbf{ij} = \mathbf{k}$ but $\mathbf{ji} = -\mathbf{k}$.

4.8.2 Quaternion Operations

Addition

Quaternion addition is component-wise:

$$(q_w + q_x\mathbf{i} + q_y\mathbf{j} + q_z\mathbf{k}) + (r_w + r_x\mathbf{i} + r_y\mathbf{j} + r_z\mathbf{k}) = (q_w + r_w) + (q_x + r_x)\mathbf{i} + (q_y + r_y)\mathbf{j} + (q_z + r_z)\mathbf{k}. \tag{4.28}$$

Hamilton Product (Multiplication)

The product of two quaternions $\mathbf{q} = q_w + \mathbf{q}_v$ and $\mathbf{r} = r_w + \mathbf{r}_v$, where $\mathbf{q}_v = [q_x, q_y, q_z]^T$ and $\mathbf{r}_v = [r_x, r_y, r_z]^T$, is:

$$\mathbf{q} \otimes \mathbf{r} = \begin{bmatrix} q_w r_w - \mathbf{q}_v^T \mathbf{r}_v \\ q_w \mathbf{r}_v + r_w \mathbf{q}_v + \mathbf{q}_v \times \mathbf{r}_v \end{bmatrix} \tag{4.29}$$

where $\times$ denotes the 3D cross product. In component form:

$$\mathbf{q} \otimes \mathbf{r} = \begin{bmatrix} q_w r_w - q_x r_x - q_y r_y - q_z r_z \\ q_w r_x + q_x r_w + q_y r_z - q_z r_y \\ q_w r_y - q_x r_z + q_y r_w + q_z r_x \\ q_w r_z + q_x r_y - q_y r_x + q_z r_w \end{bmatrix} \tag{4.30}$$

Conjugate

The **conjugate** of $\mathbf{q} = q_w + q_x\mathbf{i} + q_y\mathbf{j} + q_z\mathbf{k}$ is:

$$\mathbf{q}^* = q_w - q_x\mathbf{i} - q_y\mathbf{j} - q_z\mathbf{k} = \begin{bmatrix} q_w \\ -q_x \\ -q_y \\ -q_z \end{bmatrix} \tag{4.31}$$

Norm and Magnitude

The **norm** (or magnitude) of a quaternion is:

$$\|\mathbf{q}\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} \tag{4.32}$$

A **unit quaternion** satisfies $\|\mathbf{q}\| = 1$.

Inverse

For a unit quaternion $\mathbf{q}$, the inverse is:

$$\mathbf{q}^{-1} = \mathbf{q}^* \tag{4.33}$$

since $\mathbf{q} \otimes \mathbf{q}^* = \mathbf{q}^* \otimes \mathbf{q} = 1$ (the multiplicative identity quaternion).
For a non-unit quaternion, the inverse is $\mathbf{q}^{-1} = \mathbf{q}^*/\|\mathbf{q}\|^2$.

4.8.3 Unit Quaternions and Rotation

Theorem 4.10. Unit Quaternions Represent Rotations

Every unit quaternion $\mathbf{q}$ can be written as:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) \\ \hat{\boldsymbol{\omega}} \sin(\theta/2) \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \hat{\omega}_x \sin(\theta/2) \\ \hat{\omega}_y \sin(\theta/2) \\ \hat{\omega}_z \sin(\theta/2) \end{bmatrix} \tag{4.34}$$

where $\hat{\boldsymbol{\omega}}$ is a unit vector (the rotation axis) and $\theta \in [0, 2\pi)$ is the rotation angle. This quaternion represents a rotation of angle θ about the axis $\hat{\boldsymbol{\omega}}$.

In Plain Language 4.4. Why the Half-Angles?

You might notice $\theta/2$ in the formula instead of just θ. This is because applying a quaternion rotation to a vector $\mathbf{v}$ requires multiplying it from the front *and* the back ($\mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^{-1}$, where $\mathbf{v}$ is viewed as the quaternion $[0, v_x, v_y, v_z]^T$). The half-angles combine during this double-multiplication to give the perfect full-angle θ rotation! Note also that the quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the exact same physical rotation in 3D space.

4.8.4 Double Cover: q and $-q$ Represent the Same Rotation

Theorem 4.11. Quaternion Double Cover

The unit quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation in 3D space. Thus, the map from unit quaternions to SO(3)(3) is a 2-to-1 covering map, and we say that S^3 (the unit sphere in $\mathbb{H}$) is a *double cover* of SO(3)(3).

Proof. The rotation action of a unit quaternion $\mathbf{q}$ on a pure-imaginary quaternion $\mathbf{v} = [0, \vec{v}]$ is:

$$\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*. \tag{4.35}$$

For $-\mathbf{q}$, we compute:

$$\begin{aligned}
(-\mathbf{q}) \otimes \mathbf{v} \otimes (-\mathbf{q})^* &= (-\mathbf{q}) \otimes \mathbf{v} \otimes (-\mathbf{q}^*) && (4.36)\\
&= (-1)(\mathbf{q}) \otimes \mathbf{v} \otimes (-1)(\mathbf{q}^*) && (4.37)\\
&= (-1)(-1)\, \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^* && (4.38)\\
&= \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*, && (4.39)
\end{aligned}$$

where we used the fact that $(-\mathbf{q})^* = -\mathbf{q}^*$ and that the scalar factor (-1) commutes with quaternion multiplication and appears twice, yielding $(-1)^2 = 1$.

Thus $\mathbf{q}$ and $-\mathbf{q}$ induce the identical rotation on every vector $\vec{v} \in \mathbb{R}^3$. □

4.9 Conversion Between Quaternions and Rotation Matrices

4.9.1 Quaternion to Rotation Matrix

Theorem 4.12. Quaternion-to-Matrix Conversion

A unit quaternion $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$ with $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$ corresponds to the rotation matrix:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_wq_z) & 2(q_xq_z + q_wq_y) \\ 2(q_xq_y + q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_wq_x) \\ 2(q_xq_z - q_wq_y) & 2(q_yq_z + q_wq_x) & 1-2(q_x^2+q_y^2) \end{bmatrix} \tag{4.40}$$

Derivation. A rotation by angle θ about unit axis $\hat{\boldsymbol{\omega}} = [\omega_x, \omega_y, \omega_z]^T$ can be expressed via the Rodrigues formula [39]:

$$\mathbf{R} = \mathbf{I} + \sin\theta\,[\hat{\boldsymbol{\omega}}]_\times + (1-\cos\theta)\,[\hat{\boldsymbol{\omega}}]_\times^2. \tag{4.41}$$

With $q_w = \cos(\theta/2)$ and $\hat{\boldsymbol{\omega}}\sin(\theta/2) = [q_x, q_y, q_z]^T$, one can expand using the identities $\sin\theta = 2\sin(\theta/2)\cos(\theta/2) = 2q_w\sqrt{q_x^2+q_y^2+q_z^2}$ and $(1-\cos\theta) = 2\sin^2(\theta/2) = 2(q_x^2+q_y^2+q_z^2)$ to obtain equation (4.40). □

4.9.2 Rotation Matrix to Quaternion

Theorem 4.13. Matrix-to-Quaternion Conversion

From a rotation matrix $\mathbf{R}$, the unit quaternion can be recovered (up to sign) via:

$$q_w = \frac{1}{2}\sqrt{1 + R_{11} + R_{22} + R_{33}}, \tag{4.42}$$

with q_x, q_y, q_z determined by:

$$\begin{aligned} q_x &= \frac{R_{32} - R_{23}}{4q_w}, \\ q_y &= \frac{R_{13} - R_{31}}{4q_w}, \\ q_z &= \frac{R_{21} - R_{12}}{4q_w}. \end{aligned} \tag{4.43}$$

For numerical stability, a more robust algorithm selects the largest diagonal element to compute first, then back-solves for the others.

4.9.3 Rotation Representation Comparison

Having introduced four rotation representations, we now compare their properties systematically. The choice of representation has significant practical consequences for storage, computation, and numerical stability.

Table 4.1: Comparison of Rotation Representations in $\mathbb{R}^3$.

Property	**Rotation Matrix**	**Euler Angles**	**Axis-Angle**	**Quaternion**
Parameters	9	3	3	4
Constraints	6 (orthonormality)	0	0	1 (unit norm)
Singularities	None	Gimbal lock	$\theta = \pi$	None
Composition	Matrix multiply	Complex	Via exp map	Quaternion multiply
Interpolation	Difficult	Nonuniform	Via exp map	SLERP
Storage cost	High	Low	Low	Low
Unique	Yes	No (periodic)	No ($\theta > \pi$)	No ($\pm\mathbf{q}$)

When to use each representation:

- **Rotation matrices** are the natural choice for *composing* transformations (chain of rigid-body transforms) and for direct application to vectors. Their 9-parameter redundancy is the cost of a singularity-free, algebraically closed representation.

- **Euler angles** remain useful for *human interpretation* (roll, pitch, yaw) and low-DOF joints, but gimbal lock (Section 4.5.4) makes them unsuitable for general-purpose computation.

- **Axis-angle** (equivalently, exponential coordinates) is the natural parameterisation of the Lie algebra $\mathfrak{so}(3)(3)$ and is essential for optimisation on SO(3)(3), perturbation analysis, and control.

- **Quaternions** offer the best trade-off for *interpolation* (via SLERP, Section 6.11), *storage* (4 parameters, 1 constraint), and *numerical stability* (no singularities, cheap renormalisation). Modern simulation frameworks—including MuJoCo, Drake, and Bullet—typically store orientations as unit quaternions internally and convert to rotation matrices only when matrix operations are required.

4.10 Spherical Linear Interpolation (SLERP)

In computer graphics and animation, we often need to smoothly interpolate between two rotations. For quaternions, the natural interpolation is **Spherical Linear Interpolation (SLERP)**.

Definition 4.4. SLERP Formula

The SLERP between two unit quaternions $\mathbf{q}_0$ and $\mathbf{q}_1$ at parameter $t \in [0, 1]$ is:

$$\text{SLERP}(\mathbf{q}_0, \mathbf{q}_1; t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\mathbf{q}_0 + \frac{\sin(t\Omega)}{\sin\Omega}\mathbf{q}_1 \tag{4.44}$$

where $\Omega = \arccos(|\mathbf{q}_0 \cdot \mathbf{q}_1|)$ is the (acute) angle between the quaternions on the unit sphere.

Derivation. Two unit quaternions $\mathbf{q}_0$ and $\mathbf{q}_1$ lie on the unit sphere $S^3 \subset \mathbb{H}$. The angle between them is $\Omega = \arccos(\mathbf{q}_0 \cdot \mathbf{q}_1)$ (taking absolute value ensures we pick the shorter arc).

A geodesic (great circle arc) on S^3 is the shortest path between two points. Parameterizing the geodesic by arc length with $t \in [0, 1]$ (where $t = 0$ at $\mathbf{q}_0$ and $t = 1$ at $\mathbf{q}_1$) gives the SLERP formula.

If $\Omega \approx 0$ (very close quaternions), use linear interpolation $\text{LERP}(\mathbf{q}_0, \mathbf{q}_1; t) = (1-t)\mathbf{q}_0 + t\mathbf{q}_1$ followed by normalization to avoid numerical division by zero. □

SLERP is constant-speed interpolation and is the standard for smooth rotational animations in robotics and graphics.

4.11 Angular Velocity and the Matrix Exponential

4.11.1 Angular Velocity in the Spatial Frame

When a rigid body rotates, its instantaneous rotational motion is characterized by the **angular velocity** vector $\boldsymbol{\omega} \in \mathbb{R}^3$. The magnitude $\|\boldsymbol{\omega}\|$ is the rate of rotation (rad/s), and the direction of $\boldsymbol{\omega}$ is the instantaneous axis of rotation.

The rate of change of the rotation matrix $\mathbf{R}(t)$ with respect to time is related to angular velocity by:

Theorem 4.14. Matrix Derivative in Spatial Frame

$$\dot{\mathbf{R}}(t) = [\boldsymbol{\omega}]_\times \mathbf{R}(t) \tag{4.45}$$

where $\boldsymbol{\omega}$ is expressed in the spatial frame and $[\boldsymbol{\omega}]_\times$ is the skew-symmetric matrix defined in equation (4.16).

Proof. Differentiating the orthogonality constraint $\mathbf{R}^T(t)\mathbf{R}(t) = \mathbf{I}$ with respect to time:

$$\frac{d}{dt}[\mathbf{R}^T\mathbf{R}] = \dot{\mathbf{R}}^T\mathbf{R} + \mathbf{R}^T\dot{\mathbf{R}} = \mathbf{0}. \tag{4.46}$$

Rearranging:

$$\mathbf{R}^T\dot{\mathbf{R}} = -(\mathbf{R}^T\dot{\mathbf{R}})^T, \tag{4.47}$$

so $\mathbf{R}^T\dot{\mathbf{R}}$ is skew-symmetric. Define $[\boldsymbol{\omega}]_\times := \mathbf{R}^T\dot{\mathbf{R}}$. Then:

$$\dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}]_\times. \tag{4.48}$$

However, by convention in robotics, we often write $\dot{\mathbf{R}} = [\boldsymbol{\omega}]_\times\mathbf{R}$ where $\boldsymbol{\omega}$ is the spatial frame angular velocity. Both are correct; the difference is in the choice of frame. □

4.11.2 Angular Velocity in the Body Frame

When the angular velocity is expressed in the body-fixed frame, the differential equation becomes:

$$\dot{\mathbf{R}}(t) = \mathbf{R}(t)[\boldsymbol{\omega}_b]_\times \tag{4.49}$$

where $\boldsymbol{\omega}_b$ is the angular velocity in the body frame.

4.11.3 Proof that Angular Velocity Lives in $\mathfrak{so}(3)(3)$

Theorem 4.15. Angular Velocity is a Lie Algebra Element

For any rotation matrix $\mathbf{R}(t) \in \mathrm{SO}(3)(3)$, the quantity $\mathbf{R}^T\dot{\mathbf{R}} \in \mathfrak{so}(3)(3)$ is a skew-symmetric matrix, i.e., an element of the Lie algebra $\mathfrak{so}(3)(3)$.

Proof. From $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, differentiate:

$$\dot{\mathbf{R}}^T\mathbf{R} + \mathbf{R}^T\dot{\mathbf{R}} = \mathbf{0}. \tag{4.50}$$

Taking the transpose of the second term:

$$(\mathbf{R}^T\dot{\mathbf{R}})^T = \dot{\mathbf{R}}^T\mathbf{R}. \tag{4.51}$$

From the differentiated constraint, $\dot{\mathbf{R}}^T\mathbf{R} = -\mathbf{R}^T\dot{\mathbf{R}}$, so:

$$(\mathbf{R}^T\dot{\mathbf{R}})^T = -\mathbf{R}^T\dot{\mathbf{R}}. \tag{4.52}$$

Thus $\mathbf{R}^T\dot{\mathbf{R}}$ is skew-symmetric, confirming it belongs to $\mathfrak{so}(3)(3)$. □

4.12 Axis-Angle Representation and Euler's Rotation Theorem

Theorem 4.16. Euler's Rotation Theorem

Every rotation in 3D can be expressed as a rotation about a single axis by some angle. That is, for any $\mathbf{R} \in \mathrm{SO}(3)(3)$, there exist a unit vector $\hat{\boldsymbol{\omega}} \in \mathbb{R}^3$ (the rotation axis) and an angle $\theta \in [0, \pi]$ such that the rotation is completely described by this pair $(\hat{\boldsymbol{\omega}}, \theta)$.

Proof. The rotation axis $\hat{\boldsymbol{\omega}}$ is an eigenvector of $\mathbf{R}$ with eigenvalue 1:

$$\mathbf{R}\hat{\boldsymbol{\omega}} = \hat{\boldsymbol{\omega}}. \tag{4.53}$$

Such an eigenvector exists because $\det(\mathbf{R} - \mathbf{I}) = 0$ for any rotation (a rotation always leaves one axis invariant). The angle θ can be recovered from $\mathrm{trace}(\mathbf{R}) = 1 + 2\cos\theta$. □

The axis-angle representation is compact and intuitive but computationally less efficient than quaternions for repeated operations.

4.13 Worked Example: Quaternions and SLERP in Python

Below is a complete Python implementation of quaternion operations and rotation interpolation.

```python
"""Quaternion operations and SLERP, as developed in Volume 0 Chapter 4.

Self-contained by design: this file is typeset into the chapter as a worked
example, so it depends only on NumPy and repeats nothing the chapter has not
already derived. Equation numbers in the comments refer to Chapter 4.

Correctness is pinned by tests/test_affine_control/test_quaternion_demo.py --
the properties asserted there (unit norm, orthogonality of the rotation matrix,
agreement between quaternion rotation and matrix rotation, SLERP endpoints and
constant angular velocity) are what make this an example rather than a claim.
"""

from __future__ import annotations

import numpy as np
from numpy.typing import NDArray

type Array = NDArray[np.float64]

# Below this angle the small-angle expansion of SLERP is better conditioned
# than the closed form: sin(Omega) appears in a denominator, so the closed form
# loses precision as the two quaternions approach each other.
SLERP_LINEAR_THRESHOLD = 1.0 - 1.0e-9


def hamilton_product(p: Array, q: Array) -> Array:
    """Quaternion product, equation (eq:hamilton_product).

    Both arguments are ordered [w, x, y, z]. The product is not commutative;
    ``hamilton_product(p, q)`` applies ``q`` first, then ``p``.
    """
    pw, px, py, pz = p
    qw, qx, qy, qz = q
    return np.array(
        [
            pw * qw - px * qx - py * qy - pz * qz,
            pw * qx + px * qw + py * qz - pz * qy,
            pw * qy - px * qz + py * qw + pz * qx,
            pw * qz + px * qy - py * qx + pz * qw,
        ]
    )
```

```python

def normalize(q: Array) -> Array:
    """Project onto the unit sphere, which is where rotations live."""
    norm = float(np.linalg.norm(q))
    if norm == 0.0:
        msg = "the zero quaternion represents no rotation and cannot be normalised"
        raise ValueError(msg)
    return q / norm


def conjugate(q: Array) -> Array:
    """For a unit quaternion this is also the inverse."""
    w, x, y, z = q
    return np.array([w, -x, -y, -z])


def from_axis_angle(axis: Array, angle: float) -> Array:
    """Unit quaternion for a rotation of ``angle`` about ``axis``.

    Note the half-angle: a quaternion covers SO(3) twice, so q and -q are the
    same rotation.
    """
    unit = normalize(np.asarray(axis, dtype=float))
    return np.concatenate([[np.cos(angle / 2.0)], np.sin(angle / 2.0) * unit])


def to_matrix(q: Array) -> Array:
    """Rotation matrix of a unit quaternion, equation (eq:quat_to_matrix)."""
    w, x, y, z = normalize(q)
    return np.array(
        [
            [1 - 2 * (y * y + z * z), 2 * (x * y - w * z), 2 * (x * z + w * y)],
            [2 * (x * y + w * z), 1 - 2 * (x * x + z * z), 2 * (y * z - w * x)],
            [2 * (x * z - w * y), 2 * (y * z + w * x), 1 - 2 * (x * x + y * y)],
        ]
    )


def rotate(q: Array, v: Array) -> Array:
    """Rotate a vector by the sandwich product q v q*, without forming a matrix."""
    pure = np.concatenate([[0.0], np.asarray(v, dtype=float)])
    return hamilton_product(hamilton_product(q, pure), conjugate(q))[1:]

```

```python

def slerp(q0: Array, q1: Array, t: float) -> Array:
    """Spherical linear interpolation, equation (eq:slerp).

    Two details matter and neither is cosmetic:

    * If the dot product is negative the interpolation would take the long way
      round the sphere -- 360 degrees minus the short arc. Negating one input
      selects the short arc, which is legitimate because q and -q denote the
      same rotation.
    * As the quaternions approach each other sin(Omega) approaches zero and the
      closed form loses precision, so below a threshold we fall back to
      normalised linear interpolation, which agrees to first order.
    """
    a = normalize(np.asarray(q0, dtype=float))
    b = normalize(np.asarray(q1, dtype=float))

    dot = float(np.dot(a, b))
    if dot < 0.0:
        b, dot = -b, -dot

    if dot > SLERP_LINEAR_THRESHOLD:
        return normalize(a + t * (b - a))

    omega = float(np.arccos(np.clip(dot, -1.0, 1.0)))
    sin_omega = np.sin(omega)
    return (np.sin((1.0 - t) * omega) / sin_omega) * a + (np.sin(t * omega) / sin_omega) * b


def interpolate_point_cloud(points: Array, q0: Array, q1: Array, frames: int) -> list[Array]:
    """Carry a point cloud through the interpolated frames.

    Returns one rotated copy of ``points`` per frame, which is what the
    chapter's visualisation plots.
    """
    return [
        (to_matrix(slerp(q0, q1, t)) @ np.asarray(points, dtype=float).T).T
        for t in np.linspace(0.0, 1.0, frames)
    ]
```

Key points from the code:

1. **Quaternion representation:** We store $\mathbf{q} = [q_w, q_x, q_y, q_z]$ as a NumPy array.

2. **Hamilton product:** Custom function implements the quaternion multiplication rule (equation (4.29)).

3. **Normalization:** Unit quaternions are enforced by dividing by norm.

4. **Quaternion-to-matrix:** Uses equation (4.40).

5. **SLERP:** Implements equation (4.44) with fallback to linear interpolation for nearby quaternions.

6. **Visualization:** The example rotates a point cloud through interpolated frames.

4.14 Chapter Summary

This chapter has developed the complete mathematical framework for 3D rotations:

- **Rotation Matrices (SO(3)(3)):** We defined the Special Orthogonal Group as the set of 3×3 orthogonal matrices with determinant $+1$, proved it is both a group and a 3-dimensional smooth manifold, and derived the three elementary rotations $\mathbf{R}_x(\theta), \mathbf{R}_y(\theta), \mathbf{R}_z(\theta)$.

- **Euler Angles and Gimbal Lock:** We presented the ZYX and ZXZ conventions and rigorously proved that the Jacobian relating angular velocities to Euler angle rates becomes singular at $\theta = \pm 90°$, making Euler angles unsuitable for continuous control.

- **Lie Groups and Algebras:** We identified SO(3)(3) as a Lie Group (smooth manifold + group structure) and its Lie algebra $\mathfrak{so}(3)(3)$ as the space of skew-symmetric matrices, which represent angular velocities.

- **Quaternion Algebra:** We introduced the quaternion representation of rotations, defined all quaternion operations (addition, Hamilton product, conjugate, norm, inverse), and proved that $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation (the double cover property).

- **Conversions:** We derived explicit formulas for converting between quaternions and rotation matrices in both directions.

- **SLERP:** We presented spherical linear interpolation as the natural way to smoothly interpolate between rotations.

- **Angular Velocity:** We connected the time-derivative of rotation matrices to angular velocity vectors, showing that $\dot{\mathbf{R}} = [\boldsymbol{\omega}]_\times \mathbf{R}$ and that $\mathbf{R}^T\dot{\mathbf{R}} \in \mathfrak{so}(3)(3)$.

- **Euler's Theorem:** We stated and sketched the proof that every 3D rotation can be expressed as a rotation about a single axis by a single angle.

These tools eliminate the singularities of Euler angles while providing efficient computational representations. The next chapter combines rotation matrices with translation vectors into the full rigid body transformation group SE(3)(3).

4.15 Further Reading

The rotation group SO(3)(3) and its extension to rigid-body transformations SE(3)(3) are treated in depth by Murray, Li, and Sastry [35], whose Chapter 2 develops the group-theoretic and differential-geometric foundations that underpin this chapter. Lynch and Park [30], Chapters 3–4, provide a complementary modern treatment with numerous worked examples of rotation matrices, homogeneous transformation matrices, and their application to serial kinematic chains.

The Lie group structure of SO(3)(3) and its Lie algebra $\mathfrak{so}(3)(3)$ are part of a much larger mathematical framework. Hall [19] offers a self-contained introduction to Lie groups and Lie algebras accessible to engineers with the linear algebra background developed in Chapter 1 of this text. For the geometric and symplectic perspective on rotation groups and their role in mechanics, Marsden and Ratiu [32] provide the definitive reference.

The Rodrigues rotation formula derived in this chapter has a distinguished history: it was first published by Olinde Rodrigues in 1840 [39], predating the modern matrix formalism by nearly a century. Rodrigues' original derivation used purely geometric reasoning; the matrix exponential form $e^{[\hat{\boldsymbol{\omega}}]_\times \theta}$ that we derive in Chapter 6 provides the modern algebraic counterpart.

Finally, readers implementing rotation representations in software should be aware that quaternion arithmetic is the *de facto* standard in modern robotics and game-engine frameworks (e.g., Eigen, ROS tf2, Unity) precisely because it avoids the gimbal-lock singularity and requires only four parameters with a single unit-norm constraint. The SLERP algorithm presented in this chapter is the core interpolation primitive in all of these libraries.

Exercises 4.1. Rotations, Quaternions, and SO(3)

4.15.1 Foundation Tier

Exercise. *Verify that* $\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$ *satisfies* $\mathbf{R}_z^T \mathbf{R}_z = \mathbf{I}$ *and* $\det(\mathbf{R}_z) = 1$. *Interpret each column as a rotated basis vector.*

Exercise. *Show that the composition of two rotations* $\mathbf{R}_1$ *and* $\mathbf{R}_2$ *(applied as* $\mathbf{R}_1\mathbf{R}_2\mathbf{v}$*) is itself a rotation by verifying closure in* SO(3)(3).

Exercise. *Compute the full ZYX Euler angle rotation matrix for* $\phi = 45°$, $\theta = 30°$, $\psi = 60°$ *by multiplying* $\mathbf{R}_x(\phi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi)$. *Verify orthogonality numerically.*

Exercise. *At* $\theta = 90°$ *in ZYX Euler angles, show that* $\dot{\phi}$ *and* $\dot{\psi}$ *are indistinguishable in their effect on the body frame by examining which rows of the Jacobian become parallel.*

Exercise. *For* $\boldsymbol{\omega} = [1, 2, 3]^T$, *construct the skew-symmetric matrix* $[\boldsymbol{\omega}]_\times$ *and verify that it is skew-symmetric by checking* $[\boldsymbol{\omega}]_\times^T = -[\boldsymbol{\omega}]_\times$.

4.15.2 Intermediate Tier

Exercise. *Compute the Hamilton product* $\mathbf{q}_1 \otimes \mathbf{q}_2$ *for* $\mathbf{q}_1 = [0.7071, 0.7071, 0, 0]^T$ *and* $\mathbf{q}_2 = [0.7071, 0, 0.7071, 0]^T$ *by hand and verify that the result is unit norm.*

Exercise. *Show that* $\mathbf{q} = [0.7071, 0.7071, 0, 0]^T$ *and* $-\mathbf{q} = [-0.7071, -0.7071, 0, 0]^T$ *represent the same rotation by computing the axis and angle for each and verifying they are identical (modulo* 2π*).*

Exercise. *Convert the quaternion* $\mathbf{q} = [\cos(45°/2), \sin(45°/2), 0, 0]^T$ *(rotation about the* x*-axis by* $45°$*) to a rotation matrix using equation* (4.40) *and verify it matches* $\mathbf{R}_x(45°)$.

Exercise. *Interpolate between* $\mathbf{q}_0 = [1, 0, 0, 0]^T$ *(identity) and* $\mathbf{q}_1 = [\cos(45°/2), \sin(45°/2), 0, 0]^T$ *at* $t = 0.5$ *using SLERP and verify the result represents a* $22.5°$ *rotation about the* x*-axis.*

Exercise. *If* $\mathbf{R}(t) = \mathbf{R}_z(t)$ *(rotation about* z*-axis with unit angular velocity), compute* $\dot{\mathbf{R}}$ *and verify that* $[\boldsymbol{\omega}]_\times = \mathbf{R}^T\dot{\mathbf{R}}$ *with* $\boldsymbol{\omega} = [0, 0, 1]^T$.

Exercise. *From the rotation matrix* $\mathbf{R}_x(60°)$*, extract the rotation axis (should be* $[1, 0, 0]^T$*) and angle (should be* $60°$*) using the eigenvector method and trace*$(\mathbf{R})$ *formula.*

4.15.3 Advanced Tier

Exercise. *Prove rigorously that* $\mathrm{SO}(3)(3)$ *is a 3-dimensional manifold by counting independent constraints and showing a 3-parameter local chart. Consider both the orthogonality constraints and determinant condition.*

Exercise. *Show that the set of pure quaternions* $\{\mathbf{q} : q_w = 0\}$ *forms a 3-dimensional vector space that is isomorphic to* $\mathfrak{so}(3)(3)$*. Under what operation does this isomorphism hold?*

Exercise. *For small* t*, the matrix exponential provides* $\mathbf{R}(t) = e^{t[\boldsymbol{\omega}]_\times}$ *where* $\boldsymbol{\omega} = [0, 0, 1]^T$ *(unit rotation about* z*). Expand the series* $e^{t[\boldsymbol{\omega}]_\times} = \mathbf{I} + t[\boldsymbol{\omega}]_\times + \frac{t^2}{2}[\boldsymbol{\omega}]_\times^2 + \cdots$ *and verify it matches* $\mathbf{R}_z(t)$ *to second order in* t.

Exercise. *Derive the Rodrigues rotation formula* $\mathbf{R} = \mathbf{I} + \sin\theta\,[\hat{\boldsymbol{\omega}}]_\times + (1 - \cos\theta)\,[\hat{\boldsymbol{\omega}}]_\times^2$ *by expanding the rotation of an arbitrary vector about an axis, using the decomposition* $\mathbf{v} = (\hat{\boldsymbol{\omega}} \cdot \mathbf{v})\hat{\boldsymbol{\omega}} + \mathbf{v}_\perp$.

Exercise. *Prove that SLERP produces constant-speed interpolation: the quaternion at parameter* t *moves along the geodesic at constant angular velocity. Hint: compute the derivative* $\frac{d}{dt}$*SLERP*$(\mathbf{q}_0, \mathbf{q}_1; t)$ *and show its norm is constant.*

Chapter 5

Twists, Wrenches, and the Screw Axis

One rigid displacement can be described by a screw. A moving body generally requires a changing sequence of instantaneous screws. The distinction lets us describe a golf club precisely without assuming that its entire swing follows one fixed axis.

The purpose of this chapter is to connect three questions: where is the body, how is every point on it moving, and how do applied loads do work? A pose answers the first question, a twist the second, and a wrench the third. Their compact notation is useful because it preserves these physical relationships under a change of coordinates. It does not remove the need to specify a body, a reference point, an observer, or a force application point.

We use angular-first twists $V = (\omega, v)^T$ and moment-first wrenches $F = (n, f)^T$. Here ω is angular velocity, v is the linear coordinate of a rigid velocity field, n is moment about the specified origin, and f is resultant force. Angular velocity is measured in rad/s, linear velocity in m/s, moment in N m and force in N. Radians are dimensionless in SI, but retaining rad in pitch units helps distinguish metres per radian from metres per revolution. A six-vector therefore combines components with different physical units; its ordinary Euclidean norm is not automatically a meaningful mechanical magnitude.

5.1 Historical Context: From Chasles to Modern Screw Theory

Classical screw theory describes rotations with accompanying translations, and forces with accompanying moments. Ball's treatise [3] is a historical reference; modern robotics texts express related constructions using the Lie group of rigid transformations and its tangent algebra [35, 30]. Historical priority is not needed for the derivations below. In particular, a finite-displacement theorem must not be used as if it established a fixed axis for an arbitrary time-dependent motion.

Conventions differ across those references. Murray, Li, and Sastry use linear-first twist coordinates (v, ω) and force-first wrench coordinates (f, n). Lynch and Park use the angular-first and moment-first ordering adopted here. Featherstone's spatial motion and force vectors also put angular velocity and moment first [13]. These are ordering choices, not different mechanics. If P swaps the two three-component blocks, then $V' = PV$, $F' = PF$ and $F'^T V' = F^T V$; a matrix representing a motion map changes to PXP^T. Copying a formula while keeping the wrong vector order changes its meaning.

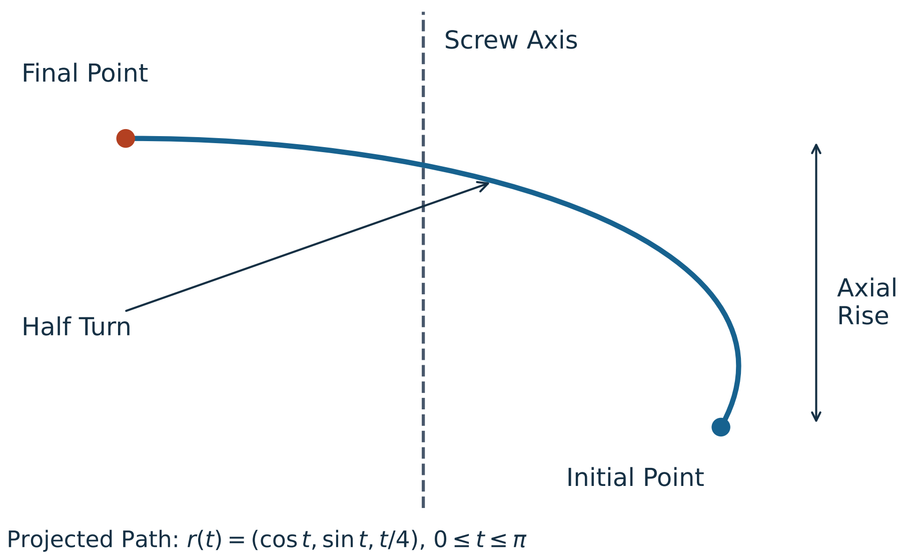


Figure 5.1: A Constant-Axis Screw Path in Projection. The Point Rotates Through Half a Turn While Rising Along the Axis; the Endpoint Theorem Does Not Require This Particular Intervening Path.

5.2 The Unified Four-by-Four Transform SE(3)

Let T_{AB} map coordinates expressed in frame B to coordinates expressed in frame A. Frame B's origin has coordinates p_{AB} in A, and R_{AB} rotates B components into A components. For a point and a free direction, respectively,

$$x_A = R_{AB}x_B + p_{AB}, \qquad d_A = R_{AB}d_B. \tag{5.1}$$

Translation is affine on three-dimensional point coordinates: it does not preserve the zero vector. No principle of linear algebra is violated. Homogeneous coordinates embed points with weight one and directions with weight zero:

$$T_{AB} = \begin{bmatrix} R_{AB} & p_{AB} \\ 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} x_A \\ 1 \end{bmatrix} = T_{AB} \begin{bmatrix} x_B \\ 1 \end{bmatrix}, \qquad \begin{bmatrix} d_A \\ 0 \end{bmatrix} = T_{AB} \begin{bmatrix} d_B \\ 0 \end{bmatrix}. \tag{5.2}$$

The set $SE(3)$ consists of these matrices with $R^T R = I$ and $\det R = 1$. Reflections, arbitrary scale changes and finite Euler approximations to rotations are not generally in this set. Separate arrays for rotation and translation are equally legitimate implementations; a homogeneous matrix is a representation, not a requirement imposed on every physics engine.

5.2.1 Group Structure of SE(3)

Following the frame subscripts gives composition and inversion:

$$T_{AC} = T_{AB}T_{BC}, \qquad T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}. \tag{5.3}$$

The product has rotation R_1R_2 and translation $p_1 + R_1p_2$, so it remains a proper rigid transform. Identity and inverses belong to the set, and associativity follows from matrix multiplication. The parameters form a smooth six-dimensional manifold: three local rotational coordinates and three translations. Multiplication and inversion are smooth, making $SE(3)$ a Lie group. Six-dimensional does not mean that one nonsingular three-angle chart covers all rotations; rotation charts still have their familiar limitations.

Composition order matters. Rotating a club and then translating it by a vector fixed in the laboratory is generally a different operation from translating it and then rotating about the laboratory origin. Subscripts and the point-mapping equation are safer guides than an unexplained instruction to "premultiply" or "postmultiply."

5.3 Chasles' Theorem and the Screw Axis

Consider a finite displacement $x' = Rx + p$ in one common spatial coordinate system. If R is not identity, this displacement can be realized as a rotation about a line, followed by a translation parallel to that line. Choose a unit direction s and a principal angle $0 < \theta \leq \pi$ with $R = \exp([s]_\times \theta)$. There are a point q on the line and an axial displacement d such that

$$x' = q + R(x - q) + ds, \qquad p = (I - R)q + ds. \tag{5.4}$$

Every point $q + \lambda s$ on the axis moves to $q + (\lambda + d)s$. Other points have a rotational displacement around the line as well as the axial displacement. The signed finite pitch is $h = d/\theta$, not d and not the total distance travelled by an off-axis point divided by angle.

This is an endpoint statement. If a tracked body has poses T_0 and T_1 relative to the same laboratory frame, the spatial displacement mapping each initial material-point position to its final position is $\Delta T = T_1 T_0^{-1}$. Decomposing T_1 alone instead describes a mapping from the chosen body reference coordinates. It generally gives a different axis. The corresponding body-coordinate relative transform is $T_0^{-1} T_1$; its coordinates must be interpreted accordingly.

5.3.1 Geometric Description and Exceptional Cases

The axis line is $\{q + \lambda s : \lambda \in \mathbb{R}\}$. Replacing q by $q + \lambda s$ does not change the line. We normally choose its closest point to the origin, $q \cdot s = 0$. For nonidentity R, this geometric line is unique. Its oriented parameterization has branch qualifications: at a half turn, s and $-s$ represent the same rotation, while the signed axial coordinate and pitch change with that choice. Additional full turns also produce alternative angle/pitch descriptions of the same endpoints.

For pure translation, $R = I$ and $p \neq 0$, every line parallel to p can serve as a translational axis. There is no unique finite rotational axis or finite pitch obtained by dividing by a zero angle. Calling this "infinite pitch" is a limiting convention, not an instruction to divide by zero. Identity displacement has neither a preferred axis nor a required nonzero motion. An object can even execute a nontrivial closed path and return to identity displacement: its endpoints alone do not reconstruct that path.

A steadily advancing ideal screw provides a useful constant-axis example. A tumbling football or precessing frisbee generally does not. Each instantaneous rigid velocity field and each nontrivial finite relative pose has its own appropriate screw description; those descriptions need not coincide over a measured time interval. This distinction is especially relevant when a changing swing plane is inferred from a sequence of club poses.

5.4 The Spatial Twist

Let $T(t)$ give the moving body's pose in a fixed laboratory frame. Differentiating T produces a tangent matrix at T, not directly a six-vector at identity. Two useful representations are

$$\widehat{V}_s = \dot{T} T^{-1}, \qquad \widehat{V}_b = T^{-1} \dot{T}, \qquad \widehat{V} = \begin{bmatrix} [\omega]_\times & v \\ 0 & 0 \end{bmatrix}. \tag{5.5}$$

The subscripts denote spatial and body representations of the same body motion relative to the laboratory. With $T = (R, p)$, direct multiplication gives

$$[\omega_s]_\times = \dot{R} R^T, \quad v_s = \dot{p} - \omega_s \times p, \qquad [\omega_b]_\times = R^T \dot{R}, \quad v_b = R^T \dot{p}. \tag{5.6}$$

In particular, v_s is not generally the velocity $\dot{p}$ of the body-frame origin. It is the velocity assigned by the body's rigid velocity field to the point located at the spatial origin. That point need not lie inside the physical body. The spatial field evaluated at any material point with current position r is

$$u(r) = v_s + \omega_s \times r. \tag{5.7}$$

For $r = p + R x_b$ with fixed material coordinates x_b, this equals $\dot{p} + \dot{R} x_b$, as it must. This identity is an immediate way to check a twist convention against measured marker velocities.

5.4.1 Screw Representation of a Twist

When $\omega \neq 0$, set $s = \omega/\|\omega\|$, choose rate $\dot{\theta} = \|\omega\|$, and define

$$h = \frac{\omega \cdot v}{\|\omega\|^2}, \qquad q = \frac{\omega \times v}{\|\omega\|^2}, \qquad V = S\dot{\theta}, \quad S = \begin{bmatrix} s \\ q \times s + hs \end{bmatrix}. \tag{5.8}$$

The vector identity $\omega \times (\omega \times v) = \omega(\omega \cdot v) - \|\omega\|^2 v$ verifies the formula. Every point on this instantaneous axis has velocity $h\omega$; off-axis points have an additional rotational contribution. Thus pitch uses the axial projection of velocity, not $\|v\|/\|\omega\|$ and not clubhead speed divided by angular speed. With an independently chosen oriented joint axis, $\dot{\theta}$ may instead be signed.

The phrase "unit screw" here means $\|s\| = 1$ in the angular block for a rotational joint. It does not mean $\|S\| = 1$ under an arbitrary mixed-unit six-dimensional norm. For pure translation, $\omega = 0$ and $v \neq 0$, one can use $S = (0, s)$ with $\|s\| = 1$ and a translational joint coordinate in metres. The zero twist does not determine an instantaneous axis.

5.4.2 Twists for Standard Joint Types

A revolute joint through q in direction s has $S = (s, q \times s)$ and zero pitch. The linear block is not zero merely because points on the axis are instantaneously stationary: their complete velocity is $q \times s + s \times q = 0$ for unit angular rate. A prismatic joint has $S = (0, s)$, and a helical joint has $S = (s, q \times s + hs)$. These are relative joint twists. In a moving mechanism, an absolute link twist also includes the predecessor's motion, expressed in a common frame.

For an axis through $(1, 0, 0)$ parallel to z, take $h = 2$ m/rad and rate 0.5 rad/s. The twist is $V = (0, 0, 0.5, 0, -0.5, 1)^T$. At the axis point, $u(q) = (0, 0, 1)$ m/s; at the origin, $u(0) = (0, -0.5, 1)$ m/s. Both velocities belong to the same rigid field. A zero-pitch revolute joint at that location would retain the -0.5 linear component but have zero velocity at q.

5.4.3 Matrix Representation and Finite Integration

The matrices $\widehat{V}$ form the Lie algebra $\mathfrak{se}(3)$. Only the upper-left rotational block is skew-symmetric; the full four-by-four matrix is generally not. Differentiating $R^T R = I$ explains that skew-symmetry. The translation column is unrestricted and the last row is zero, leaving six independent coordinates.

For a constant spatial twist, $\dot{T} = \widehat{V}_s T$ integrates to $T(t) = \exp(\widehat{V}_s t)T(0)$. For a constant body twist, $\dot{T} = T\widehat{V}_b$ integrates to $T(t) = T(0)\exp(\widehat{V}_b t)$. For a general time-dependent twist, successive increments require their correct order. Replacing them by the exponential of the simple time integral is justified only under additional commutation conditions. A single fitted endpoint screw does not show that the intervening twist was constant.

5.4.4 The Lie Bracket of Twists

The matrix commutator induces the angular-first bracket

$$[V_1, V_2] = \begin{bmatrix} \omega_1 \times \omega_2 \\ \omega_1 \times v_2 - \omega_2 \times v_1 \end{bmatrix}. \tag{5.9}$$

It describes the leading second-order effect of changing the order of small transformations. With the stated matrix order,

$$e^{\epsilon \widehat{V}_1} e^{\epsilon \widehat{V}_2} e^{-\epsilon \widehat{V}_1} e^{-\epsilon \widehat{V}_2} = I + \epsilon^2 [\widehat{V}_1, \widehat{V}_2] + O(\epsilon^3). \tag{5.10}$$

The bracket is not simply the relative velocity $V_1 - V_2$. Its role becomes important when rotations at different joints or times do not commute. It provides a local geometric description, not by itself a prediction of a finite golf-swing outcome.

5.5 The Adjoint Representation

5.5.1 Changing Reference Frames

Suppose A and B are two coordinate frames used to represent the same physical velocity field. At the instant of interest, $x_A = Rx_B + p$ with $T_{AB} = (R, p)$. Substituting the point-velocity fields gives

$$\omega_A = R\omega_B, \qquad v_A = Rv_B + p \times R\omega_B. \tag{5.11}$$

The angular-first adjoint is therefore

$$V_A = XV_B, \qquad X = \mathrm{Ad}_{T_{AB}} = \begin{bmatrix} R & 0 \\ [p]_\times R & R \end{bmatrix}. \tag{5.12}$$

Equivalently, $\widehat{V}_A = T_{AB}\widehat{V}_B T_{AB}^{-1}$. To check the sign of the offset term, evaluate $u_A(Rr_B + p)$: the $p \times R\omega_B$ term cancels the $R\omega_B \times p$ from point evaluation, leaving $Ru_B(r_B)$. Also $\mathrm{Ad}_{T_1 T_2} = \mathrm{Ad}_{T_1} \mathrm{Ad}_{T_2}$ by matrix conjugation. For the moving body's pose, $V_s = \mathrm{Ad}_T V_b$ follows from the two derivative definitions above.

Re-expression must be distinguished from changing the physical observer. If A and B themselves move relative to each other, differentiating point coordinates gives extra frame-motion terms:

$$\dot{x}_A = R\dot{x}_B + \dot{R}x_B + \dot{p}. \tag{5.13}$$

The adjoint re-expresses a specified relative motion; it does not automatically subtract an observer's own velocity. In particular, body representation V_b still describes body motion relative to the laboratory, even though the body's material coordinates are constant. A calculation of motion relative to another moving body must first specify that relative motion and then express it in the chosen frame.

5.6 The Spatial Wrench

A collection of forces f_i applied at positions r_i and free couples c_i, all expressed in one frame, has resultant wrench about that frame's origin

$$F = \begin{bmatrix} n \\ f \end{bmatrix}, \qquad f = \sum_i f_i, \quad n = \sum_i (r_i \times f_i + c_i). \tag{5.14}$$

A couple has a moment independent of the chosen origin; the moment of a nonzero resultant force generally changes with origin. Wrenches form the dual space $\mathfrak{se}(3)^*$ under the power pairing. A list of six components is not a reason to identify a wrench with a twist in $\mathfrak{se}(3)$. The distinction fixes the correct transformation law.

5.6.1 Wrenches From Forces and Moments

A force $(f_x, 0, 0)$ applied at $(0, 0, d)$ produces moment $(0, df_x, 0)$, with positive y sign for positive d and f_x. Moving the same force along its line of action leaves its moment unchanged. A force $(1, 0, 0)$ N at $(0, 2, 0)$ m instead gives $n = (0, 0, -2)$ N m. These elementary cross products are useful checks on any six-dimensional formula.

5.6.2 Wrench Transformation Under Changes of Frame

For the same B-to-A transform used for motion,

$$f_A = Rf_B, \qquad n_A = Rn_B + p \times Rf_B, \qquad F_A = X^{-T} F_B. \tag{5.15}$$

The reverse mapping is $F_B = X^T F_A$. Thus a transpose alone is correct when converting an A wrench to B, while X converts a B twist to A. Applying X and X^T in the same B-to-A direction is generally wrong. One can derive the force rule directly by writing $r_A = Rr_B + p$ and $f_A = Rf_B$ in $r_A \times f_A$, or derive it from $F_A^T V_A = F_B^T V_B$ for every V_B. Both derivations agree.

5.6.3 Physical Interpretation of Wrenches: Forces and Torques Unified

The wrench is a compact account of a body's resultant load, not a specification of how that load is distributed across skin, fingers, shafts or individual contacts. Different grip pressure distributions can have the same net wrench while producing different local stress or friction margins. Likewise, a wrench axis is not generally the instantaneous axis of the body on which it acts. Motion also depends on inertia, initial state and constraints.

For $f \neq 0$, the wrench's central axis and signed pitch are

$$q_F = \frac{f \times n}{\|f\|^2}, \qquad h_F = \frac{f \cdot n}{\|f\|^2}, \qquad n = q_F \times f + h_F f. \tag{5.16}$$

The cross-product identity verifies this decomposition. On that line the remaining moment is parallel to the resultant force. A pure force has $h_F = 0$, although its moment about a remote origin may be nonzero. A pure couple has $f = 0$ and no unique finite central line; an "infinite-pitch wrench" is again a convention. The zero wrench has no distinguished line. Although both pitches have dimensions of length when radians are suppressed, h_F and the motion pitch h describe different objects and need not be equal.

5.7 Proof of Chasles' Theorem

Here is a constructive proof. A proper rotation with $R \neq I$ has an invariant unit direction s and rotates its perpendicular plane by a nonzero angle modulo 2π. Resolve the displacement and define its pitch as

$$d = s^T p, \qquad p_\perp = p - ds, \qquad h = d/\theta. \tag{5.17}$$

Because $s^T(I - R) = 0$, any solution of $p = (I - R)q + ds$ must have this axial displacement. Restricted to $s^\perp$, the map $I - R$ is invertible. Choose $q \perp s$ and solve

$$(I - R)q = p_\perp, \qquad q = \frac{1}{2}p_\perp + \frac{1}{2}\cot(\theta/2)\, s \times p_\perp. \tag{5.18}$$

To verify the inverse, put $Jx = s \times x$ on the perpendicular plane, where $J^2 = -I$. Rodrigues' formula there is $R = \cos\theta I + \sin\theta J$. Multiplying $[(1-\cos\theta)I - \sin\theta J]$ by $\frac{1}{2}[I + \cot(\theta/2)J]$ gives I. This proves the displayed expression, including its cross-product sign. There is no additional cross product outside the restricted inverse.

The resulting q satisfies $p = (I-R)q + ds$, proving the screw representation. Adding any multiple of s gives the same axis line; no other perpendicular point solves the equation. At $\theta = \pi$, the cotangent vanishes and $q = p_\perp/2$, while the direction-sign qualification remains. As $\theta \to 0$, the inverse has scale $1/|\theta|$. A small measured perpendicular translation error can therefore produce a large axis-location error. A numerically reported distant axis near pure translation need not indicate a meaningful anatomical pivot.

For a rotation by $\pi/2$ about the vertical line through $(1,0,0)$ followed by 0.2 m of axial translation, $R = R_z(\pi/2)$ and $p = (1,-1,0.2)$. The formulas give $q = (1,0,0)$, $d = 0.2$ m and $h = 0.4/\pi \approx 0.12732$ m/rad. Substituting a point on and off the axis verifies both endpoints. This finite pitch does not determine elapsed time or the work required to realize the displacement.

5.8 Mechanical Work and the Virtual Power Principle

For loads acting on one rigid body, substitute the velocity field into the sum of physical powers:

$$P = \sum_i \{f_i \cdot [v + \omega \times r_i] + c_i \cdot \omega\} = f \cdot v + n \cdot \omega = F^T V. \tag{5.19}$$

The scalar triple-product identity converts $f_i \cdot (\omega \times r_i)$ into $(r_i \times f_i) \cdot \omega$. This derivation explains why force and moment must share a reference convention with the twist. Evaluating the linear part at the center of mass while leaving the moment about the grip would omit or double-count a term.

For an admissible virtual generalized velocity $\delta\dot{q}$, suppose the relevant relative twist is $J\delta\dot{q}$. Then virtual power is $F^T J \delta\dot{q} = (J^T F)^T \delta\dot{q}$, so the corresponding generalized force is $J^T F$. This is a kinematic duality statement; a dynamic acceleration calculation additionally requires mass, bias terms and constraints. Work over a finite interval is $\int F(t)^T V(t)\,dt$. A single instantaneous positive, negative or zero power does not determine that integral.

5.8.1 Invariance of Power and Its Limits

For a coordinate re-expression of the same motion and load, $F_A^T V_A = (X^{-T} F_B)^T X V_B = F_B^T V_B$. This is a scalar representation identity, not conservation of energy. Under a Galilean change to an observer translating with constant velocity U, point velocities become $u_i' = u_i - U$ and the applied-force power becomes $P' = P - f \cdot U$. A rotating observer introduces its own rotational terms. Energy balances must include the chosen observer and system boundary consistently; numerical equality under an adjoint change does not erase those physical choices.

5.9 Reciprocal Screws and Constraint Analysis

A wrench F is reciprocal to a twist V when $F^T V = 0$. For a subspace of admissible relative twists $\mathcal{T}$, ideal reaction wrenches lie in its annihilator: every F in that set satisfies $F^T V = 0$

for every $V \in \mathcal{T}$. If $\mathcal{T}$ has dimension d within the six-dimensional relative twist space, its annihilator has dimension $6-d$. This is a local linear statement at a specified configuration, not an automatic count of whole-mechanism mobility.

For a spatial revolute joint through q with direction s, the condition is

$$s \cdot (n - q \times f) = 0. \tag{5.20}$$

It excludes a reaction moment along the free rotational direction when moments are taken about the joint point. A motor or friction torque in that direction can perform relative work and is not part of the ideal reaction subspace. For a frictionless prismatic joint, $s \cdot f = 0$ excludes reaction force along the sliding direction; other spatial reaction components remain possible.

5.9.1 Two Geometric Screws and the Reciprocal Product

When both screws are represented in the same motion-style ordering, the corresponding reciprocal bilinear form is the cross-block expression

$$B(S_1, S_2) = \omega_1 \cdot v_2 + v_1 \cdot \omega_2. \tag{5.21}$$

This is not their ordinary Euclidean six-vector dot product. A wrench along the geometric screw encoded as (ω_2, v_2) has its force and moment roles interchanged, yielding moment-first coordinates proportional to (v_2, ω_2). Screw magnitudes and units must be supplied to interpret the resulting pairing as physical power. The geometric reciprocal product alone is not the power of two velocities.

For zero-pitch normalized axes with directions s_1, s_2 through q_1, q_2, substitution gives

$$B = (q_1 - q_2) \cdot (s_1 \times s_2). \tag{5.22}$$

Thus intersecting, parallel and coincident zero-pitch axes are reciprocal in this geometric sense; perpendicularity is not the defining condition. For finite pitches, add $(h_1 + h_2) s_1 \cdot s_2$. Constraint analysis must use the correct pairing and actual admissible motions, rather than a visual claim that the axes look orthogonal.

5.10 Physical Interpretation of Wrenches

5.10.1 Three Fundamental Wrench Types

A pure couple, a force on a line, and a force plus a parallel couple cover the central-axis cases above. For example, with gravity vector $g = (0, 0, -g_0)$, mass m, and center-of-mass position $r_C = (a, b, c)$, the gravity wrench about the origin is

$$F_g = \begin{bmatrix} -mg_0 b \\ mg_0 a \\ 0 \\ 0 \\ 0 \\ -mg_0 \end{bmatrix}. \tag{5.23}$$

Its moment vanishes when taken about the center of mass, but generally not about a grip or a joint. Summing gravity wrenches for separate bodies requires a common origin and frame, and summing them does not replace each body's dynamic balance.

5.10.2 The Wrench–Twist Duality at a Golf Grip

For hand forces f_i and couples c_i acting on the club at grip locations r_i, the club-side hand power is

$$P_{\text{hands}\to\text{club}} = \sum_i [f_i \cdot u_{\text{club}}(r_i) + c_i \cdot \omega_{\text{club}}]. \tag{5.24}$$

The same value follows from summing the hand wrenches about any one chosen origin and pairing with the club twist represented there. This is useful when comparing a clubhead-speed narrative with measured grip loads: an angular moment term and a translational force term must be interpreted together. A change in their separate numerical values after moving the origin is not evidence of a different physical energy-transfer mechanism.

If the grip is rigid and no slip occurs, equal-and-opposite interface wrenches have zero total interface power on the combined hand–club system, because their relative twist is zero. Nevertheless either subsystem can receive nonzero power from the other. With compliance or slip, the summed interface power depends on the relative motion and may represent elastic storage or dissipation. The sign must be tied to which body's load and relative velocity are used. A measured net club wrench cannot uniquely identify each hand's contribution or a neural control objective. Those claims require additional measurements and a specified contact and musculoskeletal model.

5.10.3 Transforming Wrenches Between Frames

A practical audit therefore records the sensor orientation, the point about which its moment is reported, the direction of the reported action, and the body-relative motion being paired with it. Transform all simultaneous loads to one frame and origin before adding them. The inverse-transpose rule then does the bookkeeping faithfully. It does not infer an unmeasured grip offset, calibrate a sensor, or separate muscle work from tendon storage.

5.11 Wrench Analysis and Force Polytopes

An ideal bilateral reaction subspace is usually unbounded. A unilateral frictionless point contact instead permits $f = \lambda n_c$ with $\lambda \geq 0$ along a contact normal. With isotropic Coulomb friction, a common admissibility model is $f_n \geq 0$ and $\|f_t\| \leq \mu f_n$. This is a cone; its circular boundary is not a finite-sided polyhedral cone unless approximated. Sliding friction additionally constrains force direction relative to slip velocity. A bounded actuator/contact model can yield a convex polytope, but bounds must actually be supplied. These sets should not be conflated under the word "polytope."

If admissible contact variables λ produce a resultant wrench $G\lambda$, static support of one free body requires $G\lambda + F_{\text{ext}} = 0$. The support set must contain $-F_{\text{ext}}$, not merely the external load with unchanged sign. Linkages additionally require the internal action–reaction relationships and equilibrium of every link. For moving bodies, a nonzero inertial wrench replaces the zero right-hand side. Neither convexity nor a force polygon removes those balances.

For a concrete bounded example, two vertical bilateral force actuators act on one planar rigid bar at $(0, 0)$ and $(L, 0)$, with $|u_1|, |u_2| \leq F_{\max}$. In coordinates (n_z, f_y),

$$(n_z, f_y) = (Lu_2, u_1 + u_2), \quad |n_z/L| \leq F_{\max}, \quad |f_y - n_z/L| \leq F_{\max}. \tag{5.25}$$

For $L = 1$ m and $F_{\max} = 10$ N, the vertices are $(-10, -20)$, $(10, 0)$, $(10, 20)$ and $(-10, 0)$, with moment in N m and force in N. A downward 15 N load at the midpoint requires $(n_z, f_y) = (7.5, 15)$ and is feasible with $u_1 = u_2 = 7.5$ N. A downward 25 N load there exceeds the total force capacity. If the two supports can only push upward, replace the bounds by $0 \le u_i \le F_{\max}$; the feasible set changes. This example concerns two external supports on one body, not two unspecified joints in an arbitrary mechanism.

5.12 Worked Example: Planar Four-Bar Statics

5.12.1 Setup

Use a deliberately idealized planar mechanism with ground points $A = (0, 0)$ and $D = (4, 0)$, moving joints $B = (1, 1)$ and $C = (3, 1)$, all coordinates in metres. Links AB, BC and CD are rigid and massless; all pins are frictionless. A downward load $(0, -W)$ acts at $E = (x_E, 1)$ on the coupler BC. Ground AD is fixed. A motor at A may apply a counterclockwise torque τ_A to AB; B, C and D are passive pins. The numerical example uses $W = 100$ N. Link weight, out-of-plane loading, friction, compliance and acceleration are omitted by assumption.

5.12.2 Twist Space of Each Joint

A planar pin at $q = (q_x, q_y)$ permits the relative angular-first twist

$$S_q = (0, 0, 1, q_y, -q_x, 0)^T. \tag{5.26}$$

Its reduced planar coordinates are $(\omega_z, v_x, v_y) = (1, q_y, -q_x)$. Pairing with reduced wrench (n_z, f_x, f_y) gives

$$n_z + q_y f_x - q_x f_y = 0, \qquad n_z = q_x f_y - q_y f_x. \tag{5.27}$$

Thus an ideal planar pin transmits arbitrary in-plane force but no independent couple about the pin. Force becomes directed along a link only if that link is an unloaded two-force member. CD has that property here. AB has it only when $\tau_A = 0$. The planar model does not prove that a physical spatial bearing lacks out-of-plane reactions; it has excluded those components from its idealization.

5.12.3 Constraint Wrenches and Separate Free Bodies

Let $b = (b_x, b_y)$ and $c = (c_x, c_y)$ be forces on the coupler at B and C. The forces on the side links at those joints are $-b$ and $-c$. Since each side link has no other force, the ground reactions on AB and CD are b at A and c at D. Moment balance for CD about D and for BC about B gives

$$c_x + c_y = 0, \qquad 2c_y - (x_E - 1)W = 0. \tag{5.28}$$

Coupler force balance and AB moment balance then give

$$b + c + (0, -W) = 0, \qquad \tau_A = b_y - b_x. \tag{5.29}$$

The unit lever-arm factors in the last equation are one metre; the preceding factor two is the two-metre coupler span. These equations retain consistent force and moment units. They are balances of three different bodies, not a sum of all joint forces assigned to one coupler.

5.12.4 Equilibrium and the Role of Actuation

Solving gives

$$c_y = \frac{(x_E - 1)W}{2}, \quad c_x = -c_y, \quad b_x = c_y, \quad b_y = W - c_y, \quad \tau_A = W(2 - x_E). \tag{5.30}$$

Here numerical lengths are in metres and τ_A is in N m. For the centered load $x_E = 2$ m, $b = (50, 50)$ N, $c = (-50, 50)$ N and $\tau_A = 0$. Both side links are two-force members. Their lines of force meet at $(2, 2)$, on the external load's vertical line of action. The coupler's force polygon closes, and its moment balance is satisfied. This is passive equilibrium at the specified configuration, not a proof of stability against perturbations.

For $x_E = 2.5$ m, $b = (75, 25)$ N, $c = (-75, 75)$ N and $\tau_A = -50$ N m. AB is no longer a two-force member because of the motor couple. Removing that required motor torque leaves no static equilibrium for this load at this configuration under the stated assumptions. The external forces still sum to zero; that condition alone is insufficient. For the entire assembly, the moment balance about A is $4c_y + \tau_A - W x_E = 0$, which independently confirms the result.

5.12.5 Geometric Interpretation and Virtual Power

Set the instantaneous angular speed of AB to one rad/s. Then $u_B = (-1, 1)$ m/s. Constant length BC requires $u_{C,x} = u_{B,x}$, while CD's pin motion gives $u_C = (-1, -1)$ m/s. Consequently $\omega_{BC} = -1$ rad/s and $u_{E,y} = 2 - x_E$ m/s. Ideal pin reactions contribute zero net relative power, so static virtual power requires

$$\tau_A - W(2 - x_E) = 0. \tag{5.31}$$

This reproduces the free-body torque. At the centered load, the load point's admissible instantaneous motion is horizontal, so the vertical force does no virtual work. The mechanism still has an admissible motion; load reciprocity does not make it rigid. Near other configurations, the relevant constraint or actuation matrices may lose rank and their force solution may become nonunique or infeasible. Geometry, allowed inputs and the actual external load must all be specified before interpreting a mechanism's apparent force-transmission advantage.

5.13 Python Implementation

The companion implementation checks proper rigid transforms and uses angular-first vectors throughout. It exposes the transform direction in the wrench function name and computes the inverse-transpose action by solving a linear system. Its finite-axis routine accepts resolved nonzero principal rotations; it rejects angles at or below 10^{-8} rad rather than assigning a spurious unique axis to translation. That numerical threshold is an example contract, not a universal measurement-resolution standard. At a half turn the direction follows the rotation library's branch choice, while the recovered geometric line remains the same.

Run the following from the repository root with NumPy and SciPy installed. The implementation is in `src/tools/screw_examples.py`; it reuses the common spatial cross-product helper. The four-bar solver is for the explicitly specified fixed geometry above, not a general mechanism or golf model.

```python
import numpy as np
from scipy.spatial.transform import Rotation
from src.tools.screw_examples import (
    adjoint, finite_screw, four_bar_equilibrium,
    reexpress_wrench,
)

T = np.eye(4)  # Coordinates B -> A
T[:3, :3] = Rotation.from_euler("z", np.pi / 2).as_matrix()
T[:3, 3] = [1, -1, 0.2]
V_B = np.array([0, 0, 0.5, 0, -0.5, 1.0])
F_B = np.array([0, 0, -2, 1, 0, 0.0])
V_A = adjoint(T) @ V_B
F_A = reexpress_wrench(T, F_B)
print(F_A @ V_A - F_B @ V_B)

# Here T is also a specified finite displacement in one frame.
screw = finite_screw(T)
print(screw.point, screw.pitch)
result = four_bar_equilibrium(load_x=2.5, downward_force=100)
print(result.force_b, result.force_c, result.torque_a)
```

The example returns zero power difference to floating-point accuracy, the finite axis $q = (1, 0, 0)$ with pitch approximately 0.12732 m/rad, and motor torque -50 N m for the offset load. The test suite independently compares transformed twists with Cartesian point velocities and transformed wrenches with force moments. It also reconstructs finite point displacements, exercises the half-turn and translation cases, and checks all three link balances and virtual power. These checks validate the displayed identities under their assumptions; they do not validate a biological swing model or remove calibration error from motion-capture data.

5.14 Summary

A pose, a twist and a wrench answer different mechanical questions. Homogeneous transforms compose poses; twists describe the rigid velocity field; wrenches collect resultant force and moment and act on twists through power. Finite screw decomposition describes endpoints, while a time-dependent trajectory generally has a changing instantaneous axis. Near pure translation, axis estimation becomes ill-conditioned even when the measured displacement is small and well defined.

For golf analysis, the useful outcome is a consistent chain from measured club poses to point velocities, from measured hand loads to moments about a specified point, and from their pairing to subsystem power and finite-interval work. Reciprocal reactions and actuator loads play different roles in that chain. Compact notation helps reveal those roles, but conclusions about anatomical pivots, force sharing, energy storage or neural objectives require the additional models and evidence appropriate to each claim.

5.15 Further Reading

Murray, Li, and Sastry [35], Chapter 2, develops exponential rigid displacement, spatial/body velocity, central wrench axes and reciprocal screws. Its linear-first ordering must be converted before using its block matrices here. Lynch and Park [30], Sections 3.3.2 and

3.4, provide the angular-first twist and moment-first wrench conventions used in this chapter. The accompanying explanations of body and space representations are especially useful for distinguishing a twist coordinate from a material point's velocity.

Featherstone [13] treats spatial motion/force transformations and multibody dynamics. His published spatial-vector documentation explicitly distinguishes a coordinate transformation from a displacement operator and gives inverse-transpose force transformation. Ball [3] supplies historical background to geometric screw systems. These references support the mathematical framework; the present fixed-geometry linkage calculations and golf interpretation have their own explicitly stated assumptions. No inference about a golfer's control strategy follows solely from a screw theorem.

5.16 Exercises

Unless stated otherwise, use right-handed Cartesian frames, angular-first twists, moment-first wrenches, metres, seconds and newtons. A transform T_{AB} maps B coordinates into A coordinates. Show reference points and signs, not just six numerical entries.

1. **SE(3) Inverse Computation:** Let $R = R_z(\pi/2)$ and $p = (1, 2, 3)$. Compute T_{AB}^{-1} and verify both products equal identity. Check one mapped point and one free direction to explain the different homogeneous weights.

2. **Twist From Screw Parameters:** For $q = (1, 0, 0)$, $s = (0, 0, 1)$, $h = 2$ m/rad and $\dot{\theta} = 0.5$ rad/s, compute V. Evaluate the physical velocity at q and at the origin, explaining why the linear block need not equal the axis-point velocity.

3. **Power Calculation:** For $F = (1, 0, 0, 2, 0, 0)^T$ and $V = (0, 0, 1, 0, 1, 0)^T$, compute $F^T V$ with appropriate component units. Explain why the result does not imply that either the force or motion vanishes, or that the finite-interval work must vanish.

4. **Adjoint Transformation:** Treat the twist in Exercise 2 as V_B and use T_{AB} from Exercise 1. Construct $\mathrm{Ad}_{T_{AB}}$, compute V_A, and verify the velocity at a transformed axis point directly.

5. **Reciprocal Wrench:** For the relative revolute twist $S = (0, 0, 1, 1, 0, 0)^T$, identify a point on its axis and characterize all moment-first wrenches with $F^T S = 0$. Give a nonzero example and explain the absence of an independent moment along the free axis at the joint point.

6. **Bounded Support Set:** Two vertical bilateral actuators act on one planar rigid bar at $(0, 0)$ and $(1, 0)$, each bounded by 10 N in either direction. Derive and sketch the feasible set in (n_z, f_y) about the origin. Test downward midpoint loads of 15 N and 25 N. Explain how unilateral upward-only supports change the set.

7. **Frame-Dependent Twist:** A body rotates about the laboratory x axis at one rad/s, so $V_A = (1, 0, 0, 0, 0, 0)^T$. Frame B has parallel axes and origin $p_{AB} = (0, 1, 0)$ at this instant. Compute the B representation of that same motion using the inverse adjoint, and verify its linear block by evaluating the velocity field at B's origin. State why this is not automatically motion relative to an independently moving observer B.

8. **Pitch Extraction:** Construct the normalized rotational screw for $s = (0, 0, 1)$, $q = (1, 1, 0)$ and $h = 3$ m/rad. Recover its pitch by axial projection. Compare that result with the norm of its linear block and explain the difference.

9. **Planar Mechanism Force Path:** Reproduce the centered and offset load solutions of the four-bar example, including each link's free-body balance and the required motor torque. State which assumptions justify the two-force-member simplification and which exclude out-of-plane components. Do not infer spatial bearing reactions from a planar model.

10. **Helical Joint Twist:** A helical joint through the origin has direction $s = (0, 1, 0)$, pitch 0.1 m/rad and rate 2 rad/s. Compute its twist, the axis-point velocity, and its axial travel per complete revolution. Distinguish pitch from lead per revolution.

11. **Dual Wrench Transformation:** Starting with $V_A = XV_B$, derive $F_A = X^{-T} F_B$ from power equality for every V_B. Derive the reverse rule $F_B = X^T F_A$. Use a nonzero translation and a force with a moment arm to demonstrate why applying X^T in the B-to-A direction fails.

12. **Moment Arm Counterexample:** A force $(1, 0, 0)$ N acts at $(0, 2, 0)$ m. Compute the wrench and its power on a unit-rate positive z rotation about the origin. Explain why it is not an ideal reaction wrench of a free z hinge and what balancing actuator torque would be needed in static equilibrium.

13. **Screw Axis From a Generator:** Let $\omega = (0, 0, 0.1)$ rad/s and $v = (1, 0, 0)$ m/s. Extract the instantaneous axis and pitch. Compare the exact displacement $e^{\widehat{V}\epsilon}$ with $I + \widehat{V}\epsilon$ for a nonzero time step ϵ. Show why the latter generally fails rotation orthogonality and specify the order of its local truncation error.

14. **Reciprocal Screw Pair:** For $S_1 = (0, 0, 1, 1, 1, 0)^T$ and a nonzero prismatic screw $S_2 = (0, 0, 0, a, b, c)^T$, find the condition for geometric reciprocity using the cross-block form B. Normalize a valid translational direction. Compare the condition with the ordinary six-vector dot product and explain why that dot product answers a different question.

15. **Virtual Work Principle:** For an admissible twist subspace $V = J\delta\dot{q}$, prove that zero reaction power for every $\delta\dot{q}$ is equivalent to $J^T F = 0$. Contrast this with a wrench that happens to do zero power on only one measured velocity. Explain why an ideal rigid grip may transfer power to the club while doing zero net interface work on the combined hand–club system.

Chapter 6

Exponential Coordinates and Matrix Logarithms

To move smoothly through the universe, we do not jump from matrix to matrix. We integrate continuously along the exponent of a physical axis. The exponential map wraps velocity vectors onto the curved surface of rotation space; the logarithm unfolds curved poses back into flat, linear velocity directions.

In Plain Language 6.1. Context & Use Case: Smooth Interpolation and Geodesics

The Math You Will See: We will introduce **Lie Algebras** ($\mathfrak{so}(3)(3)$ and $\mathfrak{se}(3)(3)$), the **Matrix Exponential** (*exp*), the **Matrix Logarithm** (*log*), and Rodrigues' formula. We will develop exponential coordinates, the Baker-Campbell-Hausdorff formula, and understand geodesics (shortest paths) on rotation space.
Why We Need It: Imagine a robotic arm needs to move from Pose A to Pose B. You cannot just average the 4×4 matrices of Pose A and Pose B ($[\mathbf{T}_A + \mathbf{T}_B]/2$); the resulting matrix is garbage and physically means nothing. Because 3D space is curved (due to rotation), moving smoothly requires calculating the exact "velocity" of motion and sliding along it. The Logarithm extracts the direction and speed of motion, and the Exponential walks along the geodesic path.
All Variables Defined: $\mathfrak{so}(3)(3)$ represents the tangent space of infinitesimal rotations (Lie algebra), SE(3)(3) represents the curved position manifold (Lie group), $\hat{\boldsymbol{\omega}}$ is the unit rotation axis, and θ is the rotation angle.

6.1 Historical Context: Sophus Lie and the Theory of Continuous Groups

The modern theory of Lie groups emerged in the 1870s-1890s from the groundbreaking work of Norwegian mathematician **Sophus Lie**. While studying differential equations, Lie discovered that continuous families of symmetries—transformations that preserve the structure of an equation—form special algebraic objects called **continuous groups**. These are now known as Lie groups in his honor.

Lie's central insight was that every Lie group has a **Lie algebra**—an associated linear (flat) vector space at the identity element that completely encodes the local structure of the group. The exponential map serves as the bridge: it exponentiates elements of the Lie algebra to produce group elements. Conversely, the logarithm inverts this operation.

For the rotation group SO(3)(3), Lie's theory predicts that the Lie algebra $\mathfrak{so}(3)(3)$ should consist of 3×3 skew-symmetric matrices. The exponential map then converts these skew-symmetric matrices into rotation matrices—a fact that was rediscovered and formalized by **Olinde Rodrigues** in the context of the rotation formula.

In the early 20th century, the theory was extended and generalized. By the latter half of the century, computer scientists and roboticists (notably **Richard Murray**, **Zexiang Li**, and **S. Shankar Sastry** [35], as well as **John Hauser**) recognized that Lie group and Lie algebra techniques provide the natural language for parametrizing rigid body motions and solving differential equations in robotics. A rigorous modern treatment of Lie groups and their algebras can be found in Hall [19].

The exponential-logarithm formalism is now foundational to modern control theory, computer graphics (SLERP interpolation), and numerical algorithms for rigid body dynamics. Understanding it is essential for anyone working with continuous motion in space.

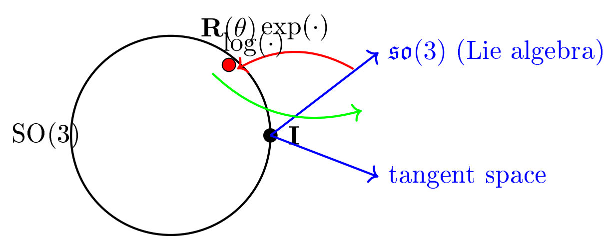


Figure 6.1: The exponential and logarithmic maps bridge the curved Lie group SO(3) and its flat tangent space (Lie algebra $\mathfrak{so}(3)$). The exponential map converts infinitesimal rotations to finite rotations; the logarithm extracts the rotation axis and angle from a rotation matrix.

6.2 Lie Algebras and the Tangent Space

Recall from Chapter 5 that SE(3)(3) and SO(3)(3) are Lie groups—they have both algebraic and smooth geometric structure. But they are curved, non-linear manifolds: you cannot simply add two rotation matrices and get a valid rotation matrix.

However, at each point on a Lie group, there exists a **tangent space**—the space of infinitesimal "velocity directions" at that point. For a Lie group, the tangent space at the identity element is particularly special: it inherits a vector space structure and can be identified with the Lie algebra.

Definition 6.1. The Lie Algebra so(3)

The **Lie algebra** $\mathfrak{so}(3)(3)$ is the set of all 3×3 skew-symmetric matrices:

$$\mathfrak{so}(3)(3) = \{\mathbf{A} \in \mathbb{R}^{3\times 3} : \mathbf{A}^T = -\mathbf{A}\} \tag{6.1}$$

Equivalently, $\mathfrak{so}(3)(3)$ can be identified with $\mathbb{R}^3$ by the isomorphism between vectors and their skew-symmetric matrix representations. A vector $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ corresponds to the matrix:

$$[\boldsymbol{\omega}] = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \tag{6.2}$$

The dimension of $\mathfrak{so}(3)(3)$ is 3 (the same as SO(3)(3)), and it is a **vector space**: sums and scalar multiples of skew-symmetric matrices are skew-symmetric.

Definition 6.2. The Lie Algebra se(3)

The **Lie algebra** $\mathfrak{se}(3)(3)$ is the set of 4×4 matrices of the form:

$$\mathfrak{se}(3)(3) = \left\{ \begin{bmatrix} \mathbf{A} & \mathbf{v} \\ 0 & 0 \end{bmatrix} : \mathbf{A} \in \mathfrak{so}(3)(3), \mathbf{v} \in \mathbb{R}^3 \right\} \tag{6.3}$$

Equivalently, $\mathfrak{se}(3)(3)$ can be identified with $\mathbb{R}^6$ by the isomorphism between 6D twist vectors $[\boldsymbol{\omega}^T, \mathbf{v}^T]^T$ and their matrix representation. The dimension is 6.

6.2.1 The Lie Bracket: Non-Commutativity of Motions

The Lie algebra structure includes a **Lie bracket** operation that measures the "failure to commute":

$$[\mathbf{A}, \mathbf{B}] = \mathbf{AB} - \mathbf{BA} \tag{6.4}$$

For skew-symmetric matrices $\mathbf{A} = [\boldsymbol{\omega}_1]$ and $\mathbf{B} = [\boldsymbol{\omega}_2]$, the bracket is itself skew-symmetric:

$$[[\boldsymbol{\omega}_1], [\boldsymbol{\omega}_2]] = [\boldsymbol{\omega}_1 \times \boldsymbol{\omega}_2] \tag{6.5}$$

Geometrically, the Lie bracket of two rotations is another rotation—a reflection of the fact that two sequential rotations about different axes can be rearranged in a non-commutative way.

6.3 Exponential Coordinates and Axis-Angle Representation

6.3.1 The Exponential Coordinate Vector

Let us begin by revisiting the concept of rotating around a single fixed unit axis $\hat{\boldsymbol{\omega}} \in \mathbb{R}^3$ (with $\|\hat{\boldsymbol{\omega}}\| = 1$) by a certain angle $\theta \in \mathbb{R}$ (measured in radians).

If we multiply the physical axis unit vector $\hat{\boldsymbol{\omega}}$ by the total angle θ, we obtain a new 3-dimensional vector:

$$\mathbf{r} = \hat{\boldsymbol{\omega}}\theta \in \mathbb{R}^3 \tag{6.6}$$

This incredibly compact vector $\mathbf{r}$ contains all the information needed to describe any 3D rotation: the direction of $\mathbf{r}$ specifies the axis of rotation, and the magnitude $\|\mathbf{r}\| = \theta$ specifies the angle. This representation is known as the **Exponential Coordinate** representation of a rotation, or the **axis-angle** representation.

In Plain Language 6.2. Why is it called "Exponential"?

In basic 1D calculus, the solution to the differential equation governing continuous exponential growth ($\dot{x} = ax$) is the scalar exponential function: $x(t) = e^{at}x(0)$. Similarly, the differential equation governing a rigid body continuously rotating around a fixed angular velocity matrix $[\hat{\boldsymbol{\omega}}]$ at speed $\dot{\theta}$ is:

$$\dot{\mathbf{R}} = [\hat{\boldsymbol{\omega}}]\dot{\theta}\mathbf{R} \tag{6.7}$$

The solution uses exactly the same mathematical structure, just upgraded for matrices: the **Matrix Exponential**:

$$\mathbf{R}(\theta) = e^{[\hat{\boldsymbol{\omega}}]\theta}\mathbf{R}(0) \tag{6.8}$$

The matrix exponential e^X is defined through its Taylor series (just like the scalar exponential):

$$e^X = \sum_{n=0}^{\infty} \frac{X^n}{n!} = I + X + \frac{X^2}{2!} + \frac{X^3}{3!} + \cdots \tag{6.9}$$

6.4 Rodrigues' Formula and the Matrix Exponential for SO(3)

Computing the matrix exponential $e^{[\hat{\boldsymbol{\omega}}]\theta}$ by summing the infinite Taylor series would be prohibitively slow for computer calculations. Fortunately, there is a closed-form formula.

Theorem 6.1. Rodrigues' Formula (Exponential Map on SO(3))

For any unit vector $\hat{\boldsymbol{\omega}} \in \mathbb{R}^3$ with $\|\hat{\boldsymbol{\omega}}\| = 1$ and angle $\theta \in [0, \pi]$, the matrix exponential is:

$$\mathbf{R} = \exp([\hat{\boldsymbol{\omega}}]\theta) = \mathbf{I} + \sin\theta\,[\hat{\boldsymbol{\omega}}] + (1 - \cos\theta)\,[\hat{\boldsymbol{\omega}}]^2 \tag{6.10}$$

Derivation of Rodrigues' Formula. We begin with the Taylor series expansion:

$$e^{[\hat{\boldsymbol{\omega}}]\theta} = \sum_{n=0}^{\infty} \frac{([\hat{\boldsymbol{\omega}}]\theta)^n}{n!} \tag{6.11}$$

The key insight is to find a pattern in the powers of $[\hat{\boldsymbol{\omega}}]$. Since $\hat{\boldsymbol{\omega}}$ is a unit vector, its skew-symmetric matrix satisfies a special property. Using the vector triple product identity, we can show:

$$[\hat{\boldsymbol{\omega}}]^2 = \hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I} \tag{6.12}$$

(This follows from the identity $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})$ applied to skew matrices.)
Crucially, we also have:

$$[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}] \tag{6.13}$$

This can be verified directly: $[\hat{\boldsymbol{\omega}}]^3 = [\hat{\boldsymbol{\omega}}] \cdot [\hat{\boldsymbol{\omega}}]^2 = [\hat{\boldsymbol{\omega}}](\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}) = \mathbf{0} - [\hat{\boldsymbol{\omega}}] = -[\hat{\boldsymbol{\omega}}]$ (using $[\hat{\boldsymbol{\omega}}]\hat{\boldsymbol{\omega}} = 0$).

The pattern repeats:

$$[\hat{\boldsymbol{\omega}}]^4 = [\hat{\boldsymbol{\omega}}]^2, \quad [\hat{\boldsymbol{\omega}}]^5 = [\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}], \quad [\hat{\boldsymbol{\omega}}]^6 = [\hat{\boldsymbol{\omega}}]^2, \quad [\hat{\boldsymbol{\omega}}]^7 = [\hat{\boldsymbol{\omega}}]^3 = [\hat{\boldsymbol{\omega}}], \quad \ldots \tag{6.14}$$

Therefore, the Taylor series can be reorganized by grouping terms with the same power modulo 3:

$$e^{[\hat{\boldsymbol{\omega}}]\theta} = \sum_{n=0}^{\infty} \frac{([\hat{\boldsymbol{\omega}}]\theta)^n}{n!} \tag{6.15}$$

$$= \sum_{k=0}^{\infty} \frac{[\hat{\boldsymbol{\omega}}]^{2k}\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-1)^k[\hat{\boldsymbol{\omega}}]\theta^{2k+1}}{(2k+1)!} \tag{6.16}$$

$$= \sum_{k=0}^{\infty} \frac{[\hat{\boldsymbol{\omega}}]^{2}\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-1)^k[\hat{\boldsymbol{\omega}}]\theta^{2k+1}}{(2k+1)!} \tag{6.17}$$

(using $[\hat{\boldsymbol{\omega}}]^0 = \mathbf{I}$, $[\hat{\boldsymbol{\omega}}]^2 = [\hat{\boldsymbol{\omega}}]^2$, and $[\hat{\boldsymbol{\omega}}]^{2k+1} = (-1)^k[\hat{\boldsymbol{\omega}}]$).
Now separate the constant terms:

$$e^{[\hat{\boldsymbol{\omega}}]\theta} = \mathbf{I} + \sum_{k=1}^{\infty} \frac{[\hat{\boldsymbol{\omega}}]^{2}\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-1)^k[\hat{\boldsymbol{\omega}}]\theta^{2k+1}}{(2k+1)!} \tag{6.18}$$

$$= \mathbf{I} + [\hat{\boldsymbol{\omega}}]^2 \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(2k)!} + [\hat{\boldsymbol{\omega}}] \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!} \tag{6.19}$$

$$= \mathbf{I} + [\hat{\boldsymbol{\omega}}]^2 \left(\sum_{k=0}^{\infty} \frac{\theta^{2k}}{(2k)!} - 1 \right) + [\hat{\boldsymbol{\omega}}] \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!} \tag{6.20}$$

Recognizing the Taylor series for $\cos\theta$ and $\sin\theta$:

$$\cos\theta = \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k}}{(2k)!}, \quad \sin\theta = \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!} \tag{6.21}$$

we get:

$$e^{[\hat{\boldsymbol{\omega}}]\theta} = \mathbf{I} + [\hat{\boldsymbol{\omega}}]^2(\cos\theta - 1) + [\hat{\boldsymbol{\omega}}]\sin\theta \tag{6.22}$$

Rearranging:

$$e^{[\hat{\boldsymbol{\omega}}]\theta} = \mathbf{I} + \sin\theta\,[\hat{\boldsymbol{\omega}}] + (1 - \cos\theta)\,[\hat{\boldsymbol{\omega}}]^2 \tag{6.23}$$

which is Rodrigues' formula. □

6.4.1 Verification That the Exponential Map Produces Valid Rotations

We must verify that Rodrigues' formula actually produces a matrix in SO(3)(3) (i.e., $\mathbf{R}^T\mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = 1$).

Theorem 6.2. The Exponential Map so(3) → SO(3) is Surjective

The map $\exp : \mathfrak{so}(3)(3) \to \mathrm{SO}(3)(3)$ defined by Rodrigues' formula is surjective (onto). That is, every rotation matrix can be expressed as the exponential of some skew-symmetric matrix.

Proof. **Orthogonality:** For any skew-symmetric matrix $[\hat{\boldsymbol{\omega}}]$, we have $[\hat{\boldsymbol{\omega}}]^T = -[\hat{\boldsymbol{\omega}}]$. The matrix exponential of a skew-symmetric matrix is orthogonal:

$$(e^{[\hat{\boldsymbol{\omega}}]\theta})^T e^{[\hat{\boldsymbol{\omega}}]\theta} = e^{-[\hat{\boldsymbol{\omega}}]\theta} e^{[\hat{\boldsymbol{\omega}}]\theta} = e^0 = \mathbf{I} \tag{6.24}$$

(using the property that $e^{A^T} = (e^A)^T$ for any matrix.)

Determinant: The determinant of a matrix exponential is $\det(e^A) = e^{\mathrm{tr}(A)}$. For skew-symmetric matrices, $\mathrm{tr}([\hat{\boldsymbol{\omega}}]\theta) = 0$, so $\det(e^{[\hat{\boldsymbol{\omega}}]\theta}) = e^0 = 1$.

Surjectivity: Any rotation matrix $\mathbf{R} \in \mathrm{SO}(3)(3)$ can be decomposed using the axis-angle representation: there exist a unit vector $\hat{\boldsymbol{\omega}}$ and angle $\theta \in [0, \pi]$ such that $\mathbf{R}$ represents rotation by angle θ about axis $\hat{\boldsymbol{\omega}}$. For this rotation, Rodrigues' formula shows that $\mathbf{R} = e^{[\hat{\boldsymbol{\omega}}]\theta}$. □

6.5 The Matrix Logarithm on SO(3)

The exponential map is not injective (one-to-one) on SO(3)(3): multiple axis-angle pairs can produce the same rotation (e.g., rotating by θ about $\hat{\boldsymbol{\omega}}$ is the same as rotating by $-\theta$ about $-\hat{\boldsymbol{\omega}}$, and rotations by θ and $\theta + 2\pi$ are identical). However, if we restrict to $\theta \in [0, \pi]$, the exponential map becomes bijective, and the logarithm is its inverse.

Definition 6.3. Matrix Logarithm on SO(3)

The **Matrix Logarithm** on SO(3)(3) is the inverse of the exponential map, mapping from SO(3)(3) back to $\mathfrak{so}(3)(3)$:

$$\log : \mathrm{SO}(3)(3) \to \mathfrak{so}(3)(3) \tag{6.25}$$

Given a rotation matrix $\mathbf{R}$, the logarithm extracts the axis-angle representation.

6.5.1 Computing the Logarithm: All Cases

The logarithm of a rotation matrix depends on the rotation angle and must be handled in multiple cases:

Theorem 6.3. Logarithm of SO(3) Rotations

For a rotation matrix $\mathbf{R} \in \mathrm{SO}(3)(3)$, the logarithm $[\hat{\boldsymbol{\omega}}]\theta = \log(\mathbf{R})$ can be computed as follows:
Case 1: Identity $(\theta = 0)$
If $\mathbf{R} = \mathbf{I}$, then:

$$\log(\mathbf{R}) = 0 \in \mathfrak{so}(3)(3) \tag{6.26}$$

Case 2: Small Angle $(\theta \ll 1)$
For small rotations, $\sin\theta \approx \theta$ and $1 - \cos\theta \approx \theta^2/2$, so Rodrigues' formula becomes:

$$\mathbf{R} \approx \mathbf{I} + \theta[\hat{\boldsymbol{\omega}}] + O(\theta^2) \tag{6.27}$$

Inverting this approximation (solving for $[\hat{\boldsymbol{\omega}}]\theta$):

$$[\hat{\boldsymbol{\omega}}]\theta \approx \mathbf{R} - \mathbf{R}^T \quad \text{(skew-symmetric part of } \mathbf{R}) \tag{6.28}$$

More precisely:

$$\log(\mathbf{R}) = \frac{1}{2}(\mathbf{R} - \mathbf{R}^T) \quad \text{for small angles} \tag{6.29}$$

Case 3: General Case
For arbitrary $\theta \in (0, \pi]$, the angle θ can be extracted from the trace of $\mathbf{R}$:

$$\mathrm{tr}(\mathbf{R}) = 1 + 2\cos\theta \tag{6.30}$$

Solving:

$$\theta = \arccos\left(\frac{\mathrm{tr}(\mathbf{R}) - 1}{2}\right) \tag{6.31}$$

Once θ is known, the axis $\hat{\boldsymbol{\omega}}$ is extracted from the skew-symmetric part:

$$\hat{\boldsymbol{\omega}} = \frac{1}{2\sin\theta}\begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} \tag{6.32}$$

The skew-symmetric matrix logarithm is:

$$\log(\mathbf{R}) = [\hat{\boldsymbol{\omega}}]\theta \tag{6.33}$$

Case 4: π Rotation
When $\theta = \pi$ (180-degree rotation), $\sin\theta = 0$, so equation (6.32) becomes singular. In this case, use:

$$\mathbf{R} = 2\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I} \tag{6.34}$$

The axis can be found from the eigenvector of $\mathbf{R}$ corresponding to eigenvalue 1:

$$\hat{\boldsymbol{\omega}} = \text{unit eigenvector of } (\mathbf{R} + \mathbf{I}) \text{ with eigenvalue } 2 \tag{6.35}$$

Example 6.1. Computing Logarithm: Example

Suppose we have a rotation matrix representing a 60° rotation about the z-axis:

$$\mathbf{R} = \begin{bmatrix} \cos(60^\circ) & -\sin(60^\circ) & 0 \\ \sin(60^\circ) & \cos(60^\circ) & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & -\sqrt{3}/2 & 0 \\ \sqrt{3}/2 & 1/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{6.36}$$

Step 1: Compute $\mathrm{tr}(\mathbf{R}) = 1/2 + 1/2 + 1 = 2$.
Step 2: Extract angle: $\theta = \arccos((2-1)/2) = \arccos(1/2) = 60^\circ = \pi/3$ rad.
Step 3: Extract axis from skew part:

$$\hat{\boldsymbol{\omega}} = \frac{1}{2\sin(60^\circ)} \begin{bmatrix} 0-0 \\ 0-0 \\ \sqrt{3}/2 - (-\sqrt{3}/2) \end{bmatrix} = \frac{1}{2 \cdot \sqrt{3}/2} \begin{bmatrix} 0 \\ 0 \\ \sqrt{3} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{6.37}$$

Step 4: Construct logarithm:

$$\log(\mathbf{R}) = \begin{bmatrix} 0 \\ 0 \\ \pi/3 \end{bmatrix} \tag{6.38}$$

As a skew matrix: $[\hat{\boldsymbol{\omega}}]\theta = \begin{bmatrix} 0 & -\pi/3 & 0 \\ \pi/3 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$.

Mathematical Principle 6.1. Geometric Distance via Logarithms

The Matrix Logarithm is computationally critical in optimal control and error correction. If your robotic hand is at orientation $\mathbf{R}_{\text{current}}$ and your target is $\mathbf{R}_{\text{target}}$, you *absolutely cannot* compute an "error" by subtracting them ($\mathbf{R}_{\text{target}} - \mathbf{R}_{\text{current}}$) because subtracting curved matrices yields garbage that does not belong to SO(3)(3). The true geometric error—the precise, shortest-path arc of rotation needed to correct the deviation—is found using the Logarithm of their relative transform:

$$\mathbf{e}_{\text{error}} = \log(\mathbf{R}_{\text{target}} \mathbf{R}_{\text{current}}^T) \tag{6.39}$$

The resulting axis-angle vector $\mathbf{e}_{\text{error}} \in \mathfrak{so}(3)(3)$ lives in the tangent space and directly tells you the minimal rotation needed to align the two frames. This is the mathematical foundation for proportional-derivative (PD) control in robotics.

6.6 The Exponential Map on SE(3)

We now extend the exponential map from rotations alone (SO(3)(3)) to full spatial rigid body transformations (SE(3)(3)).

Definition 6.4. The SE(3) Exponential Map

Given a twist $\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} \in \mathfrak{se}(3)(3)$ and a scalar θ (the parameter along the motion), the exponential map is:

$$\mathbf{T} = \exp([\mathcal{V}]\theta) \in \mathrm{SE}(3)(3) \tag{6.40}$$

where $[\mathcal{V}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}$ is the 4×4 matrix representation of the twist.

6.6.1 Derivation of the SE(3) Exponential Formula

To evaluate $\exp([\mathcal{V}]\theta)$, we use the structure of the matrix. Let $[\boldsymbol{\omega}]$ denote the skew-symmetric matrix of $\boldsymbol{\omega}$, which satisfies $[\boldsymbol{\omega}]^3 = -[\boldsymbol{\omega}]$ as before.

The Taylor series is:

$$\exp([\mathcal{V}]\theta) = \sum_{n=0}^{\infty} \frac{1}{n!} \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^n \theta^n \tag{6.41}$$

Computing powers:

$$\begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^2 = \begin{bmatrix} [\boldsymbol{\omega}]^2 & [\boldsymbol{\omega}]\mathbf{v} \\ 0 & 0 \end{bmatrix} \tag{6.42}$$

$$\begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^3 = \begin{bmatrix} [\boldsymbol{\omega}]^3 & [\boldsymbol{\omega}]^2\mathbf{v} \\ 0 & 0 \end{bmatrix} \tag{6.43}$$

The pattern is similar to the $SO(3)$ case, but now we must also track the translation vectors. Using the fact that $[\boldsymbol{\omega}]^3 = -[\boldsymbol{\omega}]$:

$$\exp([\mathcal{V}]\theta) = \begin{bmatrix} e^{[\boldsymbol{\omega}]\theta} & \mathbf{V}(\theta)\mathbf{v} \\ 0 & 1 \end{bmatrix} \tag{6.44}$$

where the **V matrix** (or screw Jacobian) is:

Theorem 6.4. The V Matrix Formula

The 3×3 matrix that relates the translation component of the twist to the spatial translation is:

$$\mathbf{V}(\theta) = \mathbf{I} + (1 - \cos\theta)\frac{[\boldsymbol{\omega}]}{\|\boldsymbol{\omega}\|^2} + (\theta - \sin\theta)\frac{[\boldsymbol{\omega}]^2}{\|\boldsymbol{\omega}\|^3} \tag{6.45}$$

For unit angular velocity $\hat{\boldsymbol{\omega}}$, this simplifies to:

$$\mathbf{V}(\theta) = \mathbf{I} + (1 - \cos\theta)[\hat{\boldsymbol{\omega}}] + (\theta - \sin\theta)[\hat{\boldsymbol{\omega}}]^2 \tag{6.46}$$

For a prismatic (pure translation) twist with $\boldsymbol{\omega} = 0$, we have $\mathbf{V} = \mathbf{I}$.

Proof. Separating the Taylor series into even and odd powers (as we did for $SO(3)$), and using the cyclic property $[\boldsymbol{\omega}]^3 = -[\boldsymbol{\omega}]$:

$$\exp([\mathcal{V}]\theta) = \sum_{n=0}^{\infty} \frac{1}{n!} \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^n \theta^n \tag{6.47}$$

$$= \begin{bmatrix} \sum_{k=0}^{\infty} \frac{[\boldsymbol{\omega}]^{2k}\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{[\boldsymbol{\omega}]^{2k+1}\theta^{2k+1}}{(2k+1)!} & \mathbf{V}(\theta)\mathbf{v} \\ 0 & 1 \end{bmatrix} \tag{6.48}$$

The rotation part gives Rodrigues' formula. For the translation part, we have:

$$\mathbf{V}(\theta)\mathbf{v} = \left(\mathbf{I} + \sum_{k=1}^{\infty} \frac{[\boldsymbol{\omega}]^2\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{[\boldsymbol{\omega}]\theta^{2k+1}}{(2k+1)!} \right) \mathbf{v} \tag{6.49}$$

$$= \left(\mathbf{I} + [\boldsymbol{\omega}] \sum_{k=0}^{\infty} \frac{\theta^{2k+1}}{(2k+1)!} + [\boldsymbol{\omega}]^2 \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(2k)!} \right) \mathbf{v} \tag{6.50}$$

$$= \left(\mathbf{I} + [\boldsymbol{\omega}] \sin\theta + [\boldsymbol{\omega}]^2(\cos\theta - 1) \right) \mathbf{v} \tag{6.51}$$

This is the $\mathbf{V}$ matrix. □

6.7 The Matrix Logarithm on SE(3)

The inverse of the exponential map is the logarithm:

Definition 6.5. Matrix Logarithm on SE(3)

Given a transformation $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \in \mathrm{SE}(3)(3)$, the logarithm extracts the twist:

$$\log(\mathbf{T}) = [\mathcal{V}] \tag{6.52}$$

where $\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix}$ is reconstructed as:

1. Extract the angular part: $[\boldsymbol{\omega}] = \log(\mathbf{R})$ using the $SO(3)$ logarithm.
2. Extract the linear part: $\mathbf{v} = \mathbf{V}^{-1}(\theta)\mathbf{p}$, where $\mathbf{V}^{-1}$ is the inverse of the $\mathbf{V}$ matrix.

For numerical implementation, the inverse of the V matrix is:

$$\mathbf{V}^{-1}(\theta) = \mathbf{I} - \frac{1}{2}[\boldsymbol{\omega}] + \frac{1}{\theta^2} \left(\frac{1}{2} - \frac{\sin(2\theta)}{4\sin\theta} \right) [\boldsymbol{\omega}]^2 \tag{6.53}$$

(and for $\theta \to 0$, use L'Hôpital's rule or series approximations to avoid numerical issues).

6.7.1 Numerical Stability of the Matrix Logarithm

The closed-form logarithm derived in Section 6.5 is exact in infinite precision but requires care in floating-point arithmetic at two boundary regimes.

Near $\theta = 0$ (identity rotation). For $\theta \ll 1$, the general formula $[\hat{\boldsymbol{\omega}}] = \frac{\theta}{2\sin\theta}(\mathbf{R} - \mathbf{R}^T)$ suffers because both θ and $\sin\theta$ approach zero. However their ratio is well-behaved: $\theta/\sin\theta \to 1$. In practice, use the first-order Taylor expansion

$$\log(\mathbf{R}) \approx \mathbf{R} - \mathbf{I} \qquad (\theta \ll 1), \tag{6.54}$$

which is the skew-symmetric part of $\mathbf{R}$ and avoids all division by small quantities.

Near $\theta = \pi$ ($180°$ rotation). The standard axis-extraction formula

$$\hat{\boldsymbol{\omega}} = \frac{1}{2\sin\theta}\begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} \tag{6.55}$$

has a $1/\sin\theta$ singularity as $\theta \to \pi$, because $\sin\pi = 0$. Near this boundary the axis must instead be extracted from the *eigenvector* of $\mathbf{R}$ corresponding to eigenvalue $+1$. A robust approach is to compute $\mathbf{S} = \mathbf{R} + \mathbf{I}$ and take the column of $\mathbf{S}$ with largest norm as $\hat{\boldsymbol{\omega}}$ (after normalization).

Condition number. The condition number of the logarithm map at angle θ scales as

$$\kappa(\theta) \approx \frac{\theta}{\sin\theta}, \tag{6.56}$$

which equals 1 at $\theta = 0$ and diverges as $\theta \to \pi$. Consequently, the logarithm is inherently ill-conditioned near half-turns: small perturbations in $\mathbf{R}$ can produce large changes in the recovered axis.

In Plain Language 6.3. Practical Recommendation

For maximum robustness, use an `atan2`-based extraction of θ (avoiding the arccos whose derivative diverges at ± 1) and switch to eigenvector-based axis recovery when $\sin\theta$ drops below a threshold (e.g. 10^{-6}). Modern libraries such as `scipy.spatial.transform` implement these guards automatically.

6.8 The Baker-Campbell-Hausdorff Formula

One of the most subtle facts about Lie algebras and Lie groups is that they do not commute under composition. That is, if $\mathbf{A}$ and $\mathbf{B}$ are two elements of a Lie algebra, then:

$$e^{\mathbf{A}} e^{\mathbf{B}} \neq e^{\mathbf{A}+\mathbf{B}} \tag{6.57}$$

in general. The difference is captured by the **Baker-Campbell-Hausdorff (BCH) formula**, which expresses the "product" of two exponentials as a single exponential:

Theorem 6.5. Baker-Campbell-Hausdorff Formula (First-Order)

For small perturbations $\mathbf{A}, \mathbf{B} \in \mathfrak{g}$ (the Lie algebra) with $\|\mathbf{A}\|, \|\mathbf{B}\| \ll 1$:

$$\log(e^{\mathbf{A}} e^{\mathbf{B}}) = \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}] + O(\|\mathbf{A}\|^3, \|\mathbf{B}\|^3) \tag{6.58}$$

where $[\mathbf{A}, \mathbf{B}] = \mathbf{AB} - \mathbf{BA}$ is the Lie bracket (commutator).

Mathematical Principle 6.2. Geometric Interpretation of BCH

The BCH formula reveals that when you compose two small rotations, the result is not simply the sum of the rotations, but the sum plus a correction term proportional to their bracket (commutator). This correction term vanishes only if the two motions commute (i.e., $[\mathbf{A}, \mathbf{B}] = 0$).
For rotations, the bracket of two twists is another twist:

$$[\mathcal{V}_1, \mathcal{V}_2] = \mathrm{Ad}_{\exp(\mathcal{V}_1)}\mathcal{V}_2 - \mathcal{V}_2 + \text{higher order} \tag{6.59}$$

This is the mathematical reason why the order of rotations matters: it is not a "software bug" but a fundamental property of 3D rotations.

The full BCH formula (for arbitrary elements) is an infinite series:

$$\log(e^{\mathbf{A}}e^{\mathbf{B}}) = \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}] + \frac{1}{12}([\mathbf{A}, [\mathbf{A}, \mathbf{B}]] + [\mathbf{B}, [\mathbf{B}, \mathbf{A}]]) + \cdots \tag{6.60}$$

In practice, the first-order approximation is sufficient for small perturbations. A thorough development of the BCH formula and its convergence properties can be found in Hall [19].

6.8.1 The Baker-Campbell-Hausdorff Formula: Higher-Order Terms

The first-order BCH approximation $\log(e^{\mathbf{A}}e^{\mathbf{B}}) \approx \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}]$ is adequate when both $\mathbf{A}$ and $\mathbf{B}$ are small. For larger rotations the higher-order Lie-bracket terms become significant. The full expansion to third order reads [19]:

Theorem 6.6. BCH Formula to Third Order

For $\mathbf{A}, \mathbf{B}$ in a Lie algebra $\mathfrak{g}$,

$$\log\big(e^{\mathbf{A}}e^{\mathbf{B}}\big) = \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}] + \frac{1}{12}\big([\mathbf{A}, [\mathbf{A}, \mathbf{B}]] + [\mathbf{B}, [\mathbf{B}, \mathbf{A}]]\big) + O(\|\cdot\|^4), \tag{6.61}$$

where $[\cdot, \cdot]$ denotes the Lie bracket and the remainder collects terms involving four or more nested brackets.

Convergence. For the rotation group SO(3)(3), the BCH series converges absolutely whenever $\|\mathbf{A}\|, \|\mathbf{B}\| < \pi$, i.e. whenever both rotation angles are less than 180°. Outside this radius the series may diverge because the exponential map on SO(3)(3) is not globally injective—rotations by θ and $\theta + 2\pi$ are indistinguishable. More precisely, the convergence radius is set by the injectivity radius of the exponential map, which equals π for SO(3)(3).

Physical meaning. The BCH formula quantifies how much two sequential rotations about different axes *interact*. The first-order bracket term $\frac{1}{2}[\mathbf{A}, \mathbf{B}]$ captures the leading coupling between the two rotation axes. The second-order terms $\frac{1}{12}[\mathbf{A}, [\mathbf{A}, \mathbf{B}]]$ and $\frac{1}{12}[\mathbf{B}, [\mathbf{B}, \mathbf{A}]]$ describe how each rotation axis "precesses" the other—an effect that grows with the magnitude of the rotations.

Mathematical Principle 6.3. Small-Angle Approximation from BCH

For small angles ($\|\mathbf{A}\|, \|\mathbf{B}\| \ll 1$), all bracket terms are negligible and

$$e^{\mathbf{A}} e^{\mathbf{B}} \approx e^{\mathbf{A}+\mathbf{B}}. \tag{6.62}$$

Rotations therefore "add like vectors" in the small-angle regime. For large angles the Lie bracket terms become significant: a 90° rotation about x followed by a 90° rotation about y differs from the reverse order by a 90° rotation about z—a dramatic non-commutativity fully predicted by the BCH series.

6.9 Numerical Computation of Matrix Exponentials

Computing the matrix exponential e^A numerically is a classic problem in scientific computing. For large matrices or large exponents, directly summing the Taylor series is inefficient and can be numerically unstable.

6.9.1 Padé Approximation

One robust approach is the **Padé approximation**, which approximates e^A as a ratio of two polynomials:

$$e^A \approx \frac{N(A)}{D(A)} = \left(I + \frac{A}{2} + \frac{A^2}{12} + \cdots\right)\left(I - \frac{A}{2} + \frac{A^2}{12} - \cdots\right)^{-1} \tag{6.63}$$

The advantage is that rational functions converge faster than power series and are more numerically stable.

6.9.2 Scaling and Squaring

For very large matrices, the **scaling and squaring** method is effective: compute $e^{A/2^k}$ (which has small norm and is easy to compute), then square the result k times:

$$e^A = (e^{A/2^k})^{2^k} \tag{6.64}$$

In Plain Language 6.4. Practical Implementation Note

Modern libraries like Eigen (C++), NumPy (Python), and MATLAB provide optimized `matrix_exp` functions that combine multiple techniques (scaling, Padé approximation, eigenvalue decomposition if applicable) to ensure numerical accuracy and efficiency. For robotics applications, it is almost always better to use these library functions than to implement your own.

6.10 Geodesics on SO(3): Shortest Rotation Paths

A **geodesic** is the shortest path between two points on a curved surface. On the surface of a sphere, geodesics are great circles. On the manifold SO(3)(3), geodesics are paths parameterized by the exponential map.

Mathematical Principle 6.4. Geodesics via the Exponential Map

The shortest path (in a Riemannian sense) from the identity rotation to a rotation $\mathbf{R} \in \mathrm{SO}(3)(3)$ is:

$$\mathbf{R}(t) = \exp(t \cdot \log(\mathbf{R})), \quad t \in [0, 1] \tag{6.65}$$

where $t = 0$ gives the identity and $t = 1$ gives $\mathbf{R}$.
Geometrically, this path rotates at constant angular velocity along the axis extracted by the logarithm. The path length is the rotation angle $\theta = \|\log(\mathbf{R})\|$.

Proof. The geodesic distance on $\mathrm{SO}(3)(3)$ is defined by the Riemannian metric inherited from the tangent space. The exponential map is a local isometry, meaning it preserves distances: the Euclidean distance in the tangent space $\mathfrak{so}(3)(3)$ equals the geodesic distance on the manifold $\mathrm{SO}(3)(3)$ (at least locally).

A straight line in the tangent space—parameterized by $t \cdot \mathbf{v}$ for a direction $\mathbf{v}$—has minimal length. Exponentiating this line gives the geodesic on the manifold. □

6.10.1 Geodesics on $\mathrm{SO}(3)(3)$: Proof via Riemannian Geometry

The informal argument above appeals to intuition; we now give a more rigorous derivation grounded in Riemannian geometry [32].

Definition 6.6. Bi-Invariant Metric on $\mathrm{SO}(3)(3)$

The **bi-invariant Riemannian metric** on $\mathrm{SO}(3)(3)$ is defined at the identity by the inner product on $\mathfrak{so}(3)(3)$:

$$\langle \mathbf{A}, \mathbf{B} \rangle = -\frac{1}{2}\,\mathrm{tr}(\mathbf{A}\mathbf{B}), \qquad \mathbf{A}, \mathbf{B} \in \mathfrak{so}(3)(3). \tag{6.66}$$

This metric is extended to the entire group by left (or right) translation: for tangent vectors $\mathbf{V}, \mathbf{W}$ at $\mathbf{R} \in \mathrm{SO}(3)(3)$,

$$\langle \mathbf{V}, \mathbf{W} \rangle_{\mathbf{R}} = -\frac{1}{2}\,\mathrm{tr}\big((\mathbf{R}^{-1}\mathbf{V})(\mathbf{R}^{-1}\mathbf{W})\big). \tag{6.67}$$

The metric is *bi-invariant*: unchanged under both left and right multiplication by any group element.

Theorem 6.7. One-Parameter Subgroups Are Geodesics

With respect to the bi-invariant metric (6.66), the geodesics through the identity $\mathbf{I} \in \mathrm{SO}(3)(3)$ are exactly the one-parameter subgroups

$$\gamma(t) = e^{t\,[\hat{\boldsymbol{\omega}}]\theta}, \qquad [\hat{\boldsymbol{\omega}}]\theta \in \mathfrak{so}(3)(3). \tag{6.68}$$

Proof. A curve $\gamma(t)$ on $\mathrm{SO}(3)(3)$ is a geodesic if and only if it satisfies the Euler–Lagrange equation for length-minimizing curves. For a bi-invariant metric this equation reduces to the condition that the **body velocity** is constant:

$$\dot{\gamma}(t)\,\gamma(t)^{-1} = \text{const} \in \mathfrak{so}(3)(3). \tag{6.69}$$

For the one-parameter subgroup $\gamma(t) = e^{t\mathbf{A}}$ with $\mathbf{A} \in \mathfrak{so}(3)(3)$, we compute

$$\dot{\gamma}(t) = \mathbf{A}\,e^{t\mathbf{A}}, \qquad \dot{\gamma}(t)\,\gamma(t)^{-1} = \mathbf{A}\,e^{t\mathbf{A}}\,e^{-t\mathbf{A}} = \mathbf{A}, \tag{6.70}$$

which is manifestly constant. Hence every one-parameter subgroup is a geodesic. Conversely, any solution of (6.69) with $\gamma(0) = \mathbf{I}$ and $\dot{\gamma}(0) = \mathbf{A}$ must satisfy $\gamma(t) = e^{t\mathbf{A}}$ by uniqueness of ODE solutions. Therefore the geodesics through $\mathbf{I}$ are *precisely* the one-parameter subgroups. □

Theorem 6.8. Geodesic Distance on SO(3)(3)

The geodesic distance between two rotations $\mathbf{R}_1, \mathbf{R}_2 \in \mathrm{SO}(3)(3)$ under the bi-invariant metric is

$$d(\mathbf{R}_1, \mathbf{R}_2) = \left\|\log(\mathbf{R}_1^T\mathbf{R}_2)\right\| = \theta, \tag{6.71}$$

where $\theta \in [0, \pi]$ is the angle of the relative rotation $\mathbf{R}_1^T\mathbf{R}_2$ and the norm is $\|\mathbf{A}\| = \sqrt{\langle \mathbf{A}, \mathbf{A}\rangle} = \sqrt{-\frac{1}{2}\,\mathrm{tr}(\mathbf{A}^2)}$.

Proof. By left-invariance of the metric, $d(\mathbf{R}_1, \mathbf{R}_2) = d(\mathbf{I}, \mathbf{R}_1^T\mathbf{R}_2)$. The geodesic from $\mathbf{I}$ to $\mathbf{R}_1^T\mathbf{R}_2$ is $\gamma(t) = e^{t\log(\mathbf{R}_1^T\mathbf{R}_2)}$, whose length is $\left\|\log(\mathbf{R}_1^T\mathbf{R}_2)\right\|$. For $\mathbf{A} = [\hat{\boldsymbol{\omega}}]\theta \in \mathfrak{so}(3)(3)$, we have $\|\mathbf{A}\| = \theta$, confirming that the geodesic distance is the rotation angle. □

6.11 SLERP: Spherical Linear Interpolation as Geodesic Interpolation

In Chapter 4, we introduced **SLERP (Spherical Linear Interpolation)**, which smoothly interpolates between two quaternions. We can now understand SLERP as a discretization of the geodesic on SO(3)(3) [30].

Theorem 6.9. SLERP as Exponential Interpolation

Given two rotation matrices $\mathbf{R}_0$ and $\mathbf{R}_1$, the SLERP interpolation:

$$\mathbf{R}(t) = \mathbf{R}_0 \exp(t\log(\mathbf{R}_0^T\mathbf{R}_1)), \quad t \in [0, 1] \tag{6.72}$$

traces a geodesic from $\mathbf{R}_0$ to $\mathbf{R}_1$. This path:

1. Is the shortest distance (in terms of rotation angle) between the two orientations.
2. Rotates at constant angular velocity.
3. Is independent of the order (left-invariant parameterization) or dependent on the parameterization (right-invariant), depending on whether we multiply on the left or right.

Proof. At $t = 0$: $\mathbf{R}(0) = \mathbf{R}_0 \exp(0) = \mathbf{R}_0$.

At $t = 1$: $\mathbf{R}(1) = \mathbf{R}_0 \exp(\log(\mathbf{R}_0^T \mathbf{R}_1)) = \mathbf{R}_0(\mathbf{R}_0^T \mathbf{R}_1) = \mathbf{R}_1$.

The rotation angle between $\mathbf{R}(t)$ and $\mathbf{R}(0)$ is $t\theta_{\max}$ where $\theta_{\max} = \|\log(\mathbf{R}_0^T \mathbf{R}_1)\|$, so the angular velocity is constant.

For quaternions, the same path can be parameterized as:

$$\mathbf{q}(t) = \frac{\sin((1-t)\theta)}{\sin\theta}\mathbf{q}_0 + \frac{\sin(t\theta)}{\sin\theta}\mathbf{q}_1 \tag{6.73}$$

where $\theta = \arccos(\mathbf{q}_0 \cdot \mathbf{q}_1)$ is the angle between the quaternions. This is the classical SLERP formula. □

Example 6.2. SLERP vs. Linear Interpolation

Consider interpolating between two rotations: - $\mathbf{R}_0 = I$ (identity, no rotation) - $\mathbf{R}_1 = e^{[0,0,1]\cdot\pi/2}$ (90-degree rotation about z-axis)

Linear Interpolation (Wrong):

$$\mathbf{R}_{\text{wrong}}(t) = (1-t)\mathbf{R}_0 + t\mathbf{R}_1 = (1-t)\mathbf{I} + t\mathbf{R}_1 \tag{6.74}$$

At $t = 0.5$, this yields a matrix that is neither orthogonal nor a valid rotation—it is geometrically meaningless.

SLERP (Correct):

$$\mathbf{R}_{\text{correct}}(t) = \mathbf{R}_0 \exp(t \log(\mathbf{R}_0^T \mathbf{R}_1)) = \exp(t[0,0,1] \cdot \pi/2) \tag{6.75}$$

At $t = 0.5$, this yields a rotation by 45 degrees about the z-axis, which is geometrically correct and lies on the shortest path between the two orientations.

6.12 Python Worked Example: Computing Exponentials and Logarithms

```python
import numpy as np
from scipy.spatial.transform import Rotation as R
from scipy.linalg import expm, logm
import warnings

def skew(v):
    """Compute the skew-symmetric matrix of a 3D vector."""
    return np.array([
        [0, -v[2], v[1]],
        [v[2], 0, -v[0]],
        [-v[1], v[0], 0]
    ])

def unskew(S):
    """Extract a 3D vector from a skew-symmetric matrix."""
    return np.array([S[2, 1], S[0, 2], S[1, 0]])

def rodrigues(omega, theta):
    """
```

```python
    Compute rotation matrix using Rodrigues' formula.
    omega: unit axis vector (3D)
    theta: rotation angle in radians
    """
    if np.linalg.norm(omega) < 1e-10:
        return np.eye(3)

    omega_hat = omega / np.linalg.norm(omega)
    omega_skew = skew(omega_hat)

    R = (np.eye(3) +
         np.sin(theta) * omega_skew +
         (1 - np.cos(theta)) * omega_skew @ omega_skew)

    return R

def log_SO3(R):
    """
    Compute the logarithm of a rotation matrix.
    Returns the axis-angle vector w = omega_hat * theta.
    """
    # Clamp to avoid numerical errors with arccos
    trace = np.trace(R)
    theta = np.arccos(np.clip((trace - 1) / 2, -1, 1))

    if theta < 1e-10:  # Identity rotation
        return np.zeros(3)
    elif np.abs(theta - np.pi) < 1e-10:  # 180-degree rotation
        # Find the axis from the eigenvector with eigenvalue 1
        eigvals, eigvecs = np.linalg.eig((R + np.eye(3)))
        idx = np.argmax(np.real(eigvals))
        omega_hat = np.real(eigvecs[:, idx])
        omega_hat = omega_hat / np.linalg.norm(omega_hat)
        return omega_hat * theta
    else:
        omega_skew = (R - R.T) / (2 * np.sin(theta))
        omega_hat = unskew(omega_skew)
        return omega_hat * theta

def exp_SE3(twist, theta):
    """
    Compute the exponential map from se(3) to SE(3).
    twist: 6D twist vector [omega; v]
    theta: parameter (typically 1.0 for unit time step)
    """
    omega = twist[:3]
    v = twist[3:]

    if np.linalg.norm(omega) < 1e-10:  # Pure translation
        R = np.eye(3)
        p = v * theta
    else:
        omega_hat = omega / np.linalg.norm(omega)
        w_magnitude = np.linalg.norm(omega)
        theta_total = w_magnitude * theta

        # Rotation part
        R = rodrigues(omega_hat, theta_total)

        # Translation part via V matrix
```

```python
        V = (np.eye(3) +
             (1 - np.cos(theta_total)) * skew(omega_hat) / w_magnitude +
             (theta_total - np.sin(theta_total)) * skew(omega_hat)**2 / (w_magnitude**2))

        p = V @ v * theta

    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = p
    return T

def log_SE3(T):
    """
    Compute the logarithm of a transformation matrix.
    Returns the twist vector [omega; v].
    """
    R = T[:3, :3]
    p = T[:3, 3]

    omega = log_SO3(R)
    w_magnitude = np.linalg.norm(omega)

    if w_magnitude < 1e-10:  # Pure translation
        v = p
    else:
        omega_hat = omega / w_magnitude
        theta = w_magnitude

        # Compute V^{-1}
        V_inv = (np.eye(3) -
                 0.5 * skew(omega_hat) +
                 (1.0/theta**2) * (0.5 - np.sin(2*theta)/(4*np.sin(theta))) * skew(omega_hat)**2)

        v = V_inv @ p

    twist = np.hstack([omega, v])
    return twist

def slerp(R0, R1, t):
    """
    Spherical linear interpolation between two rotations.
    Uses exponential coordinates for geodesic interpolation.
    """
    # Relative rotation from R0 to R1
    R_rel = R0.T @ R1
    omega = log_SO3(R_rel)

    # Interpolated relative rotation
    R_interp_rel = rodrigues(omega / np.linalg.norm(omega), t * np.linalg.norm(omega)) if np.linalg.norm(omega) > 1e-10 else np.eye(3)

    # Final rotation
    R_interp = R0 @ R_interp_rel
    return R_interp

# Example usage
if __name__ == "__main__":
    # 1. Create a rotation: 60 degrees about the z-axis
```

```python
    theta = np.pi / 3  # 60 degrees
    omega = np.array([0, 0, 1])  # z-axis
    R = rodrigues(omega, theta)
    print("Rotation matrix (60 deg about z):")
    print(R)
    print()

    # 2. Compute logarithm
    omega_recovered = log_SO3(R)
    print(f"Recovered axis-angle: {omega_recovered}")
    print(f"Expected: {omega * theta}")
    print()

    # 3. Verify round-trip
    R_recovered = rodrigues(omega_recovered / np.linalg.norm(omega_recovered)
    ,
                            np.linalg.norm(omega_recovered))
    print("Round-trip error (should be ~0):")
    print(np.linalg.norm(R - R_recovered))
    print()

    # 4. Exponential map on SE(3)
    twist = np.array([0, 0, 1, 1, 0, 0])  # rotate about z, translate in x
    T = exp_SE3(twist, 1.0)
    print("SE(3) exponential:")
    print(T)
    print()

    # 5. Log of SE(3)
    twist_recovered = log_SE3(T)
    print(f"Recovered twist: {twist_recovered}")
    print(f"Expected twist: {twist}")
    print()

    # 6. SLERP interpolation
    R0 = np.eye(3)
    theta1 = np.pi / 2  # 90 degrees
    R1 = rodrigues(np.array([0, 0, 1]), theta1)

    print("SLERP interpolation (0, 0.5, 1.0):")
    for t in [0, 0.5, 1.0]:
        R_t = slerp(R0, R1, t)
        omega_t = log_SO3(R_t)
        angle_t = np.linalg.norm(omega_t)
        print(f"  t={t}: rotation angle = {np.degrees(angle_t):.1f} deg")
    print()

    # 7. Geodesic distance
    R_test = rodrigues(np.array([1, 1, 1]) / np.sqrt(3), np.pi / 4)
    omega_log = log_SO3(R_test)
    geodesic_distance = np.linalg.norm(omega_log)
    print(f"Geodesic distance from identity to test rotation: {
    geodesic_distance:.4f} rad = {np.degrees(geodesic_distance):.2f} deg")
```

Listing 6.1: Exponential and Logarithm Computations

6.13 Summary

In this chapter, we developed the exponential and logarithmic maps that govern motion on curved manifolds like SO(3)(3) and SE(3)(3).

1. **Lie Algebras:** The tangent spaces $\mathfrak{so}(3)(3)$ and $\mathfrak{se}(3)(3)$ are flat vector spaces where standard calculus operations work perfectly. They are identified with skew-symmetric matrices (and 6D vectors in the SE(3) case).

2. **Rodrigues' Formula:** The exponential map from $\mathfrak{so}(3)(3)$ to SO(3)(3) is given in closed form by Rodrigues' formula:

$$e^{[\hat{\boldsymbol{\omega}}]\theta} = \mathbf{I} + \sin\theta[\hat{\boldsymbol{\omega}}] + (1-\cos\theta)[\hat{\boldsymbol{\omega}}]^2 \tag{6.76}$$

3. **Matrix Logarithm:** The inverse operation extracts axis-angle coordinates from rotation matrices. It handles multiple cases (identity, small angles, general angles, 180-degree rotations) carefully to avoid numerical singularities.

4. **SE(3) Exponential:** Extension to full spatial transformations introduces the V matrix, which mediates between translation and rotation.

5. **Baker-Campbell-Hausdorff:** The composition of two exponentials is not simply the exponential of the sum; the Lie bracket (commutator) introduces correction terms. This reflects the non-commutativity of rotations.

6. **Geodesics:** The exponential map traces out geodesics (shortest paths) on the manifold. SLERP interpolation of quaternions is mathematically equivalent to geodesic interpolation on SO(3)(3).

7. **Numerical Computation:** Modern libraries implement robust algorithms (scaling-and-squaring, Padé approximation) for computing exponentials and logarithms numerically.

These tools enable us to: - Smoothly interpolate between poses without averaging invalid matrices. - Extract meaningful "error vectors" from the difference between two poses (via the logarithm). - Solve differential equations governing rigid body motion. - Parameterize smooth trajectories and control laws.

With exponential coordinates and the matrix logarithm in hand, we possess the complete mathematical toolkit for describing, analyzing, and controlling rigid body motion in 3D space. We are now ready to move into Volume I, where we develop the geometric properties of SO(3)(3) and SE(3)(3) in depth.

6.14 Further Reading

For a rigorous mathematical treatment of Lie groups, Lie algebras, and the exponential map, Hall [19] is an excellent graduate-level reference that develops the theory from first principles with careful attention to the analytic subtleties. Readers seeking a deeper understanding of the algebraic structures underlying this chapter will find Hall's treatment of matrix Lie groups particularly relevant.

The robotics-specific development of exponential coordinates is best found in Murray, Li, and Sastry [35], whose Chapter 2 constructs the exponential map on SO(3)(3) and SE(3)(3) with an emphasis on applications to manipulator kinematics. Their treatment of the product of exponentials formula, which we develop in Chapter 9, builds directly on the material presented here.

Marsden and Ratiu [32] offer a geometric mechanics perspective on the exponential map, connecting it to Hamiltonian systems, symplectic structures, and momentum maps. Their framework reveals that Rodrigues' formula and the SE(3)(3) exponential are specific instances of a much broader theory governing the geometry of mechanical systems on Lie groups.

Lynch and Park [30] provide an accessible and application-oriented treatment of SLERP interpolation, geodesics on SO(3)(3), and the practical use of exponential coordinates in trajectory planning and motion control. Their worked examples are particularly valuable for building geometric intuition.

It is worth noting that the Baker-Campbell-Hausdorff formula developed in Section 6.8 has deep connections beyond robotics. In quantum mechanics, the BCH formula governs the composition of unitary operators and appears centrally in the theory of quantum gates. In differential geometry, it encodes the local structure of Lie groups and is intimately related to the curvature of the group manifold.

6.15 Exercises

1. **Rodrigues' Formula Verification:** Use Rodrigues' formula to compute the rotation matrix for a 45° rotation about the axis $\hat{\boldsymbol{\omega}} = [1,1,1]^T/\sqrt{3}$. Verify that the result is orthogonal ($\mathbf{R}^T\mathbf{R} = \mathbf{I}$) and has determinant 1.

2. **Logarithm of a 90-Degree Rotation:** Given the rotation matrix:

$$\mathbf{R} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{6.77}$$

   compute $\log(\mathbf{R})$ and extract the axis and angle.

3. **Trace-Based Angle Extraction:** Show that for any rotation matrix $\mathbf{R}$, the trace $\mathrm{tr}(\mathbf{R}) = 1 + 2\cos\theta$ where θ is the rotation angle. Explain why this formula breaks down when $\theta = \pi$ and how to handle this case.

4. **Skew Matrix Pattern:** Prove that for any unit vector $\hat{\boldsymbol{\omega}}$, the skew-symmetric matrix $[\hat{\boldsymbol{\omega}}]$ satisfies $[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}]$.

5. **Small Angle Approximation:** For small rotations with $\theta \ll 1$, show that $e^{[\hat{\boldsymbol{\omega}}]\theta} \approx \mathbf{I} + [\hat{\boldsymbol{\omega}}]\theta$ and derive the error term (to second order in θ).

6. **SE(3) Exponential:** Given a twist $\mathcal{V} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ (rotation about z, translation in x),

compute the exponential map to produce the corresponding 4×4 transformation $\mathbf{T} = \exp([\mathcal{V}])$.

7. **V Matrix for Pure Translation:** For a prismatic (pure translation) joint with twist $\mathcal{V} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, show that $\mathbf{V} = \mathbf{I}$ and thus $\exp([\mathcal{V}]) = \begin{bmatrix} \mathbf{I} & [1,0,0]^T \\ 0 & 1 \end{bmatrix}$.

8. **Logarithm Inversion:** If $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix}$, show that $\log(\mathbf{T}^{-1}) = -\log(\mathbf{T})$ (as a property of the logarithm).

9. **Baker-Campbell-Hausdorff (First Order):** For two small rotations with twists $\mathcal{V}_1 = \epsilon\mathbf{a}$ and $\mathcal{V}_2 = \epsilon\mathbf{b}$ (where $\epsilon \ll 1$), show that:

$$\log(e^{\mathcal{V}_1} e^{\mathcal{V}_2}) \approx \mathcal{V}_1 + \mathcal{V}_2 + \frac{1}{2}[\mathcal{V}_1, \mathcal{V}_2] \tag{6.78}$$

where $[\cdot, \cdot]$ is the Lie bracket.

10. **SLERP Geodesic Property:** Show algebraically that SLERP between two rotations $\mathbf{R}_0$ and $\mathbf{R}_1$ rotates at constant angular velocity. Specifically, verify that the angular velocity $\boldsymbol{\omega}(t) = \frac{d}{dt}\log(\mathbf{R}(t)\mathbf{R}_0^T)$ is constant (independent of t).

11. **Geodesic Distance:** Compute the geodesic distance (rotation angle) between the identity and a rotation by $120°$ about $[1,1,1]^T/\sqrt{3}$.

12. **Numerical Stability:** For $\theta = 0.001$ rad and $\hat{\boldsymbol{\omega}} = [0,0,1]^T$, compare the Rodrigues formula result with the first-order Taylor approximation $\mathbf{R} \approx \mathbf{I} + [\hat{\boldsymbol{\omega}}]\theta$. How large is the error?

13. **Matrix Exponential via scipy:** Using scipy's `linalg.expm`, compute $\exp([\mathcal{V}]t)$ for $\mathcal{V} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ and $t = \pi/4$. Verify the result matches Rodrigues.

14. **Interpolation Verification:** Generate two random rotation matrices $\mathbf{R}_0$ and $\mathbf{R}_1$. Interpolate between them using SLERP at $t = 0.3$ and $t = 0.7$. Verify that the rotations are on a geodesic by checking that $\theta(0.3) \approx 0.3 \cdot \theta_{\max}$ and $\theta(0.7) \approx 0.7 \cdot \theta_{\max}$.

15. **Exponential Trajectory:** A robot arm follows a screw axis with twist $\mathcal{V} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ (rotation about x, translation in y). Compute its pose at times $t = 0, \pi/4, \pi/2$ by evaluating $\mathbf{T}(t) = \exp([\mathcal{V}]t)$.

Chapter 7

Recursive Algorithms

The same equations can be expressed as a large joint-space calculation or as a sequence of local rigid-body balances. Recursion organizes that calculation around the connections between bodies. It lets us ask, in order, how motion reaches a segment and what wrench its motion requires from the rest of the mechanism. These are complementary questions: adding segment velocities without changing frames is wrong, and adding forces without shifting their moments is equally wrong.

For a golf model, this organization makes the physical dependencies inspectable. A hand changes the club's motion through an interface wrench; the equal and opposite wrench loads the arm. Ground forces enter through the feet. Joint torques are projections of transmitted wrenches onto allowed relative motions. Computing those quantities reliably is a prerequisite for studying coordination, power, constraints and delivery. It does not by itself establish which muscular strategy a golfer used, or which change would improve the next swing.

7.1 What Is Being Computed?

We distinguish three operations. Forward kinematics maps joint configuration to body poses and, with joint rates, body velocities. Inverse dynamics maps prescribed motion and known external loads to the generalized forces required by a specified mechanical model. Forward dynamics maps applied forces and the current state to acceleration. A time integrator then advances the state; an inverse-dynamics evaluation alone does not simulate a swing.

For a fixed-base rigid tree with independent scalar joints, write

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau + \sum_i J_i(q)^T f_i^{\text{ext}}. \tag{7.1}$$

Here h includes velocity-product and gravitational terms; f_i^{ext} is a wrench applied to body i, and J_i maps joint rates to that body's twist in the same frame and about the same origin as the wrench. Joint friction, springs and actuator dynamics must be specified separately. In particular, a computed net joint torque is not a uniquely identified muscle force.

Recursive Newton–Euler inverse dynamics (RNEA) evaluates the left-hand side and known-load subtraction without constructing M. Its arithmetic cost grows linearly with the number of bodies when joint dimensions and per-joint calculations are bounded. This is an algorithmic property of a tree, not a claim that Lagrangian mechanics becomes intractable beyond some number of joints. Factored expressions, automatic differentiation and numerical matrix methods also avoid enormous symbolic expansions. No universal control frequency follows from an asymptotic complexity class.

Luh, Walker and Paul's 1980 paper is an important historical reference for manipulator dynamics computation; Featherstone's spatial-vector treatment and Lynch and Park's geometric formulation provide systematic accounts of the algorithms [25, 13, 30]. The derivation below fixes its conventions explicitly because superficially similar published recursions can use opposite transform directions or external-force signs.

7.2 A Tree and Its Traversal Order

A kinematic tree is connected and has no cycles. Let body 0 be a fixed base, and let bodies $1, \ldots, N$ each attach through one joint to a parent $\lambda(i)$. Choose any numbering with $\lambda(i) < i$. This ordering is not unique: siblings can be exchanged. It ensures that a parent's motion is available during an outward pass and that every child's accumulated wrench is available during a reverse pass. The inward pass therefore visits children before parents.

For example, $\lambda = (0, 1, 1, 3)$ describes four moving bodies: body 1 attaches to the base, bodies 2 and 3 branch from body 1, and body 4 attaches to body 3. A list of parent indices is enough for the recursions; one can accumulate each child's contribution directly into its parent without searching the whole list for children. A three-body chain instead has $\lambda = (0, 1, 2)$. Adding a serial gripper joint at its end does not create a branch; two separate children attached to the same body do.

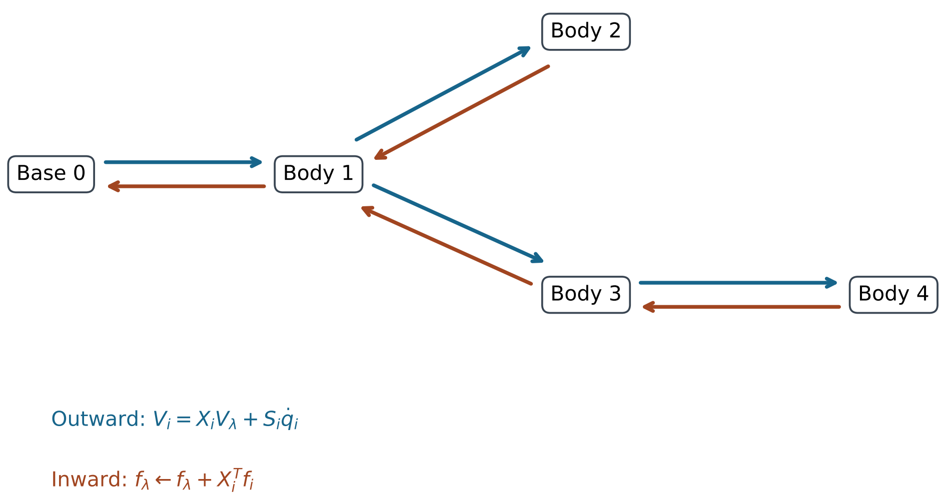


Figure 7.1: A Four-Body Tree With Parent Array (0, 1, 1, 3). Motion Propagates Outward; Subtree Wrenches Accumulate Inward.

Each body frame is rigidly attached to that body. Its origin may be at a joint, the center of mass, or another fixed point, provided its inertia, joint subspace and loads all use that choice. The base is a boundary condition, not an extra actuated joint. A floating pelvis or a robot in flight requires a moving-base formulation. Treating a foot as a permanently welded base is a physical assumption with consequences for the reaction forces and allowed motion.

7.3 Frame Changes and Their Power Dual

Define $T_{AB} = (R, p) \in SE(3)$ by $x_A = Rx_B + p$: it maps point coordinates from frame B to frame A. Thus p locates the origin of B in A. Use angular-first motion vectors $V = (\omega, v)$ and moment-first wrenches $f = (n, F)$. A rigid velocity field in B has point velocity $u_B(x_B) = v_B + \omega_B \times x_B$. Reexpressing that same field in A gives

$$\begin{aligned} u_A(x_A) &= Rv_B + (R\omega_B) \times (x_A - p), \\ \omega_A = R\omega_B, &\qquad v_A = Rv_B + p \times R\omega_B. \end{aligned} \tag{7.2}$$

This is a coordinate change of the same physical motion field. If we instead compare the velocities of two bodies moving relative to each other, their relative twist must also be included.

With $[p]_\times a = p \times a$, the motion transform is

$$X_{AB} = \mathrm{Ad}_{T_{AB}} = \begin{bmatrix} R & 0 \\ [p]_\times R & R \end{bmatrix}, \qquad X_{AB}^{-1} = \begin{bmatrix} R^T & 0 \\ -R^T[p]_\times & R^T \end{bmatrix}. \tag{7.3}$$

Every rigid transform has an invertible adjoint. The matrix is generally not orthogonal: moving an origin mixes angular motion and linear velocity. Its numerical condition number depends on the origin and on the relative scaling of angular and translational units. This is different from loss of rank of a mechanism's task Jacobian.

7.3.1 Wrenches Follow From Virtual Power

Demand that $f_A^T V_A = f_B^T V_B$ for every twist. Since $V_A = X_{AB} V_B$, the two equivalent force relations are

$$f_A = X_{AB}^{-T} f_B = \begin{bmatrix} R & [p]_\times R \\ 0 & R \end{bmatrix} f_B, \qquad f_B = X_{AB}^T f_A. \tag{7.4}$$

The force rotates as $F_A = RF_B$; the moment also acquires the lever arm $p \times RF_B$. For a child-motion transform X_i that maps parent coordinates into child coordinates, the child's wrench returns to the parent as $X_i^T f_i$. The inverse transpose maps a wrench in the same direction as the motion-coordinate change; the transpose maps it in the reverse direction. Keeping the direction explicit prevents a common sign and lever-arm error.

A force sensor reporting a force at a hand or clubhead must be converted to a wrench about the chosen body origin. If a force F acts at body-coordinate point r with applied couple n_0, then $f = (n_0 + r \times F, F)$. Merely rotating the measured force leaves out its moment. The identity $f^T J\dot{q} = (J^T f)^T \dot{q}$ then explains the generalized-load mapping in the equations of motion.

7.4 Recursive Forward Kinematics

Let $T_{i\lambda}$ map parent coordinates to body i, and set $X_i = \mathrm{Ad}_{T_{i\lambda}}$. A joint model supplies this relative transform at q_i and a local motion subspace S_i. For an ideal revolute joint through the child-frame origin about its unit z axis, $S_i = (0, 0, 1, 0, 0, 0)^T$; for translation along local x, $S_i = (0, 0, 0, 1, 0, 0)^T$. An offset rotation axis has a nonzero linear part. Revolute and prismatic coordinates have different units, so their generalized forces are torques and forces respectively.

Absolute poses and body velocities propagate as

$$T_{Wi} = T_{W\lambda(i)} T_{i\lambda}^{-1}, \qquad V_i = X_i V_{\lambda(i)} + V_{Ji}, \qquad V_{Ji} = S_i \dot{q}_i. \tag{7.5}$$

The inverse in the pose equation follows from our declared direction; using the same local matrix without inversion would describe a different mapping. Relative transforms suffice for RNEA. Absolute poses are useful for visualization, world-frame loads and contact geometry, but need not be reconstructed during every force accumulation.

At a body-fixed point r, the body's local Cartesian velocity is $v_i + \omega_i \times r$. Rotate this vector into world coordinates for a clubhead-speed calculation. Segment angular speeds alone omit both origin motion and the point's lever arm. Adding speeds as scalar magnitudes also discards directions and can turn cancellation into apparent amplification.

7.4.1 Exponentials and Numerical Representation

For fixed space screws ξ_j, a serial-chain body pose has the product-of-exponentials form $T_{Wi}(q) = \exp(\widehat{\xi}_1 q_1) \cdots \exp(\widehat{\xi}_i q_i) M_i$, with its own home pose M_i. Partial exponential products without those home poses are not generally the physical link poses. Body screws, space screws and relative joint transforms are interchangeable only through the appropriate coordinate conversions.

Pure translation is regular: if $\widehat{\xi} = \left[\begin{smallmatrix} 0 & v \\ 0 & 0 \end{smallmatrix}\right]$, then $\widehat{\xi}^2 = 0$ and $\exp(q\widehat{\xi}) = I + q\widehat{\xi}$. There is no rank test or pseudoinverse to perform on a prismatic screw. Use a stable matrix exponential or a properly implemented closed form with small-angle limits. Rotation matrices and quaternions both accumulate finite-precision error; quaternion integration does not maintain unit norm automatically. Normalization or suitable geometric integration is a numerical policy, not a replacement for consistent frames.

7.5 Acceleration and the Spatial Velocity Product

For a rigid tree with constant local S_i, differentiating the velocity recursion in the moving coordinates gives

$$a_i = X_i a_{\lambda(i)} + S_i \ddot{q}_i + c_i, \qquad c_i = \operatorname{ad}_{V_i} V_{Ji}. \tag{7.6}$$

Here a_i is spatial acceleration expressed in body coordinates. Without the gravity convention introduced below, it is the time derivative of that body's twist components. Its linear part is not the ordinary acceleration of the body origin: the latter is $\dot{v}_i + \omega_i \times v_i$ in body coordinates. For a fixed body point r, add $\dot{\omega}_i \times r + \omega_i \times (\omega_i \times r)$. These distinctions matter when comparing model arrays with an accelerometer.

For $V = (\omega, v)$, define the motion cross operator and its force dual by

$$\operatorname{ad}_V = \begin{bmatrix} [\omega]_\times & 0 \\ [v]_\times & [\omega]_\times \end{bmatrix}, \qquad V \times^* = -\operatorname{ad}_V^T. \tag{7.7}$$

The order matters: $\operatorname{ad}_{V_i} V_{Ji} = -\operatorname{ad}_{V_{Ji}} V_i$. One way to obtain the acceleration term is to use $\dot{X}_i = -\operatorname{ad}_{V_{Ji}} X_i$ for a constant local joint screw, then substitute $X_i V_\lambda = V_i - V_{Ji}$. A varying joint subspace requires an additional joint-bias term, commonly denoted c_{Ji}; spherical or other multiple-degree-of-freedom joints need their full joint model rather than an unqualified scalar formula.

Parallel angular axes do not make the six-dimensional product zero. Consider the following twists and their bracket:

$$\begin{aligned} V &= (0,0,\omega,v_x,0,0)^T, \\ V_J &= (0,0,\dot q,0,0,0)^T, \\ \mathrm{ad}_V\, V_J &= (0,0,0,0,-v_x\dot q,0)^T. \end{aligned} \tag{7.8}$$

The angular cross product vanishes, but the linear component need not. At the first joint of a fixed-base arm with its origin on the axis, $V_1 = V_{J1}$ and this particular term is zero. Farther along the arm, inherited origin velocity makes it nonzero even for perfectly planar motion. Centripetal acceleration of the center of mass also remains present when the joint-frame linear acceleration is zero.

7.6 Spatial Inertia and Rigid-Body Balance

Let a body have mass m, center-of-mass offset c measured from the body origin to the center of mass, and central rotational inertia I_C, all expressed in the body axes. Expanding the kinetic energy $\frac{1}{2}\omega^T I_C\omega + \frac{1}{2}m\|v+\omega\times c\|^2$ gives

$$\mathcal{I} = \begin{bmatrix} I_C - m[c]_\times^2 & m[c]_\times \\ -m[c]_\times & mI_3 \end{bmatrix}, \qquad K = \tfrac{1}{2}V^T\mathcal{I}V. \tag{7.9}$$

At the center of mass, $c = 0$ and $\mathcal{I} = \mathrm{diag}(I_C, mI_3)$. The spatial matrix mixes rotational inertia, first moments and mass; its blocks do not all have the same units. It is symmetric positive definite for $m > 0$ and positive-definite I_C. Ideal point masses and infinitely thin rods can have singular spatial inertias, even when a restricted planar joint-space inertia is nonsingular. A general-purpose teaching implementation may reasonably reject such spatial degeneracies while a dedicated planar model supports them.

The net wrench required to accelerate one body is

$$f^{\mathrm{net}} = \mathcal{I}a + V \times^* (\mathcal{I}V). \tag{7.10}$$

At a center-of-mass origin its moment equation is $n = I_C\dot\omega + \omega\times I_C\omega$; the gyroscopic term cannot generally be dropped. Its force equation is $F = m(\dot v + \omega\times v)$, which recovers ordinary Newtonian acceleration. Shifting the origin with the power-dual transform recovers the full matrix formula above. This also shows why force cross products use a negative transpose, while finite frame changes use the inverse transpose: they are related infinitesimal and finite operations, not the same matrix.

7.7 The Complete Recursive Newton–Euler Algorithm

Assume prescribed $q, \dot q, \ddot q$, a fixed base, ideal scalar joints, rigid inertias and known external wrenches applied to every body. All external loads must already be expressed in their respective body frames. Uniform physical gravity is a base-frame vector g. Set $V_0 = 0$ and the effective spatial base acceleration $a_0 = (0, -g)^T$. This incorporates gravity into the inertial calculation; it does not mean that the actual base accelerates, and gravity never belongs in V_0.

7.7.1 The Outward Pass: Motion and Local Wrenches

Visit bodies in parent-before-child order. Compute V_{Ji}, V_i, c_i and a_i from the recursions above. Initialize each body's own required wrench as

$$f_i = \mathcal{I}_i a_i + V_i \times^* (\mathcal{I}_i V_i) - f_i^{\text{ext}}. \tag{7.11}$$

The minus sign follows because the external load supplies part of the net wrench required for the prescribed motion. A force applied by the body to its environment has the opposite sign. Do not subtract gravity again if it is already represented through a_0.

7.7.2 The Inward Pass: Subtree Loads and Joint Efforts

Visit bodies in reverse order. Project the accumulated wrench and then propagate it to the parent:

$$\tau_i = S_i^T f_i, \qquad f_{\lambda(i)} \mathrel{+}= X_i^T f_i. \tag{7.12}$$

Equivalently, once every child has contributed,

$$f_i = \mathcal{I}_i a_i + V_i \times^* (\mathcal{I}_i V_i) - f_i^{\text{ext}} + \sum_{j:\lambda(j)=i} X_j^T f_j. \tag{7.13}$$

A leaf starts with its own inertia and its own external load; it is not initialized to zero or just an end-effector force. An external force on a nonleaf body is equally valid. Store all local wrenches before the inward pass so that child accumulation cannot overwrite a known load.

The wrench f_i includes the support required by the whole subtree across joint i. The projection $S_i^T f_i$ extracts its component doing work in that joint's allowed relative motion. Ideal constrained directions do no relative work but can transmit substantial forces and moments. A fixed base has a six-component support wrench, not an extra scalar actuator torque. If the base itself has modeled inertia or applied loads, include them when reporting its complete reaction.

For viscous joint damping $-d_i\dot{q}_i$ and an ideal reflected armature kinetic term $\frac{1}{2}A_i\dot{q}_i^2$, actuator effort becomes $S_i^T f_i + d_i\dot{q}_i + A_i\ddot{q}_i$. These additions assume diagonal joint damping and armature. Real muscle activation, tendon compliance and general rotor/body coupling require their own state and force equations.

7.8 What Linear Complexity Does and Does Not Mean

Each body has one parent edge. The outward pass performs a bounded number of fixed-size spatial operations per body; the inward pass performs one projection and one parent accumulation per edge. With N bodies the total is $O(N)$ arithmetic and $O(N)$ storage for the intermediate arrays. A node may have many children, but the sum of child counts over the whole tree is N including the base edges. A loop that scans all bodies to rediscover every node's children unnecessarily introduces quadratic work.

Computing $M\ddot{q}+h$ from an already assembled dense M costs $O(N^2)$, not a matrix inversion. Solving the dense forward problem by a generic factorization has $O(N^3)$ factorization cost, with cheaper solves if the factorization is reused. Structured sparse methods change that cost. Neither comparison justifies claiming that inverse dynamics is always thousands of times faster than another method.

One can recover M by repeated RNEA calls: subtract the same bias from responses to coordinate accelerations. There are N columns, each costing $O(N)$, so the full construction is $O(N^2)$. The composite-rigid-body algorithm (CRBA) is a different algorithm. It first accumulates locked subtree inertias as $\mathcal{I}_i^C = \mathcal{I}_i + \sum_j X_j^T \mathcal{I}_j^C X_j$, then projects and transports quantities along ancestor paths to fill the joint-space matrix. The composite-inertia pass is linear; explicitly forming a generally dense serial-chain matrix is quadratic. Its N^2 output entries already rule out a generic linear-time full-matrix claim.

The articulated-body algorithm (ABA) solves smooth tree forward dynamics with linear arithmetic by eliminating joint accelerations and recovering them in another outward pass. Alternatively, CRBA plus a suitable factorization solves the same model. Contact detection, constraint solving, actuator dynamics, integration and optimization derivatives add work beyond these rigid-tree kernels. For a concrete engine, inspect its documented pipeline: MuJoCo, for example, documents RNE bias evaluation, CRB inertia assembly and sparse factorization rather than a universal all-ABA pipeline [34].

7.9 Why RNEA Agrees With Lagrangian Mechanics

At fixed configuration, the body velocities satisfy $V_i = J_i(q)\dot{q}$ for a fixed base. Their kinetic energies therefore give

$$K = \tfrac{1}{2}\sum_i \dot{q}^T J_i^T \mathcal{I}_i J_i \dot{q}, \qquad M(q) = \sum_i J_i^T \mathcal{I}_i J_i. \tag{7.14}$$

This constructs the same mass matrix as the Lagrangian kinetic-energy definition. It is positive definite when every nonzero generalized velocity produces positive kinetic energy; massless degrees of freedom can violate that condition.

To establish the full force equivalence, use virtual velocities satisfying the joint constraints. At each interface, the parent and child contributions cancel except for the relative motion $S_i \delta\dot{q}_i$; power-preserving transforms are essential to this cancellation. Summing the rigid-body balances leaves $\sum_i \tau_i \delta\dot{q}_i$ and the external virtual power. The same sum of inertial virtual powers is the Euler–Lagrange expression of the total kinetic energy, while conservative gravity contributes the gradient of the potential. Since the independent variations are arbitrary, the generalized equations coincide. This argument uses ideal constraints and the same masses, frames, loads and gravity on both routes; merely observing an affine dependence on acceleration would not prove equivalence.

Let $\mathrm{ID}(q, v, a)$ include fixed known external loads with our sign convention. It is affine in a:

$$\begin{aligned} b &= \mathrm{ID}(q, v, 0), \\ M(q)e_j &= \mathrm{ID}(q, v, e_j) - b, \\ \mathrm{ID}(q, v, a) &= M(q)a + b. \end{aligned} \tag{7.15}$$

The loads must be held fixed when taking this difference. If a contact or actuator model itself depends on acceleration, the complete inverse map has additional terms. The columns of M do not depend on the chosen velocity, gravity or fixed applied wrench after bias subtraction. Testing that invariance is a useful implementation check.

For the conservative rigid model, energy supplies a further independent diagnostic: $\dot{E} = \tau^T\dot{q} + \sum_i (f_i^{\mathrm{ext}})^T V_i$, with ideal fixed support doing zero power. Add dissipative losses or compliant storage explicitly. Agreement of torque vectors is necessary; agreement of energy and virtual power can expose frame or sign mistakes hidden by a single special pose.

7.10 External Loads, Closed Loops, and Contact

RNEA accepts a known external wrench; it does not determine every unknown contact force from kinematics. With two feet on the ground or two hands gripping one club, force sharing may be indeterminate without measurements, compliance, constitutive laws or an additional force-selection criterion. A closed kinematic loop can be cut to produce a tree, but the missing connection must then be restored by constraints and unknown interface loads. Simply traversing the cut tree changes the physical model.

7.10.1 Bilateral Constraints and Feasibility

For a smooth stationary bilateral constraint $\phi(q) = 0$, let $J_c = \partial\phi/\partial q$. Consistent states obey $J_c\dot{q} = 0$, and accelerations obey $J_c\ddot{q} + \dot{J}_c\dot{q} = 0$. Using r for all known generalized applied forces minus the rigid-body bias, the constrained forward equations are

$$\begin{bmatrix} M & -J_c^T \\ J_c & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} r \\ -\dot{J}_c\dot{q} \end{bmatrix}. \tag{7.16}$$

If M is positive definite and the constraint rows are independent, elimination gives the positive-definite constraint matrix $W = J_c M^{-1} J_c^T$ and

$$W\lambda = -\dot{J}_c\dot{q} - J_c M^{-1} r. \tag{7.17}$$

Implement the inverse actions by solves. Redundant constraint rows make W singular and can leave individual reactions nonunique even when the acceleration is determined. Constraint stabilization addresses numerical drift; it does not justify inconsistent initial positions or guarantee accurate physical forces. A moving constraint adds explicit time derivatives and may do work through its moving support.

In constrained inverse dynamics, a prescribed acceleration must first satisfy the constraint. If $B(q)u$ represents available actuator forces, solve $M\ddot{q}+h = B(q)u + J_c^T\lambda + Q_{\text{known}}$ subject to actuation and contact limits. There can be no solution, one solution, or many force allocations. A floating base has unactuated equations that must balance; an arbitrary six-component residual cannot be silently treated as a pelvis motor. A fixed-foot model is appropriate only for the declared support regime, and additional foot or grip contacts require their own treatment.

7.10.2 Unilateral Contact, Friction, and Impact

For an ideal nonadhesive normal contact, let gap $\phi_n \geq 0$ mean separation and $\lambda_n \geq 0$ mean compression. Position-level complementarity, $\phi_n\lambda_n = 0$, states that separated bodies cannot carry this normal force. It is not a complete dynamical solver. At a closed contact with zero normal velocity, acceleration-level conditions can be written

$$0 \leq \lambda_n \ \perp \ a_n = J_n\ddot{q} + \dot{J}_n\dot{q} \geq 0. \tag{7.18}$$

These conditions concern a smooth contact mode. An approaching contact with negative normal velocity needs an impact law or a resolved compliant collision. One cannot apply the zero-normal-velocity acceleration rule indiscriminately at impact.

Coulomb friction imposes $\|\lambda_t\| \leq \mu\lambda_n$ for sticking. During nonzero slip, a conventional isotropic law selects $\lambda_t = -\mu\lambda_n v_t/\|v_t\|$. The cone inequality alone does not choose a sliding direction or a sticking force. Linear complementarity formulations often use polyhedral friction approximations; an exact circular cone leads to a different constrained formulation. Rigid frictional contact can also have existence or uniqueness difficulties. Engine-specific regularization and compliance affect predictions, so the contact model and solver tolerance belong in a reproducible report.

Across an ideal instantaneous impact, configuration is continuous while velocity can jump:

$$M(q)(\dot{q}^{+} - \dot{q}^{-}) = J_c^T P, \tag{7.19}$$

with impulse P determined by a specified restitution/friction/impact law. An impulse is not an ordinary RNEA force, and a smooth torque integral over a vanishing interval cannot be substituted without a justified collision limit. Club–ball impact therefore needs a separate event model or a contact-resolved simulation. The pre-impact delivery state and the impact constitutive assumptions jointly determine the result.

For compliant contact, a force law may depend on penetration, rate and internal deformation state. Such a load can enter the dynamics once those variables are defined. Increasing stiffness creates a faster timescale, approximately $\sqrt{m_{\text{eff}}/k}$ in a simple normal mode. A force law alone does not establish integration stability: test timestep refinement, energy and contact work, force convergence and sensitivity to damping. It is also essential to prevent unintended tensile forces when the intended contact is nonadhesive.

7.11 Worked Example: Three Planar Links

Consider a fixed-base serial mechanism in the xy plane, with $+y$ upward and physical gravity $(0, -9.81, 0)\,\mathrm{m\,s^{-2}}$. Each relative joint angle is a counterclockwise rotation about $+z$; at zero joint angles the links point along $+x$. This convention gives nonzero gravity torques. Gravity along $-z$ would instead be perpendicular to the plane and would give no generalized gravity torque about these ideal z joints.

The physical parameters, ordered from proximal to distal, are

$$\begin{aligned} L &= (1, 0.8, 0.6)\,\mathrm{m}, \\ m &= (1, 0.5, 0.2)\,\mathrm{kg}, \\ c &= (0.5, 0.4, 0.6)\,\mathrm{m}, \\ I_C &= (0.083, 0.027, 0)\,\mathrm{kg\,m^2}. \end{aligned} \tag{7.20}$$

These are the specified teaching parameters: the first two moments are rounded, and the third link is a massless rod with an endpoint mass. It is not a uniform three-rod arm, a fitted golfer or a compliant club.

Use the exact numerical inputs $q = (0.524, 0.785, 0)$ radians, approximately $(30, 45, 0)$ degrees, with $\dot{q} = (0.5, 0.3, 0.2)\,\mathrm{s^{-1}}$ and $\ddot{q} = (1, 0.5, 0.1)\,\mathrm{s^{-2}}$. The cumulative angular rates are $(0.5, 0.8, 1.0)\,\mathrm{s^{-1}}$ and accelerations $(1, 1.5, 1.6)\,\mathrm{s^{-2}}$. Rounding the input angles and then calling them exact degree values would specify a slightly different numerical problem.

For body frames at the proximal joints, the linear part of the second spatial velocity and the linear parts of the two distal joint-bias terms are

$$\begin{aligned} v_2 &= (0.353413, 0.353694, 0)\,\mathrm{m\,s^{-1}}, \\ (c_2)_{\text{lin}} &= (0.106108, -0.106024, 0)\,\mathrm{m\,s^{-2}}, \\ (c_3)_{\text{lin}} &= (0.198739, -0.070683, 0)\,\mathrm{m\,s^{-2}}. \end{aligned} \tag{7.21}$$

Both angular bias parts vanish. This is a concrete instance of nonzero planar spatial coupling despite parallel joint axes.

7.11.1 An Independent Cartesian Recursion

The planar implementation provides a second route without six-vector transforms. Let $e_i = (\cos\theta_i, \sin\theta_i)$, where $\theta_i = \sum_{j\le i} q_j$, and let $e_i^{\perp} = (-\sin\theta_i, \cos\theta_i)$. Propagate the effective joint-origin acceleration A_i from $A_1 = -g_{xy}$. With cumulative angular rate ω_i and acceleration α_i,

$$\begin{aligned} A_{Ci} &= A_i + \alpha_i c_i e_i^{\perp} - \omega_i^2 c_i e_i, \\ A_{i+1} &= A_i + \alpha_i L_i e_i^{\perp} - \omega_i^2 L_i e_i. \end{aligned} \tag{7.22}$$

In this subsection c_i is a scalar center-of-mass distance, not the spatial bias vector. Summing each segment's $m_i A_{Ci}$ force and $I_i\alpha_i$ moment inward, with the actual lever arms to the center of mass and next joint, gives the joint torques. An independent Lagrangian route assembles M from differentiated Cartesian center-of-mass positions and differentiates energy for gravity and velocity-product terms.

The resulting decomposition, in newton metres, is

$$\begin{aligned} G &= (11.411202,\ 1.218713,\ 0.304678)^T, \\ C\dot{q} &= (-0.162853,\ 0.084819,\ 0.021205)^T, \\ M\ddot{q} &= (2.655655,\ 1.104846,\ 0.344087)^T, \\ \tau &= (13.904004,\ 2.408379,\ 0.669970)^T. \end{aligned} \tag{7.23}$$

The recursive and finite-difference Lagrangian torques agree within $10^{-8}\,\mathrm{N\,m}$ for these inputs. That is a checked numerical tolerance, not a symbolic proof or a physiological validation. Gravity is the largest contribution at the proximal joint in this particular pose, while acceleration contributes slightly more than gravity at the third joint. Those ratios change with pose and prescribed acceleration; the example cannot establish a general proximal-to-distal energy-transfer strategy.

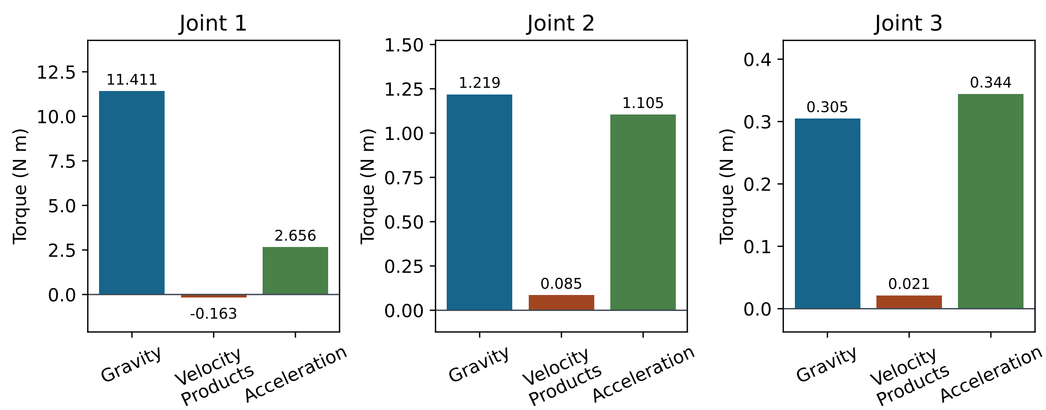


Figure 7.2: Required Torque Components for the Specified Planar Example. Signed Contributions Depend on the Chosen Pose, Rates, and Accelerations; Each Joint Uses Its Own Torque Scale.

7.12 Validation With Two Uniform Rods

Use two rods of unit length and unit mass, with centers halfway along each rod and central planar moments $1/12\,\mathrm{kg\,m^2}$. Keep the same horizontal-zero angle and upward-y gravity convention. The first center is $e_1/2$ and the second is $e_1 + e_{12}/2$, where $e_{12} = (\cos(q_1 + q_2), \sin(q_1 + q_2))$. Differentiating these positions and adding rotational kinetic energy yields

$$M(q) = \begin{bmatrix} 5/3 + \cos q_2 & 1/3 + \frac{1}{2}\cos q_2 \\ 1/3 + \frac{1}{2}\cos q_2 & 1/3 \end{bmatrix} \mathrm{kg\,m^2}. \tag{7.24}$$

The potential is $U = g[\frac{3}{2}\sin q_1 + \frac{1}{2}\sin(q_1 + q_2)]$ in SI units. The velocity-product and gravitational generalized forces are

$$\begin{aligned} b_v &= \tfrac{1}{2}\sin q_2 \begin{bmatrix} -2\dot{q}_1\dot{q}_2 - \dot{q}_2^2 \\ \dot{q}_1^2 \end{bmatrix}, \\ G &= g \begin{bmatrix} \frac{3}{2}\cos q_1 + \frac{1}{2}\cos(q_1 + q_2) \\ \frac{1}{2}\cos(q_1 + q_2) \end{bmatrix}. \end{aligned} \tag{7.25}$$

The numerical coefficients in b_v carry the unit-rod inertia factors, and those in G carry mass times length. For other dimensions, restore the physical m, L, c, I_C factors before using the formulas. The two-link mass matrix is not obtained by treating each rod as a point mass or omitting its central inertia.

At $q = (0, 0)$, $\dot{q} = (1, 0.5)\,\mathrm{s^{-1}}$ and $\ddot{q} = (0.5, 0.2)\,\mathrm{s^{-2}}$, the velocity-product generalized force happens to vanish. The gravity torques are $(19.62, 4.905)\,\mathrm{N\,m}$, the acceleration torques are $(1.5, 0.483333)\,\mathrm{N\,m}$, and the required torques are $(21.12, 5.388333)\,\mathrm{N\,m}$. Vanishing net generalized velocity-product force at this pose does not imply that every local centripetal or spatial-bias term vanishes. Tests at nonzero relative angles are necessary to expose those terms.

7.13 The Adjoint From Lie-Group Geometry

The same frame law follows by differentiating group conjugation. For a twist matrix $\widehat{\xi}$, a fixed rigid transform satisfies

$$T\exp(t\widehat{\xi})T^{-1} = \exp(tT\widehat{\xi}T^{-1}), \qquad (\mathrm{Ad}_T\,\xi)^\wedge = T\widehat{\xi}T^{-1}. \tag{7.26}$$

Conjugation acts on the group; its derivative at the identity acts on the Lie algebra. Composition immediately gives $\mathrm{Ad}_{T_1T_2} = \mathrm{Ad}_{T_1}\mathrm{Ad}_{T_2}$. This makes successive frame changes agree with a single composed change, an important numerical consistency check.

For the differential of the matrix exponential, the noncommutative formula is

$$D\exp_A[H] = \int_0^1 e^{(1-s)A}He^{sA}\,ds. \tag{7.27}$$

With $A = \widehat{\xi}$, $H = \widehat{\eta}$ and $T = \exp(\widehat{\xi})$, define the right- and left-trivialized increments explicitly:

$$\begin{aligned}(\delta T\,T^{-1})^\vee &= \int_0^1 \mathrm{Ad}_{\exp(s\widehat{\xi})}\,\eta\,ds,\\ (T^{-1}\delta T)^\vee &= \int_0^1 \mathrm{Ad}_{\exp(-s\widehat{\xi})}\,\eta\,ds.\end{aligned} \tag{7.28}$$

These operators are often named left and right Jacobians according to perturbation conventions; the multiplication order above removes that naming ambiguity. They equal the identity at $\xi = 0$. The differential can lose rank at particular exponential-coordinate values even though T and Ad_T remain invertible. Do not confuse coordinate-chart behavior, a physical task-Jacobian singularity and a rigid frame transformation.

7.14 Reproducible Python Examples

Run the following from the repository root with its Python dependencies. The first calculation uses the dedicated planar recursion, which permits the endpoint point mass. The second uses the shared spatial-tree RNEA and ABA on three uniform solid cylinders with radius 0.025 m. These are different mass distributions and therefore have different torques. The spatial example checks a forward/inverse round trip; the analytic two-rod and Cartesian-energy tests supply independent validation rather than relying on that round trip alone.

```
import numpy as np
from scripts.generate_worked_examples import (
    VOL0_CH07_CHAIN, VOL0_CH07_Q,
    VOL0_CH07_QD, VOL0_CH07_QDD,
)
from src.tools.articulated_body_examples import (
    DynamicsLoads, forward_dynamics, inverse_dynamics,
)
from src.tools.articulated_chain_examples import cylinder_chain

q, velocity, acceleration = VOL0_CH07_Q, VOL0_CH07_QD, VOL0_CH07_QDD
recursive = VOL0_CH07_CHAIN.inverse_dynamics(q, velocity, acceleration)
lagrangian = VOL0_CH07_CHAIN.inverse_dynamics_lagrangian(
    q, velocity, acceleration,
)
print("Planar torques:", recursive)
np.testing.assert_allclose(recursive, lagrangian, atol=1e-8, rtol=0)

tree = cylinder_chain(
    np.array([1., .8, .6]), np.array([1., .5, .2]), q, .025,
)
loads = DynamicsLoads(velocity, np.zeros(3))
torque = inverse_dynamics(tree, loads, acceleration)
recovered = forward_dynamics(tree, DynamicsLoads(velocity, torque))
print("Cylinder torques:", torque)
np.testing.assert_allclose(
    recovered.joint_acceleration, acceleration, atol=1e-11, rtol=0,
)
```

The cylinder torques for these inputs are approximately

$$\tau_{\text{cylinder}} = (13.447676, 1.977061, 0.315835)^T \,\mathrm{N\,m}. \tag{7.29}$$

The helper builds transforms that map parent motion into child coordinates and positive-definite cylindrical inertias. Its parent array uses zero-based Python indexing and -1 for the base. It accepts constant local one-degree-of-freedom subspaces, known body-frame external loads, uniform gravity, nonnegative diagonal damping and ideal armature. It does not solve contact, floating-base motion, closed loops or flexible-shaft dynamics. Array shape and inertia checks do not prove that an arbitrary supplied transform or inertia is a physically valid model; those remain explicit preconditions.

7.15 Interpreting a Golf or Humanoid Model

The outward and inward recursions describe two sides of one coupled mechanism. Geometry determines each joint's motion direction, each point's lever arm and the frame transformations. Mass distribution turns the resulting body velocities and accelerations into momentum and required wrench. The inward pass combines every downstream load, so proximal effort depends on what the distal bodies are doing. Conversely, in forward dynamics, a torque at one joint can accelerate several joints through the coupled inertia and constraints.

A temporal sequence of segment-speed peaks alone does not identify which term caused the next peak.

For the club considered as a separate system, hand and shaft-interface wrenches are external loads whose power changes the club's mechanical energy, alongside gravity and other modeled effects. For golfer and club considered together, the paired interface contributions are internal; relative deformation or motion determines whether their summed power transfers, stores or dissipates energy. An ideal stationary ground contact may exchange momentum through force while doing zero power at a nonmoving contact point. Large ground reaction, large joint torque and large positive mechanical work are therefore different observations. A causal argument must state the system boundary, the power pair and the measured or modeled motion.

Joint-space inverse dynamics estimates net mechanical requirements once motion, inertia and external loads are supplied. It does not separate agonist and antagonist muscle forces or determine activation from torque alone. Biarticular muscles distribute forces across joints; activation history and tendon storage can change what torques are feasible at the same mechanical state. Measurement differentiation also amplifies noise, especially in acceleration, so report filtering, uncertainty, synchronization and sensitivity to segment inertias and contact forces.

To investigate a proposed swing mechanism, first reproduce a measured or otherwise specified trajectory within the model's uncertainty. Then define an intervention: alter a particular actuator input, initial state, grip constraint or inertial parameter while stating what is held fixed. Use constrained forward dynamics to calculate the resulting trajectory, and evaluate delivery quantities at the relevant impact event. An instantaneous torque decomposition is useful evidence about required balance; a finite-horizon intervention is needed to support a claim about an outcome. Both contact-mode changes and impact-time shifts can change that conclusion.

The quality of the argument rests on a chain of checkable statements: coordinate consistency, local balance, constraint and actuator feasibility, numerically converged trajectories, and an output tied to the golf question. Recursive algorithms make that chain economical and inspectable. They do not remove uncertainty in anatomy, sensing, constitutive models or human intent.

7.16 Further Reading

Featherstone's *Rigid Body Dynamics Algorithms* develops spatial inertia and tree algorithms; the author's accompanying coordinate-transform notes distinguish motion and force directions explicitly [13, 14]. Lynch and Park's *Modern Robotics*, Chapter 8, provides a complementary body-twist derivation and separates inverse dynamics from simulation [30]. When consulting code, check whether an external wrench is applied to the body or by the end effector, and whether it is expressed in body or base coordinates. Correct implementations can differ in these conventions.

The following articulated-body chapter develops acceleration elimination and recovery for forward dynamics. Together with the state-space, configuration-space and Lagrangian chapters, it supplies the mechanical basis for later work on control, sensitivity and constrained delivery. A chosen simulator's documentation should supplement these mathematical references with its actual joint, contact, actuation and integration assumptions.

7.17 Exercises

7.17.1 Tier 1: Foundations

1. **Invertible Frame Changes.** Derive the inverse adjoint from T^{-1}. Show directly that a pure translation gives an invertible motion transform, even though its angular screw component is zero.

2. **Tree and Joint Conventions.** Give the one-based parent array for a two-link planar chain with base 0. Write its local revolute subspaces at the proximal origins and the relative motion transforms for horizontal-zero angles. Explain how the Python indexing differs.

3. **A Nonzero Planar Bracket.** For angular-first $V = (\omega, v)$, take $\omega = (0, 0, 0.5)$ and $v = (0.4, 0, 0)$. A revolute joint at the origin about local $+z$ has rate $\dot{q} = 0.2$. Calculate all six components of $\mathrm{ad}_V V_J$. Identify the physical units of the nonzero component and explain why an angular-only cross product misses it.

4. **Power and Wrench Direction.** Starting from $V_A = X_{AB} V_B$, derive both force transformation directions. Use a translated origin and a nonzero force to show why inverse-transpose child-to-parent propagation would give the wrong moment.

5. **Inertia and Degeneracy.** Construct the planar inertias of the three-link example from its masses, center offsets and central moments. Explain why its endpoint point mass is compatible with a nonsingular restricted joint-space mass matrix but does not have a positive-definite six-dimensional spatial inertia.

7.17.2 Tier 2: Mechanics and Verification

1. **Newton–Euler From Energy and Momentum.** Derive the center-of-mass force and moment equations, including the gyroscopic term, and transform them to a non-central origin. Use virtual power to explain their equivalence to the generalized Lagrangian equations.

2. **A Genuine Branch.** For parents $(0, 1, 1, 3)$, write the complete inward accumulation, including a known external load on body 1 and on both leaves. Identify which body efforts are influenced by each leaf load.

3. **Dual Operators.** Verify $V\times^* = -\mathrm{ad}_V^T$ numerically. Check preservation of power under a nontrivial rotation and translation, then compare the two uniform-rod torques with the closed forms at three nonsymmetric configurations.

4. **Like-for-Like Computational Comparisons.** For $N = 5, 10, 20, 50$, compare RNEA with a preassembled $M\ddot{q} + h$ evaluation as inverse-dynamics tasks. Separately compare ABA with a mass-matrix forward solve. Report model preparation, factorization reuse, hardware, repetitions and numerical agreement; distinguish expected arithmetic growth from measured timing.

5. **A Cut Loop.** Introduce a bilateral closure constraint into an otherwise open tree. Derive the acceleration constraint and coupled force solve. State what changes if the constraint rows are redundant or the prescribed inverse-dynamics motion is inconsistent.

7.17.3 Tier 3: Modeling and Extension

1. **Composition.** Prove the adjoint homomorphism using group conjugation and verify it with two translated, rotated frames. Show that the corresponding force transforms preserve the same composition order in the same direction.

2. **Differentiating the Exponential.** Derive the integral formula for $D\exp_A[H]$, then both explicitly ordered trivializations. Compare with centered finite differences for noncommuting A, H and explain why $e^A H$ is insufficient in general.

3. **A Declared Humanoid Tree.** Specify a teaching humanoid with 30 independent velocity degrees of freedom, stating whether six belong to a floating base. Provide a parent structure, joint types, axes, body inertias, actuation map and contact assumptions. Explain which accelerations can be prescribed consistently and which base-force equations remain unactuated.

4. **Composite Versus Repeated Inverse Dynamics.** Implement composite-inertia accumulation and ancestor projection to form M. Compare it with the bias-subtracted RNEA columns and the kinetic-energy construction. Separate the linear composite pass from worst-case quadratic full-matrix output, and identify the zero couplings between independent branches of a fixed-base tree.

5. **Compliant Contact and Delivery.** Add a specified nonadhesive compliant contact law to a simple forward-dynamics model. Perform timestep refinement and check contact work and force convergence. Explain how stiffness, damping, effective mass, actuator limits and an event-defined delivery output interact; do not infer numerical stability from the force law alone.

Chapter 8

Spatial Vector Algebra and Inertia

Why manage six tiny numbers loosely floating in space when you can forge them into an unbreakable, 6-dimensional spear?

In Plain Language 8.1. Context & Use Case: Unifying Translation and Rotation

The Math You Will See: We introduce **Spatial Vector Algebra**, defining the 6×6 **Spatial Cross Product** ($[\mathcal{V}\times]$), the **Spatial Inertia Matrix** ($\mathcal{I}$), and their role in transforming forces and accelerations across frames.
Why We Need It: In classical 3D mechanics, linear motion ($F = ma$) and angular motion ($\boldsymbol{\tau} = \mathbf{I}\boldsymbol{\alpha} + \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega}$) are computed separately, leading to a tangled web of cross products and messy offsets when bodies are connected together. Spatial Algebra fuses them. A "Spatial Vector" describes pushing and turning simultaneously. By doing math in 6D, the complex gyroscopic and Coriolis forces of a multi-joint robot swinging through space are resolved with a single matrix multiplication.
All Variables Defined: $\mathcal{V}$ is a spatial velocity (Twist), $\mathcal{F}$ is a spatial force (Wrench), and $\mathcal{I}$ is the combined 6D mass/inertia tensor.

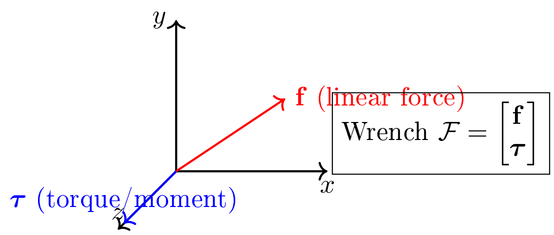
8.1 Historical Context: From Plücker to Featherstone

The roots of spatial algebra reach deep into classical geometry. In the 1860s, **Julius Plücker** studied lines in 3D space as geometric objects, discovering that a line can be represented as a 6-dimensional vector satisfying certain constraints. This **Plücker coordinate** representation remained mostly a curiosity in algebraic geometry for over a century.

In 1900, Sir **Robert Stawell Ball** [3] developed the theory of "screws"—geometric objects unifying rotation and translation—and showed that Plücker's coordinates naturally describe instantaneous screw motions. Ball's work connected to mechanics through the wrench (force + torque) dual to the twist (angular + linear velocity).

However, the true synthesis came when **Roy Featherstone**, in the 1980s and 1990s, recognized that Ball's geometric screws and Plücker's algebraic coordinates could be married with Lie algebra to create a unified computational framework for multi-body dynamics. Featherstone's **Spatial Vector Algebra** [13] standardized the representation, showed how to compose inertias of linked chains (composite rigid body algorithm), and proved that careful use of spatial algebra enables $O(N)$ inverse dynamics.

Modern robotics and physics engines (MuJoCo [48], Drake, Pinocchio) are built entirely on Featherstone's spatial framework. The companion platform accompanying this textbook builds on the same foundation.



Spatial vectors unify translation (linear) and rotation (angular) quantities

Figure 8.1: A Spatial Wrench Is a 6-Dimensional Vector Combining Linear Forces and Torques (Moments). Similarly, Spatial Twists Combine Linear and Angular Velocities. This Unification Allows Forces and Velocities to Be Manipulated Using Unified Algebraic Operations.

8.2 Plücker Coordinates and Lines in 3D

Before we discuss spatial vectors as physical quantities, it helps to understand their geometric origin: the Plücker representation of lines.

Definition 8.1. Plücker Coordinates of a Line

A line ℓ in 3D can be represented as a 6-vector in **Plücker coordinates**:

$$\mathbf{L} = \begin{bmatrix} \mathbf{d} \\ \mathbf{m} \end{bmatrix} \in \mathbb{P}^5 \tag{8.1}$$

where:

- $\mathbf{d}$ is the direction vector of the line (unit vector).
- $\mathbf{m} = \mathbf{r} \times \mathbf{d}$ is the moment (or Plücker moment), where $\mathbf{r}$ is any point on the line.

The Plücker coordinates satisfy the **Plücker relation**:

$$\mathbf{d}^T \mathbf{m} = 0 \tag{8.2}$$

This constraint encodes the fact that the direction and moment are related geometrically.
Geometric Insight: The direction $\mathbf{d}$ specifies the "orientation" of the line; the moment $\mathbf{m}$ encodes the line's location in space. Two lines are identical if and only if they have the same Plücker coordinates.

8.2.1 Connection to Spatial Vectors

A spatial velocity (Twist) $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$ can be viewed as the instantaneous motion of a rigid body. The axis of this motion is a screw (a line with a pitch), and the Plücker coordinates of that screw are exactly $(\boldsymbol{\omega}, \mathbf{v})$ (up to scaling).

Similarly, a spatial force (Wrench) $\mathcal{F} = (\mathbf{m}, \mathbf{f})$ is the dual concept: it represents a force $\mathbf{f}$ applied at some point in space, creating a torque $\mathbf{m}$ about the origin. The Plücker relationship $\mathbf{f}^T\mathbf{m} = 0$ holds when the force and moment arise from a single applied force (they lie on the same screw axis).

8.3 Spatial Vectors: Twists and Wrenches

In Chapter 6, we introduced spatial vectors as elements of the Lie algebras $\mathfrak{se}(3)(3)$ and $\mathfrak{se}(3)^*(3)$. Here we formalize them as vectors and define their algebra.

Definition 8.2. Spatial Velocity (Twist)

A spatial velocity or Twist is a 6D vector:

$$\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} \in \mathfrak{se}(3)(3) \cong \mathbb{R}^6 \tag{8.3}$$

where:

- $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity.
- $\mathbf{v} \in \mathbb{R}^3$ is the linear velocity of the origin of the reference frame.

Physical Interpretation: A point at position $\mathbf{r}$ relative to the origin has absolute velocity $\mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}$.

Definition 8.3. Spatial Force (Wrench)

A spatial force or Wrench is a 6D vector:

$$\mathcal{F} = \begin{bmatrix} \mathbf{m} \\ \mathbf{f} \end{bmatrix} \in \mathfrak{se}(3)^*(3) \cong \mathbb{R}^6 \tag{8.4}$$

where:

- $\mathbf{m} \in \mathbb{R}^3$ is the moment (torque) about the origin.
- $\mathbf{f} \in \mathbb{R}^3$ is the linear force.

Physical Interpretation: A force $\mathbf{f}$ applied at a point $\mathbf{r}$ creates a moment $\mathbf{m} + \mathbf{r} \times \mathbf{f}$ about the origin.

8.4 The Spatial Cross Product

In classical 3D mechanics, cross products appear everywhere. When a body rotates with angular velocity $\boldsymbol{\omega}$, the velocity of a point at offset $\mathbf{r}$ includes the term $\boldsymbol{\omega} \times \mathbf{r}$. When a force $\mathbf{f}$ is applied at point $\mathbf{r}$, it creates a torque $\mathbf{r} \times \mathbf{f}$.

Spatial Algebra extends the cross product to 6D, but because Twists and Wrenches live in dual spaces (with different transformation laws), there are actually *two* distinct spatial cross products.

8.4.1 Motion Cross Product

The **motion cross product** acts on Twists and related vectors (velocities, accelerations). If a frame has spatial velocity $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$, a point with position offset $\mathbf{r}$ has velocity $\mathbf{v}+\boldsymbol{\omega}\times\mathbf{r}$.

We represent this as a matrix operator:

Definition 8.4. Motion Cross Product Operator

For a spatial velocity Twist $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$, the motion cross product operator is:

$$[\mathcal{V}\times] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{0}_{3\times3} \\ [\mathbf{v}] & [\boldsymbol{\omega}] \end{bmatrix} \in \mathbb{R}^{6\times6} \tag{8.5}$$

where $[\boldsymbol{\omega}]$ and $[\mathbf{v}]$ are the standard 3×3 skew-symmetric cross product matrices.
Action on a spatial velocity: If $\mathcal{W} = (\boldsymbol{\alpha}, \mathbf{a})$ is another spatial quantity, then:

$$[\mathcal{V}\times]\mathcal{W} = \begin{bmatrix} \boldsymbol{\omega} \times \boldsymbol{\alpha} \\ \mathbf{v} \times \boldsymbol{\alpha} + \boldsymbol{\omega} \times \mathbf{a} \end{bmatrix} \tag{8.6}$$

8.4.2 Force Cross Product

Wrenches are the dual to Twists. Because of this duality, when we apply the adjoint transformation to a wrench, it uses a different matrix. Correspondingly, the force cross product is defined differently:

Definition 8.5. Force Cross Product Operator

The force cross product (coadjoint cross product) for a spatial velocity $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$ acting on Wrenches is:

$$[\mathcal{V}\times^*] = -[\mathcal{V}\times]^T = \begin{bmatrix} [\boldsymbol{\omega}] & [\mathbf{v}] \\ \mathbf{0}_{3\times3} & [\boldsymbol{\omega}] \end{bmatrix} \tag{8.7}$$

Action on a spatial force: If $\mathcal{G} = (\mathbf{m}, \mathbf{f})$ is a Wrench, then:

$$[\mathcal{V}\times^*]\mathcal{G} = \begin{bmatrix} \boldsymbol{\omega} \times \mathbf{m} + \mathbf{v} \times \mathbf{f} \\ \boldsymbol{\omega} \times \mathbf{f} \end{bmatrix} \tag{8.8}$$

8.4.3 Why Two Cross Products?

The reason for the asymmetry is that power (work rate) must be invariant:

$$\text{Power} = \mathcal{F}^T \mathcal{V} \tag{8.9}$$

Under a coordinate frame change, if Twists transform via $[\mathrm{Ad}_{\mathbf{T}}]$ and Wrenches transform via $[\mathrm{Ad}_{\mathbf{T}}]^{-T}$ (the coadjoint), then the pairing $\mathcal{F}^T \mathcal{V}$ is preserved. The cross product operators must respect this dual structure.

Mathematical Principle 8.1. Duality of Motion and Force Cross Products

The motion and force cross products are dual in the sense that:

$$\mathcal{G}^T([\mathcal{V}\times]\mathcal{W}) = ([\mathcal{V}\times^*]\mathcal{G})^T \mathcal{W} \tag{8.10}$$

for any Twist $\mathcal{V}$, Twist-like vector $\mathcal{W}$, and Wrench $\mathcal{G}$. This ensures that the bilinear form (power) is preserved under transformations.

Mathematical Principle 8.2. Ad vs. ad: Two Distinct Operators

The notation in spatial algebra distinguishes two fundamentally different operators that are easily confused:

- $[\mathrm{Ad}_{\mathbf{T}}] \in \mathbb{R}^{6\times 6}$: The **Adjoint** of a rigid body transform $\mathbf{T} \in SE(3)(3)$. This is a coordinate transformation that re-expresses a spatial velocity (or acceleration) measured in one frame into another frame. It depends on a *pose* (position and orientation).
- $[\mathrm{ad}_{\mathcal{V}}] = [\mathcal{V}\times] \in \mathbb{R}^{6\times 6}$: The **adjoint** (lowercase), also called the *Lie bracket* or *spatial motion cross product*. This maps a spatial velocity $\mathcal{V}$ to its cross-product operator, which computes Coriolis and centrifugal accelerations. It depends on a *velocity*, not a pose.

In recursive dynamics algorithms such as the RNEA, both operators appear: $[\mathrm{Ad}_{\mathbf{T}}]$ propagates velocities across frames (a coordinate change), while $[\mathrm{ad}_{\mathcal{V}}] = [\mathcal{V}\times]$ computes the velocity-dependent acceleration terms (Coriolis). Confusing the two produces incorrect dynamics [13, 35].

8.5 The Spatial Inertia Matrix

Just as scalar mass m resists linear acceleration ($F = ma$), and a 3×3 inertia tensor $\mathbf{I}_c$ resists angular acceleration ($\boldsymbol{\tau} = \mathbf{I}_c \boldsymbol{\alpha}$), Spatial Algebra unifies them into a single 6×6 **Spatial Inertia Matrix** $\mathcal{I}$.

8.5.1 Definition and Properties

Definition 8.6. Spatial Inertia Matrix

For a rigid body with:

- Mass m,
- Center of mass (CoM) at position $\mathbf{c}$ relative to the reference frame origin,
- Rotational inertia tensor $\mathbf{I}_c$ (about the CoM),

the Spatial Inertia Matrix expressed at the frame origin is:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c - m[\mathbf{c}][\mathbf{c}] & m[\mathbf{c}] \\ -m[\mathbf{c}] & m\mathbf{I}_{3\times3} \end{bmatrix} \tag{8.11}$$

Using the identity $[\mathbf{c}][\mathbf{c}] = -\mathbf{c}\mathbf{c}^T + (\mathbf{c}^T\mathbf{c})\mathbf{I}_{3\times3}$, this can be rewritten as:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c + m(|\mathbf{c}|^2\mathbf{I}_{3\times3} - \mathbf{c}\mathbf{c}^T) & m[\mathbf{c}] \\ -m[\mathbf{c}] & m\mathbf{I}_{3\times3} \end{bmatrix} \tag{8.12}$$

When the reference frame origin is *at* the center of mass ($\mathbf{c} = \mathbf{0}$), this simplifies to:

$$\mathcal{I}_{\text{CoM}} = \begin{bmatrix} \mathbf{I}_c & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & m\mathbf{I}_{3\times3} \end{bmatrix} \tag{8.13}$$

8.5.2 Symmetric and Positive Definite

Theorem 8.1. Spatial Inertia is SPD

The Spatial Inertia Matrix $\mathcal{I}$ is symmetric and positive definite.
Proof (Symmetry): Write the spatial inertia in standard form:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T & m[\mathbf{c}] \\ m[\mathbf{c}]^T & m\mathbf{I}_{3\times3} \end{bmatrix}.$$

Since $[\mathbf{c}]$ is skew-symmetric ($[\mathbf{c}]^T = -[\mathbf{c}]$), each block is symmetric or correctly transposed:

- Top-left block: $\mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T$ is symmetric (both $\mathbf{I}_c$ and the product $[\mathbf{c}][\mathbf{c}]^T$ are symmetric).
- Off-diagonal blocks: The top-right block is $m[\mathbf{c}]$, and the bottom-left block is $m[\mathbf{c}]^T = -m[\mathbf{c}]$. Their transposes satisfy $(m[\mathbf{c}])^T = m[\mathbf{c}]^T$, confirming $\mathcal{I}^T = \mathcal{I}$.
- Bottom-right block: $m\mathbf{I}_{3\times3}$ is trivially symmetric.

Proof (Positive Definite): For any non-zero spatial vector $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$, we compute:

$$\mathcal{V}^T\mathcal{I}\mathcal{V} = \boldsymbol{\omega}^T(\mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T)\boldsymbol{\omega} + 2m\boldsymbol{\omega}^T[\mathbf{c}]\mathbf{v} + m\mathbf{v}^T\mathbf{v}$$

This can be rewritten as the rotational kinetic energy plus the translational kinetic energy:

$$\mathcal{V}^T \mathcal{I} \mathcal{V} = \boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega} + m|\mathbf{v} + [\mathbf{c}]\boldsymbol{\omega}|^2$$

Since $\mathbf{I}_c$ is positive definite (non-zero $\boldsymbol{\omega}$ implies $\boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega} > 0$), the entire expression is positive whenever $\mathcal{V} \neq 0$.

8.6 The Parallel Axis Theorem in 6D

8.6.1 Derivation From First Principles

The classical parallel axis theorem states that if the inertia tensor about the center of mass is $\mathbf{I}_c$, then the inertia tensor about a point at distance $\mathbf{c}$ from the CoM is:

$$\mathbf{I}_{\text{origin}} = \mathbf{I}_c + m(c^2 \mathbf{I}_{3\times 3} - \mathbf{c}\mathbf{c}^T)$$

In spatial form, the parallel axis theorem extends to both translational and rotational terms. If we define:

$$\mathcal{I}_0 = \text{Spatial inertia at origin}$$

$$\mathcal{I}_c = \text{Spatial inertia at CoM}$$

then the relationship is:

Theorem 8.2. Parallel Axis Theorem in 6D

$$\mathcal{I}_0 = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & \mathbf{R} \end{bmatrix} \begin{bmatrix} \mathbf{I}_c & \mathbf{0} \\ \mathbf{0} & m\mathbf{I}_{3\times 3} \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & \mathbf{0} \\ \mathbf{0} & \mathbf{R}^T \end{bmatrix} + \begin{bmatrix} -m[\mathbf{c}][\mathbf{c}]^T & m[\mathbf{c}]^T \\ m[\mathbf{c}] & \mathbf{0} \end{bmatrix} \tag{8.14}$$

When $\mathbf{R} = \mathbf{I}$ (same orientation):

$$\mathcal{I}_0 = \begin{bmatrix} \mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T & m[\mathbf{c}] \\ m[\mathbf{c}]^T & m\mathbf{I}_{3\times 3} \end{bmatrix} \tag{8.15}$$

Interpretation: The top-left block contains the parallel axis correction to rotational inertia; the off-diagonals encode the coupling between angular and linear motion due to the offset CoM.

8.7 Composite Rigid Body Algorithm

In Chapter 7, we computed inverse dynamics using the RNEA. An alternative and equally important algorithm is the **Composite Rigid Body Algorithm (CRB)**, which builds up the spatial inertia of linked chains recursively.

8.7.1 Concept: Bottom-Up Inertia Assembly

Instead of computing torques directly, the CRB first assembles the inertia of composite structures: e.g., the forearm link is rigidly attached to the hand, so the hand's inertia is added to the forearm's. By recursively building composite inertias from tips to base, we can pre-compute the total mass matrix $\mathbf{M}(q)$ in $O(N)$ time.

Algorithm 8.1. Composite Rigid Body Algorithm

```
function CompositeRigidBody(links, joints, lambda):
    // Input: link inertias, joint structure
    // Output: composite inertias I_c for each link

    I_c = [None] * (N+1)          // Will store composite inertia of
    link i and all descendants

    for i <-N down to 1:          // Traverse from tips to base
        I_c[i] <-I[i]             // Start with link i's own inertia

        for j in children[i]:
            // Add inertia of child j to link i
            T_j_to_i <-relative transform from j to i
            Ad_j_to_i <-Adjoint of T_j_to_i
            I_c[i] <-I_c[i] + Ad_j_to_i^T * I_c[j] * Ad_j_to_i

    return I_c
```

8.7.2 Use in Computing the Mass Matrix

Once the composite inertias are assembled, the mass matrix can be extracted via:

$$\mathbf{M}_{ij}(q) = \mathcal{S}_i^T [\mathrm{Ad}_{\mathbf{T}_{i,j}}]^{-T} \mathcal{I}_{c,j}(q) [\mathrm{Ad}_{\mathbf{T}_{i,j}}]^{-1} \mathcal{S}_j$$

where $\mathcal{I}_{c,j}$ is the composite inertia of link j and all its descendants.

Theorem 8.3. CRB Complexity

The Composite Rigid Body Algorithm computes the mass matrix $\mathbf{M}(q)$ in $O(N^2)$ time (because there are N^2 entries), but the inertia assembly itself (bottom-up recursion) is $O(N)$. Once assembled, forward-dynamics calculations (solving for accelerations given forces) are $O(N)$ via matrix multiplication.

8.8 Spatial Algebra for Planar Systems

For systems constrained to a plane (e.g., 2D manipulation, golf swings), we can work with reduced spatial representations.

8.8.1 3D Vectors for 2D Motion

A planar motion has:

- Angular velocity ω (a scalar, rotation about the z-axis).
- Linear velocity $\mathbf{v} = (v_x, v_y) \in \mathbb{R}^2$.

We can embed this in a 3D twist:

$$\mathcal{V}_{\text{planar}} = (\omega, v_x, v_y)^T \in \mathbb{R}^3$$

The motion cross product for planar motion simplifies:

$$[\mathcal{V}\times]_{\text{planar}} = \begin{bmatrix} 0 & -v_y & v_x \\ v_y & 0 & \omega \\ -v_x & -\omega & 0 \end{bmatrix}$$

A planar spatial inertia matrix is 3×3:

$$\mathcal{I}_{\text{planar}} = \begin{bmatrix} I_z & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{bmatrix}$$

when the CoM is at the origin. With offset $\mathbf{c} = (c_x, c_y)$:

$$\mathcal{I}_{\text{planar}} = \begin{bmatrix} I_z + m(c_x^2 + c_y^2) & mc_y & -mc_x \\ mc_y & m & 0 \\ -mc_x & 0 & m \end{bmatrix}$$

8.8.2 Computational Advantage

Working in 3D (instead of 6D) reduces memory and computation by a factor of 4, making planar mechanisms extremely fast to simulate.

8.9 Newton-Euler Equations in Spatial Form

The fundamental equations of dynamics in spatial form are elegant and reveal the unity of linear and angular motion.

8.9.1 Spatial Momentum and the Equation of Motion

The spatial momentum of a rigid body with spatial inertia $\mathcal{I}$ and spatial velocity $\mathcal{V}$ is:

$$\mathcal{P} = \mathcal{I}\mathcal{V} \tag{8.16}$$

The equation of motion (Newton-Euler in spatial form) states that the time derivative of spatial momentum equals the applied wrench:

$$\frac{d\mathcal{P}}{dt} = \mathcal{F}_{\text{ext}} \tag{8.17}$$

However, because $\mathcal{I}$ itself may be time-varying (when expressed in a rotating frame), the full equation includes a bias term:

$$\frac{d}{dt}(\mathcal{I}\mathcal{V}) = \mathcal{I}\dot{\mathcal{V}} + \dot{\mathcal{I}}\mathcal{V} \tag{8.18}$$

In a *body-fixed frame* (rotating with the body), $\mathcal{I}$ is constant, but velocities are measured in the rotating frame, introducing the bias term $[\mathcal{V}\times]^*(\mathcal{I}\mathcal{V})$:

Theorem 8.4. Spatial Newton-Euler Equation

For a rigid body in a body-fixed frame, the equation of motion is:

$$\mathcal{F} = \mathcal{I}\dot{\mathcal{V}} + [\mathcal{V}\times]^*(\mathcal{I}\mathcal{V}) \tag{8.19}$$

Interpretation:

- First term: the inertial wrench (like $F = ma$ in 6D).
- Second term: the "bias" or "gyroscopic" wrench, capturing Coriolis and centrifugal effects.

Derivation: In an inertial frame:

$$\mathcal{F}_{\text{inertial}} = \frac{d}{dt}(\mathcal{I}_{\text{inertial}}\mathcal{V}_{\text{inertial}})$$

Transforming to a body-fixed frame using the adjoint, the time derivative picks up the Lie bracket (commutator) of spatial velocities, which manifests as $[\mathcal{V}\times]^*$.

8.10 Connection to Dual Quaternions

8.10.1 Brief Overview

Dual quaternions extend quaternions (Section 4.8) by using dual numbers instead of real numbers. A dual quaternion is:

$$\mathbf{q}_d = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} + \epsilon(q_4 + q_5\mathbf{i} + q_6\mathbf{j} + q_7\mathbf{k})$$

where ϵ is the dual unit satisfying $\epsilon^2 = 0$ (and $\epsilon \neq 0$).

Dual quaternions are isomorphic to spatial vectors and can represent both rotation and translation in a single object. Some robotics frameworks use dual quaternions for inertia and screw calculations, though the 6×6 matrix representation is more standard for computation.

Key property: The spatial inertia can be represented as a dual-quaternion-valued linear form, and Featherstone's spatial algorithms can be rewritten using dual quaternions, often with elegant geometric interpretations.

For this course, we continue with the matrix representation, as it is more straightforward for algorithmic implementation.

8.11 Worked Example: Spatial Inertia of a Multi-Link System

Consider a two-link arm where:

- Link 1 (upper arm): mass $m_1 = 2$ kg, CoM at $(0.3, 0, 0)$ m from joint 1, rotational inertia $\mathbf{I}_c^{(1)} = \text{diag}(0.05, 0.05, 0.01)$ kg·m^2.
- Link 2 (forearm): mass $m_2 = 1$ kg, CoM at $(0.25, 0, 0)$ m from joint 2, rotational inertia $\mathbf{I}_c^{(2)} = \text{diag}(0.02, 0.02, 0.005)$ kg·m^2.

8.11.1 Building Link Spatial Inertias

For link 1, with CoM offset $\mathbf{c}_1 = (0.3, 0, 0)^T$:

$$[\mathbf{c}_1] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -0.3 \\ 0 & 0.3 & 0 \end{bmatrix}$$

$$[\mathbf{c}_1][\mathbf{c}_1]^T = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0.09 & 0 \\ 0 & 0 & 0.09 \end{bmatrix}$$

$$\mathcal{I}^{(1)} = \begin{bmatrix} \mathbf{I}_c^{(1)} + m_1[\mathbf{c}_1][\mathbf{c}_1]^T & m_1[\mathbf{c}_1] \\ m_1[\mathbf{c}_1]^T & m_1\mathbf{I}_{3\times3} \end{bmatrix}$$

$$\mathcal{I}^{(1)} = \begin{bmatrix} 0.05 + 2(0) & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.05 + 2(0.09) & 0 & 0 & 0 & -0.6 \\ 0 & 0 & 0.01 + 2(0.09) & 0 & 0.6 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0.6 & 0 & 2 & 0 \\ 0 & -0.6 & 0 & 0 & 0 & 2 \end{bmatrix}$$

$$\mathcal{I}^{(1)} = \begin{bmatrix} 0.05 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.23 & 0 & 0 & 0 & -0.6 \\ 0 & 0 & 0.19 & 0 & 0.6 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0.6 & 0 & 2 & 0 \\ 0 & -0.6 & 0 & 0 & 0 & 2 \end{bmatrix}$$

Note the structure of the correction. Since $[\mathbf{c}][\mathbf{c}]^T = |\mathbf{c}|^2\mathbf{I} - \mathbf{c}\mathbf{c}^T$ is positive semi-definite, the parallel axis term always *adds* to the rotational inertia. With $\mathbf{c}_1 = (0.3, 0, 0)$ the correction vanishes about the offset direction itself (x) and contributes $m_1 d^2 = 2(0.3)^2 = 0.18$ about y and z — exactly the classical parallel axis theorem. A spatial inertia is symmetric positive definite by Theorem 8.1, so a negative diagonal entry would signal an algebraic error, not a physical configuration.

Similarly, $\mathcal{I}^{(2)}$ is computed with $\mathbf{c}_2 = (0.25, 0, 0)$ and $m_2 = 1$ kg.

8.11.2 Composite Inertia

If link 2 is rigidly attached to link 1 (e.g., they are the same rigid body or perfectly welded), the composite inertia is:

$$\mathcal{I}_{\text{composite}} = \mathcal{I}^{(1)} + \mathcal{I}^{(2)}$$

This is simple summation when expressed in the same reference frame.

If the links are separated by a relative transform $\mathbf{T}_{1\to2}$, we must first transform $\mathcal{I}^{(2)}$ to the frame of link 1:

$$\mathcal{I}_{\text{composite}} = \mathcal{I}^{(1)} + [\text{Ad}_{\mathbf{T}_{1\to2}}]^{-T}\mathcal{I}^{(2)}[\text{Ad}_{\mathbf{T}_{1\to2}}]^{-1}$$

8.12 Physical Interpretation of Spatial Inertia Elements

The 6×6 spatial inertia matrix has a clear physical meaning for each block:

- **Top-left** (3×3): Rotational inertia (analogous to the 3×3 inertia tensor). It relates angular acceleration to the torque required.
- **Top-right** (3×3): Coupling between linear and rotational motion due to CoM offset. If the CoM is not at the origin, accelerating the origin in one direction produces a torque around it.
- **Bottom-left** (3×3): Transpose of top-right (due to symmetry).
- **Bottom-right** (3×3): Mass matrix ($m\mathbf{I}_{3\times3}$). It relates linear acceleration of the origin to the force required.

8.12.1 Kinetic Energy and the Origin of Spatial Inertia

The 6×6 spatial inertia matrix is not an arbitrary packaging of mass properties. It is the *unique* object that makes kinetic energy a quadratic form in spatial velocity. We now prove this from first principles.

Classical kinetic energy. For a rigid body with mass m, centre-of-mass velocity $\mathbf{v}_{cm}$, angular velocity $\boldsymbol{\omega}$, and rotational inertia $\mathbf{I}_c$ about the centre of mass:

$$T = \frac{1}{2}\, m\, \|\mathbf{v}_{cm}\|^2 + \frac{1}{2}\, \boldsymbol{\omega}^T \mathbf{I}_c\, \boldsymbol{\omega}.$$

Re-express about a body-fixed frame origin. Let the body frame origin O be offset from the centre of mass by $\mathbf{c} \in \mathbb{R}^3$. Then the centre-of-mass velocity is

$$\mathbf{v}_{cm} = \mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{c},$$

where $\mathbf{v}_O$ is the linear velocity of O. Substituting into T and expanding:

$$\begin{aligned} T &= \frac{1}{2}\, m\, \|\mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{c}\|^2 + \frac{1}{2}\, \boldsymbol{\omega}^T \mathbf{I}_c\, \boldsymbol{\omega} \\ &= \frac{1}{2}\, m\, \mathbf{v}_O^T \mathbf{v}_O + m\, \mathbf{v}_O^T (\boldsymbol{\omega} \times \mathbf{c}) + \frac{1}{2}\, m\, (\boldsymbol{\omega} \times \mathbf{c})^T (\boldsymbol{\omega} \times \mathbf{c}) + \frac{1}{2}\, \boldsymbol{\omega}^T \mathbf{I}_c\, \boldsymbol{\omega}. \end{aligned}$$

Using the skew-symmetric matrix $[\mathbf{c}]_\times$ so that $\boldsymbol{\omega} \times \mathbf{c} = [\mathbf{c}]_\times^T \boldsymbol{\omega}$, and collecting the angular and linear components into the spatial velocity $\mathcal{V} = (\boldsymbol{\omega},\, \mathbf{v}_O)^T$, we obtain:

Theorem 8.5. Kinetic Energy as a Spatial Quadratic Form

The kinetic energy of a rigid body equals

$$T = \frac{1}{2}\, \mathcal{V}^T \mathcal{I}\, \mathcal{V},$$

where the 6×6 **spatial inertia matrix** is

$$\mathcal{I} = \begin{pmatrix} \mathbf{I}_c + m\, [\mathbf{c}]_\times [\mathbf{c}]_\times^T & m\, [\mathbf{c}]_\times \\ m\, [\mathbf{c}]_\times^T & m\, \mathbf{I}_{3\times3} \end{pmatrix}.$$

This derivation proves that $\mathcal{I}$ is the *correct* inertial object for spatial dynamics: it is determined entirely by the requirement that kinetic energy be a quadratic form in $\mathcal{V}$.

The **off-diagonal blocks** $m[\mathbf{c}]_\times$ encode the cross-coupling between rotation and translation that arises whenever the centre of mass does not coincide with the frame origin ($\mathbf{c} \neq \mathbf{0}$). When the frame is placed at the centre of mass ($\mathbf{c} = \mathbf{0}$), these blocks vanish and the spatial inertia becomes block-diagonal—rotational and translational energies decouple [13].

8.13 Python Implementation

```python
import numpy as np

def skew(v):
    """Convert 3D vector to skew-symmetric matrix."""
    return np.array([
        [0, -v[2], v[1]],
        [v[2], 0, -v[0]],
        [-v[1], v[0], 0]
    ])

def motion_cross(V):
    """
    Spatial cross product for motion (Twists).
    V: 6D spatial velocity (omega, v)
    Returns: 6x6 operator [Vx]
    """
    omega = V[:3]
    v = V[3:]
    Vx = np.zeros((6, 6))
    Vx[:3, :3] = skew(omega)
    Vx[3:, :3] = skew(v)
    Vx[3:, 3:] = skew(omega)
    return Vx

def force_cross(V):
    """
    Spatial cross product for forces (Wrenches).
    Returns: 6x6 operator [Vx*] = -[Vx]^T
    """
    return -motion_cross(V).T

def spatial_inertia(m, c, I_c):
    """
    Construct 6x6 spatial inertia matrix.
    Args:
        m: scalar mass
        c: 3D center of mass offset from origin
        I_c: 3x3 rotational inertia about CoM
    Returns:
        I: 6x6 spatial inertia matrix
    """
    I_spatial = np.zeros((6, 6))
    c_skew = skew(c)

    # Top-left: I_c + m[c][c]^T (parallel axis theorem).
    # [c][c]^T = |c|^2 I - c c^T is positive semi-definite, so this ADDS to
    # the rotational inertia. Equivalently I_c - m[c][c], since [c]^T = -[c].
    I_spatial[:3, :3] = I_c + m * (c_skew @ c_skew.T)
```

```python

    # Top-right: m[c]
    I_spatial[:3, 3:] = m * c_skew

    # Bottom-left: m[c]^T
    I_spatial[3:, :3] = m * c_skew.T

    # Bottom-right: m * I_3
    I_spatial[3:, 3:] = m * np.eye(3)

    return I_spatial

def transform_inertia(I, T):
    """
    Transform spatial inertia from one frame to another.
    I: 6x6 spatial inertia in source frame
    T: 4x4 transformation matrix (to target frame)
    Returns: transformed 6x6 spatial inertia
    """
    R = T[:3, :3]
    p = T[:3, 3]

    # Construct transformation for inertia
    # I' = [Ad_T]^-T I [Ad_T]^-1
    Ad = np.zeros((6, 6))
    Ad[:3, :3] = R
    Ad[3:, :3] = skew(p) @ R
    Ad[3:, 3:] = R

    Ad_inv_T = np.linalg.inv(Ad).T
    return Ad_inv_T @ I @ np.linalg.inv(Ad)

def composite_rigid_body(link_inertias, link_transforms, parent_array):
    """
    Assemble composite rigid body inertias.
    Args:
        link_inertias: list of 6x6 inertia matrices
        link_transforms: list of 4x4 transforms to parent frames
        parent_array: parent[i] = index of parent of link i (or -1 for root)
    Returns:
        composite_inertias: list of 6x6 composite inertias
    """
    N = len(link_inertias)
    composite_inertias = [np.copy(I) for I in link_inertias]

    # Process from leaves to root (reverse order)
    for i in range(N - 1, -1, -1):
        parent = parent_array[i]
        if parent >= 0:
            # Transform this link's composite inertia to parent frame
            I_in_parent = transform_inertia(
                composite_inertias[i],
                np.linalg.inv(link_transforms[i])
            )
            # Add to parent's inertia
            composite_inertias[parent] += I_in_parent

    return composite_inertias

# Example: Two-link arm
```

```
if __name__ == "__main__":
    # Link 1: upper arm
    m1 = 2.0
    c1 = np.array([0.3, 0.0, 0.0])
    I_c1 = np.diag([0.05, 0.05, 0.01])
    I1 = spatial_inertia(m1, c1, I_c1)

    # Link 2: forearm
    m2 = 1.0
    c2 = np.array([0.25, 0.0, 0.0])
    I_c2 = np.diag([0.02, 0.02, 0.005])
    I2 = spatial_inertia(m2, c2, I_c2)

    print("Link 1 Spatial Inertia:\n", np.round(I1, 4))
    print("\nLink 2 Spatial Inertia:\n", np.round(I2, 4))

    # Test motion cross product
    V = np.array([0.0, 0.0, 1.0, 1.0, 0.0, 0.0])  # Rotating around z, translating in x
    Vx = motion_cross(V)
    print("\nMotion Cross Product [Vx]:\n", np.round(Vx, 4))

    # Test duality
    F = np.array([0.1, 0.2, 0.3, 1.0, 2.0, 3.0])
    W = np.array([0.5, 0.5, 0.5, 0.1, 0.1, 0.1])

    lhs = F @ (motion_cross(V) @ W)
    rhs = (force_cross(V) @ F) @ W
    print("\nDuality test (should be equal):")
    print(f"  F^T [Vx] W = {lhs:.6f}")
    print(f"  [Vx*] F)^T W = {rhs:.6f}")
```

Listing 8.1: Spatial Algebra and Inertia Computation in Python

8.14 Summary

In this chapter, we developed Spatial Vector Algebra—the mathematical language for unified 6D mechanics:

1. **Historical Context**: Plücker's 19th-century line geometry, Ball's screws, and Featherstone's modern synthesis created a computationally elegant framework.

2. **Plücker Coordinates**: Lines in 3D are represented as 6D vectors satisfying a constraint, naturally connecting to spatial twists and wrenches.

3. **Spatial Vectors**: Twists (velocity) and Wrenches (force) are the fundamental 6D objects, living in dual spaces $\mathfrak{se}(3)(3)$ and $\mathfrak{se}(3)^*(3)$.

4. **Motion and Force Cross Products**: Two distinct 6×6 cross product operators capture the geometry of relative motion and force application; they are dual to preserve power.

5. **Spatial Inertia**: A single symmetric, positive-definite 6×6 matrix unifies mass and rotational inertia, including the parallel axis theorem as a block structure.

6. **Parallel Axis Theorem in 6D**: The full theorem includes coupling terms between angular and linear motion due to CoM offset.

7. **Composite Rigid Body Algorithm**: Inertias of linked chains are assembled bottom-up in $O(N)$ time via adjoint transformations.

8. **Planar Reduction**: 2D systems use 3D spatial algebra, reducing computation by a factor of 4.

9. **Newton-Euler in Spatial Form**: A single matrix equation $\mathcal{F} = \mathcal{I}\dot{\mathcal{V}} + [\mathcal{V}\times]^*(\mathcal{I}\mathcal{V})$ unifies linear and angular dynamics.

10. **Dual Quaternions**: Brief connection to an alternative 8D representation with elegant geometric properties.

11. **Physical Interpretation**: Each block of the spatial inertia matrix has clear mechanical meaning.

12. **Python Implementation**: Full code for inertia computation, transformation, and composite body assembly.

With Spatial Vector Algebra in hand, the Recursive Newton-Euler Algorithm of Chapter 7 becomes a straightforward loop of matrix multiplications, without a single transcendental function or explicit cross product in sight. This unity—where complex nonlinear dynamics reduce to structured linear algebra—is the deep beauty of modern computational mechanics.

8.15 Further Reading

The primary reference for spatial vector algebra is Featherstone [13], whose monograph develops the full 6×6 framework from first principles and demonstrates its application to recursive multi-body dynamics algorithms. Featherstone's treatment of spatial inertia, the motion and force cross products, and the composite rigid body algorithm is definitive, and readers seeking to implement spatial algebra in production code should consult his detailed pseudocode and numerical recipes.

The mathematical foundations connecting spatial vectors to the Lie algebra $\mathfrak{se}(3)(3)$ and its dual $\mathfrak{se}(3)^*(3)$ are developed rigorously by Murray, Li, and Sastry [35]. Their perspective clarifies why twists and wrenches live in dual vector spaces and why the adjoint and coadjoint representations have the specific transformation properties used throughout this chapter.

For the classical treatment of rigid body dynamics, inertia tensors, and the parallel axis theorem in their traditional 3×3 form, Goldstein, Poole, and Safko [15] remains the standard graduate-level reference. Comparing their development with the 6×6 spatial inertia formulation of this chapter illuminates how spatial algebra unifies and streamlines results that classically require separate angular and translational analyses.

Marsden and Ratiu [32] place spatial algebra within the broader framework of geometric mechanics, connecting spatial inertia to momentum maps and showing how the Euler equations for rigid body motion arise naturally from reduction on SE(3)(3). Their treatment is more abstract but reveals the deep geometric structures underlying the computational framework.

It is worth emphasizing that spatial algebra is the computational backbone of modern physics engines. MuJoCo [48], in particular, implements spatial vector arithmetic as its core computational primitive, using the 6×6 framework to achieve real-time simulation of complex contact-rich robotic systems. Understanding the material in this chapter is therefore essential not only for theoretical mechanics but for practical work with any modern simulation platform.

8.16 Exercises

8.16.1 Tier 1 (Foundation)

1. Construct the motion cross product $[\mathcal{V}\times]$ for $\mathcal{V} = (0, 0, \omega, v_x, v_y, 0)^T$ and verify its action on a unit spatial vector.

2. Verify that the force cross product $[\mathcal{V}\times^*]$ is the negative transpose of the motion cross product.

3. For a point mass m at location $\mathbf{c}$ with no rotational inertia (zero $\mathbf{I}_c$), compute the spatial inertia matrix and verify it is positive definite.

4. Compute the spatial inertia of a uniform rod of mass m and length L at its center of mass, and again at one end (using the parallel axis theorem).

5. Verify that power $\mathcal{F}^T\mathcal{V}$ is invariant under the frame transformation: compute it in two different frames and confirm they are equal.

8.16.2 Tier 2 (Intermediate)

1. Prove that the spatial inertia matrix is invariant under orthogonal rotations of the reference frame (i.e., only the CoM offset affects the matrix structure).

2. For a two-link arm where link 2 is offset from link 1 by transform $\mathbf{T}_{1\to2}$, use the composite rigid body algorithm to express the total inertia as a function of both link inertias and the relative pose.

3. Implement the duality property: compute $\mathcal{G}^T([\mathcal{V}\times]\mathcal{W}) = ([\mathcal{V}\times^*]\mathcal{G})^T\mathcal{W}$ numerically for random spatial vectors and verify to machine precision.

4. Derive the relationship between the classical 3D inertia tensor and the spatial inertia matrix; show how the 3x3 blocks correspond to standard mechanics quantities.

5. For a humanoid robot, conceptually design the composite inertia computation: which links would be aggregated together, and in what order?

8.16.3 Tier 3 (Advanced)

1. Prove the Plücker relation $\mathbf{d}^T\mathbf{m} = 0$ geometrically: interpret the direction and moment of a line and show why their dot product must vanish.

2. Show that the spatial inertia matrix $\mathcal{I}$ can be expressed as a dual-quaternion-valued bilinear form, and compute inertial effects using dual quaternion multiplication.

3. Derive the composite inertia algorithm from the perspective of the Adjoint map: show that $\mathcal{I}_{\text{composite}} = \sum_j [\text{Ad}_{\mathbf{T}_{j,\text{origin}}}]^{-T} \mathcal{I}_j [\text{Ad}_{\mathbf{T}_{j,\text{origin}}}]^{-1}$ reduces to the recursive algorithm.

4. For a system with articulated contact (e.g., foot on ground with rolling constraints), extend the spatial inertia formalism to include constraint forces and derive the reduced dynamics.

5. Implement a multi-body dynamics simulator using purely spatial algebra: compose RNEA (Chapter 7) with the spatial inertia formalism, and validate against a reference physics engine.

Chapter 9

The Product of Exponentials Formula

You do not need to invent abstract coordinate frames at every joint of a robot. The kinematics of the entire complex machine can be compactly described by a single product of twisting screws.

In Plain Language 9.1. Context & Use Case: Universal Forward Kinematics

The Math You Will See: We introduce the **Product of Exponentials (PoE)** formula, utilizing the Matrix Exponential ($e^{[\mathcal{S}]\theta}$) directly. **Why We Need It:** In classical robotics (taught in the 1980s), you tracked the position of a robot arm using "Denavit-Hartenberg (D-H) parameters", which required manually attaching weird local coordinate frames to every single joint using confusing perpendicular rules. The modern way (PoE) drops this completely. We just look at the robot in one global frame, find where the hinges point, and mathematically "twist" the arm around each hinge sequentially using our exact exponential maps. It is cleaner, faster, and never suffers from alignment ambiguities. **All Variables Defined:** $\mathbf{M}$ is the 4×4 "home" pose of the robot. $\mathcal{S}_i$ is the 6D Screw Axis of joint i. θ_i is how far the motor has moved.

9.1 Historical Context and the Evolution of Kinematic Formalisms

The theory of robotic kinematics has undergone a significant transformation over the past four decades. For nearly two centuries—from the mechanical linkage analysis of James Watt through the 1960s development of automated manufacturing—kinematic chains were described using local, Euclidean coordinate frames attached sequentially to each link. This frame-by-frame approach reached formal codification in 1955 with the *Denavit-Hartenberg (D-H)* parametrization [10], which became the dominant convention in robotics education and industrial practice.

The D-H convention, while algebraically complete, carries three fundamental drawbacks:

1. **Geometric Arbitrariness:** The perpendicularity rules for axis alignment create multiple valid D-H parameter sets for the same robot, requiring careful disambiguation.

2. **Notational Opacity:** Joint and link parameters are decoupled across four separate scalars $(a_i, d_i, \alpha_i, \theta_i)$, obscuring the underlying geometric intent.

3. **Singularity Avoidance:** Parallel or collinear axes demand special-case treatments, complicating software and theoretical analysis.

In the 1990s, Richard Murray, Zexiang Li, and Shankar Sastry—building on Lie group theory developed in differential geometry—published *A Mathematical Introduction to Robotic Manipulation* [35], which reformulated forward kinematics through the lens of exponential maps on the Lie group $SE(3)$ and its Lie algebra $\mathfrak{se}(3)(3)$. Their **Product of Exponentials (PoE)** formula unified kinematics under a single, coordinate-free framework: the composition of exponential twists in a global reference frame.

This approach was further refined and popularized by Kevin Lynch and Frank Park [30], whose textbook *Modern Robotics: Mechanics, Planning, and Control* established PoE as the modern standard. Today, PoE is the default kinematics formalism in advanced robotics research, computational mechanics, and biomechanics.

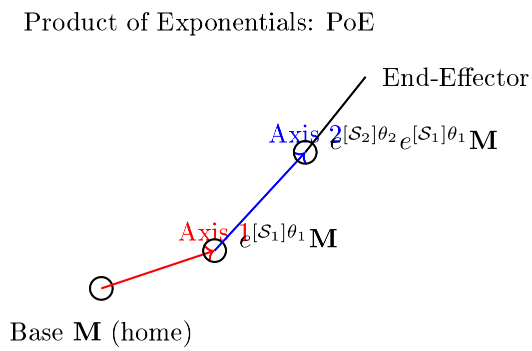


Figure 9.1: Product of Exponentials formula for forward kinematics. Each joint contributes an exponential twist applied to the previous configuration. The final end-effector pose is the product: $\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_n]\theta_n} \cdots e^{[\mathcal{S}_2]\theta_2} e^{[\mathcal{S}_1]\theta_1}\mathbf{M}$.

9.2 Abandoning Denavit-Hartenberg

The D-H convention assigns to each link i a local coordinate frame $\{\text{Frame}_i\}$ with four parameters:

$$d_i : \text{offset along the previous joint axis} \tag{9.1}$$

$$a_i : \text{length of the common perpendicular} \tag{9.2}$$

$$\alpha_i : \text{twist angle about the perpendicular} \tag{9.3}$$

$$\theta_i : \text{joint angle (for revolute joints)} \tag{9.4}$$

The combined transformation from frame $\{i-1\}$ to frame $\{i\}$ is:

$$\mathbf{A}_{i-1,i}(\theta_i) = \mathbf{Rot}_z(\theta_i)\mathbf{Trans}(d_i)\mathbf{Trans}(a_i)\mathbf{Rot}_x(\alpha_i) \tag{9.5}$$

The complete forward kinematics chain becomes:

$$\mathbf{T}(\boldsymbol{\theta}) = \mathbf{A}_{0,1}\mathbf{A}_{1,2}\cdots\mathbf{A}_{n-1,n} \tag{9.6}$$

where each transformation encodes one joint's motion relative to the previous link's local frame.

Why PoE is superior: The PoE formalism instead expresses all joint motions in a single, fixed **space frame** (the global reference frame). This eliminates the need to compute and track relative frame transformations and treats the kinematics as direct composition of rigid-body twists—a concept with deep physical meaning in mechanics.

9.3 Screws and Twists: A Geometric Foundation

Recall from Chapter 6 that a screw axis $\mathcal{S} = (\boldsymbol{\omega}, \mathbf{v}) \in \mathfrak{se}(3)(3)$ encodes:

- $\boldsymbol{\omega}$: the unit rotational axis (or zero for a purely prismatic joint)
- $\mathbf{v}$: the linear velocity component, related to the axis location and pitch

For a revolute joint (rotation only), the axis passes through a point $\mathbf{q}$ in space, and we define:

$$\mathcal{S}_{\text{rev}} = \begin{bmatrix} \boldsymbol{\omega} \\ -\boldsymbol{\omega} \times \mathbf{q} \end{bmatrix} \tag{9.7}$$

For a prismatic joint (translation along direction $\mathbf{v}$), we have:

$$\mathcal{S}_{\text{pris}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{v} \end{bmatrix} \tag{9.8}$$

The corresponding 4×4 skew-symmetric matrix representation is:

$$[\mathcal{S}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ \mathbf{0}^T & 0 \end{bmatrix} \tag{9.9}$$

9.4 The Forward Kinematics Equation: Space Frame PoE

If a robotic arm has n joints, the Product of Exponentials formula—originally formulated by Brockett [4] and developed into a complete kinematic framework by Murray, Li, and Sastry [35]—computes the final Homogeneous Transformation Matrix ($\mathbf{T}(\boldsymbol{\theta}) \in SE(3)$) of the end-effector relative to the base as:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \cdots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \tag{9.10}$$

Where:

1. $\mathbf{M} \in SE(3)$ is the **Home Configuration** matrix: the end-effector's pose when all joint angles $\theta_i = 0$. It is expressed in the space (global) frame and is typically computed by direct measurement or CAD.
2. $\mathcal{S}_i \in \mathfrak{se}(3)(3)$ is the **spatial Screw Axis** (the twist parameter) for joint i, expressed in the global Space Frame when the robot is at Home Configuration. It does *not* change as the robot moves.

3. θ_i is the actual angle (in radians, for revolute joints) or distance (in meters, for prismatic joints) the motor has moved.

4. $e^{[\mathcal{S}_i]\theta_i} \in SE(3)$ is the Matrix Exponential (developed in Chapter 6) mapping that single joint's motion into a 4×4 rigid body transform.

9.4.1 Derivation From First Principles

Imagine the robot at home configuration. As joint 1 rotates by θ_1, it rotates the entire downstream chain (all links $2, 3, \ldots, n$ plus the end-effector) by the exponential twist $e^{[\mathcal{S}_1]\theta_1}$. The end-effector moves to the configuration:

$$\mathbf{T}_1(\theta_1) = e^{[\mathcal{S}_1]\theta_1}\mathbf{M} \tag{9.11}$$

Now, while joint 1 is held fixed, joint 2 begins to move. Its axis is *fixed in the space frame*—it does not follow the rotated configuration of link 2. This is the crucial distinction between space-frame and body-frame PoE. Joint 2's twist applies to the pose resulting from joint 1:

$$\mathbf{T}_{1,2}(\theta_1, \theta_2) = e^{[\mathcal{S}_2]\theta_2}\mathbf{T}_1(\theta_1) = e^{[\mathcal{S}_2]\theta_2}e^{[\mathcal{S}_1]\theta_1}\mathbf{M} \tag{9.12}$$

Continuing inductively, the full Product of Exponentials is:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_n]\theta_n} \cdots e^{[\mathcal{S}_2]\theta_2}e^{[\mathcal{S}_1]\theta_1}\mathbf{M} \tag{9.13}$$

Note the reversed ordering: joint 1 is the leftmost exponential in the final product because it acts first (is applied closest to $\mathbf{M}$).

The elegance of this formulation lies in its physical transparency: each exponential is a pure twist in the space frame, and the product is a concatenation of rigid-body motions. No ad-hoc frame transformations are needed.

9.5 Body Frame PoE: Motion Relative to the Tool

In some contexts (particularly when working backward from the end-effector), it is convenient to express joint axes in the **body frame**—a frame attached to the end-effector and moving with it. The body frame PoE formula is:

$$\mathbf{T}(\boldsymbol{\theta}) = \mathbf{M}\, e^{[\mathcal{S}'_n]\theta_n}e^{[\mathcal{S}'_{n-1}]\theta_{n-1}} \cdots e^{[\mathcal{S}'_1]\theta_1} \tag{9.14}$$

Here, $\mathcal{S}'_i$ is the screw axis of joint i expressed in the body frame (tool frame). The key difference is that joint n acts first (is applied closest to the identity), and the exponentials are multiplied in increasing order of joint index.

9.5.1 Conversion Between Space and Body Frame

Given a body frame screw axis $\mathcal{S}'_i$ and a known transformation $\mathbf{M} \in SE(3)$, the corresponding space frame screw axis is obtained via the **Adjoint** action (see Chapter 8):

$$\mathcal{S}_i = \mathrm{Ad}_{\mathbf{T}}\mathcal{S}'_i \tag{9.15}$$

where $\mathrm{Ad}_{\mathbf{T}}$ is the adjoint representation of the transformation $\mathbf{T}$. In 6D spatial vector form:

$$\mathrm{Ad}_{\mathbf{T}} = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ [\mathbf{p}]\mathbf{R} & \mathbf{R} \end{bmatrix} \tag{9.16}$$

where $\mathbf{T} = (\mathbf{R}, \mathbf{p})$.

Equivalently, if we know all screw axes in space frame and wish to express them in body frame, we use:

$$\mathcal{S}_i' = \mathrm{Ad}_{\mathbf{M}^{-1}} \mathcal{S}_i \tag{9.17}$$

The two PoE formulations are mathematically equivalent; the choice depends on which frame is more convenient for the application or computation.

9.6 Spatial Jacobian From the Product of Exponentials

The **Spatial Jacobian** $\mathbf{J}_s(\boldsymbol{\theta})$ relates the time derivatives of joint angles to the spatial velocity (twist) of the end-effector:

$$\mathcal{V}_{\mathrm{ee}} = \mathbf{J}_s(\boldsymbol{\theta})\dot{\boldsymbol{\theta}} \tag{9.18}$$

where $\mathcal{V}_{\mathrm{ee}} = [\boldsymbol{\omega}, \mathbf{v}]^T \in \mathbb{R}^6$ is the 6D twist of the end-effector.

To derive the spatial Jacobian from the PoE formula, we differentiate Equation 9.10 with respect to time:

$$\dot{\mathbf{T}} = \frac{d}{dt}\left(e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \cdots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \right) \tag{9.19}$$

Using the product rule and chain rule on matrix exponentials (and noting that $[\mathcal{S}]$ is constant):

$$\dot{\mathbf{T}} = [\mathcal{S}_1]\dot{\theta}_1\, e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \cdots e^{[\mathcal{S}_n]\theta_n}\mathbf{M} \,+\, e^{[\mathcal{S}_1]\theta_1}[\mathcal{S}_2]\dot{\theta}_2\, e^{[\mathcal{S}_2]\theta_2} \cdots + \cdots \tag{9.20}$$

The spatial velocity (twist) at the end-effector is related to the time derivative of the transformation by:

$$\mathcal{V}_{\mathrm{ee}} = \dot{\mathbf{T}}\mathbf{T}^{-1} \tag{9.21}$$

where the product $\dot{\mathbf{T}}\mathbf{T}^{-1}$ extracts the velocity in the space frame (as opposed to the body frame).

After algebraic manipulation, we obtain the **spatial Jacobian columns**:

$$\mathbf{J}_s(\boldsymbol{\theta}) = \begin{bmatrix} \mathcal{S}_1 & e^{[\mathcal{S}_1]\theta_1}\mathcal{S}_2 & \cdots & e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_{n-1}]\theta_{n-1}}\mathcal{S}_n \end{bmatrix} \tag{9.22}$$

Each column is the corresponding screw axis, transformed by all the exponential twists that precede it in the PoE product. This encodes the fact that earlier joints affect the effective direction of later joints.

9.6.1 Direct Numerical Computation

In practice, the spatial Jacobian is computed numerically by:

1. Compute $\mathbf{T}(\boldsymbol{\theta})$ using the PoE formula.

2. For each joint i, compute the partial derivative using finite differences or automatic differentiation:
$$\frac{\partial \mathbf{T}}{\partial \theta_i} \approx \frac{\mathbf{T}(\boldsymbol{\theta} + \boldsymbol{\epsilon}_i) - \mathbf{T}(\boldsymbol{\theta} - \boldsymbol{\epsilon}_i)}{2\epsilon} \tag{9.23}$$
where $\boldsymbol{\epsilon}_i$ is a small perturbation in joint i only.

3. Extract the spatial velocity from each partial derivative.

9.7 Body Jacobian and the Adjoint Relationship

The **Body Jacobian** $\mathbf{J}_b(\boldsymbol{\theta})$ expresses the end-effector velocity in the body (tool) frame:

$$\mathcal{V}_{\text{ee}}^{\text{body}} = \mathbf{J}_b(\boldsymbol{\theta})\dot{\boldsymbol{\theta}} \tag{9.24}$$

The body Jacobian is related to the spatial Jacobian via the adjoint of the end-effector transformation:

$$\mathbf{J}_b(\boldsymbol{\theta}) = \mathrm{Ad}_{\mathbf{T}(\boldsymbol{\theta})^{-1}}\mathbf{J}_s(\boldsymbol{\theta}) \tag{9.25}$$

where the adjoint action on a Jacobian matrix (with columns as 6D vectors) is:

$$\mathrm{Ad}_{\mathbf{T}}\mathbf{J} = \mathrm{Ad}_{\mathbf{T}}[\mathrm{col}_1(\mathbf{J}), \mathrm{col}_2(\mathbf{J}), \ldots] \tag{9.26}$$

applied column-wise.

The body frame Jacobian is often more useful in control applications because the body frame is attached to the end-effector and its axes remain intuitively meaningful as the tool moves.

Having established both the spatial and body Jacobians, we now turn to the critical question of when these Jacobians break down. The Jacobian's rank determines the robot's instantaneous ability to generate motion in all directions, and configurations where this rank drops—called singularities—impose fundamental limits on the robot's dexterity. Understanding and detecting singularities is essential for safe and effective motion planning.

9.8 Singularity Analysis Using the PoE Jacobian

A robotic configuration is **singular** when the Jacobian loses rank, i.e., when $\mathrm{rank}(\mathbf{J}_s) < n$. At a singularity:

1. The robot cannot move in certain directions (loss of controllability).

2. Joint velocities required to achieve a desired end-effector motion become infinite.

3. The robot exhibits locked degrees of freedom.

9.8.1 Types of Singularities

- **Boundary Singularities:** The end-effector is at or beyond the reachable workspace boundary. Typically occur at full extension of the arm.
- **Interior Singularities:** Within the reachable workspace, occur when multiple joint axes become aligned. For example, in a planar 2R arm, singularity occurs when both links are collinear (fully extended or fully retracted).
- **Redundant Degeneracies:** In redundant robots (more DOFs than required for a task), the null space of the Jacobian is non-trivial, allowing motion without changing the end-effector pose.

9.8.2 Detection and Handling

Singularities are detected by:

1. Computing $\det(\mathbf{J}_s^T \mathbf{J}_s)$ (the condition number of the Jacobian squared).
2. If $\det(\mathbf{J}_s^T \mathbf{J}_s) < \epsilon_{\text{tol}}$ for a small threshold ϵ_{tol}, the configuration is near-singular.
3. The ratio $\det(\mathbf{J}_s^T \mathbf{J}_s)$ is called the **manipulability measure**; high values indicate dexterous configurations.

Numerically, near-singular configurations are handled by:

1. Adding damping to the inverse: $\mathbf{J}_{\text{damp}}^{-1} = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T + \lambda I)^{-1}$ (Levenberg-Marquardt damping).
2. Using the pseudo-inverse with a singular value threshold: keep only singular values above $\tau\sigma_{\max}$, where τ is a threshold ratio (e.g., $\tau = 10^{-4}$).

9.9 Comparison With D-H: A Worked Example

To illustrate the practical differences, consider a 3-DOF planar robot with:

- Link 1: length $L_1 = 1$ m
- Link 2: length $L_2 = 0.8$ m
- Link 3: length $L_3 = 0.6$ m
- All joints: revolute, rotating about the Z-axis (perpendicular to the table)

9.9.1 D-H Parametrization

The D-H parameters are:

Link i	a_{i-1}	d_i	α_{i-1}	θ_i
1	0	0	0	θ_1
2	L_1	0	0	θ_2
3	L_2	0	0	θ_3

The transformation from base to end-effector is:

$$\mathbf{T}_{\text{D-H}}(\boldsymbol{\theta}) = \mathbf{A}_{0,1}\mathbf{A}_{1,2}\mathbf{A}_{2,3} \tag{9.27}$$

where each $\mathbf{A}_{i,i+1}$ is computed using the D-H convention. At $\boldsymbol{\theta} = [45°, -30°, 60°]^T$, one obtains a specific end-effector pose.

9.9.2 PoE Parametrization

The home configuration ($\boldsymbol{\theta} = \mathbf{0}$) has the arm fully extended along the X-axis:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 + L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.28}$$

The three screw axes (all revolute about Z):

$$\mathcal{S}_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathcal{S}_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ -L_1 \\ 0 \end{bmatrix}, \quad \mathcal{S}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ -(L_1 + L_2) \\ 0 \end{bmatrix} \tag{9.29}$$

The PoE formula gives:

$$\mathbf{T}_{\text{PoE}}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} e^{[\mathcal{S}_3]\theta_3} \mathbf{M} \tag{9.30}$$

When computed at the same configuration $\boldsymbol{\theta} = [45°, -30°, 60°]^T$, the PoE and D-H formulations yield identical end-effector poses (as they must, since both are valid descriptions of the same kinematic chain).

9.9.3 Computational Advantages of PoE

1. **Consistency:** The PoE formula is coordinate-free; no ambiguity in frame attachment.
2. **Scalability:** Adding a new joint requires only specifying its screw axis; no recomputation of previous parameters.
3. **Jacobian Clarity:** The Jacobian columns have direct physical meaning: the i-th column is the screw axis of joint i as "seen" from the space frame after the effects of earlier joints.
4. **Software Simplicity:** A single, unified algorithm computes forward kinematics for any serial chain, regardless of link lengths or orientations.

With the comparison between the classical D-H convention [10] and the modern PoE approach now clear, we can extend the PoE framework to handle all joint types—including prismatic (sliding) joints—before returning to the question of how joint velocities propagate through the kinematic chain. The velocity kinematics that follow are a direct consequence of differentiating the PoE formula, unifying position and velocity analysis under a single mathematical roof.

9.10 PoE for Prismatic Joints

A prismatic (sliding) joint moves along a direction vector $\mathbf{d}$ without rotating. Its screw axis is:

$$\mathcal{S}_{\text{pris}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{d} \end{bmatrix} \tag{9.31}$$

where $\mathbf{d}$ is the unit direction of translation.

The exponential of a purely prismatic screw is:

$$e^{[\mathcal{S}_{\text{pris}}]d} = \begin{bmatrix} \mathbf{I}_{3\times 3} & d\mathbf{d} \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{9.32}$$

a pure translation matrix.

9.10.1 Example: 3D Cartesian Robot

A 3D Cartesian robot with three prismatic joints (X, Y, Z directions) has screw axes:

$$\mathcal{S}_x = [0, 0, 0, 1, 0, 0]^T \tag{9.33}$$

$$\mathcal{S}_y = [0, 0, 0, 0, 1, 0]^T \tag{9.34}$$

$$\mathcal{S}_z = [0, 0, 0, 0, 0, 1]^T \tag{9.35}$$

and home configuration $\mathbf{M} = \mathbf{I}$ (end-effector at the origin when all sliders are at zero position).

The forward kinematics is:

$$\mathbf{T}(d_x, d_y, d_z) = e^{[\mathcal{S}_x]d_x} e^{[\mathcal{S}_y]d_y} e^{[\mathcal{S}_z]d_z} = \begin{bmatrix} \mathbf{I} & [d_x, d_y, d_z]^T \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{9.36}$$

which correctly expresses a point displaced by (d_x, d_y, d_z) in the global frame.

9.11 Velocity Kinematics From PoE Differentiation

The instantaneous spatial velocity of the end-effector is obtained by differentiating the forward kinematics. Starting from:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \tag{9.37}$$

Differentiate with respect to time:

$$\dot{\mathbf{T}} = \sum_{i=1}^{n} \frac{d}{dt} \left(e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_i]\theta_i} \cdots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \right) \dot{\theta}_i \tag{9.38}$$

Using the derivative rule for matrix exponentials $\frac{d}{dt} e^{A(t)} = A(t) e^{A(t)}$ when $A(t)$ is linear in a parameter:

$$\dot{\mathbf{T}} = [\mathcal{S}_1]\dot{\theta}_1 e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} + e^{[\mathcal{S}_1]\theta_1} [\mathcal{S}_2]\dot{\theta}_2 e^{[\mathcal{S}_2]\theta_2} \cdots + \cdots \tag{9.39}$$

The spatial velocity is:

$$\mathcal{V} = \dot{\mathbf{T}}\mathbf{T}^{-1} \tag{9.40}$$

After collecting terms and using the adjoint action, the spatial velocity can be written as:

$$[\mathcal{V}] = \sum_{i=1}^{n} [\mathcal{S}_i^{\text{transformed}}]\dot{\theta}_i \tag{9.41}$$

where $\mathcal{S}_i^{\text{transformed}}$ is the screw axis of joint i after being advected by all preceding joints. In matrix form, this is precisely the spatial Jacobian:

$$\mathcal{V} = \mathbf{J}_s(\boldsymbol{\theta})\dot{\boldsymbol{\theta}} \tag{9.42}$$

9.12 Worked Example: Complete PoE for a Spatial 3R Robot

Consider a 3-DOF spatial manipulator (Spatial 3R) with:

- Base at origin; first joint axis along Z
- Link 1 length: $L_1 = 0.5$ m
- Second joint axis along Y (at the end of link 1)
- Link 2 length: $L_2 = 0.3$ m
- Third joint axis along X (at the end of link 2)
- Link 3 length: $L_3 = 0.2$ m

9.12.1 Home Configuration

At $\boldsymbol{\theta} = \mathbf{0}$, the robot is fully extended in the positive X direction:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 + L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1.0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.43}$$

9.12.2 Screw Axes

$$\text{Joint 1 (Z-axis, at origin):} \quad \mathcal{S}_1 = [0,0,1,0,0,0]^T \tag{9.44}$$

$$\text{Joint 2 (Y-axis, at } [L_1,0,0]^T = [0.5,0,0]^T) : \quad \mathcal{S}_2 = [0,1,0,0\cdot 0 - 0\cdot 0.5, 0\cdot 0.5 - 0\cdot 0, 1\cdot 0.5 - 0\cdot 0]^T \tag{9.45}$$

$$= [0,1,0,0,0,0.5]^T \tag{9.46}$$

$$\text{Joint 3 (X-axis, at } [L_1+L_2,0,0]^T = [0.8,0,0]^T) : \quad \mathcal{S}_3 = [1,0,0,0\cdot 0 - 0\cdot 0.8, 0\cdot 0.8 - 0\cdot 0, 0\cdot 0.8 - 0\cdot 0]^T \tag{9.47}$$

$$= [1,0,0,0,0,0]^T \tag{9.48}$$

9.12.3 Numerical Evaluation at a Test Configuration

At $\boldsymbol{\theta} = [\pi/4, \pi/6, -\pi/3]^T$ (45°, 30°, −60°):

Compute the matrix exponentials:

$$e^{[\mathcal{S}_1]\pi/4} = \begin{bmatrix} \cos(\pi/4) & -\sin(\pi/4) & 0 & 0 \\ \sin(\pi/4) & \cos(\pi/4) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.707 & -0.707 & 0 & 0 \\ 0.707 & 0.707 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.49}$$

(Rotation by $\pi/4$ about Z-axis; since the screw axis $\mathcal{S}_1$ passes through the origin, there is no translation component.)

For $e^{[\mathcal{S}_2]\pi/6}$, the screw axis has a rotation part (Y-axis) and a translation part ($v = [0, 0, 0.5]^T$). The exponential yields a rotation about the Y-axis combined with a translation. Detailed computation yields:

$$e^{[\mathcal{S}_2]\pi/6} \approx \begin{bmatrix} 0.966 & 0 & 0.259 & 0 \\ 0 & 1 & 0 & 0.067 \\ -0.259 & 0 & 0.966 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.50}$$

Similarly, $e^{[\mathcal{S}_3](-\pi/3)}$ produces a rotation about the X-axis plus a translation.

The full forward kinematics is:

$$\mathbf{T}(\pi/4, \pi/6, -\pi/3) = e^{[\mathcal{S}_1]\pi/4} e^{[\mathcal{S}_2]\pi/6} e^{[\mathcal{S}_3](-\pi/3)} \mathbf{M} \tag{9.51}$$

which, when computed numerically, yields a specific 3D position and orientation of the end-effector. The exact numerical result depends on careful matrix multiplication and is best computed programmatically (see Section 9.15).

9.13 Inverse Kinematics Using the Jacobian: Newton-Raphson

The inverse kinematics problem seeks to find joint angles $\boldsymbol{\theta}$ that place the end-effector at a desired pose $\mathbf{T}_{\text{desired}}$. Unlike forward kinematics, inverse kinematics has no closed-form solution for general serial chains and requires numerical optimization.

The Newton-Raphson method formulates IK as an optimization problem:

$$\min_{\boldsymbol{\theta}} \|\mathbf{T}(\boldsymbol{\theta}) - \mathbf{T}_{\text{desired}}\|_F^2 \tag{9.52}$$

where $\|\cdot\|_F$ is the Frobenius norm of the pose error.

9.13.1 Pose Error Representation

The error is typically decomposed into position and orientation components:

1. **Position Error:** $\mathbf{e}_p = \mathbf{p}_{\text{desired}} - \mathbf{p}(\boldsymbol{\theta})$, where $\mathbf{p}$ is the translation part of $\mathbf{T}$.
2. **Orientation Error:** $\mathbf{e}_o = \text{vex}(\mathbf{R}_{\text{desired}}\mathbf{R}(\boldsymbol{\theta})^T)$, where vex is the inverse of the skew-symmetric operator (extracting the axis-angle representation).

The combined error vector is:

$$\mathbf{e}(\boldsymbol{\theta}) = \begin{bmatrix} \mathbf{e}_p \\ \mathbf{e}_o \end{bmatrix} \in \mathbb{R}^6 \tag{9.53}$$

9.13.2 Newton-Raphson Update Rule

The Jacobian of the error is:

$$\frac{\partial \mathbf{e}}{\partial \boldsymbol{\theta}} = \begin{bmatrix} \frac{\partial \mathbf{p}}{\partial \boldsymbol{\theta}} \\ \frac{\partial \mathbf{e}_o}{\partial \boldsymbol{\theta}} \end{bmatrix} \tag{9.54}$$

For small errors, the relationship is approximately linear, and the Newton-Raphson update is:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \Delta\boldsymbol{\theta} \tag{9.55}$$

where $\Delta\boldsymbol{\theta}$ solves:

$$\mathbf{J}_s(\boldsymbol{\theta}_k)\Delta\boldsymbol{\theta} = \mathbf{e}(\boldsymbol{\theta}_k) \tag{9.56}$$

In practice, the pseudo-inverse is used to handle rank deficiency:

$$\Delta\boldsymbol{\theta} = \mathbf{J}_s^{\dagger}(\boldsymbol{\theta}_k)\mathbf{e}(\boldsymbol{\theta}_k) \tag{9.57}$$

where $\mathbf{J}_s^{\dagger}$ is the Moore-Penrose pseudo-inverse.

9.13.3 Algorithm and Convergence

1. Initialize $\boldsymbol{\theta}_0$ with an initial guess (e.g., the previous solution or a random configuration).
2. For $k = 0, 1, 2, \ldots, K_{\max}$:
   (a) Compute the forward kinematics $\mathbf{T}_k = \mathbf{T}(\boldsymbol{\theta}_k)$.
   (b) Compute the spatial Jacobian $\mathbf{J}_s(\boldsymbol{\theta}_k)$.
   (c) Compute the pose error $\mathbf{e}_k = \mathbf{e}(\boldsymbol{\theta}_k)$.
   (d) If $\|\mathbf{e}_k\| < \epsilon_{\text{tol}}$, terminate and return $\boldsymbol{\theta}_k$ (solution found).
   (e) Compute the update $\Delta\boldsymbol{\theta} = \mathbf{J}_s^{\dagger}(\boldsymbol{\theta}_k)\mathbf{e}_k$.
   (f) Update $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha\Delta\boldsymbol{\theta}$, where $0 < \alpha \leq 1$ is a step size (often $\alpha = 1$).
3. If convergence is not achieved within $K_{\max}$ iterations, the initial guess was infeasible or the target pose is outside the reachable workspace.

Convergence: Near a non-singular configuration, Newton-Raphson converges quadratically. Far from a solution or near singularities, the convergence may be slow or absent. For robust IK, multiple random restarts or more sophisticated optimization methods (e.g., BFGS with line search) are often employed.

9.14 Workspace Analysis and Manipulability

The Jacobian reveals not only instantaneous velocities but also the *quality* of motion at a given configuration. Two fundamental questions arise: *where* can the robot reach, and *how well* can it move once there?

Definition 9.1. Reachable Workspace

The **reachable workspace** of a manipulator with forward kinematics f_{kin} and joint space $\mathcal{Q}$ is the set of all end-effector poses attainable:

$$\mathcal{W} = \{f_{kin}(\mathbf{q}) \mid \mathbf{q} \in \mathcal{Q}\}. \tag{9.58}$$

The **dexterous workspace** is the subset of $\mathcal{W}$ reachable with *arbitrary* end-effector orientation.

While the workspace tells us *if* a pose is reachable, the **manipulability measure** quantifies *how easily* the robot can move at a given configuration.

Definition 9.2. Manipulability Measure (Yoshikawa)

The **manipulability measure** at configuration $\mathbf{q}$ is [42]:

$$w(\mathbf{q}) = \sqrt{\det(\mathbf{J}(\mathbf{q})\,\mathbf{J}(\mathbf{q})^T)}. \tag{9.59}$$

When $w(\mathbf{q}) = 0$, the Jacobian is rank-deficient and the robot is at a **kinematic singularity**.

The geometric interpretation comes from the **singular value decomposition** of the Jacobian. Let $\mathbf{J} = \mathbf{U\Sigma V}^T$, where $\mathbf{\Sigma} = \mathrm{diag}(\sigma_1, \ldots, \sigma_m)$ with $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m \geq 0$.

Definition 9.3. Velocity Manipulability Ellipsoid

The set of end-effector velocities achievable with unit joint velocity ($\|\dot{\mathbf{q}}\| \leq 1$) forms an ellipsoid in task space. Its principal axes are the columns of $\mathbf{U}$, with semi-axis lengths equal to the singular values σ_i. The manipulability measure equals the ellipsoid volume: $w = \sigma_1\sigma_2\cdots\sigma_m$.

Definition 9.4. Force Manipulability Ellipsoid

The **force manipulability ellipsoid** is the dual of the velocity ellipsoid. Unit joint torques ($\|\boldsymbol{\tau}\| \leq 1$) map to end-effector forces within an ellipsoid whose semi-axes have lengths $1/\sigma_i$. The axes of easy motion become axes of weak force, and vice versa.

In Plain Language 9.2. Velocity–Force Duality

Key insight: Where the robot is fast, it is weak; where it is slow, it is strong. A configuration that allows rapid end-effector motion along a direction (large σ_i) can exert only small forces along that same direction ($1/\sigma_i$), and conversely. This is a direct consequence of the principle of virtual work [30].

Example 9.1. Manipulability of a Planar 2R Arm

Consider a planar 2R arm with link lengths $L_1 = L_2 = 1$ m. The Jacobian is:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix} \tag{9.60}$$

where $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$.
Case 1: Elbow extended ($q_1 = 0$, $q_2 = \pi/4$). Numerical evaluation gives $w \approx 0.71$. The velocity ellipsoid is nearly circular—the robot can move comparably well in all directions.
Case 2: Elbow nearly straight ($q_1 = 0$, $q_2 = 0.1$). Now $w \approx 0.10$. The ellipsoid collapses along the radial direction—the robot can barely move inward or outward (approaching a workspace boundary singularity), but can still swing laterally. The force ellipsoid shows the opposite: enormous radial force capability but weak lateral force.

9.14.1 Redundancy and Null-Space Motion

When a manipulator has more joints than task-space dimensions ($n > m$), the Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$ has a nontrivial **null space**: joint velocities $\dot{\mathbf{q}}_0$ satisfying $\mathbf{J}\dot{\mathbf{q}}_0 = \mathbf{0}$ produce *zero* end-effector motion. This internal motion—reconfiguration without moving the tool—is the hallmark of **kinematic redundancy**.

Definition 9.5. Null-Space Projector

Given the Moore–Penrose pseudoinverse $\mathbf{J}^\dagger = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1}$, the **null-space projector** is:

$$\mathbf{N} = \mathbf{I}_n - \mathbf{J}^\dagger \mathbf{J}. \tag{9.61}$$

Any vector $\mathbf{N}\dot{\mathbf{q}}_0$ lies in the null space of $\mathbf{J}$, regardless of $\dot{\mathbf{q}}_0$.

The general solution for a desired end-effector velocity $\mathbf{v}_{des}$ is then [42]:

$$\dot{\mathbf{q}} = \mathbf{J}^\dagger \mathbf{v}_{des} + \mathbf{N}\dot{\mathbf{q}}_0 \tag{9.62}$$

where the first term achieves the primary task (end-effector motion) and the second term exploits the redundancy for a **secondary objective** without disturbing the primary task.

Example 9.2. 7-DOF Arm with Manipulability Optimization

A 7-DOF robot arm reaching a 6-DOF target (position + orientation) has a one-dimensional null space at non-singular configurations. A common strategy sets $\dot{\mathbf{q}}_0 = \alpha \nabla_{\mathbf{q}} w(\mathbf{q})$, the gradient of the manipulability measure, so that the "elbow" reconfigures to maximize dexterity while tracking the desired trajectory:

$$\dot{\mathbf{q}} = \mathbf{J}^\dagger \mathbf{v}_{des} + \alpha\, \mathbf{N}\, \nabla_{\mathbf{q}} w(\mathbf{q}). \tag{9.63}$$

This keeps the arm away from singularities and ensures good force/velocity capability throughout the motion. Industrial 7-DOF arms (e.g., KUKA iiwa, Franka Emika

Panda) exploit this technique routinely.

9.15 Python Worked Example

Here is a complete Python implementation computing PoE forward kinematics and the spatial Jacobian for the 3R robot.

```python
import numpy as np
from scipy.linalg import expm
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

class SpatialRobot3R:
    """3-DOF spatial manipulator using Product of Exponentials"""

    def __init__(self, L1=0.5, L2=0.3, L3=0.2):
        self.L1 = L1
        self.L2 = L2
        self.L3 = L3

        # Home configuration (fully extended along X-axis)
        self.M = np.eye(4)
        self.M[0, 3] = L1 + L2 + L3

        # Screw axes in space frame (at home)
        self.S1 = np.array([0, 0, 1, 0, 0, 0])  # Z-axis at origin
        self.S2 = np.array([0, 1, 0, 0, 0, L1])  # Y-axis at (L1, 0, 0)
        self.S3 = np.array([1, 0, 0, 0, 0, 0])  # X-axis at (L1+L2, 0, 0)

    def screw_to_matrix(self, screw):
        """Convert 6D screw to 4x4 skew-symmetric matrix"""
        w = screw[:3]  # Angular velocity
        v = screw[3:]  # Linear velocity
        S = np.zeros((4, 4))
        S[:3, :3] = np.array([[0, -w[2], w[1]],
                              [w[2], 0, -w[0]],
                              [-w[1], w[0], 0]])
        S[:3, 3] = v
        return S

    def forward_kinematics(self, theta):
        """Compute end-effector transformation T(theta)"""
        T = np.eye(4)

        # Apply each joint's exponential twist
        for i, S in enumerate([self.S1, self.S2, self.S3]):
            S_skew = self.screw_to_matrix(S)
            exp_S_theta = expm(S_skew * theta[i])
            T = exp_S_theta @ T

        # Apply home configuration
        T = T @ self.M
        return T

    def spatial_jacobian(self, theta, eps=1e-6):
        """Compute spatial Jacobian using finite differences"""
        J = np.zeros((6, 3))
```

```
        T = self.forward_kinematics(theta)

        for i in range(3):
            theta_plus = theta.copy()
            theta_minus = theta.copy()
            theta_plus[i] += eps
            theta_minus[i] -= eps

            T_plus = self.forward_kinematics(theta_plus)
            T_minus = self.forward_kinematics(theta_minus)

            # Compute velocity from finite differences
            dT = (T_plus - T_minus) / (2 * eps)
            V = dT @ np.linalg.inv(T)  # Spatial velocity

            # Extract twist from V
            J[0:3, i] = V[0:3, 3]  # Linear velocity
            J[3, i] = V[2, 1]      # Angular velocity components
            J[4, i] = V[0, 2]
            J[5, i] = V[1, 0]

        return J

    def end_effector_position(self, theta):
        """Return (x, y, z) position of end-effector"""
        T = self.forward_kinematics(theta)
        return T[:3, 3]

# Create robot and test
robot = SpatialRobot3R()
theta_test = np.array([np.pi/4, np.pi/6, -np.pi/3])

T_ee = robot.forward_kinematics(theta_test)
print("End-effector transformation:\n", np.round(T_ee, 4))

pos = robot.end_effector_position(theta_test)
print("End-effector position (x, y, z):", np.round(pos, 4))

J = robot.spatial_jacobian(theta_test)
print("Spatial Jacobian:\n", np.round(J, 4))

# Test manipulability
manip = np.sqrt(np.linalg.det(J @ J.T))
print("Manipulability measure:", np.round(manip, 6))
```

Listing 9.1: PoE Forward Kinematics and Spatial Jacobian

9.16 Chapter Summary

The Product of Exponentials formula provides a modern, geometrically transparent approach to forward kinematics:

1. **PoE Formula:** $\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_n]\theta_n}\mathbf{M}$ expresses forward kinematics as a product of matrix exponentials in the space frame.

2. **Screw Axes:** Each joint is characterized by a single 6D screw $\mathcal{S}_i$, eliminating frame attachment ambiguities of D-H.

3. **Spatial Jacobian:** Differentiation of the PoE formula yields the spatial Jacobian, relating joint velocities to end-effector twists.

4. **Body vs. Space Frame:** Two equivalent formulations exist; conversion via the Adjoint relates them.

5. **Singularities:** The rank of the Jacobian indicates dexterity; singular configurations arise when the Jacobian loses rank.

6. **Inverse Kinematics:** Newton-Raphson iterations using the pseudo-inverse Jacobian solve IK numerically.

7. **Computational Simplicity:** A single PoE algorithm handles revolute, prismatic, and spatial robots uniformly.

9.17 Further Reading and References

The Product of Exponentials formula has its intellectual roots in the Lie-theoretic treatment of rigid-body kinematics pioneered by Brockett [4], who first showed that the forward kinematics of a serial chain could be expressed as a product of matrix exponentials on $SE(3)$. Murray, Li, and Sastry [35] developed this insight into a comprehensive framework for robotic manipulation, providing the definitive mathematical treatment of screw theory, spatial velocities, and the PoE formula that forms the backbone of this chapter. For a modern, pedagogically oriented presentation of PoE-based kinematics—including extensive worked examples and software implementations—the reader is referred to Lynch and Park [30], whose *Modern Robotics* textbook has become the standard reference for graduate instruction.

The classical Denavit-Hartenberg convention, introduced in its original form by Denavit and Hartenberg [10], remains historically important and is still widely used in industrial robotics; understanding its relationship to PoE deepens one's appreciation of why the modern formalism is preferred. For singularity analysis and manipulability theory beyond what is covered here, Siciliano et al. [42] provide an excellent treatment that bridges classical and modern perspectives. Numerical methods for inverse kinematics—including damped least-squares, pseudo-inverse techniques, and optimization-based approaches—are discussed in depth by Spong, Hutchinson, and Vidyasagar [44], who also cover the stability and convergence properties of Newton-Raphson iterations in the context of robotic systems.

9.18 Exercises

1. **Planar 2R Arm:** A planar 2R arm has link lengths $L_1 = 1.5$ m and $L_2 = 1$ m. Write the PoE formula in space-frame form, and compute the end-effector position at $\boldsymbol{\theta} = [60°, -30°]$.

2. **D-H Comparison:** For the same 2R arm, compute the D-H parameters and verify that the forward kinematics matches the PoE result at the same configuration.

3. **Screw Axis Identification:** For a revolute joint rotating about the Y-axis and passing through the point $(2, 0, 1)$, write the screw axis $\mathcal{S}$.

4. **3D Cartesian Robot:** A 3D Cartesian robot has three prismatic joints (X, Y, Z directions). Write its PoE formula and forward kinematics. What is the spatial Jacobian?

5. **Spatial Jacobian:** For the planar 2R arm of Exercise 1, compute the spatial Jacobian at $\boldsymbol{\theta} = [45°, 0°]$ and identify any singular configurations.

6. **Manipulability:** For the 2R arm, plot the manipulability measure $\sqrt{\det(\mathbf{J}\mathbf{J}^T)}$ as a function of $\theta_1 \in [0, 180°]$ with θ_2 fixed at $0°$.

7. **Singularity Analysis:** For the planar 2R arm, find all configurations where the Jacobian rank drops below 2.

8. **Body Frame PoE:** Rewrite the space-frame PoE of Exercise 1 in body-frame form using the Adjoint action.

9. **Inverse Kinematics:** Implement Newton-Raphson inverse kinematics for the planar 2R arm. Given a desired end-effector position $(x_d, y_d) = (2, 0.5)$, find $\boldsymbol{\theta}$ starting from an initial guess.

10. **Velocity Kinematics:** For the 2R arm at $\boldsymbol{\theta} = [30°, 60°]$ with $\dot{\boldsymbol{\theta}} = [1, -0.5]$ rad/s, compute the end-effector linear and angular velocities.

11. **Prismatic-Revolute Arm:** A robot has a prismatic joint (Z-direction) followed by a revolute joint (Y-axis). Write the screw axes and PoE formula.

12. **Numerical Jacobian:** Implement the spatial Jacobian computation using finite differences and compare it to analytical differentiation for the 2R arm.

13. **Constraint Jacobian:** For the 2R arm, compute the task-space Jacobian relating joint angles to end-effector orientation ϕ (yaw angle in the plane).

14. **Null-Space Motion:** For a redundant planar 3R arm, identify the null space of the Jacobian at a non-singular configuration and describe a motion in null space.

15. **Configuration Space Visualization:** For the planar 2R arm, plot the reachable workspace (set of all possible end-effector positions) and overlay the singular configurations.

Chapter 10

The Articulated Body Algorithm

A torque at the wrist does not accelerate only the hand. The resulting joint accelerations depend on the configuration and inertia of the entire connected mechanism, the other applied torques, and its constraints. The articulated body algorithm (ABA) computes this coupled response efficiently for a rigid-body tree. Its value for golf is that it makes the mechanics calculable. It does not, by itself, identify a player's intention, solve ground contact, or determine which intervention improves club delivery.

10.1 Historical Context: Recursive Forward Dynamics

Featherstone's articulated-body formulation is a foundational recursive method for forward dynamics [11, 12]. It exploits the tree structure of interconnected rigid bodies, eliminating one joint acceleration at a time and then recovering all accelerations. For a bounded number of degrees of freedom per joint, the unconstrained calculation takes work proportional to the number of bodies. This is an algorithmic statement, not a measured wall-clock time or a guarantee of accuracy for a chosen anatomical model.

Many useful dynamics calculations instead assemble the generalized mass matrix and solve a linear system. Neither method requires an explicit numerical matrix inverse. The right choice depends on the model, the quantities required, repeated right-hand sides, constraints and implementation. A fast solver cannot compensate for an incorrect inertia, omitted joint torque or unrealistic contact assumption.

10.2 The Forward Dynamics Problem

10.2.1 Problem Formulation

For a fixed-base mechanism with independent scalar joint coordinates, write

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau + Q_{\text{ext}}. \tag{10.1}$$

Here M is the generalized inertia, h contains velocity-dependent and gravitational terms, τ is the applied generalized joint force, and Q_{ext} represents other known external wrenches after projection into joint coordinates. A revolute joint's generalized force is a torque; a prismatic joint's is a force. All terms must use the same coordinate convention. Positive-definite M gives a unique acceleration for these specified loads. It does not imply that the motion is dynamically stable.

Forward dynamics predicts acceleration from the state and loads. Inverse dynamics computes the generalized loads required by a prescribed acceleration. With contact or closed loops, one must additionally specify or solve for constraint reactions. With muscles, activation, force–length–velocity behavior and moment arms determine which joint torques

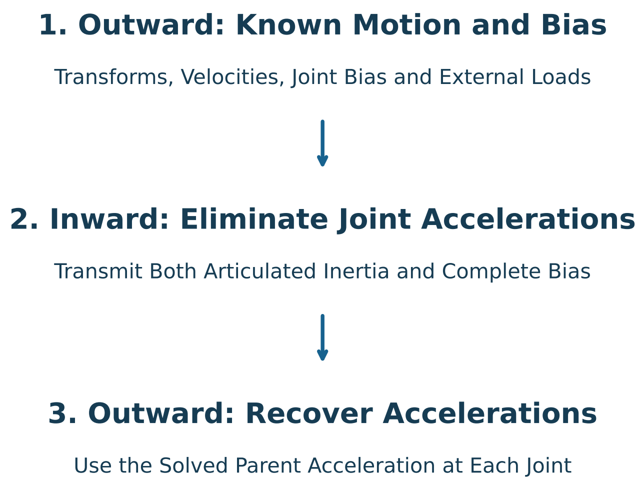


Figure 10.1: Three Passes Through a Rigid-Body Tree: Compute Known Motion and Bias, Eliminate Joint Accelerations Inward, Then Recover Accelerations Outward.

can actually occur. Treating τ as a known input is a mechanical calculation, not a complete physiological model.

10.2.2 What the Complexity Comparison Means

A generic dense positive-definite solve uses an $O(N^3)$ factorization followed by $O(N^2)$ work per right-hand side. That does not make a thirty-joint calculation impossible. Constants, hardware, numerical libraries, sparsity and reuse matter. Conversely, $O(N)$ does not imply that every implementation meets a particular control deadline. An operation count of 10^3 at an assumed 10^9 operations per second would correspond to one microsecond, not one nanosecond; real processors do not convert every mathematical operation into one clock cycle.

ABA avoids constructing the complete mass matrix when the immediate requirement is a tree acceleration solve. Collision detection, constraint solving, muscle dynamics, flexible shafts, integration and output derivatives are additional work. Their cost must not be hidden inside a claim that the whole golf simulation is linear-time.

10.3 Articulated Inertia: The Key Insight

10.3.1 A Free Hinge Changes the Apparent Load

Imagine accelerating the proximal end of two connected links. If the hinge is locked, both links must satisfy that extra kinematic restriction. If it is free and its torque is specified, the distal link can accelerate relative to the proximal link. These two experiments generally produce different proximal reactions. Articulated inertia describes the second experiment after the distal joint accelerations have been eliminated. Composite rigid-body inertia describes the locked collection used in a different calculation.

This distinction matters when discussing a wrist release. A freely moving wrist, a commanded joint torque, a stiff feedback controller and a mechanically locked joint are different boundary conditions. They should not be represented by exchanging descriptive words while leaving the equations unchanged. The inertia appearing in ABA is an instantaneous mechanical response at a specified configuration and specified joint-force model; it is not a frequency-dependent closed-loop impedance.

10.3.2 Mathematical Definition and Frames

Use angular-first spatial motion $v_i = (\omega_i, v_{O_i})$ and moment-first spatial force $f_i = (n_{O_i}, F_i)$, expressed in body frame i. Their pairing $f_i^T v_i$ is power. Let X_i map motion coordinates from parent $p(i)$ into child i for the same physical motion. Consequently a child wrench contributes $X_i^T f_i$ to its parent, and a child inertia contributes $X_i^T I_i X_i$. The transpose direction follows power invariance, not a visual resemblance between matrices.

After eliminating all joints strictly below body i, its subtree obeys

$$f_i^A = I_i^A a_i + p_i^A. \tag{10.2}$$

The unknown wrench f_i^A is the wrench applied to that subtree through its proximal connection. The subtree bias p_i^A includes known velocity terms, external loads and the effects of specified descendant torques. It is not merely friction or a force "dragging" the link. I_i^A depends on geometry and inertias, and on any included ideal armature terms; it is generally not the spatial inertia of one rigid body.

The scalar joint direction S_i maps $\dot{q}_i$ to relative spatial motion. It is a six-component motion vector, not an adjoint transformation. The derivation below assumes a constant subspace in the chosen joint/body frame, as for a standard one-degree-of-freedom revolute, prismatic or helical joint. General joint models need their own kinematic bias terms.

10.4 Derivation From First Principles

10.4.1 Eliminating One Joint Acceleration

At a fixed state, the kinematic acceleration relation is

$$a_i = X_i a_{p(i)} + c_i + S_i \ddot{q}_i. \tag{10.3}$$

Write $\bar{a}_i = X_i a_{p(i)} + c_i$. The unknown parent acceleration is kept separate from the known bias c_i. An ideal scalar joint satisfies $\tau_i = S_i^T f_i^A$. Substitute the subtree wrench relation to obtain

$$\tau_i = S_i^T I_i^A \bar{a}_i + S_i^T I_i^A S_i \ddot{q}_i + S_i^T p_i^A. \tag{10.4}$$

Define

$$U_i = I_i^A S_i, \qquad d_i = S_i^T U_i, \qquad u_i = \tau_i - S_i^T p_i^A. \tag{10.5}$$

Symmetry of I_i^A then gives

$$\ddot{q}_i = \frac{u_i - U_i^T \bar{a}_i}{d_i}. \tag{10.6}$$

The scalar u_i is a residual joint effort, not a six-dimensional wrench. For a physically nondegenerate model $d_i > 0$. Dividing by a small or nonpositive value without investigating the model is not a stability remedy.

Substituting this acceleration back into the wrench relation gives

$$f_i^A = I_i^a X_i a_{p(i)} + p_i^a, \qquad I_i^a = I_i^A - \frac{U_i U_i^T}{d_i}, \tag{10.7}$$

$$p_i^a = p_i^A + I_i^a c_i + \frac{U_i u_i}{d_i}. \tag{10.8}$$

Both terms are essential. The remainder I_i^a represents the load transmitted after joint i is allowed to accelerate under its specified torque. The transmitted bias includes the joint's torque effect even when velocities vanish. Dropping that term would prevent a distal applied torque from correctly influencing the proximal acceleration.

For positive-definite I_i^A, set $z = (I_i^A)^{1/2} S_i$. Then

$$I_i^a = (I_i^A)^{1/2} \left(I - \frac{zz^T}{z^T z} \right) (I_i^A)^{1/2} \succeq 0, \qquad I_i^a S_i = 0. \tag{10.9}$$

The projection occurs in these inertia-weighted coordinates. It is not a Euclidean projection that says only motion along the joint contributes to the parent load. Rather, one freely accelerating direction has been eliminated. This is a Schur complement, the same algebraic operation used in block Gaussian elimination.

10.5 The Articulated Body Algorithm: Three Passes

Number bodies so that parents precede children. The base is body 0, and each nonbase body has one incoming joint. Known external wrenches must be expressed about and in the same frame as each body's inertia. Here gravity is included through a fictitious base acceleration, so it must not also be included as a body external load.

10.5.1 Pass 1: Outward Velocity and Bias Calculation

Spatial Velocity Propagation

Start with $v_0 = 0$ for a fixed base. For each body in outward order,

$$v_i^J = S_i \dot{q}_i, \qquad v_i = X_i v_{p(i)} + v_i^J. \tag{10.10}$$

The transformed parent velocity already carries the effect of the parent origin's offset. Adding untransformed six-vectors would mix different physical reference points as well as orientations.

Coriolis Acceleration and Bias Wrench

For angular-first $v = (\omega, v_O)$, define

$$v\times = \begin{bmatrix} [\omega]_\times & 0 \\ [v_O]_\times & [\omega]_\times \end{bmatrix}, \qquad v\times^* = -(v\times)^T. \tag{10.11}$$

The cross operator acting on motion differs from its dual acting on momentum or force. Initialize

$$c_i = v_i \times v_i^J, \qquad I_i^A = I_i, \qquad p_i^A = v_i \times^* (I_i v_i) - f_i^{\mathrm{ext}}. \tag{10.12}$$

For a more general joint, add its joint-specific acceleration bias to c_i. The Newton–Euler velocity wrench contains gyroscopic and centrifugal effects; it cannot be replaced by an arbitrary scalar multiple of angular speed. The external wrench is subtracted here because the unknown proximal wrench supplies only the part not already supplied externally.

10.5.2 Pass 2: Inward Subtree Elimination

Leaf Boundary Condition

A leaf starts with its own rigid inertia and the bias just calculated. It has no child contribution. Process the bodies in reverse order so that all children have already contributed before their parent is eliminated.

Recursive Inertia Update

Compute $U_i, d_i, u_i, I_i^a, p_i^a$ from the one-joint derivation and accumulate

$$I_{p(i)}^A \mathrel{+}= X_i^T I_i^a X_i. \tag{10.13}$$

The update adds every child's contribution. A branching tree is handled by summation, not by assuming one child or by overwriting an earlier contribution. Preserve U_i, d_i, u_i for the final outward pass.

Required Bias Propagation

The matching update is

$$p_{p(i)}^A \mathrel{+}= X_i^T \left(p_i^A + I_i^a c_i + U_i u_i / d_i\right). \tag{10.14}$$

This update is part of ABA, not an optional refinement. Inertia alone cannot describe a subtree's reaction at nonzero velocity or under a nonzero descendant torque. A static distal torque already changes the force balance transmitted proximally.

10.5.3 Pass 3: Outward Acceleration Recovery

Fixed-Base Acceleration and Gravity

For a stationary base in a uniform gravitational field g, set the algorithmic base acceleration to $a_0 = (0, -g)$. Alternatively, use the physical zero base acceleration and supply gravity as external body forces consistently. Do not use both conventions simultaneously. The fixed base has a reaction wrench, generally nonzero, that can be recovered by inverse dynamics once the accelerations are known.

Spatial acceleration is also not simply a stack of ordinary angular and origin linear accelerations. In body coordinates its linear part is the ordinary origin acceleration minus $\omega_i \times v_{O_i}$, before accounting for the fictitious gravity convention. Comparing these arrays with accelerometer data therefore requires restoring the physical gravity/reference convention and the sensor's offset.

Joint Acceleration Recovery

For each body in outward order, compute

$$\bar{a}_i = X_i a_{p(i)} + c_i, \qquad \ddot{q}_i = (u_i - U_i^T \bar{a}_i)/d_i, \qquad a_i = \bar{a}_i + S_i \ddot{q}_i. \tag{10.15}$$

No trigonometric approximation has been introduced by this elimination. It solves the stated rigid-body equations up to arithmetic error. Its agreement with an independent formulation checks the implementation, while experimental agreement checks the physical model. Those are separate questions.

10.6 Why the Tree Calculation Is Linear-Time

Each body requires a bounded number of operations on six-vectors and 6×6 matrices. Each edge transmits one inertia and one bias contribution. The three traversals therefore use $O(N)$ arithmetic and $O(N)$ storage for scalar joints. A joint with several degrees of freedom replaces d_i by the block $S_i^T I_i^A S_i$ and requires a small linear solve. The linear-in-body-count statement assumes that block size is bounded.

Allocation and data structures also matter. Preallocated arrays or append-only lists followed by reverse traversal preserve the intended scaling. Repeated insertion at the front of a growing list can add quadratic data movement. Computing a complete dense derivative matrix, detecting contacts or solving a large contact system is not covered by the three-pass count.

10.7 Floating Base: A Six-Degree-of-Freedom Root

10.7.1 The Root Articulated Solve

For a freely moving base, its acceleration is unknown. Initialize the base's own inertia and bias, include its known external wrench, and accumulate all child contributions. With the same fictitious-gravity convention and no unknown root constraint wrench,

$$I_0^A a_0 + p_0^A = 0, \qquad a_0 = -(I_0^A)^{-1} p_0^A. \tag{10.16}$$

Implement this expression as a 6×6 solve. The matrix is the entire articulated subtree's inertia at the current configuration, not the isolated base's constant rigid inertia. Its factorization can be reused only while the quantities on which it depends remain unchanged. The remaining outward acceleration recovery is the same as before.

10.7.2 Contact Does Not Automatically Fix the Base

A foot touching the ground may impose one normal constraint, several sticking constraints, or a finite contact-patch wrench model. It is not automatically a six-degree-of-freedom weld. A no-slip constraint is valid only while the required reaction satisfies the admitted friction and unilateral conditions. If the contact wrench is unknown, the root solve and contact equations must be resolved together; one cannot first assume an arbitrary base acceleration and then declare the resulting contact physically feasible.

10.8 Floating-Base Humanoids and Momentum

10.8.1 Configuration Coordinates and Generalized Velocity

A humanoid model may use a position and unit quaternion for the pelvis, together with joint coordinates. The quaternion has four stored components but only three rotational degrees of freedom. Write the state kinematics as $\dot{q} = N(q)\nu$, and dynamics in the generalized velocity ν. In general $\nu \neq \dot{q}$. A six-dimensional root joint is a local motion description, not a requirement to use three globally nonsingular orientation angles; no such global three-angle chart exists.

10.8.2 Root Equation With a Known Applied Wrench

If a known additional root wrench w_0 is kept outside the bias convention, the root relation is

$$I_0^A a_0 = w_0 - p_0^A. \tag{10.17}$$

Count a wrench exactly once. An unknown ground reaction belongs to the constraint calculation, while a measured ground wrench can be supplied as known data within its measurement uncertainty. These choices answer different forward or inverse questions.

10.8.3 Centroidal Momentum and Whole-Body Dynamics

Keep the angular-first ordering and define momentum about the total center of mass c:

$$h_G = \begin{bmatrix} k_c \\ p \end{bmatrix} = A_G(q)\nu, \qquad \dot{h}_G = \begin{bmatrix} \sum_j ((r_j - c) \times F_j + n_j) \\ m_{\text{tot}} g + \sum_j F_j \end{bmatrix}. \tag{10.18}$$

The sums contain nongravitational external forces and couples. Uniform gravity has zero resultant moment about the total COM. Internal joint forces and torques cancel in the whole-system balance, but joint motion redistributes momentum among segments and changes A_G, the COM and contact moment arms. "Internal torques cancel" does not mean configuration is irrelevant.

For a golfer–club system, hand–club wrenches are internal only if both hands and the club are included in the system boundary. Ground reactions remain external. After ball contact, the ball must either be included or represented through an external interaction. This bookkeeping is necessary before attributing a rise in club momentum to any particular segment or source of work.

10.9 Joint Friction and Motor Inertia

10.9.1 Viscous Joint Friction

For an ideal viscous coefficient $b_i \geq 0$, use $\tau_i^{\text{eff}} = \tau_i^{\text{act}} - b_i \dot{q}_i$ in the joint elimination. The associated power is $-b_i \dot{q}_i^2 \leq 0$. The induced acceleration is coupled through the dynamics; subtracting $b_i \dot{q}_i$ directly from a computed acceleration has both the wrong units and the wrong mechanical effect. Dry friction at zero velocity requires a separate set-valued or regularized model, not an arbitrary choice of sign presented as a universal law.

10.9.2 An Ideal Reflected Armature Model

If a transmission is modeled by an independent rotor kinetic energy $J_i(r_i\dot{q}_i)^2/2$, its reflected armature is $a_i^{\mathrm{rot}} = r_i^2 J_i$. Under precisely that idealization, the joint equation includes $a_i^{\mathrm{rot}}\ddot{q}_i$, and the pivot becomes

$$d_i = S_i^T I_i^A S_i + a_i^{\mathrm{rot}}. \tag{10.19}$$

Use this pivot in the remainder, transmitted bias and acceleration recovery. The remainder no longer generally annihilates S_i. General rotor/body coupling, flexible transmissions and configuration-dependent gearing require a fuller kinetic model. Muscles are not rotary motors with a freely chosen reflected rotor inertia; their activation and force-generation dynamics must be modeled on their own terms.

10.10 ABA and Composite Rigid-Body Methods

The composite rigid-body algorithm (CRBA) constructs $M(q)$ from locked-subtree inertias. Its worst-case assembly cost is $O(N^2)$ for a scalar-joint tree. In a serial chain, distant joint velocities can contribute to the same body's kinetic energy, so the mass matrix is generally dense. A branched tree has an ancestor-dependent sparsity pattern, not a universally bounded bandwidth.

For one unconstrained right-hand side, ABA can avoid forming M. If an application already needs M for constraints, identification or optimization, factorization may be useful. At fixed configuration, a dense factorization is reused at $O(N^2)$ per additional right-hand side; a suitably organized articulated factorization can apply the inverse in $O(N)$ per right-hand side. Constructing R complete solutions still requires their output and load processing. No numeric runtime comparison follows from stating these asymptotic orders alone.

10.11 Choosing Forward or Inverse Dynamics

10.11.1 When the Acceleration Is Prescribed

Use inverse dynamics to find the generalized force required by a specified motion and external-load model. In a measured swing, differentiation noise, inertial estimates and ground/contact measurements strongly affect this calculation. A net joint torque is not a unique decomposition into individual muscle forces. If external forces are unknown, kinematics alone may not determine all loads.

10.11.2 When the Applied Loads Are Specified

Use forward dynamics to compare predicted accelerations or trajectories under explicit loading or control changes. Holding all other joint torques fixed while changing one torque gives a well-defined instantaneous intervention in the unconstrained model. Holding a feedback policy fixed over a new trajectory is a different experiment from replaying the original torque history. A claimed improvement in club delivery must state which experiment was performed and how ground, grip and muscle feasibility were maintained.

10.12 Constraint Handling: Contact and Joint Limits

10.12.1 Unilateral Contact

Let $\phi(q)$ be a signed normal gap: $\phi > 0$ means separation and $\phi = 0$ means touching. An ideal rigid contact requires $\phi \geq 0$ and a compressive normal reaction $\lambda_n \geq 0$. A separated contact does not supply a force merely because its unconstrained acceleration points toward the surface. A closing trajectory needs event detection or a consistent time-stepping scheme; sustained-contact acceleration equations do not describe an instantaneous impact.

10.12.2 Joint Limits

A hard lower or upper joint limit can be represented by a unilateral gap in configuration space, with its own reaction or impact rule. A soft limit instead supplies a specified force law. Abruptly clipping the angle after integration can violate momentum and inject or remove energy without a stated model. Anatomical range limits, passive tissue forces and active control are related but distinct phenomena.

10.13 Connection to Gaussian Elimination

The one-joint derivation is block elimination of coupled wrench and acceleration equations. ABA performs this elimination on a spatial tree representation. It does not require the generalized mass matrix to be band diagonal. The Schur complement is exact for the stated algebra; its numerical evaluation still depends on scaling and conditioning. Very small inertias, inconsistent units, nonphysical mass parameters and cancellation can produce poor numerical results in a recursive or matrix formulation.

For example, partition a three-joint mass matrix into proximal coordinate q_1 and distal coordinates q_d. At zero velocity and gravity, with $\tau_d = 0$,

$$M_{\text{eff}} = M_{11} - M_{1d} M_{dd}^{-1} M_{d1}, \qquad \ddot{q}_1 = \tau_1 / M_{\text{eff}}. \tag{10.20}$$

With the distal joints locked instead, the corresponding coefficient is M_{11}. The difference expresses the mechanical effect of allowing relative motion, not evidence that a golfer should minimize effective inertia throughout the swing. Club orientation, feasible torque, shaft deformation and impact conditions impose additional objectives and restrictions.

10.14 Worked Example: A Fully Specified Three-Link Arm

Consider three uniform solid cylinders of lengths $(0.4, 0.3, 0.25)$ m, masses $(2.0, 1.2, 0.6)$ kg and common radius 0.025 m. Their local x axes run along the links; all joints rotate about local z. Each body frame originates at its proximal joint. This is an ideal teaching mechanism, not measured anatomy or a fitted golf club. The finite radius gives physically consistent positive rotational inertias.

For each cylinder, $c_i = (L_i/2, 0, 0)$ and

$$I_{C_i} = \operatorname{diag}\left(\frac{m_i r^2}{2}, \frac{m_i (L_i^2 + 3r^2)}{12}, \frac{m_i (L_i^2 + 3r^2)}{12} \right), \tag{10.21}$$

$$I_i = \begin{bmatrix} I_{C_i} + m_i[c_i]_\times [c_i]_\times^T & m_i[c_i]_\times \\ m_i[c_i]_\times^T & m_i I_3 \end{bmatrix}. \tag{10.22}$$

The upper-left parallel-axis contribution is added. The off-diagonal blocks follow from the COM offset and must not be invented independently.

10.14.1 Configuration and Loads

Use relative angles $q = (0.4, -0.7, 0.2)$ rad, rates $\dot{q} = (0.8, -0.5, 1.1)$ rad/s and actuator torques $\tau = (4, -1, 0.3)$ N m. Gravity is $(0, -9.81, 0)$ m/s^2. There are no other external wrenches, no damping and no armature in this particular calculation.

Let $E_i = R_z(q_i)^T$ reexpress parent-oriented components in child axes, and let $r_i = (L_{i-1}, 0, 0)$ be the parent-frame joint offset, with $r_1 = 0$. Then

$$X_i = \begin{bmatrix} E_i & 0 \\ -E_i[r_i]_\times & E_i \end{bmatrix}, \qquad S_i = (0, 0, 1, 0, 0, 0)^T. \tag{10.23}$$

The joint angles therefore enter every transform. The relative angles are not interchangeable with the links' absolute orientations, which are $(0.4, -0.3, -0.1)$ rad.

10.14.2 Pass 1: Velocities and Bias

All out-of-plane components vanish. Listing each spatial velocity as the planar triple (ω_z, v_x, v_y), the outward recursion gives

$$\begin{aligned} v_1 &= (0.8, 0, 0), \\ v_2 &= (0.3, -0.206149660, 0.244749500), \\ v_3 &= (1.4, -0.135535933, 0.369032412). \end{aligned} \tag{10.24}$$

The angular components sum relative rates, while the linear components also require rotations and joint-origin offsets. Form $c_i = v_i \times S_i \dot{q}_i$ and the initial $p_i^A = v_i \times^* I_i v_i$ from these complete velocities. There is no arbitrary bias-force estimate.

10.14.3 Pass 2: Articulated Inertias and Residual Efforts

The planar subblocks of the articulated inertias, before eliminating each body's own joint, are approximately

$$I_3^A\big|_{zxy} = \begin{bmatrix} 0.012593750 & 0 & 0.075000000 \\ 0 & 0.600000000 & 0 \\ 0.075000000 & 0 & 0.600000000 \end{bmatrix}, \tag{10.25}$$

$$I_2^A\big|_{zxy} = \begin{bmatrix} 0.051575604 & 0.026090063 & 0.231293680 \\ 0.026090063 & 1.782370942 & 0.086966875 \\ 0.231293680 & 0.086966875 & 1.370978934 \end{bmatrix}, \tag{10.26}$$

$$I_1^A\big|_{zxy} = \begin{bmatrix} 0.260428859 & -0.284953502 & 0.783624232 \\ -0.284953502 & 3.143841922 & -0.712383755 \\ 0.783624232 & -0.712383755 & 2.959060579 \end{bmatrix}. \tag{10.27}$$

The rotational, mixed and translational entries carry different SI units. These subblocks are sufficient to display this planar example, but the implementation uses full six-dimensional quantities. The pivots and residual joint efforts are

$$\begin{aligned} d &= (0.260428859, 0.051575604, 0.012593750), \\ u &= (6.518800728, -1.799610488, 0.314231273). \end{aligned} \tag{10.28}$$

Gravity has not yet entered these residual efforts because it enters through the fictitious base acceleration in pass 3. The residuals differ from the supplied torques because the inward bias carries descendant loads and velocity effects.

10.14.4 Pass 3 and an Independent Check

Outward recovery gives, in rad/s^2,

$$\ddot{q} = (2.023035078, -79.465503634, 175.204404572). \tag{10.29}$$

These large distal accelerations reflect the particular light-link model and loads. They are not plausible-muscle bounds or a prediction for a human swing. To check the result independently, form the Cartesian COM Jacobians J_{C_i} and angular Jacobians J_{ω_i} from the link positions. Kinetic energy gives

$$M = \sum_i \left(m_i J_{C_i}^T J_{C_i} + J_{\omega_i}^T I_{C_i}^{\text{world}} J_{\omega_i} \right). \tag{10.30}$$

For these planar coordinates, differentiating this mass matrix gives the Christoffel velocity terms, and differentiating gravitational potential gives the gravitational terms. The independent calculation produces

$$M = \begin{bmatrix} 0.814792916 & 0.283348998 & 0.060972725 \\ 0.283348998 & 0.146884246 & 0.034645248 \\ 0.060972725 & 0.034645248 & 0.012593750 \end{bmatrix}, \tag{10.31}$$

$$h = (14.185426211, 4.029005575, 0.723271650)^T. \tag{10.32}$$

Solving $M\ddot{q} = \tau - h$ agrees with ABA. The displayed values are rounded; residual checks use full precision. Tests additionally check nonzero external wrenches, damping and armature, multiple configurations, branched prismatic motion and reexpression in different body frames. Agreement across these cases is stronger evidence than checking an example against a second transcription of the same inward recursion.

10.15 Reproducible Python Example

The repository implementation is split between the generic prepared-tree solver and a cylinder-chain model builder. It uses the existing checked spatial-inertia and cross-product functions. Its inputs are explicitly restricted to a fixed-base rigid tree with constant local scalar-joint subspaces. It does not silently treat a contact or a flexible link as such a joint. The following example runs from the repository root:

```python
import numpy as np
from src.tools.articulated_chain_examples import cylinder_chain
from src.tools.articulated_body_examples import (
    DynamicsLoads, forward_dynamics, inverse_dynamics,
)

tree = cylinder_chain(
    np.array([0.4, 0.3, 0.25]),
    np.array([2.0, 1.2, 0.6]),
    np.array([0.4, -0.7, 0.2]),
    0.025,
)
loads = DynamicsLoads(
    velocity=np.array([0.8, -0.5, 1.1]),
    torque=np.array([4.0, -1.0, 0.3]),
)
result = forward_dynamics(tree, loads)
print(result.joint_acceleration)
print(result.pivot)
recovered = inverse_dynamics(
    tree, loads, result.joint_acceleration
)
np.testing.assert_allclose(recovered, loads.torque)
```

The diagnostic result exposes velocities, articulated inertias, biases, pivots and residual efforts for inspecting the derivation. Array shapes, finite loads, ordered parents and positive-definite body inertias are checked. Physical rigid-body inertia structure and valid rigid motion transforms remain modeling preconditions. The supplied cylinder builder constructs them from positive dimensions. Invalid or nonpositive articulated pivots raise an error; assigning zero acceleration would conceal a broken or degenerate model.

In NumPy, a one-dimensional vector's transpose is still one-dimensional. To form $U_i U_i^T$, call `np.outer` with the vector as both arguments. For a one-dimensional array, `U @ U.T` instead returns a scalar, which can silently broadcast into an incorrect matrix update.

10.15.1 Connection to Simulation Frameworks

Different simulators use different combinations of recursive dynamics and constraint solvers. MuJoCo's documented computation path forms its mass matrix with CRB and uses sparse $L^T DL$ factorization and back-substitution [34]. It must not be described as using ABA for every timestep. Pinocchio provides ABA and analytical dynamics derivatives, along with other algorithms [37]. These statements identify documented methods; they do not rank accuracy or runtime. A reproducible comparison must identify software versions, model parameters, contact and integration settings, hardware and the actual quantity timed.

10.16 Contact Forces and Impact Dynamics

During smooth motion, a force changes acceleration. An ideal instantaneous impact changes velocity while configuration remains continuous. For a single frictionless contact with normal Jacobian J_n, closing velocity $v_n^- = J_n \nu^- < 0$ and restitution coefficient $0 \le e \le 1$,

$$M(\nu^+ - \nu^-) = J_n^T \Lambda_n, \qquad J_n \nu^+ = -e J_n \nu^-, \tag{10.33}$$

$$\Lambda_n = -\frac{(1+e)J_n\nu^-}{J_n M^{-1} J_n^T}. \tag{10.34}$$

Here M and J_n are evaluated at the impact configuration and the denominator is positive for a nonzero contact direction and positive-definite M. The resulting impulse is compressive. It has impulse units, unlike the sustained reaction force. Multiple simultaneous contacts, friction and restitution require a specified coupled impact law; independent application of this scalar formula need not be consistent.

Ball impact also involves ball spin, clubface geometry and material deformation. This scalar illustration does not replace a validated ball–club collision model. It explains why comparing only pre-impact accelerations cannot establish a post-impact delivery or ball-flight claim.

10.17 Contact Dynamics and Complementarity

10.17.1 Non-Penetration and Sustained Contact

At a closed contact with zero normal velocity, define normal acceleration $a_n = J_n\dot{\nu} + \dot{J}_n\nu$. An ideal frictionless acceleration-level condition is

$$a_n \geq 0, \qquad \lambda_n \geq 0, \qquad a_n\lambda_n = 0. \tag{10.35}$$

The alternatives are separating acceleration with zero reaction, or a compressive reaction maintaining zero normal acceleration. These conditions apply to the specified closed, non-impacting candidate contacts. They are not a universal replacement for the gap and velocity conditions along an arbitrary trajectory.

10.17.2 Friction and Contact Wrenches

For a point contact, the Coulomb force cone is

$$\|\lambda_t\| \leq \mu\lambda_n, \qquad \lambda_n \geq 0. \tag{10.36}$$

This inequality bounds the tangential force but does not specify its value during sliding. A sliding law ordinarily also imposes opposition to slip or a maximum-dissipation condition. A finite foot patch can transmit moments subject to geometry and pressure restrictions; a three-dimensional force cone is not the complete admissible six-dimensional wrench set. Rigid frictional contact can exhibit nonuniqueness or inconsistency, so no simple cone equation should be advertised as solving all contact physics exactly.

10.17.3 The Frictionless Acceleration LCP

For a chosen set of closed, zero-normal-velocity contacts, write

$$M\dot{\nu} + h = \tau + J_n^T\lambda_n. \tag{10.37}$$

Let $a_{\text{free}} = M^{-1}(\tau - h)$, $W = J_n M^{-1} J_n^T$ and $b = J_n a_{\text{free}} + \dot{J}_n\nu$. Then

$$a_n = W\lambda_n + b, \qquad 0 \leq \lambda_n \perp a_n \geq 0. \tag{10.38}$$

The matrix W is positive semidefinite. Redundant contact directions can make reaction forces nonunique even when the resulting acceleration is determined. Friction introduces additional modeling choices; this frictionless LCP is not the full frictional problem.

ABA can supply products with M^{-1} for constructing or applying W. Each such product is a tree solve, but assembling and solving the contact problem has its own cost. The number and arrangement of contacts and the chosen numerical method matter. Soft-contact formulations, including MuJoCo's documented model, make different assumptions from exact rigid complementarity.

10.17.4 Time-Stepping and Numerical Order

A velocity step using an impulse Λ may take the form

$$M(q_k)(\nu_{k+1} - \nu_k) = \Delta t\,(\tau_k - h_k) + J(q_k)^T \Lambda. \tag{10.39}$$

This frozen-coefficient expression is not, by itself, full implicit Euler. The contact law, evaluation points and configuration update define the method. Restitution and stabilization also affect the result.

The familiar velocity-first Euler update is first order. In canonical separable Hamiltonian coordinates the corresponding symplectic Euler method is symplectic; a velocity-first update for a configuration-dependent multibody mass matrix does not inherit that claim automatically. Velocity Verlet is second order for its appropriate position-force/separable setting. General velocity-dependent multibody dynamics and impacts require an integrator suited to those equations. Check convergence as the timestep decreases, constraint residuals and energy/momentum behavior under the stated force model. An accurate acceleration solve does not remove integration error.

10.18 From Coupled Dynamics to a Golf-Swing Argument

For smooth unconstrained motion at a fixed state, the immediate torque-to-acceleration sensitivity is $\delta\ddot{q} = M^{-1}\delta\tau$. With bilateral constraints $J\dot{\nu} + b_c = 0$ and full-row-rank J, solving for the reaction gives the constrained acceleration sensitivity

$$\delta\dot{\nu} = P_M \delta\tau, \qquad P_M = M^{-1} - M^{-1}J^T(JM^{-1}J^T)^{-1}JM^{-1}. \tag{10.40}$$

The reaction changes when a torque changes. Holding the old contact force fixed would answer a different question and can violate the constraint. Both hands on a rigid club close a kinematic loop through the arms and torso; a cut-tree ABA calculation must restore that loop through constraints or an explicitly compliant grip model.

This instantaneous response is only one component of delivery. If x includes configuration, velocity, muscle activation and any shaft states, an implemented policy produces a field $\dot{x} = F_\pi(x,t)$. For an additional input perturbation with local input derivative B_π, the finite-interval variational equation is

$$\delta\dot{x} = D_x F_\pi\,\delta x + B_\pi\,\delta u. \tag{10.41}$$

The state derivative includes the feedback policy's response to state changes. The club's selected output at a fixed time depends on the propagated state perturbation and the output derivative. At a state-triggered impact, event-time and reset sensitivities are also required. A torque change may improve one output while worsening face angle, path, contact location or robustness. Segment speed peaks alone do not identify which muscles caused those changes.

A strong swing analysis therefore states its system boundary, inertia and joint model, admitted ground/grip reactions, actuator policy, perturbation and delivery outcome. It checks mechanical consistency before drawing a coaching conclusion. Energy and momentum balances supply complementary checks: internal joint action can redistribute momentum and perform work, but a sequence of angular-speed maxima is neither an energy-flow measurement nor proof of a unique optimal release strategy.

10.19 Differentiating the Dynamics

For a smooth, nonsingular fixed model, differentiating $M\ddot{q} + h = \tau$ gives

$$\delta\ddot{q} = M^{-1}\left(\delta\tau - \delta h - (\delta M)\ddot{q}\right). \tag{10.42}$$

This identity clarifies the derivative target independently of a software implementation. Automatic differentiation can propagate derivatives through a correctly implemented smooth recursion. Analytical derivative algorithms provide another route. Neither produces exact real-number arithmetic or an automatic derivative through every switching contact event. Differentiating the integrated or hybrid map requires its additional chain rules and event assumptions.

The full dense Jacobian has quadratically many entries in the number of joints. A directional derivative or reverse-mode derivative of a scalar objective can have a different cost from materializing every entry. Benchmark the actual requested derivative and include contact, controller and integration work before asserting a learning or control rate.

10.20 Chapter Summary

ABA solves the specified rigid-tree forward problem through outward kinematics, inward elimination and outward recovery. The essential quantities are the articulated inertia and its complete bias, transformed through power-consistent frames. Articulating and locking a subtree are different mechanical experiments. Floating bases, actuator models, contact and impact require their own explicit assumptions. The independently checked example establishes computational consistency; applying the method to golf additionally requires a validated model and a clearly defined causal or predictive question.

10.21 Further Reading and References

Featherstone's original formulation and later treatment provide the recursive dynamics foundation [11, 12]. His Spatial_v2 documentation provides inspectable reference algorithms and coordinate-transform conventions [14]. MuJoCo's computation documentation and Pinocchio's software description document distinct implementation choices [34, 37]. The calculations here are derived explicitly and checked independently; consulting these sources does not substitute for stating the model and experiment used in a golf argument.

10.22 Exercises

1. **Complexity Analysis:** Count the bounded-size operations in all three passes for scalar joints. Identify the assumptions needed for $O(N)$ work and storage. Explain why constructing all dense acceleration derivatives or solving contact is additional work.

2. **Articulated and Composite Inertia:** Use three cylinders with $m_i = 1$ kg, $L_i = 1$ m, radius 0.05 m and $q = (0.2, -0.4, 0.3)$ rad. Calculate every articulated inertia with the distal joints free, and every composite inertia with them locked. Explain the difference without treating a singular point-mass inertia as a generic rigid body.

3. **Motor Inertia:** Add 0.04 kg m^2 of ideal armature to joint 1 of the worked example. Recalculate all accelerations. Derive the sensitivity using the rank-one change in M, and explain why distal accelerations can change even though their own armatures are unchanged.

4. **Floating Base:** Describe the extra root quantities and solve for a thirty-joint floating-base mechanism. Separate its additional bounded-size root work from the effects of adding contact constraints. Do not infer an exact runtime ratio from the degree-of-freedom count.

5. **Repeated Right-Hand Sides:** At fixed q and $\dot{q}$, compare reuse of a dense mass factorization and an articulated factorization for one hundred torque vectors. State preparation, per-solve and storage costs separately. Explain which quantities must be refreshed if $\dot{q}$ or q changes.

6. **Bias Verification:** Restrict the worked model to its first two links at $q = (0.4, -0.7)$ and $\dot{q} = (0.8, -0.5)$. Calculate velocity forces from the spatial recursion and from Christoffel terms of the Cartesian kinetic-energy mass matrix. Compare gravity separately to avoid confusing force conventions.

7. **Singular Pivots:** Prove $d_i > 0$ for a nonzero S_i and positive-definite I_i^A without armature. Explain how massless idealizations, degenerate coordinates or nonphysical parameters can defeat this condition. Why is replacing a zero pivot by zero acceleration not a valid solution?

8. **Joint Damping:** Add coefficients $(0.2, 0.1, 0.03)$ N m s/rad to the worked example. Verify $\tau^{\text{eff}} = \tau - B\dot{q}$ before the solve and that the damping power is nonpositive. Compare against the incorrect post-solve acceleration subtraction.

9. **Gravity Simulation:** Construct four identical cylinders of length 0.3 m, mass 1 kg and radius 0.025 m, initially at $q = (0.4, -0.2, 0.3, -0.1)$ with zero rates. Simulate gravity-only motion with a fixed proximal base. State the integrator, refine the timestep, and monitor total mechanical energy; a fixed-base pendulum chain is not whole-body ballistic free fall.

10. **Contact Impulse:** For the first two worked links at $q = (0.4, -0.7)$, define a horizontal surface through the distal endpoint and use $\dot{q} = (-0.8, -0.5)$. Form the endpoint's upward normal Jacobian. Compute the single frictionless impulse for $e = 0$ and verify the post-impact normal velocity and nonnegative impulse. Distinguish this velocity jump from subsequent sustained-contact acceleration.

11. **Integration Order:** Compare a first-order velocity-first method with a suitable second-order method on a smooth problem. Verify observed convergence against a refined reference. Explain when the words symplectic and Verlet apply, and why an impact can invalidate a smooth-step order argument.

12. **Matrix Formulation:** Build a specified ten-link cylinder chain and compare ABA with a positive-definite linear solve using its assembled M and bias. Report residuals and conditioning; do not explicitly invert M merely to apply it to one vector.

13. **Floating-Base Reactions:** For a specified humanoid model, choose either a point-foot or a finite-patch contact model. Write the root and contact equations together. State the rank, friction and unilateral conditions required for the computed reaction, and explain what measured contact data would remove from the unknowns.

14. **Redundancy:** Choose a task Jacobian J for the worked model. Examine the map $JM^{-1}\delta\tau$ and construct a torque perturbation with zero immediate task acceleration when such a null space exists. Explain why that does not imply zero state change or unchanged delivery at a later time.

15. **Constrained Projection:** Let $M \succ 0$ and full-row-rank J be fixed, and require $Ja + b_c = 0$. Minimize $(a - a_{\text{free}})^T M(a - a_{\text{free}})/2$ under that constraint. Derive $a = a_{\text{free}} - M^{-1}J^T(JM^{-1}J^T)^{-1}(Ja_{\text{free}} + b_c)$. Explain why this is a mass-weighted closest acceleration, not an arbitrary Euclidean minimum-norm forward solution, and discuss redundant constraints separately.

Chapter 11

Lagrangian Mechanics

A golf swing combines posture-dependent inertia, muscle and joint action, gravity, contact reactions and elastic storage. An energy formulation helps keep those contributions consistent. It does not remove forces from the physical explanation. It organizes their effects in coordinates that respect the chosen geometry, so that a model's acceleration, power and conservation statements can be checked against one another.

The central question is whether the same declared model gives the same predictions through kinetic energy, virtual work and Newton–Euler balance. A model that produces a plausible trajectory but admits negative kinetic energy has already failed that test. A model that conserves an incorrectly defined energy has not passed it.

11.1 What the Lagrangian Formulation Assumes

Lagrange's analytical mechanics, the calculus of variations and Hamilton's later action and phase-space formulations provide complementary tools. Hamilton's stationary-action principle should not be treated as a synonym for every historical least-action principle. Here we use a finite-dimensional classical mechanical model; no claim about quantum paths or a universal optimization performed by nature is needed.

Let q be a local configuration chart, $v = \dot{q}$ its coordinate velocity, and Q a generalized-force covector. Coordinates can include passive joints, floating-base variables and elastic modes; they need not be motor angles. A regular chart can incorporate independent holonomic constraints locally. Closed loops, singular configurations and changing contacts can require redundant coordinates, multipliers or separate contact modes.

For an autonomous natural mechanical system we write

$$\mathcal{L}(q, v) = T(q, v) - V(q), \qquad T(q, v) = \tfrac{1}{2} v^T M(q) v. \tag{11.1}$$

This form assumes homogeneous quadratic kinetic energy in coordinate velocities and conservative forces from the stated potential. Moving coordinate systems or prescribed base motion can introduce terms linear in velocity and explicit time dependence. Velocity-dependent interactions may require a more general Lagrangian. Dissipation and actuation normally enter through generalized forces, not by relabeling them as potential energy.

A rigid body's kinetic energy is its center-of-mass translation plus rotation about that center. With all vectors expressed consistently,

$$T_i = \tfrac{1}{2} m_i \|v_{C_i}\|^2 + \tfrac{1}{2} \omega_i^T I_{C_i} \omega_i. \tag{11.2}$$

The same frame must be used for the inertia and angular velocity; the inertia is about the center of mass. Near a uniform gravitational field, $V_g = \sum_i m_i g y_{C_i}$; a linear spring adds $kx^2/2$. Neither expression includes the metabolic cost of producing muscle force.

11.2 Virtual Work and the Role of Constraints

Newton's laws do not require solving every reaction force before obtaining motion. Projecting force balance onto admissible displacements can eliminate ideal reactions. Lagrange–d'Alembert mechanics makes that projection systematic. For particle positions $r_a(q,t)$, the fixed-time virtual displacement is $\delta r_a = (\partial r_a/\partial q)\delta q$. Applied forces define

$$Q_i = \sum_a F_a \cdot \frac{\partial r_a}{\partial q_i}, \qquad \delta W = Q^T \delta q. \tag{11.3}$$

A spatial wrench similarly contributes $J^T F$, using a velocity Jacobian and dual wrench with matching frame, reference point and component ordering. This is why coordinate changes must transform forces as well as positions.

D'Alembert's equation is $\sum_a (F_a - m_a \ddot{r}_a) \cdot \delta r_a = 0$ after ideal reaction terms vanish. The word ideal is an assumption: reaction covectors annihilate the allowed virtual displacements. Friction, compliance, bearing losses or a driven contact can require additional terms. A normal direction in Cartesian space becomes a constraint covector in generalized coordinates; calling it perpendicular without specifying a metric can obscure that distinction.

For fixed holonomic constraints and independent reduced coordinates, virtual displacements are arbitrary in those coordinates. Newton's projection then yields the forced Euler–Lagrange equations. For moving constraints, virtual displacement still holds time fixed. Actual velocity has an additional prescribed-motion component, so zero virtual work need not mean zero actual power. We derive that distinction below.

11.3 Hamilton's Principle and Variation

For an unforced variational system, fix both endpoint configurations and their times. A smooth trajectory is stationary for

$$\mathcal{S}[q] = \int_{t_0}^{t_1} \mathcal{L}(q, \dot{q}, t)\, dt, \qquad \delta\mathcal{S} = 0. \tag{11.4}$$

Stationary means zero first variation. It does not assert a minimum, a maximum or an extremum; saddles are possible. The physical initial-value problem also needs an initial configuration and velocity. Stationarity of an endpoint problem is not a substitute for those data.

For $q_\epsilon = q + \epsilon\eta$ with $\eta(t_0) = \eta(t_1) = 0$, differentiation and integration by parts give

$$\begin{aligned} \delta\mathcal{S} &= \int_{t_0}^{t_1} (\mathcal{L}_q^T \eta + \mathcal{L}_v^T \dot{\eta})\, dt \\ &= [\mathcal{L}_v^T \eta]_{t_0}^{t_1} + \int_{t_0}^{t_1} \left(\mathcal{L}_q - \frac{d}{dt}\mathcal{L}_v \right)^T \eta\, dt. \end{aligned} \tag{11.5}$$

The endpoint term is zero. The fundamental lemma of the calculus of variations therefore gives

$$\frac{d}{dt}\mathcal{L}_v - \mathcal{L}_q = 0. \tag{11.6}$$

For nonconservative generalized force Q, the Lagrange–d'Alembert statement is

$$\delta \mathcal{S} + \int_{t_0}^{t_1} Q^T \eta \, dt = 0. \tag{11.7}$$

The same integration by parts gives

$$\frac{d}{dt}\mathcal{L}_v - \mathcal{L}_q = Q. \tag{11.8}$$

This is a forced virtual-work principle, not a claim that every dissipative trajectory is stationary for the unmodified action. Nonholonomic admissible variations require their own restriction and cannot generally be replaced by an unconstrained endpoint variation.

For a point-mass pendulum with a massless rigid rod, angle θ measured counterclockwise from downward vertical, $T = m\ell^2\dot{\theta}^2/2$ and $V = mg\ell(1-\cos\theta)$. Equation 11.8 gives $m\ell^2\ddot{\theta} + mg\ell\sin\theta = u$. Tension does no work in this fixed-length, fixed-pivot model, but its force remains physically necessary. The bob's tangential speed is $\ell|\dot{\theta}|$, not $\dot{\theta}$ in metres per second.

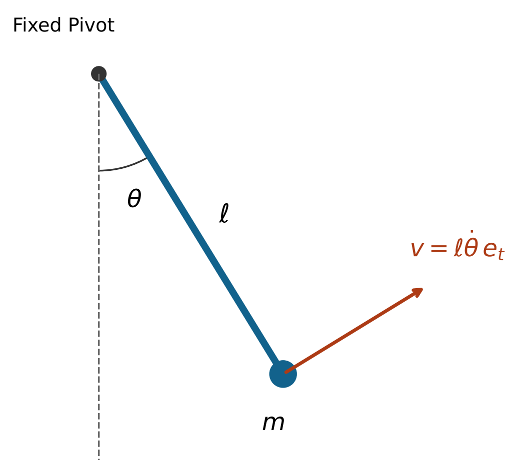


Figure 11.1: Point-Mass Pendulum With a Fixed Pivot. Tangential Velocity Depends on Length and Angular Rate; the Equation Also Requires Initial State and Applied Torque.

11.4 Deriving the Manipulator Equations

For the autonomous natural Lagrangian, the conjugate momentum is $p = \mathcal{L}_v = Mv$. Differentiate the entire product:

$$\dot{p} = M\dot{v} + \dot{M}v, \qquad \dot{M}_{ij} = \sum_k \partial_k M_{ij}\, v_k. \tag{11.9}$$

The velocity-dependent force in the equations of motion is not simply $\dot{M}v$. Euler–Lagrange also subtracts the configuration gradient of kinetic energy,

$$(\partial_q T)_i = \tfrac{1}{2}\sum_{j,k} \partial_i M_{jk}\, v_j v_k. \tag{11.10}$$

Combining these terms and symmetrizing the dummy velocity indices gives

$$\begin{aligned} \gamma_{ijk} &= \tfrac{1}{2}(\partial_k M_{ij} + \partial_j M_{ik} - \partial_i M_{jk}), \\ c_i(q,v) &= \sum_{j,k} \gamma_{ijk} v_j v_k, \qquad C_{ij}(q,v) = \sum_k \gamma_{ijk} v_k. \end{aligned} \tag{11.11}$$

Here γ_{ijk} are Christoffel symbols of the first kind for the kinetic metric. Thus $c = Cv$, with C linear and c quadratic in velocity. The resulting standard form is

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = Q, \qquad G = \partial_q V. \tag{11.12}$$

For gravity alone, the applied generalized gravitational force is $-G$. The positive vector G on the left is the holding-force demand in a static unconstrained model. If V also contains springs, its gradient is not exclusively a gravity vector.

Squared-velocity and mixed-velocity terms are often called centrifugal/centripetal and Coriolis terms. Their partition depends on coordinates and conventions. Centripetal acceleration points inward in circular motion; a rotating-frame centrifugal force points outward. Mixed products require velocities, not the acceleration of another joint. Inertia coupling also appears in $M\ddot{q}$, so a claim about a distal segment's acceleration cannot be assigned to $C\dot{q}$ alone.

The force side can contain $Bu + J^T F_{\text{ext}} - Dv$. The actuator map B can depend on configuration and can be rectangular. With quaternion positions and independent angular velocities, software commonly uses $\dot{q} = N(q)v$; those v are not coordinate derivatives. The formulas above must then be transformed with the appropriate velocity representation rather than applied entry by entry unchanged. Tedrake's manipulator-equation notes discuss this distinction and force mapping [47].

11.5 Mass, Coriolis and Gravity Properties

11.5.1 When the Mass Matrix Is Positive-Definite

Define M as the velocity Hessian of a twice continuously differentiable kinetic energy. Equality of mixed partial derivatives proves symmetry. Merely observing that a scalar equals its transpose cannot prove symmetry of a proposed matrix: its skew part is invisible in $v^T M v$.

Let each body's linear/angular velocity be $J_i(q)v$ and its positive kinetic quadratic form be $\mathcal{I}_i$. Then

$$M = \sum_i J_i^T \mathcal{I}_i J_i, \qquad v^T M v = \sum_i (J_i v)^T \mathcal{I}_i (J_i v). \tag{11.13}$$

This is nonnegative. It is strictly positive for nonzero coordinate velocity when the aggregate velocity map has no kinetic null direction. Independent regular coordinates and physically nondegenerate inertias provide a standard sufficient condition. Massless variables, redundant coordinates or singular charts can violate it. Nonnegative energy alone proves only semidefiniteness.

Under a time-independent coordinate change $q = \psi(z)$, $v = J\dot{z}$ and $M_z = J^T M_q J$. Force transforms as $Q_z = J^T Q_q$, preserving $Q^T v$. Positive-definiteness is preserved under nonsingular chart changes, while individual matrix entries and their numerical condition numbers depend on coordinate units and scaling.

11.5.2 The Skew Identity and Nonunique Factors

For the Christoffel factor in Equation 11.11, symmetry of M gives

$$(\dot{M} - 2C)_{ij} = \sum_k (\partial_i M_{jk} - \partial_j M_{ik}) v_k. \tag{11.14}$$

Interchanging i and j changes the sign, proving the matrix identity. It follows that $v^T C v = \frac{1}{2} v^T \dot{M} v$. The converse argument from that one contracted scalar is insufficient because C itself depends on v.

The Christoffel factor is a convenient choice, not the only factor with the skew property. If $Kv = 0$, then $(C + K)v = Cv$. A generic such K need not preserve the matrix identity. However, a skew K does preserve it; in three coordinates $K(v) = [v]_\times$ is a nonzero example with $Kv = v \times v = 0$. A controller proof must establish the identity for its chosen factor, not assert uniqueness of that factorization.

Even the skew check is not complete model validation. It can pass for a consistently wrong inertia model. Independent kinetic-energy, force, potential-gradient and coordinate-transformation checks are needed. The numerical example uses these complementary tests.

11.5.3 Gravity Sign and Energy Units

For $V_g = \sum_i m_i g y_i(q)$, $G_i = \partial V_g / \partial q_i$. An angular coordinate gives a generalized torque; a translational coordinate gives a generalized force. A mass-matrix entry has corresponding coordinate-dependent units. Multiplying an inertia entry by $g \cos q$ does not generally produce potential energy: a lever length is required in the correct place. Derive V from physical center-of-mass heights before differentiating it.

11.6 Energy Balance Before Energy Conservation

For a general smooth Lagrangian define the energy function $E = v^T p - \mathcal{L}$, with $p = \mathcal{L}_v$. Direct differentiation gives

$$\begin{aligned} \dot{E} &= v^T (\dot{p} - \mathcal{L}_q) - \mathcal{L}_t \\ &= v^T Q - \mathcal{L}_t. \end{aligned} \tag{11.15}$$

For an autonomous natural system, homogeneity gives $v^T p = 2T$ and $E = T + V$. In that setting $\dot{E} = v^T Q$, and energy is conserved when net generalized power is zero. Zero actuator torque alone is insufficient if there are other forces, changing constraints, prescribed motion or unmodeled dissipation. Conversely, a nonzero force can do zero power if its pairing with velocity vanishes.

With $Q = Bu - Dv$ and symmetric $D \succeq 0$, the balance is $\dot{E} = u^T B^T v - v^T D v$. Damping makes energy nonincreasing; strict decrease requires positive dissipation at that instant. At rest the damping power is zero. For physical joint dampers, D must be transformed with the coordinate map just like other forces.

A moving coordinate system changes the relation between E and laboratory mechanical energy. For a free particle with $x = q + a(t)$, $\mathcal{L} = m(\dot{q} + \dot{a})^2/2$. Its energy function is $m\dot{q}^2/2 - m\dot{a}^2/2$, while laboratory kinetic energy is $m(\dot{q} + \dot{a})^2/2$. Conflating them would manufacture or conceal apparent energy transfer. The balance identity remains valid for the correctly declared Lagrangian.

11.7 Hamiltonian Mechanics and Phase Space

If the velocity Hessian of $\mathcal{L}$ is nonsingular, the inverse function theorem locally solves $p = \mathcal{L}_v$ for $v = v(q, p, t)$. Define $H(q, p, t) = p^T v - \mathcal{L}(q, v, t)$. Differentiating while keeping the appropriate independent variables fixed cancels the dv terms and yields $H_p = v$, $H_q = -\mathcal{L}_q$ and $H_t = -\mathcal{L}_t$. Consequently,

$$\dot{q} = H_p, \qquad \dot{p} = -H_q + Q. \tag{11.16}$$

In the natural SPD case, $H = \frac{1}{2} p^T M^{-1}(q) p + V(q)$ equals mechanical energy. Global invertibility needs more than a local regularity assertion. Degenerate Lagrangians require constrained Hamiltonian analysis, not an ordinary inverse of a singular mass matrix.

The canonical phase space is T^*Q, with local coordinates (q, p). Its symplectic form is

$$\omega = \sum_i dq_i \wedge dp_i. \tag{11.17}$$

A Hamiltonian flow preserves this form, including when H depends explicitly on time; time dependence can nevertheless destroy energy conservation. General applied forces need not preserve symplectic structure. For $\dot{q} = p/m$, $\dot{p} = -kq - bp/m$, phase-space divergence is $-b/m$, so viscous damping contracts area. Energy preservation, volume preservation and symplecticity are distinct properties.

11.8 Noether's Theorem and Momentum Balance

A continuous symmetry of the Lagrangian gives a conserved quantity along unforced Euler–Lagrange trajectories. The symmetry must act on velocities as well as configurations and respect the model's domain and constraints. For forced trajectories it gives a balance law with the force projected onto the symmetry direction. This formulation makes conservation useful for grounded and actuated systems without pretending that they are isolated.

11.8.1 Proof for a Configuration Symmetry

Let Φ_ϵ be a smooth one-parameter group of configuration diffeomorphisms, with

$$\xi(q) = \left.\frac{d}{d\epsilon} \Phi_\epsilon(q)\right|_{\epsilon=0}. \tag{11.18}$$

Its tangent action maps (q, v) to $(\Phi_\epsilon(q), D\Phi_\epsilon(q)v)$. Assume exact invariance at fixed time:

$$\mathcal{L}(\Phi_\epsilon(q), D\Phi_\epsilon(q)v, t) = \mathcal{L}(q, v, t). \tag{11.19}$$

The candidate momentum is

$$I(q, v, t) = p^T \xi(q). \tag{11.20}$$

Differentiating the invariance condition at zero gives

$$\mathcal{L}_q^T \xi + p^T D\xi\, v = 0. \tag{11.21}$$

Along the forced equations, the product rule therefore gives

$$\dot{I} = (\mathcal{L}_q + Q)^T \xi + p^T D\xi\, v = Q^T \xi. \tag{11.22}$$

Thus the charge is conserved if the projected generalized force vanishes. Ideal reaction forces drop out when the generator is an admissible displacement. A symmetry that changes the constraint surface is not covered. If the Lagrangian changes by a total derivative dF/dt under the infinitesimal transformation, the corresponding charge becomes $p^T\xi - F$; exact invariance is sufficient but not necessary.

11.8.2 Which Quantity Is Actually Conserved?

A cyclic coordinate satisfies $\mathcal{L}_{q_i} = 0$, so $\dot{p}_i = Q_i$. Its momentum need not equal a single link's $I_i\dot{q}_i$. In a coupled chain it generally includes several angular rates. For a particle, translation symmetry in a fixed Cartesian direction gives that component of linear momentum. A simultaneous rigid rotation of all relevant positions gives the corresponding total angular-momentum component, including all masses and their contributions about the stated origin.

In uniform downward gravity, $\mathcal{L} = m(\dot{x}^2+\dot{y}^2+\dot{z}^2)/2 - mgz$ is invariant under horizontal translations and vertical-axis rotations, not vertical translations. Thus p_x, p_y and vertical angular momentum are conserved for a free particle, while $\dot{p}_z = -mg$. Time independence gives conservation of mechanical energy, not conservation of every momentum component.

A free particle described in a frame rotating at constant Ω has planar polar Lagrangian $\mathcal{L} = m[\dot{r}^2 + r^2(\dot{\theta}+\Omega)^2]/2$. It remains independent of θ. The cyclic momentum is $p_\theta = mr^2(\dot{\theta}+\Omega)$, the inertial angular momentum, not $mr^2\dot{\theta}$. A table that exerts friction supplies an external torque and changes that balance; changing coordinate frames alone does not.

Time-translation symmetry is handled by the energy identity above, which includes a time transformation rather than the fixed-time configuration action proved here. Unactuated does not automatically mean autonomous, frictionless or isolated. For a golfer, the ground can change whole-system momentum while ideal stationary contact does no contact-point power. Momentum and energy answer different questions.

11.9 Constrained Lagrangian Systems

For independent holonomic constraints $\phi(q,t) = 0$, let $A = D_q\phi$. Ideal reaction force is $A^T\lambda$, so

$$\frac{d}{dt}\mathcal{L}_v - \mathcal{L}_q = Q + A^T\lambda. \tag{11.23}$$

Velocity consistency is $Av + \phi_t = 0$. Differentiating again gives $A\dot{v} + b = 0$, where component a of the known bias is

$$b_a = v^T D^2_{qq}\phi_a\, v + 2D^2_{qt}\phi_a\, v + \partial_{tt}\phi_a. \tag{11.24}$$

For the natural-system equation define $F = Q - c - G$. The coupled solve is

$$\begin{pmatrix} M & -A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} \dot{v} \\ \lambda \end{pmatrix} = \begin{pmatrix} F \\ -b \end{pmatrix}. \tag{11.25}$$

If M is SPD and A has full row rank, $AM^{-1}A^T$ is SPD, and elimination yields

$$\lambda = -(AM^{-1}A^T)^{-1}(b + AM^{-1}F). \tag{11.26}$$

Numerical implementations solve linear systems rather than form dense inverses. Redundant constraints can make individual multipliers nonunique even when motion is determined. Scaling a constraint rescales its multiplier; the physical generalized reaction is the invariant object.

Acceleration consistency alone does not restore initially violated position or velocity constraints. Consistent initial data, projection, stabilization or a suitable differential-algebraic integrator is needed. Unilateral contacts also require nonpenetration and admissible reaction signs; friction introduces inequalities and possible slip. An equality multiplier solve alone does not certify a feasible foot-ground contact.

At fixed time, $A\delta q = 0$ makes reaction virtual work zero. Actual reaction power is

$$v^T A^T \lambda = -\lambda^T \phi_t. \tag{11.27}$$

It vanishes for stationary holonomic constraints, but a moving support can supply or absorb energy. A prescribed hand path can therefore do work on a club subsystem through its constraint reactions. Treating that subsystem as energy-conserving would exclude the very mechanism under investigation.

For an ideal nonholonomic constraint $A(q,t)v + a(q,t) = 0$, Lagrange–d'Alembert uses $A\delta q = 0$ and reaction $A^T\lambda$. In general the allowed distribution is not tangent to any lower-dimensional configuration surface. Varying an action with velocity constraints imposed on every neighboring path instead gives a vakonomic problem, usually different dynamics. Use the appropriate physical constraint law; do not obtain rolling dynamics merely by substituting velocities into $\mathcal{L}$ and varying freely.

11.10 The Kinetic Metric and Geodesic Motion

When M is SPD it defines a Riemannian metric on configuration space. Raise the first index of the first-kind symbols to obtain the connection coefficients

$$\Gamma^i{}_{jk} = \sum_\ell (M^{-1})_{i\ell}\gamma_{\ell jk}. \tag{11.28}$$

For no applied force and constant potential, the equation is $\ddot{q}^i + \sum_{jk} \Gamma^i{}_{jk}\dot{q}^j\dot{q}^k = 0$. This describes a geodesic parameterized by physical time. With forces, covariant acceleration is $\nabla_{\dot{q}}\dot{q} = M^{-1}(Q - G)$. A free geodesic need not be a globally shortest route between its endpoints.

The connection terms are not geodesic deviation. Geodesic deviation concerns the relative evolution of neighboring geodesics and involves curvature. Nonzero Christoffel symbols do not prove nonzero curvature: the Euclidean plane in polar coordinates has $M = \mathrm{diag}(1, r^2)$ and nonzero coefficients for $r > 0$, yet it is flat. The equations $\ddot{r} - r\dot{\theta}^2 = 0$ and $\ddot{\theta} + 2\dot{r}\dot{\theta}/r = 0$ can describe an ordinary straight Cartesian trajectory.

This distinction matters for swing interpretation. A change in the numerical Coriolis terms under a new coordinate chart does not imply a new physical force source. Posture changes physical kinetic coupling, while coordinates change its representation. A kinetic metric is also not automatically a contraction metric: contraction requires a differential inequality for the actual forced, dissipative or controlled state dynamics, with the relevant domain and input assumptions.

11.11 Worked Example 1: Two Uniform Pendulum Rods

11.11.1 Geometry and Distributed Kinetic Energy

Use two uniform slender rods of masses $m_1, m_2 > 0$, lengths $\ell_1, \ell_2 > 0$, a fixed base and ideal planar hinges. Both angles α_1, α_2 are absolute, counterclockwise from downward vertical. Define $e(\alpha) = (\sin\alpha, -\cos\alpha)^T$. The centers are $r_{C_1} = \ell_1 e(\alpha_1)/2$ and $r_{C_2} = \ell_1 e(\alpha_1) + \ell_2 e(\alpha_2)/2$.

Differentiation gives $\|v_{C_1}\|^2 = \ell_1^2\dot\alpha_1^2/4$ and

$$\|v_{C_2}\|^2 = \ell_1^2\dot\alpha_1^2 + \tfrac{1}{4}\ell_2^2\dot\alpha_2^2 + \ell_1\ell_2\cos(\alpha_1-\alpha_2)\dot\alpha_1\dot\alpha_2. \tag{11.29}$$

Each rod also has center rotational inertia $I_{C_i} = m_i\ell_i^2/12$. Add both terms from Equation 11.2; treating each rod as a point at its center would be a different physical model. With $\Delta = \alpha_1 - \alpha_2$ and $k = m_2\ell_1\ell_2/2$,

$$M_\alpha = \begin{pmatrix} (m_1/3+m_2)\ell_1^2 & k\cos\Delta \\ k\cos\Delta & m_2\ell_2^2/3 \end{pmatrix}. \tag{11.30}$$

The cross term in $T = \dot\alpha^T M_\alpha \dot\alpha/2$ is $k\cos\Delta\,\dot\alpha_1\dot\alpha_2$. Its coefficient is the off-diagonal entry because the quadratic form contains both symmetric entries and the leading factor one-half.

The determinant has the positive lower bound

$$\begin{aligned} \det M_\alpha &= \ell_1^2\ell_2^2\left(\frac{m_1m_2}{9} + \frac{m_2^2}{3} - \frac{m_2^2}{4}\cos^2\Delta\right) \\ &\geq \ell_1^2\ell_2^2\left(\frac{m_1m_2}{9} + \frac{m_2^2}{12}\right) > 0. \end{aligned} \tag{11.31}$$

Together with a positive diagonal entry this proves SPD for the declared positive parameters, including aligned rods.

11.11.2 Bias, Potential and Motor Torques

Potential relative to pivot height and its gradient are

$$\begin{aligned} V &= -(m_1/2+m_2)g\ell_1\cos\alpha_1 - m_2g\ell_2\cos\alpha_2/2, \\ G_\alpha &= \begin{pmatrix} (m_1/2+m_2)g\ell_1\sin\alpha_1 \\ m_2g\ell_2\sin\alpha_2/2 \end{pmatrix}. \end{aligned} \tag{11.32}$$

The derivative of the off-diagonal mass entry is $\dot M_{12} = -k\sin\Delta(\dot\alpha_1 - \dot\alpha_2)$. The Christoffel factor and its contracted bias are

$$C_\alpha = k\sin\Delta\begin{pmatrix} 0 & \dot\alpha_2 \\ -\dot\alpha_1 & 0 \end{pmatrix}, \qquad c_\alpha = k\sin\Delta\begin{pmatrix} \dot\alpha_2^2 \\ -\dot\alpha_1^2 \end{pmatrix}. \tag{11.33}$$

Direct substitution gives

$$\dot M - 2C = k\sin\Delta(\dot\alpha_1+\dot\alpha_2)\left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right). \tag{11.34}$$

This matrix is skew.

Let u_1 be the base motor torque and u_2 the elbow torque on rod 2, whose equal opposite reaction acts on rod 1. Their virtual work is $u_1\delta\alpha_1 + u_2(\delta\alpha_2 - \delta\alpha_1)$, so

$$Q_\alpha = \begin{pmatrix} u_1 - u_2 \\ u_2 \end{pmatrix}, \qquad \dot{E} = u_1\dot{\alpha}_1 + u_2(\dot{\alpha}_2 - \dot{\alpha}_1). \tag{11.35}$$

Writing (u_1, u_2) directly on the right in absolute coordinates would change the actuation model. Internal joint torques can do mechanical work because the connected bodies have different angular velocities. Equal and opposite torque does not mean zero summed power.

For unit masses and lengths with $\Delta = 0$, the correct matrix is $\left(\begin{smallmatrix} 4/3 & 1/2 \\ 1/2 & 1/3 \end{smallmatrix}\right)$, with determinant $7/36$. This supplies a quick numerical checkpoint. The distributed-particle integration in the companion tests checks the whole quadratic form, not just this special pose.

11.12 Worked Example 2: A Two-Link Robot Arm

Let q_1 be the first rod's angle from horizontal and q_2 the relative elbow angle. The same physical rods have $\alpha = Sq + (\pi/2, \pi/2)^T$, with $S = \left(\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right)$. Therefore $\dot{\alpha} = S\dot{q}$, $M_q = S^T M_\alpha S$, $C_q = S^T C_\alpha S$ and $G_q = S^T G_\alpha$. The positive offset is required by the downward-vertical convention.

The endpoint is

$$(\ell_1 \cos q_1 + \ell_2 \cos(q_1 + q_2), \ell_1 \sin q_1 + \ell_2 \sin(q_1 + q_2))^T. \tag{11.36}$$

Its position Jacobian is

$$J = \begin{pmatrix} -\ell_1 \sin q_1 - \ell_2 \sin(q_1 + q_2) & -\ell_2 \sin(q_1 + q_2) \\ \ell_1 \cos q_1 + \ell_2 \cos(q_1 + q_2) & \ell_2 \cos(q_1 + q_2) \end{pmatrix}. \tag{11.37}$$

With $b = m_2\ell_2^2/3$ and $d = m_2\ell_1\ell_2 \cos q_2/2$, the mass matrix is

$$M_q = \begin{pmatrix} (m_1/3 + m_2)\ell_1^2 + b + 2d & b + d \\ b + d & b \end{pmatrix}. \tag{11.38}$$

In particular the distal squared-length contribution to M_{12} is $m_2\ell_2^2/3$, not $m_2\ell_2^2/6$. It includes both translation and rod rotation. For $h = m_2\ell_1\ell_2 \sin q_2/2$, one Christoffel factor gives

$$C_q = h \begin{pmatrix} -\dot{q}_2 & -(\dot{q}_1 + \dot{q}_2) \\ \dot{q}_1 & 0 \end{pmatrix}, \qquad c_q = h \begin{pmatrix} -2\dot{q}_1\dot{q}_2 - \dot{q}_2^2 \\ \dot{q}_1^2 \end{pmatrix}. \tag{11.39}$$

Let $a_g = (m_1/2 + m_2)g\ell_1$ and $b_g = m_2 g\ell_2/2$. Then

$$\begin{aligned} V &= a_g \sin q_1 + b_g \sin(q_1 + q_2), \\ G_q &= \begin{pmatrix} a_g \cos q_1 + b_g \cos(q_1 + q_2) \\ b_g \cos(q_1 + q_2) \end{pmatrix}. \end{aligned} \tag{11.40}$$

Now $Q_q = (u_1, u_2)^T$, and motor power is $u^T\dot{q}$. The two coordinate descriptions produce identical Cartesian motion and power. Their different individual coefficients are not competing physical explanations.

11.13 Lagrangian Equations and Recursive Computation

A formulation and an algorithm answer different questions. Euler–Lagrange and Newton–Euler equations describe the same declared classical model. Either can support simulation, inverse dynamics, linearization or controller design; an explicit symbolic expression for every matrix is not required for feedback or optimization.

Recursive Newton–Euler evaluates inverse dynamics: given motion, it computes generalized force. The articulated-body algorithm solves forward dynamics: given force and state, it computes acceleration. For a tree with bounded joint dimension, both have linear complexity for their respective tasks. Building a dense mass matrix and factoring it is another forward-dynamics strategy, with cubic dense factorization cost; that cost is not inherent to the Lagrangian formulation. Closed-loop/contact solves add their own work.

Comparing an inverse-dynamics recursion with a forward-dynamics matrix solve is not an equal-task benchmark. A claim of a hundredfold speedup for a 15-coordinate golf model needs measured implementations, hardware, tolerances and identical tasks. The chapter makes no such performance claim. The useful connection is structural: energy-derived expressions provide independent checks on recursive results, and recursive algorithms provide efficient evaluations for larger models [13].

11.14 Symplectic Integration and Its Limits

For a conservative Hamiltonian system, a symplectic one-step map preserves the canonical form. It need not preserve the original Hamiltonian exactly. Phase errors can accumulate even when energy error stays small, and a chaotic trajectory can diverge from a reference while remaining on a nearby energy level. Contacts, dissipation and control inputs need a method appropriate to their actual structure.

11.14.1 Symplectic Euler for a General Hamiltonian

One symplectic Euler variant is

$$p_{n+1} = p_n - hH_q(q_n, p_{n+1}), \tag{11.41}$$

$$q_{n+1} = q_n + hH_p(q_n, p_{n+1}). \tag{11.42}$$

The first equation is generally implicit in the new momentum. The generating function is

$$F_2(q_n, p_{n+1}) = q_n^T p_{n+1} + hH(q_n, p_{n+1}). \tag{11.43}$$

It gives $p_n = \partial_{q_n} F_2$ and $q_{n+1} = \partial_{p_{n+1}} F_2$, establishing symplecticity where the implicit map is regular. Evaluating the first gradient at old momentum instead does not generally preserve the form. Nonlinear solve tolerances also matter in an implementation.

For a nondimensional example, set $x = \exp q$ for a free unit-mass particle on $x > 0$. Its Hamiltonian is $H = e^{-2q}p^2/2$. Set $a = e^{-2q_n}$. The momentum equation is $p_{n+1} = p_n + hap_{n+1}^2$. On the branch continuous from $h = 0$,

$$p_{n+1} = \frac{2p_n}{1 + \sqrt{1 - 4hap_n}}, \qquad q_{n+1} = q_n + hap_{n+1}. \tag{11.44}$$

Require a positive discriminant for a regular real local branch. This example isolates the implicit-momentum requirement without a complicated robot. Its map preserves area; replacing p_{n+1}^2 by p_n^2 in the first equation yields determinant $1 - 6z^2 - 4z^3$, where $z = hap_n$, which generally differs from one.

11.14.2 Separable Verlet and Step-Size Restrictions

For a separable $H(q, p) = K(p) + V(q)$, compose exact kick and drift substeps:

$$p_{n+1/2} = p_n - \tfrac{h}{2}\nabla V(q_n), \tag{11.45}$$

$$q_{n+1} = q_n + h\nabla K(p_{n+1/2}), \tag{11.46}$$

$$p_{n+1} = p_{n+1/2} - \tfrac{h}{2}\nabla V(q_{n+1}). \tag{11.47}$$

This Störmer–Verlet map is second order, reversible and symplectic. A robot Hamiltonian $p^T M^{-1}(q)p/2 + V(q)$ is generally nonseparable, so this explicit kick–drift formula cannot simply be reused. Implicit midpoint is another symplectic option for a smooth general canonical Hamiltonian when its implicit equations are solved consistently.

Even a symplectic scheme can be unstable. For the unit-frequency harmonic oscillator, momentum-first symplectic Euler has matrix

$$A_h = \begin{pmatrix} 1 - h^2 & h \\ -h & 1 \end{pmatrix} \tag{11.48}$$

in (q, p) order. Its determinant is one, but its eigenvalues leave the unit circle for $h > 2$. For $0 < h < 2$ it preserves the positive quadratic form $(q^2 + p^2 - hqp)/2$, a modified energy different from $(q^2 + p^2)/2$. At the boundary $h = 2$, generic solutions can grow through a nontrivial Jordan block. Symplecticity alone gives no unconditional energy bound.

Backward error analysis explains near energy preservation for sufficiently small fixed steps under appropriate smoothness, boundedness and regularity assumptions. Analytic Hamiltonians and suitable methods can yield exponentially long estimates; the modified series is generally formal or truncated, not a guarantee that every numerical orbit exactly solves a globally convergent nearby system. Hairer's review states the small-step/highest-frequency regime and the limits of that analysis [18]. Neither stability for arbitrarily long times nor exact phase accuracy follows. Ordinary state-dependent step adaptation can also destroy the original symplectic property.

11.15 Python Example: Dynamics and Power Checks

The importable example uses the uniform rods and absolute-angle motor map derived above. It solves linear systems rather than forming an inverse. Run from the repository root with its Python dependencies. The integration uses an adaptive high-order ODE solver as a finite-horizon accuracy check; it is not presented as symplectic. The independently tested functions also expose the relative-coordinate arm and the one-dimensional implicit example.

```python
import numpy as np
from scipy.integrate import solve_ivp
from src.tools.lagrangian_examples import RodPair, variable_mass_step

```

```python
model = RodPair()
initial = np.array([0.6, -0.2, 0.3, -0.4])
times = np.linspace(0, 5, 501)
for torque in (np.zeros(2), np.array([0.2, -0.1])):
    for tolerance in (1e-9, 1e-11):
        result = solve_ivp(
            lambda t, state: np.r_[
                state[2:], model.acceleration(state, torque)
            ],
            (times[0], times[-1]), initial,
            method="DOP853", t_eval=times,
            rtol=tolerance, atol=tolerance / 100,
        )
        if not result.success:
            raise RuntimeError(result.message)
        energy = np.array([model.energy(s) for s in result.y.T])
        joints = np.vstack((result.y[0], result.y[1] - result.y[0]))
        work = torque @ (joints - joints[:, :1])
        residual = energy - energy[0] - work
        print(torque, tolerance, np.max(np.abs(residual)))
print(model.mass_matrix(np.zeros(2)))
print(variable_mass_step(np.array([0.2, 0.4]), 0.1))
```

For constant physical motor torques, define $\Delta\alpha_i = \alpha_i - \alpha_{i,0}$. The work is

$$W = u_1\Delta\alpha_1 + u_2(\Delta\alpha_2 - \Delta\alpha_1). \tag{11.49}$$

Comparing that analytic work with sampled energy change checks the force-coordinate pairing. For zero torque the same residual becomes energy error. Inspect the maximum error over the interval, not merely the difference between two endpoints; repeat at tighter tolerances before interpreting fine features.

The tests independently integrate particle kinetic energy over each uniform rod, check the mass matrix's positive eigenvalues, differentiate potential and inertia numerically, verify coordinate transformation, and compare energy change with motor work. An energy check by itself would miss a consistently wrong model. None of these numerical checks validates a particular golfer's inertial parameters, muscle forces or swing strategy.

11.16 Golf and Humanoid Modeling

Geometry determines which configurations and contact motions are admissible. The kinetic model then assigns inertia to those motions; actuator and contact maps determine which forces can produce them. The resulting dynamics determine how power, momentum and state evolve. These are connected stages of one explanation. A posture may change clubhead sensitivity or joint coupling without proving that a particular muscle sequence caused the observed speed.

Define the system boundary before discussing energy transfer. For a club alone, moving-hand force and moment can supply power. For golfer plus club, grip interactions are internal. Their equal and opposite wrenches cancel in total power when the paired interface velocities agree, as in an ideal rigid grip. Relative slip, deformation or other interface motion requires its own dissipation or storage accounting. Muscle action can increase mechanical energy while drawing on chemical energy; shaft bending can store and release energy. A rigid-joint model cannot quantify omitted elastic storage.

At an ideal stationary sticking ground contact, contact-point velocity is zero and contact force does no instantaneous mechanical power, even though ground reaction changes total momentum and enables muscle forces to accelerate the body. Real shoes, turf, slip and compliance can change the local power balance. A force multiplied by center-of-mass velocity is not automatically ground-contact power; the point of application matters.

Within a selected coordinate chart, $M\ddot{q}$, $C\dot{q}$ and G help organize calculations. Their individual entries are not coordinate-invariant labels for independent energy channels. To attribute a change in club delivery to a modeled action, perturb a declared input or parameter, retain the physical constraints, recompute the coupled motion and compare the relevant output at the same event. Include actuator limits, feedback, state uncertainty and impact timing before drawing performance conclusions.

This leads to a practical writing standard: state the boundary, coordinates, inertia assumptions, force maps and power balance before assigning a causal story to a velocity peak. Energy consistency supplies a necessary foundation; it does not by itself identify a unique mechanism, a globally optimal swing or a stable controller.

11.17 Further Reading

Goldstein, Poole and Safko [15] and Arnold [1] treat classical variational and Hamiltonian mechanics. Marsden and Ratiu [32] develop symmetry, momentum maps and reduction; Murray, Li and Sastry [35] connect geometric mechanics with robotics.

For accessible primary supplements, Tedrake's multibody notes discuss manipulator equations, force maps and velocity representations [47]; Hairer's numerical-analysis review gives the implicit symplectic Euler formulas and qualified long-time energy reasoning [18]. Featherstone [13] treats recursive computation, while Siciliano et al. and Spong et al. develop robot dynamics and control applications [42, 44]. The worked formulas, counterexamples and numerical checks in this chapter are derived explicitly so that a citation is not asked to substitute for their verification.

11.18 Exercises

1. For $\mathcal{S} = \int_0^T (q^2 + \dot{q}^2)\, dt$, $T > 0$, with $q(0) = 0$ and $q(T) = 1$, derive $\ddot{q} - q = 0$ and solve for $q(t)$. Compute the second variation and determine whether the stationary path is a strict minimum.

2. Derive the point-mass pendulum equation from the stated downward-vertical convention. Add a motor torque u and viscous hinge damping $-b\dot{\theta}$, with $b \geq 0$, and verify $\dot{E} = u\dot{\theta} - b\dot{\theta}^2$.

3. Starting from $\mathcal{L} = m[\dot{r}^2 + r^2(\dot{\theta} + \Omega)^2]/2$ for a free planar particle in a frame rotating at constant Ω, identify the cyclic momentum. Relate it to inertial angular momentum. Then add a specified table friction torque and derive the new momentum balance.

4. Consider a three-link horizontal planar arm with a fixed base, no gravity in its plane and no applied torques. Use relative joint angles and rotational invariance to identify a

cyclic coordinate and its conserved momentum. Explain why the other two generalized momenta are not generally conserved. State what changes when gravity acts in the motion plane.

5. Prove symmetry from the kinetic Hessian and positive-definiteness from an injective aggregate velocity map with positive body kinetic forms. Give a massless-coordinate or redundant-coordinate example for which nonnegative physical kinetic energy does not imply a positive-definite coordinate matrix.

6. Verify $\dot{M}-2C$, including the time derivative, for both two-rod coordinate descriptions. Construct a nonzero skew addition $K(v)$ in three coordinates with $Kv = 0$ to show that the skew-preserving factor is not unique.

7. Derive the complete energy of the uniform-rod pendulum and check its balance for arbitrary initial velocities. Explain why release from rest is not required for conservation in the autonomous unforced model. Compare the kinetic quadratic form with independent integration along the rods.

8. For $\mathcal{L} = m\dot{x}^2/2 - kx^2$ with $m, k > 0$, compute p, H and Hamilton's equations. Identify the oscillator frequency and distinguish the coefficient k in this potential from the conventional spring constant multiplying $x^2/2$.

9. A bead of mass m slides frictionlessly on the stationary parabola $y = ax^2$, with downward gravity. Derive the equation using $\delta y = 2ax\delta x$ and independently using the reduced Lagrangian. Identify the effective mass, velocity bias and potential gradient.

10. A particle moves on $r^T r = R^2$ in a specified applied force F. Set reaction $2\lambda r$, derive λ from acceleration consistency, and explain the required initial position/velocity constraints. Verify that the reaction does no power on the stationary sphere.

11. Re-derive the relative-coordinate uniform-rod arm's M, C, G directly from center translation and center rotation. Compare with the constant transformation from absolute coordinates. Verify the distal squared-length contribution to the off-diagonal mass entry.

12. Put viscous dampers at the physical base and elbow hinges. Their torques are $u_1 = -b_1\dot{\alpha}_1$ and $u_2 = -b_2(\dot{\alpha}_2 - \dot{\alpha}_1)$. Find the generalized damping matrix in absolute coordinates and the energy dissipation rate. Specify when the rate is strictly negative.

13. Compute first- and second-kind Christoffel symbols for the polar-plane kinetic metric $M = \mathrm{diag}(1, r^2)$, $r > 0$. Derive the free-motion equations and verify that a straight Cartesian path satisfies them. Explain why nonzero connection coefficients do not establish curvature or geodesic deviation.

14. For an unconstrained particle in uniform vertical gravity, identify horizontal translation, vertical-axis rotation and time-translation symmetries. Derive the corresponding conserved quantities and the nonzero rate of vertical momentum. State the external-force assumptions.

15. Set the point-pendulum potential to zero at the bottom. Find the top's potential barrier and the bottom speed with that energy. For an unforced lossless pendulum, distinguish asymptotic arrival at the unstable top from finite-time crossing. Explain how active control, final velocity and losses change a swing-up requirement.

16. Implement the relative-coordinate arm and compare it with the absolute-coordinate example under identical physical torques and initial conditions. Check sampled energy-minus-work residuals at two tolerances. Independently differentiate the variable-mass symplectic map and its old-momentum alternative to test area preservation; do not use energy error alone as a test of symplecticity.

Chapter 12

Machine Learning and Neural Networks

12.1 Scope of This Chapter

Learning is useful when measured examples can improve a model, an estimator or a decision rule. It does not remove the need to define the physical system and the question being asked. A network that predicts a golfer's motion, a network that proposes muscle excitations, and a network that estimates joint torques solve different problems. Success at one is not evidence of success at the others.

This chapter develops neural function approximation, manual backpropagation, reinforcement learning and learned dynamical models in one consistent scope. It connects policy optimization to the geometric mechanics of the preceding chapters and provides a reproducible regression example. The pendulum and golf discussions specify models and experiments; they do not report a trained swing controller or new measurements of human performance.

Use q for configuration, v for generalized velocity, x for the complete modeled state, o for observations and u for the chosen physical input. In the learning equations, θ denotes network parameters, never the pendulum angle. A loss ℓ is an optimization criterion; a Lagrangian L is a mechanical function. Their similar notation in the literature does not make them the same quantity.

12.2 The Limits of Analytical Control

Rigid-body algorithms are exact evaluations of declared mathematical models up to numerical error. Real predictions also depend on identified inertias, joint definitions, contact laws, actuator dynamics, measurement errors and the appropriateness of the rigid approximation. Even a manufactured robot is not described perfectly merely because its equations are analytical. Conversely, deformable bodies can have analytical constitutive equations and numerical models. Complexity and nonlinearity alone do not force a choice between physics and learning.

For a selected smooth mechanical mode, one useful model is

$$\begin{aligned} \dot{q} &= N(q)v, \\ M(q)\dot{v} + c(q,v) + g(q) &= B(q)u + J(q)^T\lambda + f_{\text{other}}. \end{aligned} \tag{12.1}$$

Here M is inertia, c contains velocity-dependent inertial forces, g is gravity, B maps inputs to generalized forces, and $J^T\lambda$ represents constraint reactions. N maps generalized velocities to configuration rates; it need not be an identity for quaternion coordinates. Contact feasibility and actuator or tissue states complete the model. Adding a learned residual to these equations can be more informative than replacing every term with an unconstrained network, provided the residual is identifiable and does not duplicate forces already included.

12.2.1 Prediction, Inversion and Policy Are Different Maps

A forward model predicts $\dot{x} = F_\theta(x, u)$ or a sampled transition $x_{k+1} = T_{\theta,h}(x_k, u_k)$ for a declared sampling interval h and input hold. These maps have different units and different meanings. Fitting x_{k+1} directly does not make the output an acceleration. A learned transition may also depend on the integration scheme and sampling interval used to produce its data.

An inverse model might estimate u from $(q, v, \dot{v}_{\text{desired}})$ and the contact mode. For full actuation and known external loads, a torque-level inverse can be well-defined. Under-actuation, redundant muscles and unknown reactions can make an inverse nonexistent or nonunique. A least-squares torque estimate then depends on an explicitly chosen allocation or regularization criterion. A small residual is not proof that its individual muscle forces were measured correctly.

A policy instead selects an action given information and a task. Let i_k contain the observation history, target and current step. A deterministic policy gives $u_k = \mu_\theta(i_k)$; a stochastic policy specifies a conditional distribution $\pi_\theta(u_k \mid i_k)$. Identical posture can require different actions when velocity, activation, target, remaining time or disturbances differ. There is no universal hidden function from posture alone to the uniquely correct motor torque.

12.2.2 Why State Choice Matters for Golf

Position and velocity can be a sufficient state for a specified rigid torque-driven model. A muscle-driven swing may also require activation, fiber or tendon variables and shaft deformation. Two swings with the same visible geometry can therefore have different subsequent responses. Observations of joint angles, club motion and ground forces provide incomplete and noisy information about these states.

A partially observed Markov decision process retains a Markov latent state; the observation alone need not be Markov. An estimator, belief distribution or history-dependent network can use past observations to infer missing information. Recurrence does not guarantee recovery of an unobservable state. Sensor placement, synchronization and experimental excitation remain essential.

This distinction links preparation to impact. An earlier action can change configuration, velocity, stored elastic energy and activation before a later interval begins. A policy may exploit that retained state. Explaining its benefit requires controlled comparisons with the retained quantities declared, not a label such as "the network discovered passive dynamics." Classical optimal control can exploit the same dynamics when its model and objective include them.

12.3 Neural Networks as Function Approximators

With column vectors, a feedforward network composes affine maps and nonlinear activations:

$$h_0 = x, \qquad z_j = W_j h_{j-1} + b_j, \qquad h_j = \sigma_j(z_j). \tag{12.2}$$

For a real-valued regression output, the last layer may be affine: $\hat{y} = W_L h_{L-1} + b_L$. If every activation is an identity, the whole network is affine, including a composed bias. The nonlinear activations create a richer function class. The weights have shapes determined by the input and output sizes of their respective layers.

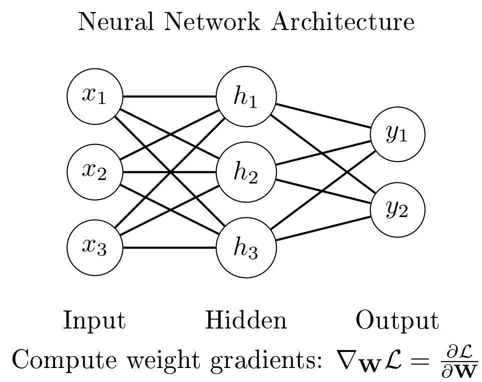


Figure 12.1: A Fully Connected Three-Input, Three-Hidden-Unit, Two-Output Network. All Fifteen Inter-Layer Weights Are Shown; the Five Biases Are Omitted.

The network in the figure has three inputs, three hidden units and two outputs: nine input-to-hidden and six hidden-to-output weights, plus five biases. Its architecture alone specifies no task or training data. A torque policy, value function and forward-model residual can share an architecture while having entirely different input definitions, losses and validation requirements.

12.3.1 Universal Approximation and Its Limits

For continuous $f : [0,1]^n \to \mathbb{R}$ and any $\epsilon > 0$, Cybenko's theorem permits a finite-width single-hidden-layer network with a continuous sigmoidal activation such that

$$\sup_{x \in [0,1]^n} \left| f(x) - \sum_{i=1}^{H} c_i \sigma(w_i^T x + b_i) \right| < \epsilon. \tag{12.3}$$

The c_i are output coefficients and b_i are hidden biases. Width and parameter magnitudes are not fixed in advance. This is an existence result, not an algorithm for finding the coefficients or a bound on the data needed. It does not guarantee uniform approximation of a discontinuous policy by continuous networks. ReLU networks also have universal approximation results with sufficient width; a blanket requirement for extra hidden depth is inappropriate. Depth can help represent particular compositional functions efficiently, but cannot rescue missing information in the inputs.

For an elementary constructive example, $|x| = \max(0, x) + \max(0, -x)$. Two ReLU hidden units and one linear output represent this function exactly. That identity is a statement about representation. Learning those parameters from a finite noisy dataset is a separate optimization and generalization problem.

12.4 Activation Functions

ReLU is $\sigma(z) = \max(0, z)$, with derivative zero for $z < 0$ and one for $z > 0$; an implementation must choose a derivative convention at zero. A unit inactive on every training input supplies no local gradient through that branch. Whether it remains inactive depends on future inputs, upstream changes and the update rule. "Dead forever" is not a theorem for every network.

The logistic sigmoid satisfies

$$s(z) = \frac{1}{1+e^{-z}}, \qquad s'(z) = s(z)(1-s(z)) \le \frac{1}{4}, \tag{12.4}$$

with equality at $z = 0$. Tanh has derivative $1 - \tanh^2 z$, reaching one at zero and tending to zero in either tail. Both activations can saturate. GELU is $z\Phi(z)$, where Φ is the standard normal cumulative distribution; its derivative is $\Phi(z) + z\phi(z)$, with ϕ the corresponding density. Smoothness does not imply uniformly large or positive derivatives.

For a stack of layers, the input Jacobian contains products $D_j W_j$, where D_j is the diagonal activation-derivative matrix. Thus

$$\left\| \frac{\partial h_L}{\partial h_0} \right\| \le \prod_{j=1}^{L} \|D_j\| \, \|W_j\|. \tag{12.5}$$

Ten sigmoid derivative factors alone are at most $4^{-10} = 9.5367 \times 10^{-7}$. The actual gradient also depends on weights, inputs and loss derivatives. Large weight norms can counteract that attenuation or cause explosion; multiplying ten sigmoid bounds is not a universal estimate of a first-layer parameter gradient.

12.5 Loss Functions and What They Estimate

For m scalar examples, use the half mean squared error

$$\ell(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2. \tag{12.6}$$

Under squared loss, the ideal unrestricted predictor is the conditional mean of the target given the input. If two hidden physiological states produce different torques at the same observed posture, averaging their labels need not yield a successful action for either state. This is a representation and information problem, not necessarily inadequate network width. Supervised labels can come from calibrated measurements, a specified inverse-dynamics procedure or simulation; a human need not supply every correct answer manually.

Huber loss with threshold $\delta > 0$ is

$$\ell_\delta(e) = \begin{cases} e^2/2, & |e| \le \delta, \\ \delta(|e| - \delta/2), & |e| > \delta. \end{cases} \tag{12.7}$$

The threshold has the same units as the residual, unless residuals have been normalized. Linear growth reduces the influence of large residuals relative to squared loss; it does not identify which observations are erroneous. A contact event, a sensor fault and a rare valid swing can all create a large residual for different reasons.

Multiple outputs require a declared metric. Squared radians, angular velocities and newtons cannot be added meaningfully without scale factors or a covariance model. Normalize using training data only and retain the transformations for deployment. Split evaluation by independent trajectories, sessions or participants as appropriate; random neighboring frames from one swing can leak almost identical states into both training and testing.

12.6 Backpropagation: Chain Rule for Neural Networks

Consider a scalar output $\hat{y} = W_2 h + b_2$, $h = \tanh(W_1 x + b_1)$, and a single-example loss $\ell = (\hat{y} - y)^2/2$. Set $e = \hat{y} - y$. Applying the chain rule gives

$$\begin{aligned} \nabla_{W_2}\ell &= eh^T, & \nabla_{b_2}\ell &= e, \\ d_1 &= (W_2^T e) \odot (1 - h \odot h), & & \\ \nabla_{W_1}\ell &= d_1 x^T, & \nabla_{b_1}\ell &= d_1. \end{aligned} \tag{12.8}$$

For a batch mean, sum the individual gradients and divide by the actual number of examples in that batch. A shorter final batch uses its own size. Compute all derivatives using the same parameter values before applying updates: changing W_2 before computing d_1 would differentiate a different network.

Backpropagation efficiently accumulates these derivatives in reverse order through a computational graph. Gradient descent, momentum and Adam are optimizers that use the derivatives to update parameters. The gradient calculation itself is not a training algorithm, a guarantee of decreasing loss or a method for choosing a physically appropriate loss.

Finite differences provide an independent check. For one parameter component θ_j, compare backpropagation with

$$\frac{\ell(\theta + \varepsilon e_j) - \ell(\theta - \varepsilon e_j)}{2\varepsilon}. \tag{12.9}$$

Choose a step large enough to avoid cancellation and small enough to control truncation error. Check every parameter group, including biases, and avoid ReLU kinks for an ordinary smooth derivative check. Passing this test validates differentiation at the checked inputs; it does not certify generalization or control stability.

12.7 Gradient Descent and Optimization

For a differentiable objective with gradient $g = \nabla\ell(\theta)$, the directional derivative along $-g$ is $-\|g\|^2$. A sufficiently small step decreases the same smooth objective when $g \neq 0$. A fixed large step can overshoot. A mini-batch gradient is a noisy estimate, so even a reasonable stochastic step need not decrease the full-data loss on every iteration.

SGD updates $\theta_{k+1} = \theta_k - \alpha_k \hat{g}_k$. A momentum convention is $m_k = \beta m_{k-1} + \hat{g}_k$, followed by $\theta_{k+1} = \theta_k - \alpha_k m_k$. Other conventions multiply the new gradient by $1 - \beta$; the learning-rate interpretation changes accordingly. Momentum can reduce oscillation in some directions and accelerate progress in others, but it can also overshoot.

Adam maintains coordinatewise first and second moments:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1-\beta_1)\hat{g}_k, \\ v_k &= \beta_2 v_{k-1} + (1-\beta_2)\hat{g}_k \odot \hat{g}_k, \\ \hat{m}_k &= m_k/(1-\beta_1^k), \quad \hat{v}_k = v_k/(1-\beta_2^k), \\ \theta_{k+1} &= \theta_k - \alpha_k \hat{m}_k/(\sqrt{\hat{v}_k} + \varepsilon). \end{aligned} \tag{12.10}$$

Here $m_0 = v_0 = 0$, operations in the last line are elementwise, and $\varepsilon > 0$ prevents division by zero. Moment adaptation is different from a schedule for α_k. Neither Adam nor momentum universally converges for every nonconvex or changing reinforcement-learning objective. Learning rate, normalization, initialization and optimizer settings remain part of the experiment.

12.8 Markov Decision Processes

For clarity, first use a finite horizon of T decisions. At step t, the agent observes state s_t, samples action a_t, receives reward r_t, and transitions to s_{t+1}. The distribution $P(s_{t+1}, r_t \mid s_t, a_t)$ is the environment model. The reward index here names the action that generated it; some texts use r_{t+1} for the same event.

The Markov property means this conditional distribution does not additionally depend on earlier history when s_t is complete. Include time in the state or use time-indexed policies when the task has a deadline. A terminal state is part of the task definition, not just an accidental interruption of data collection.

12.8.1 Return and Value

Define $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ and $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$, with $0 \le \gamma \le 1$. Rewards must have finite expectations. For an infinite discounted task, bounded rewards and $\gamma < 1$ give a finite return; $\gamma = 1$ requires additional termination or summability conditions. Continuing average-reward control is a different formulation and needs its own assumptions.

Under a specified policy π, define

$$V_t^\pi(s) = \mathbb{E}[G_t \mid s_t = s], \qquad Q_t^\pi(s,a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]. \tag{12.11}$$

The advantage $A_t^\pi(s,a) = Q_t^\pi(s,a) - V_t^\pi(s)$ compares an action with the policy's conditional average, not necessarily with doing nothing. Since $\mathbb{E}_{a\sim\pi}[A_t^\pi(s,a)] = 0$, it expresses relative merit under the chosen reward, horizon and future policy.

12.8.2 Policy Evaluation Versus Optimality

With $V_T^\pi = 0$, conditioning on the next transition gives

$$V_t^\pi(s) = \sum_a \pi_t(a \mid s)\mathbb{E}[r_t + \gamma V_{t+1}^\pi(s_{t+1}) \mid s, a]. \tag{12.12}$$

This Bellman equation evaluates the fixed policy. It does not optimize that policy. For finite action sets, the optimal recursion replaces the average with a maximum:

$$V_t^*(s) = \max_a \mathbb{E}[r_t + \gamma V_{t+1}^*(s_{t+1}) \mid s, a]. \tag{12.13}$$

Continuous spaces require integrals, and a supremum replaces the maximum unless an optimizer exists. Policy iteration alternates evaluation and improvement; value iteration applies an optimality operator. Approximate neural versions do not inherit every convergence property of finite tabular algorithms.

For a one-decision example, actions return 0 and 2. The policy choosing each equally has value 1; the optimal value is 2. Solving the evaluation equation more accurately cannot improve the equal-choice policy. This simple distinction prevents a common misunderstanding when interpreting a learned critic.

12.9 Policy Gradient Methods

Let $\zeta = (s_0, a_0, r_0, \ldots, s_T)$ be a trajectory. Assume the initial-state and environment distributions are independent of θ, the policy has differentiable positive likelihood on the sampled support, and differentiation can pass through the expectation. Its likelihood factors as

$$p_\theta(\zeta) = p_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1}, r_t \mid s_t, a_t). \tag{12.14}$$

The log derivative is a sum of policy scores. Therefore

$$\nabla_\theta J = \mathbb{E}\left[G_0 \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]. \tag{12.15}$$

Past rewards have zero expected product with a later score, because the score has conditional mean zero before the action is sampled. Removing those terms yields the return-to-go form

$$\nabla_\theta J = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right]. \tag{12.16}$$

This derivation accounts for the effect of actions on subsequent state visitation through the trajectory likelihood. It does not simply hold a policy-dependent state distribution constant. If the initial distribution, environment or reward depends directly on θ, additional derivative terms are needed. The explicit γ^t belongs to the stated start-state objective; uniform time-step averages used in practical implementations can correspond to a different weighting convention.

12.9.1 Baselines and REINFORCE

For an action-independent baseline $b_t(s_t)$,

$$\mathbb{E}_{a_t \sim \pi_\theta}[b_t(s_t) \nabla_\theta \log \pi_\theta(a_t \mid s_t)] = b_t(s_t) \nabla_\theta \int \pi_\theta(a \mid s_t)\, da = 0. \tag{12.17}$$

Thus $G_t - b_t(s_t)$ can replace G_t without changing the expected score estimator under these assumptions. Treat the baseline and sampled returns as constants in the actor loss; differentiating through a learned baseline adds unwanted terms. A baseline fitted using the same action's outcome can also introduce finite-sample dependence unless the estimator is designed appropriately.

REINFORCE samples complete returns to estimate the score-weighted update. A useful baseline can reduce variance, but an arbitrary baseline can increase it. The variance-minimizing scalar baseline generally weights returns by score magnitude; it is not always exactly V^π. Reward scale, horizon, rare successes and stochastic transitions affect estimator variance.

12.10 Actor-Critic Architecture

An actor represents the policy and a critic estimates value. For a transition that genuinely terminates the task, let $d_t = 1$; otherwise $d_t = 0$. A one-step target and temporal-difference residual are

$$y_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}), \qquad \delta_t = y_t - V_\phi(s_t). \tag{12.18}$$

The usual semi-gradient critic step minimizes $(V_\phi(s_t) - \text{stopgrad}(y_t))^2/2$. An actor loss uses a detached advantage estimate, for example $-\log \pi_\theta(a_t \mid s_t)\,\text{stopgrad}(\delta_t)$ with the objective's appropriate weighting. A low critic training loss is not a proof that δ_t estimates the true advantage accurately on all visited states.

An artificial rollout cutoff differs from task termination. If the underlying task continues, bootstrap from the final observed state; do not use a reset observation as that state. If a deadline defines the task itself, include remaining time and use zero continuation at its terminal boundary. This choice can materially change a swing policy trained around an impact deadline.

12.10.1 Multi-Step Returns and Bias-Variance Tradeoffs

Before a terminal boundary, an n-step target is

$$G_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j r_{t+j} + \gamma^n V_\phi(s_{t+n}). \tag{12.19}$$

Stop the sum at termination and omit continuation there. A longer return typically relies less on an imperfect bootstrap and uses more sampled rewards, often reducing bootstrap bias while increasing variance. These are tendencies, not universal monotonic laws: reward correlations, critic errors and environment noise matter. With an exact critic under the same policy, the shorter target need not have bootstrap bias.

Generalized advantage estimation forms a truncated weighted sum of TD residuals, $\hat{A}_t = \sum_{j=0}^{K-1} (\gamma\lambda)^j \delta_{t+j}$, with boundary masks and final bootstrap handled consistently. λ changes the mixture of multi-step estimates. Neither this estimator nor nonlinear actor-critic training guarantees convergence without further assumptions.

12.11 Proximal Policy Optimization

PPO gathers data under a frozen old policy and optimizes a surrogate using those data. Define the likelihood ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$, keeping the stored old likelihood and advantage fixed during the update. Its clipped sample objective is

$$L_t^{\text{clip}} = \min\left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon)\hat{A}_t\right). \tag{12.20}$$

For positive advantage this is $\hat{A}_t \min(\rho_t, 1+\epsilon)$; for negative advantage it is $\hat{A}_t \max(\rho_t, 1-\epsilon)$. The first branch stops rewarding a sufficiently large increase in the sampled action's likelihood. The second stops rewarding a sufficiently large decrease. Harmful changes in the opposite direction are still penalized.

With $\epsilon = 0.2$ and $\hat{A} = 2$, ratios 1.2 and 2 both give 2.4. With $\hat{A} = -2$, ratios 0.8 and 0.1 both give -1.6. These plateaus do not forbid those ratios. Shared parameters, other samples and optimizer steps can move probabilities outside the interval. The clipped quantity is no greater than the corresponding unclipped sampled surrogate, but is not a lower-bound certificate for true future performance. Clipping does not enforce a hard ratio or KL-divergence constraint or guarantee that a policy cannot collapse.

An actor-critic implementation can maximize the average clipped objective minus c_V times a value-error loss plus c_H times an entropy bonus. Equivalently, minimize its negative. Define signs and coefficients explicitly. Monitor actual likelihood ratios, KL estimates, entropy, critic errors and independent rollout performance; a plateau in the surrogate is not sufficient evaluation.

12.11.1 Stochastic Bounded Actions

A deterministic tanh network alone does not provide PPO's sampling likelihood. One possible continuous policy draws $z \sim \mathcal{N}(\mu_\theta(s), \operatorname{diag}(\sigma_\theta(s)^2))$ with positive standard deviations and maps $a_i = a_{\max,i} \tanh z_i$. Inside the bounds, change of variables gives

$$\log \pi_\theta(a \mid s) = \log p_\theta(z \mid s) - \sum_i \log \left[a_{\max,i}(1 - \tanh^2 z_i)\right]. \tag{12.21}$$

The action scales are positive and fixed here. Use numerically stable log-Jacobian evaluation near saturation. For old and new policies with the same transform and the same stored action, the common Jacobian cancels in their ratio; it still matters for the actual action density and entropy. Sampling an unbounded Gaussian and clipping afterward produces a different distribution, with probability mass at the boundaries, and cannot silently use this tanh density.

Collect new rollouts after a bounded number of optimization epochs and freeze a new old policy for the next update. Excessive reuse of the old data, changed normalization statistics or inconsistent likelihood calculations can invalidate the intended comparison. PPO is an algorithmic design, not a guarantee that these implementation details are automatically correct.

12.12 Value-Based Methods and Continuous Actions

Tabular Q-learning uses the sampled optimality target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma(1 - d_t) \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right]. \tag{12.22}$$

Its standard discounted finite-MDP convergence results require suitable learning-rate schedules and repeated exploration of the relevant state-action pairs. Replacing the table with a neural network changes the problem substantially.

DQN fits a network to targets from a separate target network:

$$\ell_Q = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}} \left[\left(Q_\theta(s, a) - \operatorname{stopgrad}[r + \gamma(1 - d) \max_{a'} Q_{\text{target}}(s', a')]\right)^2\right]. \tag{12.23}$$

Replay changes how stored experience is reused and can reduce correlations within training batches. A delayed target slows movement of the bootstrap. Neither device eliminates all statistical dependence, approximation error or instability. Exploration still determines what information enters the buffer.

For continuous actions, maximizing a general nonlinear Q can be an expensive nonconvex optimization at every state. It is not categorically impossible; special quadratic or concave forms can be tractable. DDPG uses a deterministic actor μ_θ and a learned critic. A practical actor update follows

$$\hat{g} = \mathbb{E}_{s\sim\mathcal{D}} \left[(D_\theta \mu_\theta(s))^T \nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)} \right]. \tag{12.24}$$

This is a gradient through a learned critic over replay states, not an exact global argmax or automatically an unbiased start-state performance gradient. The deterministic policy-gradient theorem uses a specified state-visitation weighting and true or suitably compatible action values. Exploration must be supplied during data collection because a deterministic actor itself does not sample alternatives.

TD3 addresses particular actor-critic estimation errors using two critics with a minimum target, delayed actor updates and target-policy smoothing. Taking a minimum can also create underestimation. SAC uses an entropy-regularized objective, schematically

$$J_{\text{soft}} = \mathbb{E} \sum_t \gamma^t \left[r_t + \alpha_{\text{ent}} \mathcal{H}(\pi(\cdot \mid s_t)) \right]. \tag{12.25}$$

The entropy coefficient changes the tradeoff being optimized. In continuous action coordinates this is differential entropy, which depends on action scaling and can be negative. These methods differ in policy class, objective and data reuse; there is no universal ranking of their stability or sample efficiency for every swing model.

12.13 Exploration Strategies

In a finite action set, epsilon-greedy exploration chooses a random action with probability ϵ and otherwise a greedy action. A uniform distribution over an unbounded continuous action space does not exist; continuous exploration requires a defined distribution and admissible bounds. For a finite action set, entropy is

$$\mathcal{H}(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s), \tag{12.26}$$

with $0 \log 0 = 0$ by continuity. Entropy regularization encourages spread according to its chosen density and coordinates, but broad action noise alone does not ensure useful coverage of long-horizon states.

A curiosity bonus can depend on forward-prediction error,

$$r_t^{\text{int}} = \eta \|\hat{T}_\theta(s_t, a_t) - s_{t+1}\|_W^2. \tag{12.27}$$

The weighting W declares the state metric. Large error can indicate a new learnable transition, sensor noise, stochasticity or a deficient model class. Rewarding all such error can repeatedly attract an agent to irreducible noise. Adding the bonus also changes the optimization objective unless its role and limiting behavior are specified.

For golf, define an exploratory simulation task before assigning human significance to its discoveries. A policy that gains club speed by exploiting unphysical contact penetration or unlimited torque has found a weakness in the model or reward, not a coaching principle.

12.14 Worked Example: Pendulum Swing-Up Objective

Use a point mass m on a massless rod of length l, a fixed pivot, viscous rotational damping $b \geq 0$, and torque u. The angle $q = 0$ points downward and $q = \pi$ upward. With $I = ml^2$,

$$I\ddot{q} = u - b\dot{q} - mgl\sin q, \qquad |u| \leq u_{\max}. \tag{12.28}$$

The model's mechanical energy above hanging and its rate are

$$E = \tfrac{1}{2}I\dot{q}^2 + mgl(1 - \cos q), \qquad \dot{E} = u\dot{q} - b\dot{q}^2. \tag{12.29}$$

These formulas make signs and input power checkable. A sample implementation must also declare its integration method, step size, torque hold and terminal conditions. A physical pendulum with distributed rod mass would have a different inertia and center-of-mass gravity coefficient.

Let $e_q = \operatorname{atan2}(\sin(q - \pi), \cos(q - \pi))$. One dimensionless upright reward is

$$r(q, \dot{q}, u) = -\left[e_q^2 + w_v(\dot{q}/\omega_*)^2 + w_u(u/u_{\max})^2\right], \tag{12.30}$$

where angular measure is in radians, $\omega_* > 0$ is a declared speed scale, and $w_v, w_u \geq 0$ are dimensionless weights. The wrapped error is discontinuous in sign at the opposite orientation, though its square is continuous there; its derivative has a cusp. The smooth periodic alternative $1 + \cos q$ also has its minimum at upright. In either case, the objective must be examined at both equilibria before training.

For illustrative $m = l = 1$, $g = 9.81$, $b = 0$, $u_{\max} = 2$, $\omega_* = 1$, $w_v = 0.1$, and $w_u = 0.01$, upright at rest with zero torque gives reward 0, while hanging gives $-\pi^2$. Replacing e_q with the raw downward angle would reverse the intended preference. At $q = \pi/2$, $\dot{q} = 1$ and $u = 2$, acceleration is -7.81 rad/s^2 and input power is 2 W. The energy difference between the two resting equilibria is 19.62 J.

An action-squared penalty is an effort regularizer, not mechanical work. Work is $\int u\dot{q}\,dt$; it can be positive or negative, whereas u^2 cannot. Metabolic expenditure requires an additional physiological model. A torque-free interval may retain kinetic energy from earlier actuation; low instantaneous effort does not show that preparation was free.

For a reproducible training experiment, specify an initial-state distribution, sampling interval, finite horizon, policy distribution, optimizer settings and random seeds. Compare against a declared baseline controller and evaluate unseen initial states and parameter perturbations. Report success criteria such as dwell time near upright, speed tolerance, torque feasibility and energy balance in addition to return. No PPO training run is reported here, so no learned swing-up trajectory or success rate is claimed.

12.15 Physics-Informed Neural Networks

A neural network can approximate a solution while a loss penalizes a specified differential-equation residual. For example, a nondimensional viscous Burgers problem is

$$\begin{aligned} w_t + ww_\xi - \nu w_{\xi\xi} = 0, &\qquad \xi \in [-1, 1], \quad t \in [0, 1], \\ w(0, \xi) = -\sin(\pi\xi), &\qquad w(t, -1) = w(t, 1) = 0, \end{aligned} \tag{12.31}$$

with $\nu = 0.01/\pi$ in the cited example. We use w rather than control input u to avoid a notation collision. Given an approximation $w_\theta(t, \xi)$, its residual is $R_\theta = w_{\theta,t} + w_\theta w_{\theta,\xi} -$

$\nu w_{\theta,\xi\xi}$. A training loss combines initial/boundary data errors and squared residuals at collocation points. The nonlinear transport sign and viscosity term must agree with the stated PDE.

This is different from fitting an unknown vector field directly to derivative labels. For $\dot{x} = F_\theta(x, u)$, minimizing $\sum_i \|F_\theta(x_i, u_i) - \dot{x}_i\|_W^2$ is derivative regression. It becomes physics-informed only through additional known equations, restrictions or priors. Noisy numerical differentiation can bias those labels, and a full-state derivative includes configuration rates as well as accelerations.

For the known forced pendulum, an appropriate trajectory residual is $I\ddot{q}_\theta + b\dot{q}_\theta + mgl \sin q_\theta - u(t)$. Observations and initial conditions constrain the solution as well. A small residual at finitely many points does not establish a global solution error bound, unique parameters or valid behavior at impacts. Enforcing an incorrect conservative equation on a driven dissipative swing can produce a physically misleading fit despite an apparently principled loss.

12.16 Neural ODEs and the Adjoint Method

A Neural ODE parameterizes a continuous vector field $\dot{h} = f_\theta(h, t)$ and computes states with an ODE solver. The state can be physical or latent; a latent dimension does not automatically correspond to measured joint coordinates. A policy evaluated at discrete observations is not a Neural ODE merely because it controls a continuous mechanical system.

For a terminal loss $\ell = \Psi(h(T), \theta)$ with fixed endpoints, define a column adjoint $a(t)$ by

$$\dot{a} = -(D_h f_\theta)^T a, \qquad a(T) = \nabla_h \Psi(h(T), \theta). \tag{12.32}$$

Differentiating $a^T \delta h$ gives the parameter contribution after the state-variation terms cancel. The result is

$$\nabla_\theta \ell = \partial_\theta \Psi + (D_\theta h(0))^T a(0) + \int_0^T (D_\theta f_\theta)^T a \, dt. \tag{12.33}$$

If the initial state is parameter-independent and the terminal loss has no direct parameter dependence, only the integral remains. Writing it as $-\int_T^0$ is equivalent; that reversed-limit notation is not itself a sign error. Running costs add source terms to the adjoint and parameter integral, while losses at intermediate observation times introduce adjoint jumps. Contact events need their own event and reset sensitivities.

These equations differentiate the continuous initial-value problem. Differentiating a particular numerical solver produces the derivative of its discrete computation; the two agree only to the relevant numerical accuracy. Backward reconstruction of a forward-dissipative state can be unstable. Checkpointing and solver tolerances affect memory use, accuracy and cost, so a universal constant-memory exact-gradient claim is unwarranted.

As a direct check, let $\dot{h} = \theta h$, $h(0) = 1$, $T = 1$ and $\Psi = h(T)^2/2$. Then $h(T) = e^\theta$ and $d\ell/d\theta = e^{2\theta}$. The adjoint integral gives the same value. This verifies the sign convention on a solvable example, not the accuracy of every adaptive-solver implementation.

12.17 Structure-Preserving Neural Networks

12.17.1 Hamiltonian Models

An autonomous Hamiltonian neural network learns a scalar $H_\theta(q, p)$ in canonical coordinates and defines

$$\dot{q} = \nabla_p H_\theta, \qquad \dot{p} = -\nabla_q H_\theta. \tag{12.34}$$

Along a smooth exact solution, $dH_\theta/dt = 0$ because the two cross-products cancel. The conserved quantity is the learned Hamiltonian; data and physical calibration must establish whether it equals the real energy. Canonical momentum is not generally equal to velocity. A numerical integrator need not conserve H_θ exactly or preserve the symplectic form merely because its vector field is Hamiltonian.

For an explicit force f_p added to $\dot{p}$, the balance becomes $\dot{H}_\theta = \nabla_p H_\theta^T f_p$, plus $\partial_t H_\theta$ when time dependence is present. Dissipation and actuation therefore call for power balances, not a claim that an actively driven golf swing conserves total mechanical energy.

12.17.2 General Lagrangian Models

For a learned smooth $L_\theta(q, v)$ with $v = \dot{q}$, the forced Euler–Lagrange equation is

$$\frac{d}{dt}\nabla_v L_\theta - \nabla_q L_\theta = \tau. \tag{12.35}$$

Let $H_{vv} = D_v(\nabla_v L_\theta)$ and $H_{vq} = D_q(\nabla_v L_\theta)$. If H_{vv} is invertible,

$$\dot{v} = H_{vv}^{-1}\left[\tau + \nabla_q L_\theta - H_{vq} v\right]. \tag{12.36}$$

A generic Lagrangian's velocity Hessian can depend on both q and v and can be indefinite or singular. It is not automatically a positive-definite mass matrix depending only on configuration. A pseudoinverse may produce a numerical vector, but does not establish that a singular system has unique physically consistent dynamics. Time-dependent Lagrangians require an additional $\partial_t \nabla_v L$ term.

For an autonomous regular model, the energy function is $E_L = v^T \nabla_v L - L$, and the forced balance is $\dot{E}_L = v^T \tau$. Lagrangian gauge freedoms also mean that fitting trajectories need not identify one unique scalar Lagrangian. Do not interpret every learned coefficient as a measured anatomical parameter.

12.17.3 Natural Mechanical Models and DeLaN

A more restricted construction sets

$$M_\theta(q) = R_\theta(q) R_\theta(q)^T + \varepsilon I, \qquad L_\theta = \tfrac{1}{2} v^T M_\theta(q) v - U_\theta(q), \tag{12.37}$$

where R_θ is lower triangular and $\varepsilon > 0$ is defined in appropriately scaled inertia units. This guarantees a positive-definite inertia matrix. Deriving both inertial bias and gravity from this same L_θ gives a coherent natural mechanical model. It does not guarantee identified parameters or accurate contact forces outside the training regime.

The original DeLaN paper parameterizes a triangular inertia factor and a separate gravity-force network. Learning a scalar potential and taking its gradient, as in the construction above, adds an integrability restriction; it should not be attributed to every original

DeLaN implementation. A separately learned force vector is not automatically conservative. Likewise, an independently learned Coriolis matrix must not be assumed consistent with a learned mass matrix without verification.

Friction, muscle activation, tendon storage, contact and input limits require explicit representation beyond the conservative component. For the swing, a useful learned model should expose which quantities exchange power, which dissipate energy and which retain history. Structural priors can reduce certain impossible predictions while still encoding the wrong physiological mechanism.

12.18 Sim-to-Real Transfer and System Identification

A known analytical model or a learned forward model can support model-based planning. Model-free learning does not explicitly require that forward model in its policy/value update, but can still use physical representations, simulation, constrained actions and engineered rewards. Neither category is universally more data-efficient. Model errors, exploration cost and the precision required near impact determine the relevant tradeoffs.

Domain randomization trains over a specified distribution of plausible parameters, delays, sensor errors and disturbances. The distribution is part of the model. Mass and inertia should vary consistently, and correlated subject parameters should not be replaced automatically by independent uniform noise. Terrestrial gravity is usually much better known than muscle strength or sensor bias; arbitrary gravity variation is not a default model of differences among golfers.

Evaluate on withheld parameter combinations and, when claiming transfer, on appropriately governed physical observations. Training on diverse simulations does not substitute for real validation. Report failures and uncertainty rather than only mean reward. A policy may be robust to the sampled uncertainties and fragile to an omitted contact or actuator mechanism.

12.18.1 What Can the Data Identify?

For an unforced ideal point-mass pendulum,

$$\ddot{q} = -\kappa \sin q, \qquad \kappa = g/l. \tag{12.38}$$

Any pair (cg, cl) with $c > 0$ gives the same angle dynamics. Angle-only free-motion data therefore cannot identify g and l separately. At an exact resting equilibrium they may provide no information even about κ. A network cannot remove this structural ambiguity by having more parameters.

An independent length measurement, known gravity or a calibrated forced experiment can add information. With measured torque, acceleration depends on both $1/(ml^2)$ and g/l; which individual parameters become identifiable still depends on what else is known and whether the input sufficiently excites the system. In a golfer, redundant muscle forces and unknown external loads create analogous ambiguities. Design the measurement and intervention before giving a fitted parameter a mechanistic interpretation.

12.19 Python Worked Example: Backpropagation From Scratch

The accompanying example fits the scalar function $y = x^2$ on $[-1, 1]$ using a tanh hidden layer and a linear output. Its implementation is shared between the print and web lessons in the repository module `src.tools.learning_examples`. This is supervised regression, not reinforcement learning, inverse dynamics or a pendulum controller.

The implementation uses row samples, so X has shape $(m, 1)$, the input weights have shape $(1, H)$, hidden activations have shape (m, H) and output weights have shape $(H, 1)$. These are the transposed storage conventions of the earlier column-vector derivation. The actual gradient core is reproduced below; training and validation use that same source.

```python
prediction = predict(network, features)
hidden = np.tanh(features @ network.input_weights + network.hidden_bias)
error = prediction - targets
output_derivative = error / features.shape[0]
hidden_derivative = (output_derivative @ network.output_weights.T) * (1 -
    hidden**2)
gradient = Network(
    features.T @ hidden_derivative,
    hidden_derivative.sum(axis=0, keepdims=True),
    hidden.T @ output_derivative,
    output_derivative.sum(axis=0, keepdims=True),
)
loss = float(np.mean(error**2) / 2)
```

Mini-batches are shuffled with an explicit random generator. Every nonempty slice is processed, including a partial final batch or one batch larger than the whole dataset. After all updates, the reported epoch loss is recomputed on the full dataset at the final parameters. Averaging batch losses computed at changing parameter values would answer a different question, and dividing by the number of full batches would misweight a remainder or divide by zero for an oversized batch.

```python
from src.tools.learning_examples import regression_demo

result = regression_demo()
print(result)  # Interactive lesson output; the library performs no printing.
```

The declared example uses seed 2026, 12 hidden units, 41 equally spaced training inputs, batch size 16, learning rate 0.05 and 1,200 epochs. It evaluates additional interval midpoints never used as training inputs. The numerical report is:

Initial half MSE: 0.158078703; final training half MSE: 0.001261210; unseen-midpoint half MSE: 0.001021767.

Tests compare manual gradients with centered finite differences for every weight and bias, check partial and oversized batches against the actual final full-data loss, and verify deterministic reproduction. Midpoint accuracy tests interpolation within this example's interval. It is not evidence for extrapolation to arbitrary inputs, a simulation throughput claim, or successful control of a physical system.

12.20 Deep Reinforcement Learning for Robot and Golf Control

The technical chain begins with the task: desired impact position and time, clubhead velocity and orientation, balance or contact requirements, and allowable actuator and tissue loads. Those quantities determine what the policy must know and what its reward should value. A scalar reward is a modeling choice, not an automatic expression of "good golf." Distinct combinations of speed, accuracy, repeatability and loading can produce different preferred actions.

12.20.1 State Representation and Action Parameterization

Use consistent frames, units and phase conventions. A representation containing pelvis orientation, joint configuration, generalized velocity, club state and target information still may omit activation, shaft vibration or contact history. Test the consequences of these omissions. A periodic angle representation such as $(\sin q, \cos q)$ avoids a raw wrap discontinuity but does not uniquely represent every three-dimensional orientation; rotation encodings need their own constraints and continuity analysis.

Direct joint torque, muscle excitation and desired joint position are different action ports. A desired position normally passes through a feedback controller, so its force response depends on gains and current state. Muscle excitation passes through activation and contraction dynamics. Document these layers and their limits before comparing policies. A low excitation at impact can coexist with retained activation or tendon force, and a bounded position command need not imply a bounded joint torque.

When comparing actions from the same frozen state in a torque-affine mechanical model, instantaneous acceleration increments can add linearly. Policies applied over time change the state, feedback and contact mode, so their full trajectories need not add. Learned strategies should respect this distinction rather than treating trajectory interaction as evidence against force addition.

12.20.2 Reward Shaping and Evidence

Adding speed bonuses, torque penalties or trajectory-tracking terms can change the optimal task. Potential-based shaping has a special telescoping structure: with $r_t' = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t)$,

$$G_0' = G_0 - \Phi(s_0) + \gamma^T \Phi(s_T). \tag{12.39}$$

For a fixed initial distribution and an action-independent terminal term, such as zero potential at every terminal state, the added terms do not change policy ranking. In an infinite discounted task a bounded potential makes the terminal term vanish. Arbitrary penalties do not share this property. A finite-horizon impact bonus or a nonzero terminal potential must be checked explicitly.

A large penalty for exceeding a limit is not a guarantee that the limit is never exceeded. Hard constraints, feasibility checks and validated safety mechanisms have separate roles. A reported simulation experiment should name its numerical tolerances, failure conditions, episode resets and contact handling. If an outcome depends on a very narrow impact timing interval, small integration or observation delays can alter the apparent strategy.

To explain a learned swing, compare matched models and controlled interventions: retain the initial state, define the altered input port, preserve unrelated parameters and measure the specified downstream outcome. Assess whether the explanation survives plausible modeling errors and withheld observations. A zero-torque counterfactual removes the declared torque channel; it does not erase prior work, stored energy, gravity or other active channels. Sensitivity analysis can reveal interactions, while identifiability analysis tells us which of those interactions the available measurements can distinguish.

12.21 Chapter Summary and Further Reading

Function approximation supplies a flexible representation; differentiation supplies parameter sensitivities; optimization uses those sensitivities to change a stated objective. Reinforcement learning adds the consequences of decisions over time. Learned mechanics adds structural assumptions about how states evolve. Each layer can be mathematically correct while the overall physical explanation remains unsupported because the state, input, objective or evidence is wrong.

The preceding geometry and recursive-dynamics chapters provide coordinate maps and consistent mechanics. This chapter adds tools for fitting uncertain components and optimizing policies. Their connection is especially useful in golf because earlier actions change the conditions under which later actions operate. Technical rigor comes from making those dependencies testable, not from assigning a universal sequence or physiological interpretation to a network's behavior.

The references below identify methods and assumptions, not empirical validation of a human golf policy. Cybenko's Theorem 2 supports the bounded approximation statement. Goodfellow, Bengio and Courville's chapters on feedforward networks and optimization distinguish gradient computation from training. Sutton and colleagues' policy-gradient paper explains the state-visitation weighting and compatible-critic conditions. The PPO paper's clipped surrogate and actor-critic algorithm motivate the update described here; they do not prove hard clipping or safety.

Cybenko (1989), Theorem 2 [9]. Uniform approximation on the unit cube; no training guarantee.

Goodfellow, Bengio and Courville (2016), Chapters 6 and 8 [16]. Feedforward models, loss functions, backpropagation and optimization; see the book navigation for Chapter 8.

Sutton and Colleagues (1999), Policy Gradients [46]. State-visitation weighting and compatible function approximation; not a blanket deep actor-critic convergence theorem.

Schulman and Colleagues (2017), PPO [40]. Sections 3 and 5 define the clipped surrogate and rollout/update procedure.

Mnih and Colleagues (2015), DQN. Replay and target-network method; its game benchmarks are not golf evidence.

Silver and Colleagues (2014), Deterministic Policy Gradients. The deterministic policy-gradient framework and exploration distinction.

Lillicrap and Colleagues, Continuous Control. The DDPG actor, critic, replay and target-network implementation.

Fujimoto, Van Hoof and Meger (2018), TD3. Twin critics, delayed updates and target-policy smoothing; Algorithm 1.

Haarnoja and Colleagues (2018), SAC. Entropy-regularized objective and approximate soft actor-critic; theoretical assumptions differ from practical approximation.

Raissi, Perdikaris and Karniadakis, PINNs [38]. The authors' specified Burgers problem and collocation residual; no general solution-error certificate.

Chen and Colleagues (2018), Neural ODEs [7]. Section 2, numerical considerations and Appendix B explain adjoint calculation and its numerical limits.

Greydanus, Dzamba and Yosinski (2019), HNNs [17]. Section 2 constructs a learned conserved quantity in canonical coordinates.

Cranmer and Colleagues (2020), LNNs [8]. Euler–Lagrange derivative construction and its distinction from a natural mechanical model.

Lutter, Ritter and Peters (2019), DeLaN [26]. Sections 3–4 parameterize inertia and a separate gravity force; distinguish a scalar-potential variant.

12.22 Exercises

1. **Representation:** Construct $|x|$ from two ReLU units. Then explain why continuous networks cannot uniformly approximate a jump of nonzero height to arbitrarily small error on a domain containing the jump.

2. **Backpropagation:** Derive all four parameter gradients for the tanh network. Compare them with centered finite differences at three nonzero inputs. State the loss normalization and array shapes.

3. **Activation Functions:** Plot sigmoid, tanh, ReLU and GELU and their derivatives. Identify saturation regions and nondifferentiability. Discuss advantages and disadvantages for a specified regression problem without asserting one activation is universally best.

4. **Gradient Bounds:** Derive the product-of-Jacobians bound for ten sigmoid layers. Evaluate the bound when every weight norm is one and when every weight norm is eight. Explain why neither is a measured parameter gradient.

5. **Pendulum Task:** Specify an MDP with the downward angle convention, torque bounds, input hold, integrator, observation, horizon and upright reward. Check reward and energy at both resting equilibria and at the numerical state in the text.

6. **Evaluation and Improvement:** In a 5-by-5 grid, movements are deterministic; attempted off-grid moves leave the state unchanged. The upper-right cell is absorbing with zero future reward; every preceding action earns -1, including the action entering the goal. Use $\gamma = 0.9$. Compare uniform-random policy evaluation with optimal value iteration from all-zero initial values.

7. **Policy Gradient:** Derive the finite-horizon score estimator for the stated discounted start-state objective. Show explicitly where the γ^t factor and the cancellation of past rewards enter.

8. **Bootstrap Boundaries:** For rewards $(1, 2, 3)$, $\gamma = 0.9$, critic values $V(s_1) = 4$, $V(s_2) = 5$, and true termination at s_3, compute the one-, two- and three-step targets from s_0. Explain what repeated trajectories would be needed to compare their variance.

9. **Actor-Critic:** Write pseudocode that keeps bootstrap targets and actor advantages detached. Distinguish a genuine terminal transition from a truncated continuing roll-out and specify the final observation used for bootstrapping.

10. **PPO:** Plot the clipped sample objective for $\epsilon = 0.2$ and advantages 2 and -2. Give ratios outside the interval that lie on its plateaus. Explain why the plot does not establish a hard probability bound or prevent policy collapse.

11. **Physics Residual:** Write a trajectory-fitting loss for the forced damped pendulum with known input and initial conditions. Give a residual that would be incorrect if actuation or damping were omitted. Propose held-out residual checks.

12. **Domain Randomization:** Define a physically consistent uncertainty distribution for the pendulum's mass, length, torque calibration and observation delay. Explain how inertia changes with mass and length, and separate training draws from held-out evaluation cases.

13. **Identification:** Show that scaling both g and l by the same positive factor leaves free angle trajectories unchanged. Identify an additional measurement or calibrated experiment and explain which ambiguity it removes.

14. **Adjoint and Conservation:** Verify the scalar Neural ODE derivative in the text analytically and with finite differences. Separately show that explicit Euler applied to $H = (q^2 + p^2)/2$ multiplies H by $1 + h^2$ per step, despite the exact Hamiltonian flow conserving it.

15. **Reproducible Learning:** Run the shared regression example with batch sizes that divide the dataset, leave a remainder and exceed its size. Then design, but do not claim to have executed, a multi-seed PPO pendulum comparison with explicit success and failure criteria.

16. **Golf Interpretation:** A simulated policy uses less commanded torque near impact after a stronger early action. List the retained states and power flows needed to interpret that result. Design a matched counterfactual and identify which conclusions would still require independent human measurements.

Glossary of Important Concepts

These definitions distinguish geometric objects, physical models, and the guarantees obtained from a calculation. A definition does not establish that a particular golf model satisfies its assumptions.

Articulated Body Algorithm
A recursive forward-dynamics algorithm for a rigid-body tree. With bounded joint dimension, its arithmetic cost scales linearly with the number of bodies. Closed-loop constraints and contact require additional treatment; the tree algorithm alone does not solve every constrained contact problem.

Configuration Space
The set of configurations allowed by a model, including positions, orientations, and declared geometric constraints. It is a smooth manifold when suitable regularity conditions hold; redundant coordinates and constraint singularities require care. Velocities are additional state information.

Contraction Analysis
Analysis of convergence between trajectories under stated common inputs or closed-loop dynamics, in a specified metric and domain. A differential contraction inequality needs regularity, metric bounds, and domain/existence conditions to imply the corresponding finite-distance result. Contraction does not by itself prove input feasibility or a desired contact outcome.

Control Contraction Metric (CCM)
A positive-definite differential metric satisfying control-dependent contraction conditions that support feedback construction for feasible reference trajectories. The result depends on the theorem's domain, regularity, metric bounds, and controller construction. Actuator saturation and state constraints require explicit treatment.

Degrees of Freedom (DOF)
The number of locally independent coordinates needed to specify a configuration. Independent constraints can reduce it; redundant parameters can exceed it. The number of actuators and the dimension of the full state space are different quantities.

Drift
The evolution of the declared effective plant when the declared control channel is zero. It includes the configuration-velocity kinematics and all retained uncontrolled dynamics, such as gravity, inertial coupling, passive elasticity, and damping. Its meaning depends on the chosen state, control channel, and environment; zero control does not mean zero acceleration or zero velocity.

Drift-Control Ratio (DCR)
A model-based comparison of drift and bounded control authority in a declared acceleration

or task-projected space:

$$\mathrm{DCR}_{W,\mathcal{U}}(x) = \frac{\|W a_{\mathrm{drift}}(x)\|_2}{\sup_{u \in \mathcal{U}(x)} \|W B_a(x) u\|_2 + \varepsilon}.$$

State the projection/scaling W, admissible input set $\mathcal{U}$, acceleration input map B_a, and regularizer ε, with compatible units. This scalar magnitude ratio omits directional alignment and does not by itself establish controllability, cancellation feasibility, or a universal coaching threshold.

Euler Angles
Three parameters describing an orientation using a declared sequence of rotations. Different sequences have different singular sets. Angle rates are related to angular velocity by a configuration-dependent map and are not generally its Cartesian components.

Exponential Map
For a Lie group, the map from a Lie-algebra element to the group element reached at parameter one along its generated one-parameter subgroup; for a matrix Lie group it is the matrix exponential. It need not be globally one-to-one. It is not general integration of an arbitrary time-varying velocity. The Riemannian exponential defined by geodesics is a related but distinct construction requiring a connection or metric.

Flow
The solution map of an ordinary differential equation where solutions exist uniquely. For an autonomous smooth vector field it forms a local one-parameter family with the composition property. Time-dependent systems use a two-time solution map; hybrid resets need not belong to a single smooth flow.

Gimbal Lock
A singularity of an Euler-angle rate-to-angular-velocity map. The parameterization loses local rank; the rigid body's physical rotational freedom does not disappear. Quaternions or overlapping coordinate charts handle orientation differently and have their own representation conventions.

Jacobian
The derivative of a map expressed in coordinates. For a task map $y = h(q)$, $\dot{y} = J(q)\dot{q}$. Acceleration also contains $\dot{J}\dot{q}$. Spatial and body Jacobians use different frame conventions, which must match the represented velocities and forces.

Kinematics
Description of position, orientation, velocity, and acceleration relationships without using forces to determine the motion. A kinematically allowed path is not automatically dynamically feasible under available actuation.

Lagrangian Mechanics
A formulation using a Lagrangian, often $L = T - U$, together with applied generalized forces and any constraints. For unconstrained coordinates, $d(\partial L/\partial \dot{q})/dt - \partial L/\partial q = Q$. Dissipation, actuation, nonconservative loads, and contact must be specified; kinetic and potential energy alone do not determine a forced system's motion.

Lie Algebra
A vector space equipped with a bilinear antisymmetric bracket satisfying the Jacobi identity. For a Lie group, it is the tangent space at the identity with the induced bracket. Brackets of smooth vector fields are another important example; accessibility arguments require the appropriate system and theorem assumptions.

Lie Group
A smooth manifold that is also a group, with smooth multiplication and inversion. Examples include the rotation group $SO(3)$ and rigid-transformation group $SE(3)$. A choice of coordinates represents these objects without changing the group itself.

Lyapunov Stability
For an equilibrium, the property that sufficiently small initial perturbations remain small for all future times of the stated system. It does not require return to equilibrium. Asymptotic stability additionally requires attraction; orbital and finite-horizon stability statements refer to different targets and must be defined separately.

Manifold
A space locally described by Euclidean coordinates, with the usual topological regularity conditions. A smooth manifold also has smoothly compatible charts. Its dimension is local and does not depend on how many coordinates are used to embed or redundantly represent it.

Screw Axis
For a rigid-body twist with nonzero angular velocity, an instantaneous axis and pitch describing rotation together with translation along that axis. Pure translation is a limiting case, often represented as an axis at infinity; a finite unique rotational axis is then unavailable.

State Space
The set of variables sufficient to determine future evolution under specified inputs and a model. Mechanical models often include configuration and velocity or momentum, and may additionally include actuator states, flexible modes, estimators, or discrete contact modes.

Tangent Bundle
The union of tangent spaces T_qQ over configurations $q \in Q$, with its bundle and smooth-manifold structure. For an n-dimensional smooth configuration manifold, TQ has dimension $2n$. It represents configuration-velocity states for a basic mechanical model; momentum states instead lie in the cotangent bundle T^*Q.

Tangent Space
The vector space of instantaneous derivatives of smooth curves through a point on a manifold. Its vectors are local velocities, not finite displacements between arbitrary configurations. Applied differential constraints can restrict the admissible velocities further.

Twist
A six-component representation of an instantaneous rigid-body velocity field, combining angular and linear terms with a declared reference point and frame. In one frame, $v(r) = v_0 + \omega \times (r - r_0)$. Spatial and body representations are related by the appropriate adjoint transformation; the linear term must not be confused with the velocity of an unspecified point.

Wrench
The dual force representation consisting of a moment about a declared reference point and a resultant force in a declared frame. With matching twist conventions, mechanical power is $\tau \cdot \omega + F \cdot v_0$. Changing point or frame requires the corresponding moment shift and dual transformation so that power is unchanged.

Zero-Torque Counterfactual (ZTCF) Family
Model interventions setting the declared applied generalized-control channel to zero while preserving the declared effective plant. Pointwise samples, stitched traces, forward trajectories, and branched trajectories answer different questions. A stitched pointwise acceleration trace is not automatically a feasible unforced trajectory.

Zero-Velocity Counterfactual (ZVCF)
The instantaneous acceleration after the declared velocity and control intervention:

$$a_{\mathrm{ZVCF}}(q, z) = -M(q)^{-1} h(q, 0, z_0; p).$$

Specify the auxiliary-state reset z_0, retained loads, parameters, and contact mode. This separates the retained configuration-dependent loads under that intervention; it is not necessarily gravity alone, and it is not a forward simulation of a stationary body.

Further Reading and References

The following books develop complementary parts of the framework used here. Choose a source for the particular claim or calculation being investigated, and consult its assumptions, notation, edition, and errata. A robotics or control theorem does not become evidence about human golf performance merely because it can be applied to a simplified swing model.

Geometry, Kinematics, and Mechanical Models

- **A Mathematical Introduction to Robotic Manipulation**, Murray, Li, and Sastry [35], treats rigid-body motion, manipulator and hand kinematics, dynamics, and nonholonomic motion planning. The authors' companion site provides chapter information and corrections. Use it to connect Lie groups and Jacobians to explicitly defined frames and mechanical systems.

- **Geometric Control of Mechanical Systems**, Bullo and Lewis [6], develops differential-geometric modeling and control of mechanical systems. Its companion page includes contents, selected material, examples, and errata. This is a suitable route into connections, mechanical control structures, and the hypotheses behind geometric analysis.

- **Robot Modeling and Control**, Spong, Hutchinson, and Vidyasagar [44], covers kinematics, Jacobians, trajectory planning, dynamics, joint and multivariable control, force control, and vision-based methods. The publisher's chapter listing identifies the scope of the cited edition. Its robot models still require adaptation and validation before supporting a biomechanical interpretation.

Computational Rigid-Body Dynamics

- **Rigid Body Dynamics Algorithms**, Featherstone [13], develops spatial-vector conventions and recursive dynamics algorithms. The author's teaching materials and software resources support implementation. Distinguish the linear-cost articulated-body calculation for a tree from the additional work needed for closed loops, constraints, and contacts.

Nonlinear Stability and Feedback

- **Applied Nonlinear Control**, Slotine and Li [43], develops nonlinear stability and feedback methods. Slotine's accompanying course materials identify the relevant book sections and separate the later contraction lectures and papers. Check the domain and model conditions of a stability argument before extending it to saturated or hybrid systems.
- **Contraction Theory for Dynamical Systems**, Bullo [5], develops contractivity and incremental stability using specified norms and metrics. The author's book page provides version information and materials. The bibliography cites the 2024 edition; later revisions may reorganize or extend the treatment. Contraction claims must retain their metric, input, domain, and existence assumptions.

The chapter-level primary references supply more specialized results for transverse dynamics, phase constraints, nonlinear funnels, stochastic models, and learning. Keep mathematical derivations, numerical verification, model calibration, and experimental validation identifiable when combining those sources.

Index

Bibliography

[1] Vladimir I. Arnold. *Mathematical Methods of Classical Mechanics*. Springer, 2nd edition, 1989.

[2] Karl J. Åström and Richard M. Murray. Feedback systems: Transfer functions, 2020. https://www.cds.caltech.edu/~murray/books/AM08/pdf/fbs-xferfcns_24Jul2020.pdf. Author-hosted chapter dated 24 July 2020, section 9.2, page 9-13; accessed 2026-09-08.

[3] Robert Stawell Ball. *A Treatise on the Theory of Screws*. Cambridge University Press, 1900. Legacy duplicate key retained for backwards compatibility.

[4] R. W. Brockett. Robotic manipulators and the product of exponentials formula. *Mathematical Theory of Networks and Systems*, pages 120–129, 1984. Legacy duplicate key retained for backwards compatibility.

[5] F. Bullo. *Contraction Theory for Dynamical Systems*. Kindle Direct Publishing, 2024.

[6] F. Bullo and A. D. Lewis. *Geometric Control of Mechanical Systems*. Springer, 2004.

[7] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[8] Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, and Shirley Ho. Lagrangian neural networks. *ICLR Workshop on Integration of Deep Neural Models and Differential Equations*, 2020. Neural networks that learn Lagrangian structure for physical systems.

[9] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.

[10] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955. Legacy duplicate key retained for backwards compatibility.

[11] R. Featherstone. The calculation of robot dynamics using articulated-body inertias. *International Journal of Robotics Research*, 2(1):13–30, 1983.

[12] R. Featherstone. *Robot Dynamics Algorithms*. Kluwer Academic Publishers, 1987. Legacy duplicate key retained for backwards compatibility.

[13] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.

[14] Roy Featherstone. Spatial vector and rigid-body dynamics software: Spatial_v2, 2012. Reference algorithms and coordinate-transform documentation; accessed 2026-09-07.

[15] Herbert Goldstein, Charles Poole, and John Safko. *Classical Mechanics*. Addison-Wesley, 3rd edition, 2002.

[16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Legacy duplicate key retained for backwards compatibility.

[17] Sam Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. *NeurIPS*, 2019. Neural networks that learn to conserve Hamiltonian structure.

[18] Ernst Hairer. Challenges in geometric numerical integration. https://www.unige.ch/~hairer/preprints/hairer-pisa.pdf. Author-hosted review; sections 1–2 and the step-size discussion in section 3; accessed 2026-09-07.

[19] Brian C. Hall. *Lie Groups, Lie Algebras, and Representations*. Springer, 2nd edition, 2015. Legacy duplicate key retained for backwards compatibility.

[20] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[21] Hassan K. Khalil. Nonlinear systems and control: Lecture 10, the invariance principle. https://www.egr.msu.edu/~khalil/NonlinearSystems/Sample/Lect_10.pdf. Author lecture notes, pendulum example and LaSalle hypotheses; accessed 2026-09-08.

[22] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.

[23] Solmaz Kia. MAE270A: Concepts of controllability and reachability for LTV systems. https://solmaz.eng.uci.edu/Teaching/MAE270A/Lec/ControllabilityReachability_StateFeedback.pdf. University of California, Irvine lecture notes, especially pages 5–6; accessed 2026-09-08.

[24] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.

[25] J. Y. S. Luh, M. W. Walker, and R. P. C. Paul. On-line computational scheme for mechanical manipulators. *Journal of Dynamic Systems, Measurement, and Control*, 102(2):69–76, 1980. Legacy duplicate key retained for backwards compatibility.

[26] Michael Lutter, Christian Ritter, and Jan Peters. Deep lagrangian networks: Using physics as model prior for deep learning. In *International Conference on Learning Representations*, 2019.

[27] Kevin M. Lynch and Frank C. Park. Modern robotics: Controllability of wheeled mobile robots, part 4. Official companion video transcript; accessed 2026-09-07.

[28] Kevin M. Lynch and Frank C. Park. Modern robotics: Numerical inverse kinematics, part 1. Official companion video transcript; accessed 2026-09-07.

[29] Kevin M. Lynch and Frank C. Park. Modern robotics: Task space and workspace. Official companion video transcript; accessed 2026-09-07.

[30] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.

[31] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.

[32] Jerrold E. Marsden and Tudor S. Ratiu. *Introduction to Mechanics and Symmetry*. Springer, 2nd edition, 1999. Legacy duplicate key retained for backwards compatibility.

[33] MathWorks. c2d: Convert model from continuous to discrete time. https://www.mathworks.com/help/control/ref/dynamicsystem.c2d.html. Official reference, zero-order-hold input assumption; accessed 2026-09-08.

[34] MuJoCo Contributors. Computation: MuJoCo documentation, 2026. Online algorithm and contact-model documentation; accessed 2026-09-07.

[35] R. M. Murray, Z. Li, and S. S. Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.

[36] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. Legacy duplicate key retained for backwards compatibility.

[37] Pinocchio Contributors. Pinocchio: Rigid body dynamics algorithms and their analytical derivatives, 2026. Software description and feature documentation; accessed 2026-09-07.

[38] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[39] Olinde Rodrigues. Des lois géométriques qui régissent les déplacements d'un système solide. *Journal de Mathématiques Pures et Appliquées*, 5:380–440, 1840. Legacy duplicate key retained for backwards compatibility.

[40] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. Legacy duplicate key retained for backwards compatibility.

[41] A. S. Shiriaev, L. B. Freidovich, and S. V. Gusev. Transverse linearization for controlled mechanical systems with several passive degrees of freedom. *IEEE Transactions on Automatic Control*, 55(4):893–906, 2010.

[42] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009. Legacy duplicate key retained for backwards compatibility.

[43] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.

[44] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2006.

[45] Gilbert Strang. *Linear Algebra and Its Applications*. Wellesley-Cambridge Press, 5th edition, 2019. Legacy duplicate key retained for backwards compatibility.

[46] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999.

[47] Russ Tedrake. Underactuated robotics: Multi-body dynamics. https://underactuated.mit.edu/multibody.html. Author lecture notes; accessed 2026-09-07.

[48] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. Legacy duplicate key retained for backwards compatibility.